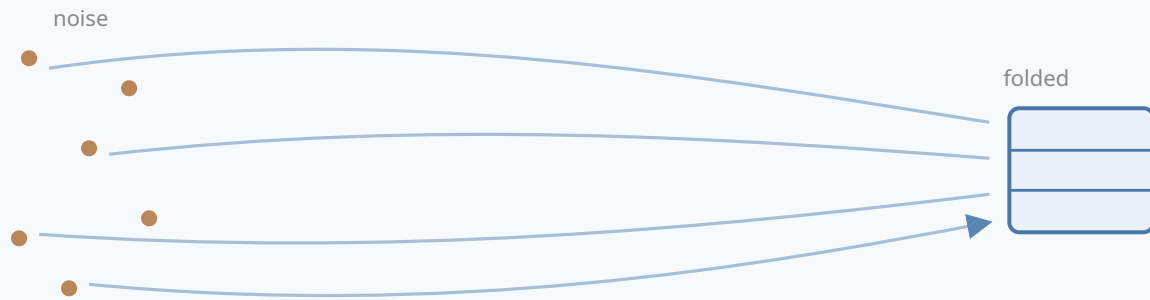




Reinforcement Learning for Real Robots

From PPO to diffusion, flow matching, and vision-language-action models — the theory behind the LeHome Challenge, and a path to replicating it

Second edition · compiled for Lucas · August 2026

Built around the open-sourced LeHome Challenge (ICRA 2026) solution and video series by Ilia Larchenko, and the papers it stands on.

Contents

PART I · FOUNDATIONS

1	The reinforcement learning problem	6
2	Policy gradients	17
3	Trust regions, PPO, and advantage estimation	31

PART II · LEARNING FROM DATA YOU ALREADY HAVE

4	RL as weighted behavior cloning	43
----------	---------------------------------	----

PART III · GENERATIVE POLICIES: THE NEW WORLD

5	Why one Gaussian is not a policy	67
6	Diffusion policies from zero	79
7	Flow matching	93
8	Vision-language-action models	104
9	PPO versus VLA-RL: the two worlds side by side	116
10	Learning by conditioning: RECAP	128

PART IV · MAKING IT WORK

11	Reward and advantage engineering	141
12	Inference-time optimization	154
13	Imitation and the human in the loop	166
14	Simulation, sim-to-real, and the asynchronous RL loop	178

PART V · DOING IT

15	Capstone: replicating LeHome	193
-----------	------------------------------	-----

APPENDICES

A	Paper-by-paper reading guide	207
B	The experiment ladder	213

Preface

This book answers one question: **what do you need to understand, and in what order, to train a modern learned policy on a real robot** — concretely, to replicate the winning solution of the LeHome Challenge (ICRA 2026), in which a bimanual SO-ARM101 robot learned to fold garments, first in simulation and then in the real world.

The LeHome solution is the running example because it is unusually complete and unusually open: code, tech report, checkpoints, and a three-part video series explaining every choice. One project touches nearly everything that matters in robot learning today — policy gradients and their limits, offline RL as weighted behavior cloning, diffusion and flow-matching action policies, vision-language-action models, reward and advantage engineering, inference-time optimization, human-in-the-loop data collection, and sim-to-real transfer. Follow the thread of this one problem and you cover the field.

About this second edition

The first edition was correct but compressed: it stated results where it should have built them. This edition roughly triples the explanation while keeping every equation. Three things changed in particular.

Nothing is asserted before you feel the problem. Each idea now arrives in the order it was actually discovered: here is what we want; here is the obvious thing to try; here is precisely how it fails; here is the fix, and why it works. The fix alone is a fact to memorize. The sequence is understanding.

Generative policies get a real on-ramp. The first edition compressed multimodality, flow matching, VLAs, and auxiliary heads into a single chapter. They are now five: Chapter 5 explains what a generative model even is and why a Gaussian stopped being enough; Chapter 6 teaches diffusion from zero; Chapter 7 does flow matching; Chapter 8 covers vision-language-action models; and Chapter 9 puts PPO and VLA-RL side by side, component by component, so you always know which world you are standing in.

Every equation is unpacked in prose. Mathematics is the compressed form of an idea, not the idea itself. Each important equation is followed by a plain-words box that names every symbol and says what the formula does. You should never have to decode notation alone.

How to read it

Chapters build strictly on one another; read them in order. Parts I and II take you from the MDP to the single most important update rule in the book. Part III is the conceptual pivot and the longest stretch — go slowly there, especially if diffusion and transformers are new to you. Part IV is engineering, Part V is your project.

Six kinds of box recur, and each has a job:

- **In plain words** — the preceding equation, restated in English, every symbol named.
- **Words you will hear** — vocabulary you will meet in papers and codebases around this topic, including terms this book does not otherwise use.
- **In the LeHome solution** — where the idea appears in the running example, with the real numbers.
- **On your rig** — how it connects to your own rover, HIL-SERL, and SO-101 work.
- **Pitfalls** — the mistakes that cost people weeks, including ones you have already hit.
- **Checkpoint** — at each chapter's end, what you should now be able to explain. If a bullet does not feel true, that section is worth a second pass before moving on.

Long derivations are sketched and referenced out to the original papers. **Appendix A** tells you which papers to read deeply, which to read once, and which to skim. **Appendix B** pairs every chapter with one concrete experiment on hardware you already own — reading a chapter without running its rung is half the lesson.

What this book assumes

Engineer-level probability and calculus: you can read an expectation, a gradient, and a Gaussian without looking anything up, and you are not expecting measure theory. Working Python. Some hands-on RL: you have trained an agent before and watched its learning curves. Everything past that — and all of Part III in particular — is built here from the ground up. No prior exposure to diffusion models, transformers, or vision-language models is assumed anywhere.

This book is one half of a pair. The other half is the roadmap (ROADMAP.md) that schedules these chapters against five months of building. The book tells you how things work; the roadmap tells you when to care.

Notation

One table, used everywhere. The only notational trap in the book is the difference between the environment's timestep t and the generative model's internal time u (or step index k); Chapters 6 and 7 warn about it again where it bites.

Symbol	Meaning
s, o, a	state, observation, action
$t; a_{t:t+H}$	environment timestep; action chunk with horizon H
$\pi_\theta, \pi_\beta, \pi_{\theta_{old}}$	learned policy, behavior/demonstrator policy, pre-update policy
τ, r_t, γ, G_t	trajectory, reward, discount, return $G_t = \sum_k \gamma^k r_{t+k}$
$V^\pi, Q^\pi, A^\pi, \hat{A}_t$	value, action-value, advantage $Q^\pi - V^\pi$, advantage estimate
δ_t, λ	TD residual $r_t + \gamma V(s_{t+1}) - V(s_t)$; GAE parameter
$J(\theta), \rho_t(\theta), \epsilon$	RL objective, importance ratio $\pi_\theta / \pi_{\theta_{old}}$. PPO clip parameter
$w_i = \exp(\hat{A}_i / \beta)$	AWR weight with temperature β
$\pi^*, \delta, Z(s)$	solution of the KL-constrained problem (Chapter 4), its leash radius, its normalizer; the star is unrelated to $\pi_{0.6}^*$
$k, \bar{\alpha}_k$	diffusion noise-level index and cumulative signal coefficient (Chapter 6)
u, z, a^u, v_θ, K	flow time $u \in [0, 1]$, noise $z \sim \mathcal{N}(0, I)$, interpolant, velocity field, integration steps
c, ϵ_A, g	advantage condition, its threshold, classifier-free guidance scale
$\hat{p}_{succ}, R_{acc}, \alpha, \rho_{GV}$	predicted success probability, accumulated reward, baseline damping, return–value correlation



PART I

Foundations

The problem every method in this book is trying to solve, and the classical machinery — policy gradients, PPO, advantage estimation — that solves it when samples are cheap.

Chapter 1 — The reinforcement learning problem

Every method in this book answers one underlying problem. The PPO agent that drove your rover attacks it — PPO is proximal policy optimization, developed in Chapters 2 and 3. So does the HIL-SERL loop on your SO-101 (human-in-the-loop sample-efficient robot RL, Chapter 13). So does the flow-matching policy (a generative action model: one that samples actions from a learned distribution instead of emitting a single number per joint — Chapter 7) that folded garments well enough to take first place in the LeHome Challenge's online round and second in its real-robot final. They differ in what they assume and what they cost, not in the problem they answer.

This chapter states that problem precisely, fixes the notation later chapters reuse, explains why physical robots make it far harder than a benchmark suggests, and draws the map you will navigate by. If you have trained a PPO agent, some of this is familiar. Read it anyway: Chapters 5 through 10 bend this vocabulary into unfamiliar shapes.

1.1 From supervised learning to decisions

Start with something concrete. Put a wooden cube on the table in front of your SO-101 and ask the arm to pick it up. Thirty times a second, some piece of software must choose six numbers — target angles, or deltas from the current angles — and send them to the servos. A three-second reach is ninety such decisions, five hundred and forty numbers. Nobody writes those by hand. This book is about where they come from.

1.1.1 The obvious thing to try, and where it breaks

The obvious first answer, and a good one, is supervised learning. Teleoperate with the leader arm, record fifty successful grasps, and you have a dataset of (camera image, joint command) pairs; train a network to map one to the other. This is **behavior cloning** — supervised learning of a controller from recorded demonstrations, treating each frame as an independent labeled example. It is the honest starting point for almost every robot learning project, LeHome included, and Chapter 4 develops it properly.

Two comforts hide in that setup, and control removes both. First, the feedback is **instructive**: the label says exactly what you should have produced, so the error is a vector pointing at the right answer. Second, mistakes have no consequences. Mislabel one ImageNet image and the next arrives unchanged; the data does not care what your model thinks.

Now watch behavior cloning fail on the cube, in the way it always fails. Your policy closes the gripper two centimeters early. The cube tips and rolls four centimeters left. The next camera frame shows a scene appearing nowhere in your fifty demonstrations, because all fifty succeeded and no successful demonstration contains a tipped cube. The network extrapolates, a polite word for guesses, and the error compounds: each wrong action moves the robot further from its training data, making the next action worse. This is not a capacity problem. It is structural, and it follows from the fact supervised learning ignores: *your actions determine your future inputs*.

The first comfort — instructive feedback — dies with the second. When the cube ends up on the floor, nobody tells you which of the ninety decisions was wrong. You get a score — failed — and nothing else. Feedback of this kind is **evaluative** (a number saying how good the outcome was) rather than instructive (a label saying what to do), and it is **delayed**: it arrives long after the decision that caused it. That combination creates the **credit assignment problem**: deciding which past actions deserve credit or blame for an outcome that shows up much later. Nearly everything technical in this book is a strategy for credit assignment.

1.1.2 An analogy, and its limit

Supervised learning is a graded exam with an answer key: you write an answer, the key says what it should have been, you adjust. Reinforcement learning is a game of chess. You make forty moves, you are told “you lost”, and you must work out yourself whether the loss traces to the pawn structure on move nine or the blunder on move thirty-seven. One number for the whole game, evaluative rather than instructive.

Now the limit, because an analogy without a stated limit misleads. Chess shows you the complete state of the world on the board, and its transitions are deterministic. A robot gets a camera image, a shadow of the state, and its transitions are noisy: the same joint command applied twice to the same apparent scene gives different results, because friction, backlash, and cable tension are not in the picture. The analogy captures delayed evaluative feedback exactly, and hidden state and stochastic dynamics not at all. Those two extras are most of what makes robot RL hard.

The formal picture is a loop between an **agent** (the thing that chooses) and an **environment** (everything else, including the robot's own body), drawn in Figure 1.1. At each environment timestep t the agent gets an observation, emits an action a_t , and the environment answers with a reward r_t and the next observation.

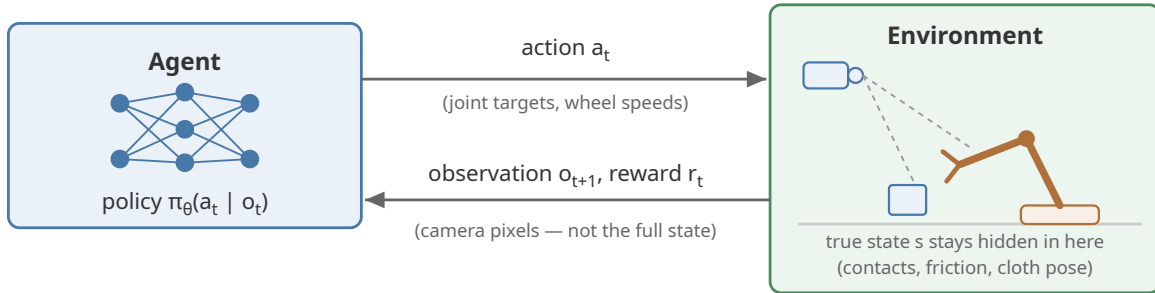


Figure 1.1 — the agent–environment loop. Notice that only three things cross the boundary: an action going right, an observation and a reward coming back. Notice what does *not* cross: the true state, which stays inside the environment. Almost every difficulty in this book follows from those two facts.

1.2 The MDP, built piece by piece

Textbooks introduce the formal model as a five-element tuple, all at once. That is efficient, and it is also why people bounce off it. We will take the pieces one at a time and write the tuple only at the end.

1.2.1 The five pieces

The state space $\mathcal{S}$ is the set of every situation the world can be in. For your rover a state is roughly the pose in the room, the wheel velocities, and the obstacle layout. For an SO-101 holding a cube it includes six joint angles, six joint velocities, the cube's pose, and the fingertip friction. Notice two things: the state is continuous rather than a list of cells, and it includes quantities you cannot measure.

The action space $\mathcal{A}$ is the set of choices at each step. Your rover's was a pair of wheel commands. The SO-101's is six joint targets, or six deltas from the current position; LeHome uses two arms, so its action is twelve numbers (Larchenko, 2026). These are continuous, which rules out the tabular value iteration of a gridworld tutorial.

The transition distribution $p(s_{t+1} | s_t, a_t)$ says what the world does in response. It is a distribution, not a function, because the same command in the same apparent situation gives different outcomes: the servo overshoots differently, the cloth catches on a wrinkle, a wheel slips. Simulators are no exception, and not for the reason people usually give: a contact solver is sensitive to ordering and thread timing, and domain randomization resamples friction, mass, and latency between episodes, so the same command in the same nominal state genuinely lands you in different next states. Seen from the policy's side it is worse still, because the policy conditions on an observation rather than the state, and many different states hide behind one image.

The reward function $r(s, a)$ emits one scalar per step saying how good that step was. Here is the underrated point: unlike p , the reward is not a fact about the world. You write it, and the agent optimizes what you wrote rather than what you meant. Chapter 11 is devoted to that.

The discount factor $\gamma \in [0, 1]$ says what a later reward is worth relative to one arriving now. It is a knob you choose, and the rest of this section is about it, because choosing it badly quietly ruins runs.

Together those are a **Markov decision process** (MDP): the tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$. Every algorithm here assumes some version of it.

1.2.2 The Markov property, and a shirt that violates it

The word “Markov” carries the defining assumption: the next state depends only on the current state and action, not on anything deeper in the past. Formally $p(s_{t+1} \mid s_t, a_t)$ is the whole story; also conditioning on $s_{t-1}, a_{t-1}, \dots$ tells you nothing more.

An analogy from your own trade: a Markov state is a complete save file. Serialize it, kill the process, restore it elsewhere, resume, and the future unfolds statistically the same way. Now the limit, and it is a big one: a camera image is not a save file, it is a screenshot. A screenshot shows the rendered scene, not the health counters, the seed, or what is behind the wall.

Make that concrete. One overhead photograph of a crumpled long-sleeved shirt does not determine what happens when you pull the visible corner. If the left sleeve is folded underneath, the pull drags the whole garment; if it lies free, the pull peels the top layer away. The pixels look nearly identical either way, so the image is not a Markov state — and training cannot fix that, because the information is not in the input.

A **trajectory** is one run through the loop: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$. An **episode** is a trajectory that ends — at a terminal state (shirt folded, rover crashed) or at a time limit. Episodes are where the pain lives on hardware: an episode boundary is a reset, and somebody has to perform it.

1.2.3 Returns: why we discount, and by how much

What the agent cares about is not one step's reward but everything that follows. The obvious definition is the plain sum $r_t + r_{t+1} + r_{t+2} + \dots$, and for a task that always ends in ten steps it is fine. It fails in two cases you will meet. If the task runs indefinitely — a rover patrolling a room — the sum can be infinite, and infinity minus infinity gives you nothing to optimize. And even for finite episodes, it calls a reward six hundred steps away as urgent as one right now.

The fix is to weight later rewards less, geometrically, by a factor γ per step. That gives the **return**:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k} = r_t + \gamma G_{t+1}.$$

IN PLAIN WORDS

G_t — the return — is the one number summarizing everything good that happens from timestep t onward. r_t is the reward collected right now, at full value. r_{t+1} is next step's reward at γ of face value, r_{t+2} at γ^2 , and a reward k steps away at γ^k . The middle form compresses that into a sum over the offset k . The last form says it recursively: today's return is today's reward plus a discounted copy of tomorrow's return.

Geometric weighting is not arbitrary: it is the only weighting that makes the recursion on the right true, and that recursion seeds nearly every algorithm here. Read forward it is a definition; read backward over a stored episode it is an algorithm — accumulate from the end, one multiply-add per step.

```
def discounted_returns(rewards, gamma):
    out = [0.0] * len(rewards)
    G = 0.0
    for t in reversed(range(len(rewards))): # sweep backward from the last step
        G = rewards[t] + gamma * G          # this line IS the recursion above
        out[t] = G
    return out                             # one return per input step, same length
```

How should you pick γ ? Use the **effective horizon** $N = 1/(1 - \gamma)$, roughly how far ahead the agent looks. That number is worth deriving rather than memorizing, because it is the one heuristic in this section you will use every time you start a project.

All the weights together, $1 + \gamma + \gamma^2 + \dots$, sum to $1/(1 - \gamma)$. The first N of them sum to $(1 - \gamma^N)/(1 - \gamma)$. Divide the second by the first and the $1/(1 - \gamma)$ cancels, so the fraction of the total weight that lands inside the first N steps is simply $1 - \gamma^N$. Now substitute $N = 1/(1 - \gamma)$. The quantity $\gamma^{1/(1-\gamma)}$ approaches $e^{-1} \approx 0.37$ as γ approaches 1 — at $\gamma = 0.9$ it is $0.9^{10} = 0.35$, at $\gamma = 0.99$ it is $0.99^{100} = 0.366$, at $\gamma = 0.999$ it is $0.999^{1000} = 0.368$. So the fraction inside the effective horizon is about $1 - 0.37 = 0.63$, and it is about that for every γ you might pick.

That is the content of the heuristic: roughly two-thirds of everything the agent is optimizing lies within $1/(1 - \gamma)$ steps of now, and rewards well past that barely move the objective at all. The horizon is not a hard cutoff — nothing is ignored exactly — but it tells you where the agent's attention is.

γ	effective horizon $1/(1 - \gamma)$	weight γ^{600} on a reward 600 steps away
0.9	10 steps	4×10^{-28} (effectively zero)
0.95	20 steps	4×10^{-14}
0.99	100 steps	0.002
0.999	1000 steps	0.55

The last column is the argument. A six-hundred-step folding episode paying out only at the end is invisible at $\gamma = 0.99$: the terminal reward reaches the first state scaled by $0.99^{600} \approx 0.002$, drowning in the noise of your value estimates. LeHome used $\gamma = 0.999$, keeping 55 percent alive at step zero (Larchenko, 2026). The discount is not a number to copy from a tutorial; it states how long your task is.

1.2.4 Worked example: returns by hand

Take a four-step gridworld episode like your rl-boarding project. The agent wanders two steps for nothing, clips an obstacle for -1 , then reaches the goal for $+10$: $(r_0, r_1, r_2, r_3) = (0, 0, -1, +10)$. Sweep backward with $\gamma = 0.9$:

```
gamma = 0.9

G_3 = r_3                = 10
G_2 = r_2 + 0.9 * G_3    = -1 + 0.9*10 = -1 + 9.0 = 8.0
G_1 = r_1 + 0.9 * G_2    = 0 + 0.9*8.0 = 0 + 7.2 = 7.2
G_0 = r_0 + 0.9 * G_1    = 0 + 0.9*7.2 = 0 + 6.48 = 6.48

check, summing forward from t=0:
gamma^0 * 0 = 1.000 * 0 = 0
gamma^1 * 0 = 0.900 * 0 = 0
gamma^2 * -1 = 0.810 * -1 = -0.81
gamma^3 * 10 = 0.729 * 10 = +7.29
total                = 6.48 (matches)
```

Two things to notice. The returns climb as the agent nears the goal — 6.48, 7.2, 8.0, 10 — though three of the four rewards are zero or negative; that gradient in G_t , not the sparse reward itself, is what value functions estimate and

policies climb. And seen from the start, the collision costs only $\gamma^2 \cdot 1 = 0.81$ while the goal contributes $\gamma^3 \cdot 10 = 7.29$, a ratio of nine to one.

Now run the identical episode at $\gamma = 0.5$ to see what the discount controls:

```
gamma = 0.5

G_3 = r_3 = 10
G_2 = r_2 + 0.5 * G_3 = -1 + 0.5*10 = -1 + 5.0 = 4.0
G_1 = r_1 + 0.5 * G_2 = 0 + 0.5*4.0 = 0 + 2.0 = 2.0
G_0 = r_0 + 0.5 * G_1 = 0 + 0.5*2.0 = 0 + 1.0 = 1.0

collision seen from t=0: gamma^2 * 1 = 0.25
goal      seen from t=0: gamma^3 * 10 = 1.25   ratio 5 : 1   (was 9 : 1)
```

The ordering of the two events did not change, but their relative weight did: the goal now outweighs the collision five to one instead of nine to one. That ratio is 10γ , so a smaller γ makes the nearby penalty loom larger against the distant prize. An impatient agent takes fewer risks for a far-off payoff — a real behavior change caused by one scalar in a config file.

WORDS YOU WILL HEAR

Rollout — one run of a policy in an environment, producing a trajectory; code calls it *traj* or *episode*, used interchangeably. **Episode** — a rollout that terminates; papers say “episodic” for tasks with resets. **Return** — the discounted sum G_t , also called *reward-to-go*. **Horizon** — how far ahead you care, either episode length or $1/(1 - \gamma)$; in this book H instead means the length of an action chunk, the number of future actions a policy emits in one shot, defined in the next section. **On-policy** — training only on current-policy data; *off-policy* reuses older data from a *behavior policy* π_β . **POMDP** — a partially observable MDP; a “belief state” is a distribution over hidden states.

1.3 Policies and value functions

A **policy** $\pi_\theta(a \mid s)$ is a distribution over actions given the state, with parameters θ — for us, network weights. One bookkeeping point before we go on: Figure 1.1 drew the policy as $\pi_\theta(a_t \mid o_t)$, conditioned on the observation, and here it is conditioned on the state s . That is deliberate, not a slip; Section 1.4 explains which letter the theory uses, which one your hardware actually hands you, and what it costs you that they differ.

That the policy is a distribution rather than one action usually strikes engineers as a defect. Surely we want the *best* action, not a sample?

Three reasons randomness is a feature. First, **exploration**: a deterministic policy never discovers a better action than the one it takes, because it never takes anything else. Second, the whole basis of Chapter 2: a stochastic policy makes expected return a smooth, differentiable function of θ even when the action choice is discrete. Third, visible only in Part III: some tasks genuinely have several correct answers — grasp the shirt at the left collar or the right, both fine — and a policy forced to commit to one number averages them and reaches for the middle, where there is no cloth. That is Chapter 5, and why the second half of this book replaces Gaussians with **generative models** — networks that sample from a learned distribution of arbitrary shape rather than outputting one number, or one mean and spread, per joint.

On your rover the policy was a Gaussian over wheel commands: the network output a mean and a spread, and you sampled. From Chapter 5 onward it becomes a generative model emitting a whole **action chunk** $a_{t:t+H}$ — the next H actions in one shot — rather than one action per step.

1.3.1 Value, action-value, advantage

Value functions summarize what a policy can expect. The **state value** $V^\pi(s) = \mathbb{E}[G_t \mid s_t = s]$ is the expected return from s when acting with π thereafter: on average, how well does this situation end? The **action value** $Q^\pi(s, a)$ is the

same, except the first action is forced to be $\mathbf{a}$: how well does it end if I do $\mathbf{a}$ right now?

Now the question that matters for training: given a state and an action, was that action any good? The obvious answer is to judge it by what followed. Run the policy, record the return that came after each action, and rank actions by that return. It is direct, it needs no extra machinery, and it is what the plain policy gradient of Chapter 2 does in its rawest form.

Take the four-step episode from Section 1.2 to see what that ranking really measures. With $(\mathbf{r}_0, \mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3) = (0, 0, -1, +10)$ and $\gamma = 0.9$ the returns were $G_0 = 6.48$, $G_1 = 7.2$, $G_2 = 8.0$, $G_3 = 10$. The action taken at the last step scores 10. The action taken at the first step scores 6.48. Ranked by return, the last action wins by a wide margin — and yet nothing about that comparison involved comparing choices.

Standing one step from the goal, every action that does not actively throw the episode away collects something near 10, the lazy ones included. Standing four steps out, even the best available action collects about 6.48, because that is all the situation is worth from there. So the ranking is mostly a readout of which situation the agent happened to be in, not which choice it made. A mediocre move in a winning position outscores an excellent move in a hard one, and if you train on that signal you reinforce having been lucky rather than having chosen well. On a real robot the spread across situations is far larger than in this gridworld — a folding policy that starts with the shirt half-flat is in a completely different position from one starting with it bunched — so this effect does not average out, it dominates.

The fix is to give every state its own baseline and grade the action against that baseline rather than against zero. The baseline is already sitting on the page: $V^\pi(\mathbf{s})$, what this situation is worth to this policy on average. Subtract it from $Q^\pi(\mathbf{s}, \mathbf{a})$ and what remains is the part attributable to the choice. That difference is the single most important derived quantity in this book, the **advantage**:

$$A^\pi(\mathbf{s}, \mathbf{a}) = Q^\pi(\mathbf{s}, \mathbf{a}) - V^\pi(\mathbf{s}).$$

IN PLAIN WORDS

$A^\pi(\mathbf{s}, \mathbf{a})$ asks: in state $\mathbf{s}$, is taking action $\mathbf{a}$ better or worse than what I would usually do here? $Q^\pi(\mathbf{s}, \mathbf{a})$ is the expected return if I take $\mathbf{a}$ now and behave normally afterward. $V^\pi(\mathbf{s})$ is the expected return if I just behave normally from now — my own average in this state. Subtracting removes everything about how good the situation itself is, leaving only the part attributable to the choice. Positive means better than my average, negative worse, zero typical. That baseline-relative reading is why almost every method here consumes the advantage rather than the raw return.

Put numbers on it, back in the gridworld, so you can watch the two rankings disagree. In the start state the policy averages $V^\pi(\mathbf{s}_0) = 6.48$, because its usual route clips the obstacle. Consider an action there that avoids the obstacle and lands in a state worth 8.0, collecting nothing on the way: $Q^\pi(\mathbf{s}_0, \mathbf{a}) = 0 + 0.9 \times 8.0 = 7.2$, so $A^\pi(\mathbf{s}_0, \mathbf{a}) = 7.2 - 6.48 = +0.72$. Now consider a state next to the goal, worth $V^\pi(\mathbf{s}_2) = 8.0$, and a dawdling action there that wastes a step and leaves the agent in a state worth 8.0 again: $Q^\pi(\mathbf{s}_2, \mathbf{a}') = 0 + 0.9 \times 8.0 = 7.2$, so $A^\pi(\mathbf{s}_2, \mathbf{a}') = 7.2 - 8.0 = -0.8$. Both actions score exactly 7.2 by raw return, so raw return cannot tell them apart at all. By advantage, one is clearly worth doing more often and the other clearly worth doing less, and the sign alone says which.

Hold on to that last phrasing: *how much better than my own average*. It scales gradients in PPO (Chapter 3), becomes an exponential weight $w_i = \exp(\hat{A}_i/\beta)$ in AWR (Chapter 4), a conditioning input in RECAP (Chapter 10), and the object of deliberate engineering in Chapter 11.

Two pieces of decoration in that weight will recur throughout the book, so meet them now. The hat on $\hat{A}_i$ marks an *estimate* computed from collected data, as opposed to the true A^π , which you never have; from here on, a hat always means “our best guess at this, built from samples”. And β is a temperature: it sets how sharply the weighting favors good actions, with small β making the exponential pick out only the very best and large β flattening it toward treating

all data equally (Chapter 4). Note that this β has nothing to do with the behavior policy π_β in the jargon box above. The letter is overloaded in the literature, and this book keeps both uses because you will meet both in papers.

Values obey a one-step recursion of their own, the **Bellman expectation equation** — the return identity $G_t = r_t + \gamma G_{t+1}$ with expectations applied to both sides:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot | s), s' \sim p(\cdot | s, a)} [r(s, a) + \gamma V^\pi(s')].$$

IN PLAIN WORDS

The value of being in state s equals the reward you expect to collect on the next step, plus γ times the value of wherever you land. $\mathbb{E}$ means average over two sources of randomness, named underneath it: $a \sim \pi(\cdot | s)$ is the action your own policy samples here, and $s' \sim p(\cdot | s, a)$ is the next state the environment produces in response. The centered dot means the whole distribution rather than its value at one point, so $\pi(\cdot | s)$ reads “the distribution over actions the policy would use in state s ” and $\sim$ reads “drawn from”. $r(s, a)$ is that immediate reward and $V^\pi(s')$ is the value of the landing state. The consequence is large: you can learn V from single transitions, without waiting for the episode to end.

That is what the TD residual $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ of Chapter 3 measures: how much one observed transition disagrees with the Bellman equation.

Finally, the objective. All of reinforcement learning is the search for parameters that maximize expected return from the start of the episode:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G_0] = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \gamma^t r_t \right].$$

IN PLAIN WORDS

$J(\theta)$ is one number: how good the policy with weights θ is. $\tau \sim \pi_\theta$ means “run the current policy and get a trajectory”, and $\mathbb{E}$ averages over all trajectories it could produce. G_0 is the return measured from the first step, which expands into the discounted sum on the right, γ^t shrinking rewards that arrive at later timesteps t . Training means climbing J in θ . The catch that defines the field: you can never evaluate $J(\theta)$ exactly, only estimate it by running the robot and averaging — and each estimate costs real seconds of hardware time.

Chapters 2, 3, 4, 9, and 10 are five strategies for climbing $J(\theta)$ when you can only sample it: differentiate through the sampling (2), do that safely in bounded steps (3), turn it into weighted imitation of data you already have (4), see which of those survive generative policies (9), and condition the policy on the advantage instead of weighting by it (10).

1.4 Why robot RL is hard

On paper the MDP is clean and the objective is a scalar to maximize. Five features of real robots make it hostile — five places where a method that works on a benchmark stops working on your bench.

1.4.1 Continuous, high-dimensional spaces

The tabular tools of gridworlds — enumerate states, store one value per cell, iterate — do not survive contact with hardware. One SO-101 has six continuous joints; a bimanual pair has twelve. And modern policies emit a chunk rather than an action: LeHome's predicts thirty timesteps at once, so one output is a $30 \times 12 = 360$ -dimensional vector (Larchenko, 2026).

So function approximation is mandatory, and it imports every instability neural networks have: the value function you train is wrong in ways that correlate with the states your policy visits, and the policy then optimizes against those errors. Much of Chapter 3 keeps that loop from exploding.

1.4.2 Partial observability

The robot never sees the state $\mathbf{s}$. It sees an **observation** $\mathbf{o}$ — camera pixels, encoder ticks, a sonar sweep. When observations do not determine the state, the problem is formally a **POMDP** (partially observable Markov decision process), and the guarantees MDP algorithms carry no longer apply.

This is the crumpled shirt of Section 1.2 under its formal name. The practical responses all smuggle the missing information back in: stack recent frames so velocity becomes visible, give the policy memory, add sensors, or accept the ambiguity and learn a policy whose action distribution covers both possibilities — again Chapter 5.

1.4.3 Which letter, $\mathbf{s}$ or $\mathbf{o}$? The book's convention

You may have noticed a tension. Section 1.3 wrote everything on states: the policy $\pi_\theta(\mathbf{a} \mid \mathbf{s})$, the value $V^\pi(\mathbf{s})$, the action value $Q^\pi(\mathbf{s}, \mathbf{a})$, the Bellman equation. Figure 1.1 wrote the policy on observations, $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$. And this section has just told you the robot never sees $\mathbf{s}$ at all. So are the value functions of Section 1.3 computable on real hardware, and which letter will later chapters mean?

Here is the convention, and it is worth fixing in your head now. The theory is stated on states, because the Markov property is a statement about states and the guarantees are proved there, so every formula in this book is written with $\mathbf{s}$. On hardware, every $\mathbf{s}$ you see in a formula is really an $\mathbf{o}$. Your network is fed pixels and encoder ticks; it computes something we write as $V(\mathbf{s}_t)$ but that is in fact $V(\mathbf{o}_t)$, and your policy is $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ no matter how the equation above it is printed.

The formulas do not stop working when you make that substitution — people train excellent policies this way every day — but they stop being exactly true, and the size of the error is precisely the POMDP tax described above. A value function on observations is trying to predict a return that depends on things the observation does not contain, so it is systematically wrong wherever two different states share one image. That is why the practical responses listed above are all attempts to make $\mathbf{o}_t$ more nearly a state. When a later chapter needs the distinction sharp — Chapter 5 on multimodality, Chapter 9 on what a generative policy can and cannot estimate — it will write $\mathbf{o}$ explicitly and say why.

ON YOUR RIG

Your rover environment is a POMDP by design: the policy sees a sonar sweep and recent velocities, never its pose in the map. Now you can name the choice — $\mathbf{o}_t$ is a thin sensor slice of a hidden $\mathbf{s}_t$. When the RPLIDAR and ROS 2 iteration lands, ask of every input: if two genuinely different situations produce the same reading, can any policy tell them apart? If one scan does not determine what happens next, stack scans or add memory before blaming the algorithm or the reward. Debugging a representation problem as a tuning problem costs weeks.

1.4.4 Sparse rewards over long horizons

“One if the shirt is folded, else zero” is an honest reward. It is also nearly information-free: hundreds of steps go by, the policy commits thousands of joint targets, and one bit comes back. Figure 1.2 draws that ratio to scale.

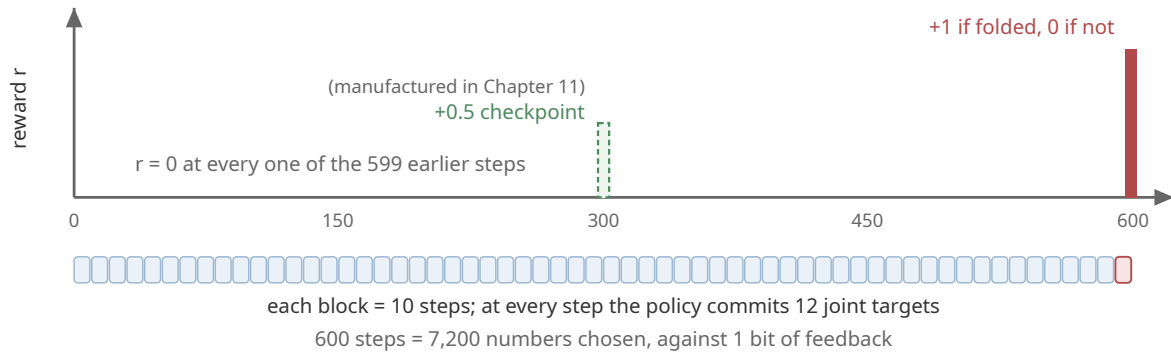


Figure 1.2 — one episode, one bit. The red spike at the far right is the entire learning signal for everything before it, and the blue strip shows the volume of decisions it must explain. Random exploration essentially never triggers that spike, so a from-scratch learner sees a flat line forever; the dashed green checkpoint is what Chapter 11 manufactures to break the silence.

Two different problems follow. First, exploration: random flailing essentially never folds a shirt, so a from-scratch learner sees reward zero forever and has nothing to climb. Second, credit assignment, which bites even once you succeed: the bit says the episode worked, not which of six hundred steps mattered. Chapter 11 manufactures denser signals without corrupting the task — and that second clause is the hard half, since an agent handed a proxy optimizes the proxy.

1.4.5 Sample cost and resets

PPO cheerfully consumes tens of millions of simulator steps, and in a simulator that is fine. A real arm at 30 Hz yields roughly a hundred thousand transitions per hour if nothing goes wrong. Things go wrong: motors heat and lose torque, cables snag, printed parts fatigue.

Worse, somebody must reset the scene — between folding episodes a human un-folds the shirt. You have felt this: your HIL-SERL run reached a 98 percent intervention rate, another way of saying a human was in the loop for essentially every step. Sample efficiency is not a nicety here; it decides which algorithms can exist on hardware at all, and it is why Chapters 4 and 13 work so hard to extract signal from data you already paid for.

1.4.6 The sim-to-real gap

Simulation buys cheap samples and free resets, which is why LeHome's online round ran in Isaac Sim. But simulated contact, latency, friction, and rendering differ from reality, and an optimizer finds and exploits exactly those differences — a policy can come to rely on a rendering artifact that does not exist in your kitchen.

The gap can be startlingly narrow and still fatal. In the LeHome real-world round, zero-shot transfer failed, and the policy proved sensitive even to the image-resize pipeline: 640 to 224 pixels directly, versus 640 to 320 to 224, was distinguishable to the model and dropped its success rate (Larchenko, 2026). Chapter 14 develops the two levers — make the domains more similar, or the training data more diverse.

PITFALLS

Treating observations as states. A memoryless policy reading $\mathbf{o}_t$ can only be as good as $\mathbf{o}_t$ is Markov, and no algorithm repairs a missing history; if two situations look identical to your network, tuning will not save you. *Setting γ by habit.* The discount fixes the effective horizon $1/(1 - \gamma)$. Copying $\gamma = 0.99$ onto a 600-step sparse task scales the terminal reward by $0.99^{600} \approx 0.002$ before it reaches the first state. Match γ to the task's horizon before tuning anything else, and re-check it whenever episode length changes.

1.5 A map of the field

The literature looks like a hundred unrelated acronyms. It is not. Four binary distinctions organize nearly all of it, and once you can place a method on them you can usually predict its strengths without reading the paper.

Model-free versus model-based. Does the algorithm learn the transition model p and plan through it, imagining futures before acting, or learn only a policy and some value estimates? Everything here is model-free. LeHome comes close to an exception: instead of a full world model it trains a cheap head predicting a few task-relevant keypoint distances — a world model shrunk to the numbers that matter (Chapter 8).

On-policy versus off-policy. Must training data come from the current policy π_θ , or may it come from older versions and other sources, written π_β ? On-policy methods like PPO throw data away after a few gradient steps — affordable in simulation, painful on hardware. Off-policy methods reuse a replay buffer and pay in extra machinery.

Online versus offline. Can you collect new data during training, or is the dataset frozen? This axis is about access to the robot, not the math, and people constantly conflate it with the previous one. Your HIL-SERL setup keeps them apart: it is online (fresh episodes continuously) and off-policy (it trains from a buffer of older data and human interventions).

RL versus imitation. Is the objective to maximize return or to match a demonstrator? They fail differently: imitation is stable but capped by the demonstrator and fragile off-distribution; RL can exceed the demonstrator but needs a reward and far more data. Figure 1.3 arranges the book's methods on the two axes that matter most day to day.

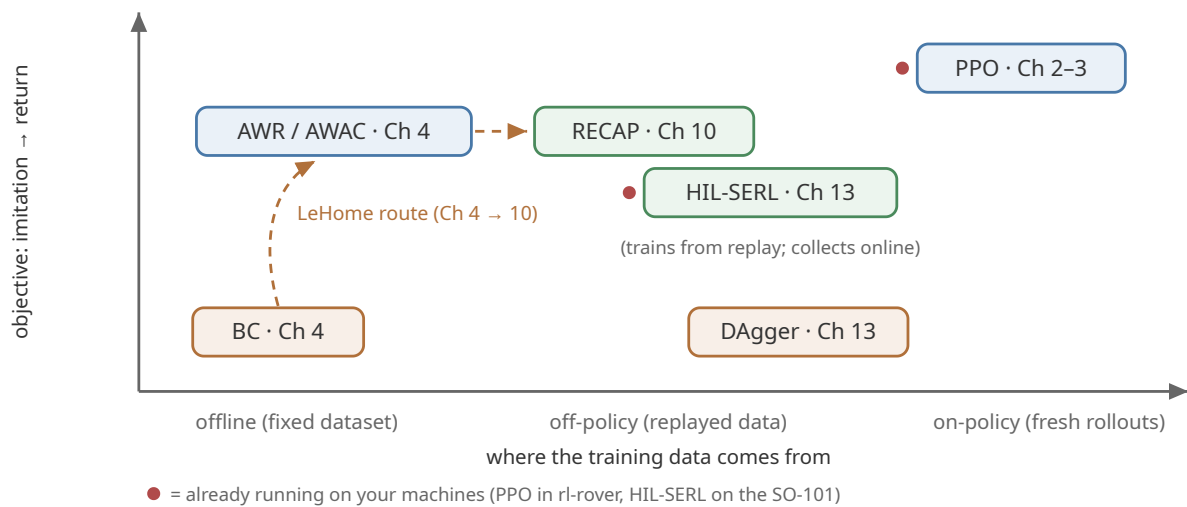


Figure 1.3 — the map of the field, and the book's route through it. Horizontal position says where training data comes from; vertical position says whether the objective is imitation or return. Orange chips are pure imitation, blue pure RL, green hybrids; all are model-free. The red dots mark where you already stand, and the dashed arrow is the path Chapters 4 and 10 take — note that it deliberately avoids the top-right corner.

Now walk the map, chip by chip. **Behavior cloning** sits bottom-left: supervised learning on fixed demonstrations, pure imitation, fully offline — and vulnerable to exactly the drift you saw with the tipped cube (Chapter 4). **DAGger** — dataset aggregation — keeps the imitation objective but moves right, going online: it rolls out the learner and asks an expert what to do in the states the learner actually reaches (Ross et al., 2011; Chapter 13). **HIL-SERL**, your pipeline, sits in the middle: online, off-policy RL with human interventions in the buffer, hence green (Luo et al., 2024; Chapter 13).

PPO occupies the top-right: online, on-policy return maximization, happiest in simulation where fresh rollouts are nearly free (Schulman et al., 2017; Chapters 2 and 3). That is the corner you know from rl-rover. **AWR** — advantage-weighted regression — sits high on the objective axis but far left on data: it maximizes return from off-policy or

offline data by recasting RL as *weighted* behavior cloning, which is the hinge of this book and why Chapter 4 is its own chapter (Peng et al., 2019). **AWAC**, advantage-weighted actor-critic, is AWR's online sibling: the same weighted-imitation update, but fed by a replay buffer that keeps growing as the policy collects fresh episodes (Nair et al., 2020). **RECAP** — RL with experience and corrections via advantage-conditioned policies — also learns from replayed rollouts, but injects the advantage as a conditioning input rather than a sample weight (Physical Intelligence, 2025b; Chapter 10).

Read the figure as the book's route. You have lived in the top-right corner with rl-rover and in the off-policy middle with hil-serl. Chapters 5 through 8 move you into the world of generative policies, and Chapter 9 delivers the bad news: for a flow-matching policy — one that produces an action by starting from noise and integrating a learned velocity field, rather than sampling from a Gaussian whose mean and spread the network printed out (Chapter 7) — the top-right corner is effectively unreachable. PPO-style updates need the probability the policy assigned to the action it took, and a policy built that way has no closed-form probability to hand you; getting one costs an expensive integration you cannot afford per gradient step. The dashed path — clone, reweight into RL with AWR, then condition on advantage with RECAP — is how LeHome extracted RL-grade improvement from BC-grade machinery, finishing first of sixty-two teams at 79.63 percent in the challenge's online round (Larchenko, 2026). Chapters 11 and 12 engineer the advantage and the inference procedure; Chapters 13 to 15 add the human, the simulator, and the capstone on your own hardware.

IN THE LEHOME SOLUTION

Garment folding is this chapter's MDP made concrete. Observations $\mathbf{o}_t$ are three RGB images, overhead plus one per wrist — the cloth's true configuration is never observed. Actions are chunks $\mathbf{a}_{t:t+H}$ of twelve-dimensional bimanual joint deltas, emitted by a policy derived from $\pi_{0.5}$, a vision-language-action model — a network mapping images and a text instruction to actions (Physical Intelligence, 2025a; Chapter 8). The raw reward is one bit, folded or not, after hundreds of steps, with $\gamma = 0.999$ to keep it alive. Everything else in Larchenko's system — checkpoint rewards (Chapter 11), AWR (Chapter 4), RECAP (Chapter 10), human-in-the-loop DAGger (Chapter 13) — is machinery for making this sparse, partially observed, long-horizon MDP tractable.

CHECKPOINT

- Explain, with the tipped-cube example, why supervised learning on demonstrations is not enough, and name the two comforts control removes.
- Write down the five parts of an MDP with an instance of each from your rover or SO-101, and say which part you author rather than discover.
- Compute discounted returns by hand with the backward sweep $G_t = r_t + \gamma G_{t+1}$, and predict how they change when γ shrinks.
- Choose γ from episode length via the effective horizon $1/(1 - \gamma)$, and say why **0.99** is the wrong default for a 600-step sparse task.
- Define V^π , Q^π , and A^π , and state the advantage in the “better than my own average” form without the formula.
- Place BC, DAGger, HIL-SERL, PPO, AWR, and RECAP on the online/offline and imitation/return axes, naming the chapter for each.

Where to go deeper. Sutton and Barto (2018), chapters 3 and 4, remain the canonical treatment of MDPs, returns, and Bellman equations; read them alongside this chapter if the value definitions still feel slippery. Kaelbling, Littman, and Cassandra (1998) formalize POMDPs, and Ibarz et al. (2021) catalog why deep RL on physical robots fails in practice. Both are optional context beyond this book's core list. Appendix A sequences Sutton and Barto against the papers of the coming chapters, and Appendix B gives you the first experiment to run on your own rig.

Chapter 2 — Policy gradients

Chapter 1 left you with an optimization problem: find the parameters θ of a policy π_θ that maximize expected return. This chapter develops the most direct attack anyone has found. Write expected return as a scalar function $J(\theta)$, estimate its gradient from trajectories you actually sampled, and take a step uphill.

Everything descended from that idea shares one load-bearing assumption. REINFORCE assumes it, actor-critic assumes it, and the PPO you have already run on rl-rover assumes it. We expose that assumption at the very end of the chapter, because it is exactly the assumption that stops holding for the generative policies of Part III. If you take one thing from this chapter, take what the assumption is and why it is invisible in classical deep RL.

This chapter goes slowly on purpose. The derivation in Section 2.2 is where most people first bounce off reinforcement learning — not because it is hard, but because it is usually presented in four lines with no commentary. We will spend several pages on those four lines.

2.1 The problem: the reward comes from the world

2.1.1 What we want

The quantity to maximize is the expected return of the policy:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right].$$

Read that carefully. τ is one trajectory, meaning one complete episode $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$. The subscript $\tau \sim \pi_\theta$ says the trajectories are drawn by actually running the policy in the environment. Inside, r_t is the reward at environment timestep t and $\gamma \in [0, 1]$ is the discount factor from Chapter 1. T is the length of the episode — the timestep at which the task ends or the time limit cuts it off. The whole expression is a single number for any given θ , and we want the θ that makes it large.

2.1.2 The obvious thing to try

You are a software engineer, so your instinct is to build the computation graph, call `.backward()`, and let autograd sort it out. In supervised learning that works because the loss is a differentiable function of the network's own output: you emit a logit, the loss compares it to a label, and the chain rule walks the derivative back through every layer.

So write the analogous thing for RL. Roll out an episode, sum the rewards, negate it to make a loss, call `.backward()`. Try it in PyTorch and you get an error, or worse, a silent zero. The reason is that the rewards are Python floats that came back from `env.step()`. They crossed a boundary that has no derivative attached to it. There is no graph node connecting r_t back to θ .

2.1.3 Why it fails, concretely

This is not a software limitation you could fix with a better library. It is a statement about where the rewards physically come from. Consider three rewards you might actually collect:

A motor controller saturates because you commanded more current than the driver can deliver, and the arm undershoots; the path from commanded torque to joint angle runs through firmware you did not write and cannot differentiate. A piece of cloth settles onto a table after you release it, and whether it landed folded or crumpled depends on thousands of contact events between fibers. And on your own SO-101 rig, the success reward is a human deciding the task looked done and pressing a button. None of these has a derivative with respect to your network weights, because none is a mathematical function of your weights at all. They are things that happened in the world.

The usual objection is that differentiable simulators exist — so why not train in one and backpropagate straight through the physics? Two answers. First, gradients through contact are genuinely pathological, not merely inconvenient. Move a gripper finger one millimeter left and it misses the object entirely, so the gradient of the outcome with respect to finger position is zero. Move it one millimeter right and it makes contact, and the gradient becomes enormous and stiff. The true derivative flips between zero and huge across a knife-edge, which makes it close to useless as a descent direction at any step size you would actually take. Second, a perfect differentiable simulator gives you the gradient of the *simulator*, and Chapter 14 is a whole chapter on why that is not the gradient of reality.

Here is the pivot the rest of the chapter turns on. The one thing θ genuinely controls is not the reward. It is *which trajectories you are likely to sample*. Change the weights and you change the probability distribution over episodes; the world then does whatever it does to each episode and hands back numbers. So we will stop trying to differentiate the reward and instead differentiate the sampling distribution. That single change of target is the entire content of the next section, and it is what makes reinforcement learning possible on hardware that no autograd system will ever see inside.

2.2 The score-function trick, in slow motion

2.2.1 The identity everything rests on

Start with a fact from first-year calculus. For any probability density $p_\theta(\mathbf{x})$ that is positive and differentiable in θ , the chain rule applied to the logarithm gives

$$\nabla_\theta \log p_\theta(\mathbf{x}) = \frac{\nabla_\theta p_\theta(\mathbf{x})}{p_\theta(\mathbf{x})}.$$

Multiply both sides by $p_\theta(\mathbf{x})$ and you have the identity we need:

$$\nabla_\theta p_\theta(\mathbf{x}) = p_\theta(\mathbf{x}) \nabla_\theta \log p_\theta(\mathbf{x}).$$

That is the whole trick. It is true by definition of the logarithm's derivative, there is nothing statistical about it, and you can verify it in one line as we just did. The quantity $\nabla_\theta \log p_\theta(\mathbf{x})$ has a name that you will see constantly in papers: it is the **score function** (the gradient of the log-probability with respect to the parameters). Some papers call the whole method the log-derivative trick or the likelihood-ratio estimator; they mean the same thing.

2.2.2 Why that identity is worth anything

Here is why we care. On the left is the gradient of a probability, which is not something you can average, because you never observe probabilities directly — only samples. On the right is a probability *multiplied by* something, and anything of the form " $p_\theta(\mathbf{x})$ times a function of $\mathbf{x}$ " is an expectation waiting to happen. The identity converts a derivative you cannot compute into an average you can.

2.2.3 The derivation, one step at a time

Write $R(\tau) = \sum_t \gamma^t r_t$ for the total discounted return of trajectory τ , and $p_\theta(\tau)$ for the probability of that trajectory when running π_θ . Then $J(\theta) = \int p_\theta(\tau) R(\tau) d\tau$, and:

$$\nabla_\theta J = \nabla_\theta \int p_\theta(\tau) R(\tau) d\tau = \int \nabla_\theta p_\theta(\tau) R(\tau) d\tau = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) R(\tau) d\tau.$$

Take those three equalities one at a time, because each does exactly one job. The first is just the definition of J written as an integral over trajectories. The second moves the gradient inside the integral, which is legitimate here because the space of trajectories does not itself change with θ — the same episodes remain possible, only the probability the policy assigns to each one moves. Note also that $R(\tau)$ is not differentiated, because the return of a *fixed* trajectory does not depend on θ at all. The third applies the identity: replace $\nabla_\theta p_\theta$ with $p_\theta \nabla_\theta \log p_\theta$. Now the integrand is once again "density times function", so we can write it back as an expectation:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau) \nabla_{\theta} \log p_{\theta}(\tau)].$$

IN PLAIN WORDS

The left side is what we want: the direction in weight space that most increases expected return. θ is the vector of all your network's parameters. τ is one complete episode. $p_{\theta}(\tau)$ is how likely that episode was under the current policy, and $R(\tau)$ is the total discounted reward it collected — just a number the world handed you. The score $\nabla_{\theta} \log p_{\theta}(\tau)$ is the direction that would make *this exact episode* more likely next time. $\mathbb{E}_{\tau \sim \pi_{\theta}}$ means "average over episodes you get by running the current policy". So: sample episodes, and nudge the weights toward each one in proportion to the reward it earned.

2.2.4 The moment the dynamics disappear

We are not finished, because $\nabla_{\theta} \log p_{\theta}(\tau)$ still looks like it needs to know the environment. Expand the trajectory probability into its factors:

$$p_{\theta}(\tau) = p(s_0) \prod_t \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t).$$

The episode starts from some initial state with probability $p(s_0)$; at each step you pick an action with probability $\pi_{\theta}(a_t | s_t)$; the world then moves you to a new state with probability $p(s_{t+1} | s_t, a_t)$. Taking the logarithm turns that product into a sum:

$$\log p_{\theta}(\tau) = \log p(s_0) + \sum_t \log \pi_{\theta}(a_t | s_t) + \sum_t \log p(s_{t+1} | s_t, a_t).$$

Now differentiate with respect to θ . The initial-state term contains no θ , so its gradient is zero. Every dynamics term $\log p(s_{t+1} | s_t, a_t)$ also contains no θ — the world's transition probabilities do not care about your network weights — so all of those gradients are zero too. What is left is only the policy terms:

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

Stop and appreciate this, because it is the "aha" of the whole subject. The transition model $p(s_{t+1} | s_t, a_t)$ — the thing you do not know, cannot write down, and would need a physics engine to approximate — has vanished from the gradient entirely. It was needed to *generate* the trajectory, and the robot generated it for us by simply existing. It is not needed to compute the gradient. That property has a name: the method is **model-free** (it requires no model of the environment's dynamics, only the ability to interact with it). This is why policy gradients work on a physical arm at all.

2.2.5 The loaded die: the whole method in one object

If the last three subsections felt like symbol-pushing — you followed each line but could not say what the thing *does* — that is the normal experience, and it is not a failure of yours. The derivation is four lines of calculus that encode one physical idea, and the calculus does a poor job of showing you the idea. This subsection is the idea. If you keep only one thing from this chapter, keep this.

The setup. You have a three-sided die. Each face pays a fixed amount stamped on it: face A pays \$1, face B pays \$2, face C pays \$9. The die is loaded, and you control the loading — a dial that sets how probable each face is. Right now it is fair: each face comes up a third of the time, so your average payout per roll is $(1 + 2 + 9)/3 = \$4$.

Two rules, and they are the whole of reinforcement learning in miniature. **Rule one: you cannot change the payouts.** They are painted on plastic. That is the environment. **Rule two: you cannot read the payouts either** — the only way to learn what a face pays is to roll it and get paid. You may only turn the dial.

2.2.6 The move, and the objection you should have

REINFORCE's instruction is startlingly crude. Roll the die. Whatever face comes up, turn the dial to make *that face* more likely, by an amount proportional to what it just paid. Repeat.

Suppose you roll B and collect \$2. You now turn the dial to make B more likely. Here is the objection, and if you had it you were reading correctly: *B is not the good face*. C pays four and a half times more. You just spent your update making a mediocre outcome more likely, using information (\$2) that says nothing about whether \$2 is good or bad. How can a rule this blind ever find C?

Three facts answer it, and they have to be taken together.

First, you are not only pushing B. The rule applies to every roll, so across many rolls every face gets pushed up — A when A comes up, C when C comes up. No face is singled out for promotion.

Second, the pushes have different strengths. A's push is worth 1, B's is worth 2, C's is worth 9. Each face pushes with a force equal to its own payout.

Third, and this is the part the equations hide: the pushes compete, because probabilities must sum to one. The dial has no absolute setting, only shares. Making B more likely necessarily makes A and C less likely. So the faces are in a tug of war over a rope of fixed length, and each pulls with a force equal to what it pays. C pulls nine times harder than A. It does not matter that no single roll knew C was best; the outcome of the tug of war is decided by the forces, and the forces are the payouts.

Now watch the bookkeeping do the rest. Roll three hundred times from the fair setting. Each face comes up about a hundred times. Face A accumulates about $100 \times 1 = 100$ units of upward push, B about $100 \times 2 = 200$, and C about $100 \times 9 = 900$. The totals are in the ratio 1 : 2 : 9 — exactly the ratio of the payouts — and you never enumerated the faces, never knew their probabilities, and never read a single number off the plastic. *The rolling did the weighting for you.* That is what an expectation is, and it is why "average over samples" is allowed to stand in for "expected value" in the estimator you just derived.

2.2.7 Watching the average go up, in numbers

The tug of war is easy to assert, so let us verify it with arithmetic small enough to check by hand. Take the average effect of one round of pushes: multiply each face's weight by $1 + 0.05 \times (\text{its payout})$, which is what "push proportional to payout" amounts to when each face has come up equally often. Starting from $1/3$ each:

```
face A: (1/3) x (1 + 0.05 x 1) = (1/3) x 1.05 = 0.3500
face B: (1/3) x (1 + 0.05 x 2) = (1/3) x 1.10 = 0.3667
face C: (1/3) x (1 + 0.05 x 9) = (1/3) x 1.45 = 0.4833
                                sum = 1.2000    <-- not 1: must renormalize

A: 0.3500 / 1.2 = 0.2917      B: 0.3667 / 1.2 = 0.3056      C: 0.4833 / 1.2 = 0.4028
```

Look at what happened to B. Its own roll pushed it *up* — and its share still fell, from 0.333 to 0.306, because C was pushed harder and the rope is only so long. That is the answer to the objection in one line: an update that raises a mediocre action can still lower its share. Face A, the worst, fell furthest, from 0.333 to 0.292. Face C rose from 0.333 to 0.403.

And the payoff, which is the only thing we were ever trying to move:

```
before: (1/3)(1) + (1/3)(2) + (1/3)(9)          = $4.00
after:   0.2917(1) + 0.3056(2) + 0.4028(9)
         = 0.2917 + 0.6111 + 3.6250              = $4.53
```

The average payout rose by fifty-three cents, from a rule that never compared two faces, never knew which was best, and made a mediocre face more likely on the roll we watched. Figure 2.1 shows the same round as a picture. Run the

update again and again and the dial walks toward C. That is REINFORCE, and there is nothing else in it.

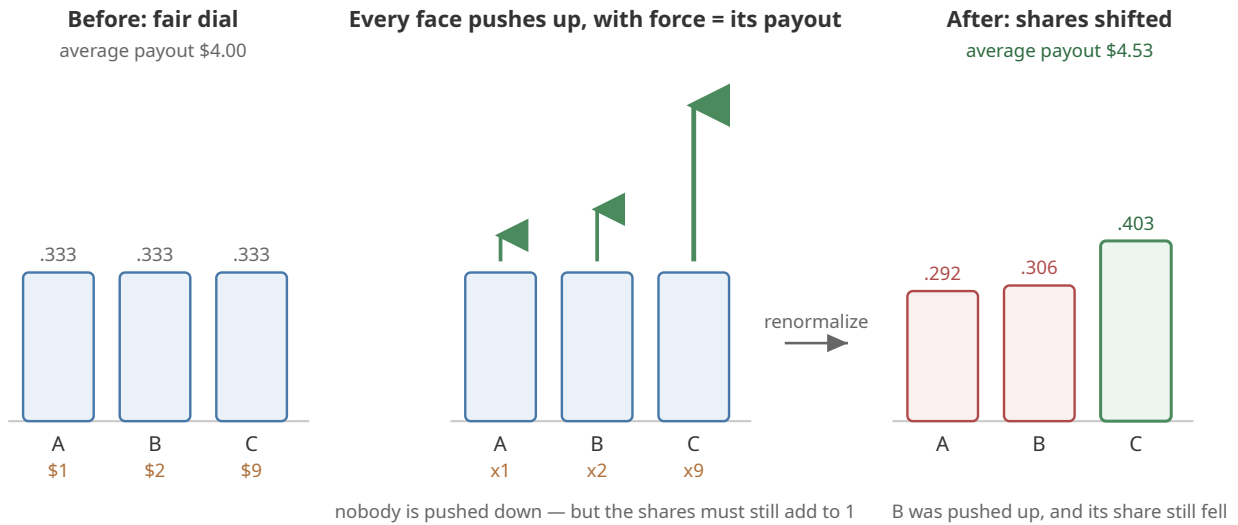


Figure 2.1 — One round of REINFORCE on a three-sided die with hidden payouts. The thing to notice is the middle panel: every face is pushed *up*, none is pushed *down*, and yet the right panel shows A and B losing share. That is because probability is a fixed budget — the arrows are competing for the same rope, and C pulls nine times harder than A. No step of this compares two faces or knows which is best; the payouts settle it by arithmetic. The average rises from \$4.00 to \$4.53 as a consequence, not as a goal any single roll pursued.

2.2.8 Reading the analogy back into the equation

Every piece of the estimator you derived is somewhere in that die, so line them up explicitly.

In the die	In the equation
The dial setting	The parameters θ
Rolling once	Sampling a trajectory $\tau \sim \pi_\theta$
The face that came up	The actions you actually took
"Turn the dial toward this face"	The score $\nabla_\theta \log \pi_\theta(a s)$
The payout stamped on it	The return $R(\tau)$, which you cannot differentiate
Push strength = payout	The product $R(\tau) \nabla_\theta \log \pi_\theta$
Faces come up as often as they are likely	$\mathbb{E}_{\tau \sim \pi_\theta}[\cdot]$, estimated by averaging
Shares must add to one	π_θ is a normalized distribution

The last row is worth dwelling on, because it is the one with no visible symbol. Nothing in the estimator explicitly says "push the bad actions down" — and with all-positive returns, nothing does. The downward motion is a side effect of normalization, which for a softmax or a Gaussian is baked into the policy's own definition. Improvement lives entirely in the *differences* between pushes.

That should strike you as a fragile way to run an algorithm, and it is. Take the same die and add a million dollars to every face: now they pay \$1,000,001, \$1,000,002 and \$1,000,009. Nothing about the *decision* has changed — C is still the face you want, by the same margin — but the pushes are now nearly identical in strength, the shares barely move, and you would need an astronomical number of rolls to see the difference through the sampling noise. A constant that carries no information about which face is better has wrecked the method.

The fix is to stop working with raw payouts and start working with payouts *relative to what you normally get*. Subtract the average from each. On the original die the average is \$4, giving pushes of $1 - 4 = -3$, $2 - 4 = -2$ and

$9 - 4 = +5$. On the million-dollar die the average is \$1,000,004, giving -3 , -2 and $+5$ — *the same three numbers*. The centered version cannot tell the two dice apart, which is exactly right, because as decision problems they are identical. Notice also what centering bought: A and B now carry *negative* pushes, so bad faces are driven down explicitly instead of merely being out-pulled. That subtraction is the **baseline**, it is the entire subject of Section 2.4, and you now know why it exists before you meet it.

2.2.9 Where the die breaks down

An analogy earns its keep by having a stated limit, so here are three.

Three faces, or infinitely many. A die has faces you can enumerate and will see within a few rolls. A policy over continuous joint torques has infinitely many "faces", and you only ever push the ones you sampled. The estimator learns literally nothing about actions never tried — which is why exploration matters so much, and why a sparse reward that fires only on a motion you have never stumbled into gives you exactly zero gradient, forever, no matter how long you run.

The die has no situations. One dial, rolled repeatedly under identical conditions. A policy must choose differently depending on what it sees, so "the dial" is really a function from observations to distributions, and the network's job is to share what it learns across similar situations. The die captures the credit-assignment mechanism and skips generalization entirely.

The payout is immediate. You roll, you get paid. In an episode you act ninety times and get one number at the end, and the estimator's honest position is that all ninety actions share the credit equally. Section 2.3 recovers what it can — an action cannot be responsible for rewards that arrived before it — but the fundamental difficulty stays, and it is what Chapter 3's advantage estimation and Chapter 11's reward shaping are both built to attack.

2.3 REINFORCE and reward-to-go

Putting the last two results together gives $\nabla_{\theta} J = \mathbb{E}[R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$. This is correct, but it contains an obvious piece of nonsense: it weights the action taken at timestep 50 by the *whole* episode return, including the rewards collected at timesteps 0 through 49. An action cannot cause a reward that was already banked before it was taken.

Removing those terms is not an approximation; it is exact. The reason is close to the baseline argument of the next section: multiplying a score function by something that does not depend on the action gives an expectation of zero. The argument needs one extra move over the baseline proof, though, because a past reward like r_3 is not a function of the current state s_{10} — it depends on states and actions from earlier in the episode. The extra move is to condition on everything that happened up to time t . Given that history, the past rewards are fixed numbers, and the average of the score over the action you are about to take is zero, so the product of the two averages to zero as well. Then average over histories and it is still zero. So we may drop the past rewards for free. What remains attached to each action is $\gamma^t G_t$, where $G_t = \sum_k \gamma^k r_{t+k}$ is the **reward-to-go** (the discounted return counted from timestep t onward, ignoring everything earlier).

Implementations then make one further simplification that is *not* free: they drop the leading γ^t . Start with why anyone would want to. The factor γ^t shrinks fast. At $\gamma = 0.99$, by timestep 300 it is down to about **0.05**, and by timestep 1000 it is about **0.00004**. Keeping it would therefore mute the gradient coming from everything late in a long episode, almost to silence. A robot that learns nothing from how its episodes actually *end* is not much use to you, so implementations drop the factor on purpose. You will not find γ^t in any codebase you read.

There is a price, and it is worth stating exactly. For $\gamma < 1$ that factor genuinely belongs in the true gradient, so removing it changes the expectation and the resulting estimator is mildly biased. Concretely, what you get instead is an estimator that counts each state by how often you actually visit it, rather than by how often you visit it discounted by how late in the episode you get there. That difference has a name attached to it — the factor d^{π} , which we will

meet in Section 2.4 when we state the policy gradient theorem. File the whole thing as a deliberate, well-understood approximation rather than an oversight. The estimator everyone actually uses is therefore:

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right]$$

IN PLAIN WORDS

For every timestep t in every episode you collected: look at the action a_t you took in state s_t , and ask what small change to the weights θ would have made that action more likely. That direction is $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$. Scale it by G_t , the total discounted reward that arrived *after* that moment. Add all of those scaled directions up, average over episodes, and step that way. In one sentence: make each action more likely in proportion to how well things went afterwards.

Averaging that over N sampled trajectories is the **REINFORCE** estimator (Williams, 1992). Figure 2.2 is the picture worth keeping in your head. Notice what the estimator does *not* require: a differentiable reward, a known transition model, or a dense signal. It requires exactly two things — the ability to sample from π_{θ} , and the ability to compute $\nabla_{\theta} \log \pi_{\theta}(a | s)$ for an action you sampled. Hold onto that second requirement. Section 2.6 is entirely about it.

The trick generalizes far beyond RL; the same estimator shows up in variational inference and anywhere else you must differentiate through a sampling step. But its generality is also its weakness. The reward enters only as a scalar multiplier on the whole score vector, so the estimator learns nothing from a trajectory's internal *structure*. It cannot tell you which of the two hundred actions in a failed fold attempt was the fatal one; it pushes all two hundred down a little and waits for averaging to sort out which were to blame. That is the **credit assignment** problem, and leaving it entirely to averaging is where the variance disaster of the next section comes from.

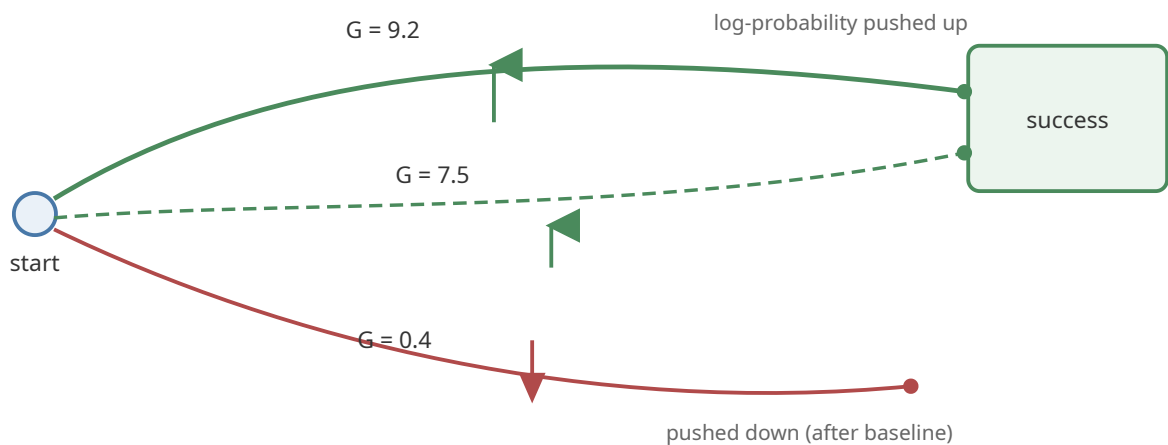


Figure 2.2 — The policy gradient in one picture. Three episodes sampled from the same start state; each is scored by the return G it happened to collect, and probability mass shifts toward the high scorers. Notice that nothing here requires knowing *why* the red trajectory failed — the method only reweights whole behaviors it has already seen, which is both its generality and its inefficiency. With a baseline (Section 2.4), below-average trajectories are actively pushed down rather than merely pushed up less.

PITFALLS

REINFORCE does not need a differentiable reward — a very common misreading. You never backpropagate through r_t or through the simulator; the gradient flows only through $\log \pi_{\theta}$, and the return is a constant multiplier. Second misreading: with all-positive returns, every sampled action is pushed *up*, and improvement comes only from actions being pushed up by different amounts while the probabilities renormalize. That is correct in expectation but has dreadful variance. Third: in code, always detach the advantage or return before multiplying, or autograd will happily backpropagate into your critic through the actor loss.

2.4 Variance, and why the naive version is unusable

Set aside the small γ^t bias we just discussed, which is not what this section is about; the exact reward-to-go estimator, before that simplification, is unbiased. Run it on infinitely many episodes and you get the true gradient. It is also so noisy that on a real robot it is worthless. That combination — correct on average, useless in practice — is worth sitting with, because the entire engineering history of policy gradients is about variance reduction, not correctness.

2.4.1 The 100/101 problem, worked out

To keep the arithmetic small, work at the level of whole trajectories rather than individual timesteps. Write $\mathbf{g}$ for the summed score of one episode, $\mathbf{g} = \sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$, and weight it by the return of that whole episode. That is the Section 2.2 form of the estimator, not the reward-to-go form we just built. Nothing in the variance argument depends on which of the two you use; the trajectory-level version simply has fewer numbers to carry.

Suppose your rover samples two episodes. Episode A returns 100; episode B returns 101. Say the score vectors happen to be, in some two-parameter toy policy,

$$\mathbf{g}_A = \nabla_{\theta} \log p_{\theta}(\tau_A) = (1.0, -0.4), \quad \mathbf{g}_B = \nabla_{\theta} \log p_{\theta}(\tau_B) = (-0.8, 1.2).$$

REINFORCE averages the score vectors weighted by the returns:

```
100 * g_A = (100 * 1.0, 100 * -0.4) = ( 100.0, -40.0)
101 * g_B = (101 * -0.8, 101 * 1.2) = ( -80.8, 121.2)
sum                                     = ( 19.2, 81.2)
average (divide by 2)                 = ( 9.6, 40.6)   <- the update
```

Now split that same update into two pieces by writing the weights as $100 = 100.5 - 0.5$ and $101 = 100.5 + 0.5$. That **100.5** is the average of the two returns you actually sampled. Strictly, a baseline computed from the same batch it is applied to is very mildly biased, because it peeks at the returns you happened to draw; everyone does it anyway, and the bias shrinks away as the batch grows.

```
common part: 100.5 * (g_A + g_B)/2 = 100.5 * ( 0.1, 0.4) = (10.05, 40.20)
signal part:  0.5 * (g_B - g_A)/2 =  0.25 * (-1.8, 1.6) = (-0.45, 0.40)
check: (10.05, 40.20) + (-0.45, 0.40) = (9.60, 40.60)   <- matches above
```

Look at the two pieces. The common part has length $\sqrt{10.05^2 + 40.20^2} = \sqrt{101.0 + 1616.0} \approx 41.4$. The signal part has length $\sqrt{0.45^2 + 0.40^2} = \sqrt{0.2025 + 0.16} \approx 0.60$. The common part is about 69 times larger than the signal part, and it points in a completely different direction.

Here is the sting. The common part contains no information at all. It is **100.5** times an average of score vectors, and by the identity we prove below, the expected value of a score vector is exactly zero. So the piece that dominates your update by a factor of 69 has mean zero: it is pure noise. The only piece that says "episode B was better than episode A" is the little **(-0.45, 0.40)** vector, half a percent of the total weight.

2.4.2 What that costs on real hardware

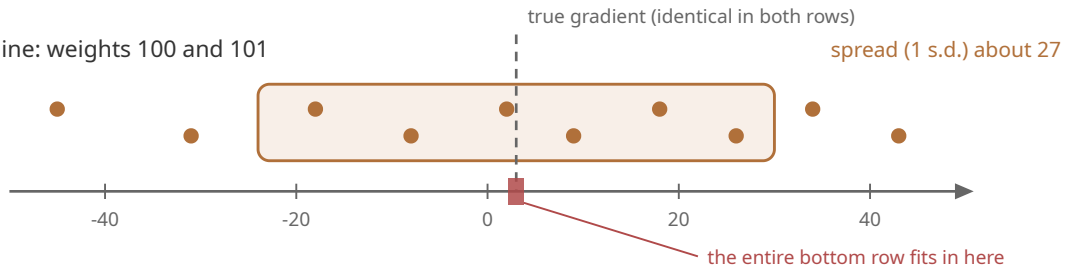
Noise averages out, but slowly. The standard error of a sample mean shrinks as $1/\sqrt{N}$, so shrinking the per-sample noise by a factor of k buys you the same accuracy with k^2 times fewer samples. Now run that backwards. Treat the 69 above as roughly the factor by which a baseline cuts the noise from batch to batch. It is an estimate, not an identity: 69 is the ratio of two vector lengths in one particular two-episode sample, and the honest version of the claim is that the batch-to-batch spread falls by something of that order. Even taken loosely, $69^2 \approx 4,800$, so without the fix you need something like four or five thousand times as many episodes to see through the noise.

On an Isaac Sim farm with four thousand parallel environments, that is annoying. On your single SO-101 arm it is fatal, and the arithmetic is worth doing out loud. Suppose 100 episodes with a baseline would get you a usable

gradient. At perhaps thirty seconds per episode including the reset, that is 3,000 seconds, under an hour. Without the baseline you need $100 \times 4,800 = 480,000$ episodes. At the same thirty seconds each that is 14.4 million seconds, or about 167 days: five and a half months of continuous running, resets included, with no failures. Manipulation makes it worse still: with the sparse binary success rewards typical of real tasks (Chapter 11), most episodes return exactly zero and contribute nothing but noise to the average. This is why nobody runs plain REINFORCE on hardware. Figure 2.3 shows the same phenomenon as a picture.

One component of the gradient estimate, over many repeated two-episode batches

without a baseline: weights 100 and 101



with baseline $b = 100.5$: weights -0.5 and $+0.5$

horizontal axis magnified 10x below (same units, stretched)



Figure 2.3 — What a baseline actually does. Each dot is one gradient estimate from a fresh two-episode batch, and the dashed line is the true gradient, which is *identical* in both rows because subtracting an action-independent baseline does not change the expectation. What changes is the spread: it shrinks by a factor of about seventy, which by the $1/\sqrt{N}$ rule means roughly five thousand times fewer episodes to reach the same accuracy. Notice that the bottom row had to be drawn at ten times the horizontal magnification of the top row to be visible at all — at the top row's scale, all ten of its dots would fall inside that small red segment on the axis. The lesson to carry forward: variance reduction, not bias correction, is what makes these methods run.

2.4.3 The fix: subtract a baseline

The fix is to subtract a number from G_t before using it as a weight. Call it a **baseline**, written $b(s_t)$: any quantity that may depend on the state but must *not* depend on the action. With $b = 100.5$ in the example above, the weights become -0.5 and $+0.5$, and the update becomes exactly the small signal vector $(-0.45, 0.40)$. The common-mode noise is gone. Each pair of samples now expresses a *preference* rather than a shared enthusiasm.

An analogy from grading. Telling a student "you scored 85" mixes two things: how hard the exam was, and how well the student did. Telling them "you scored 3 points above the class average" strips out the difficulty and keeps only the part that is about them. The class average is the baseline. Where this breaks down: the curve must be computed *without* looking at which answer this particular student gave. If you let the baseline depend on the action — say, by curving against only the students who gave the same answer — you destroy the comparison you were trying to make and the gradient becomes biased. That is the single condition the whole method rests on.

2.4.4 Why subtracting it is free

The remarkable fact is that *any* action-independent baseline leaves the gradient exactly unbiased. The proof is five lines and every one of them is elementary. For a fixed state s , with discrete actions:

$$\begin{aligned}
\mathbb{E}_{a \sim \pi_\theta} [b(s) \nabla_\theta \log \pi_\theta(a | s)] &= \sum_a \pi_\theta(a | s) b(s) \nabla_\theta \log \pi_\theta(a | s) \\
&= b(s) \sum_a \pi_\theta(a | s) \frac{\nabla_\theta \pi_\theta(a | s)}{\pi_\theta(a | s)} \\
&= b(s) \sum_a \nabla_\theta \pi_\theta(a | s) \\
&= b(s) \nabla_\theta \sum_a \pi_\theta(a | s) = b(s) \nabla_\theta 1 = 0.
\end{aligned}$$

Line by line: the first line just writes the expectation as a sum over actions weighted by their probabilities. The second pulls $b(s)$ out front, because it does not depend on a , and expands the score using our identity. The third cancels $\pi_\theta(a | s)$ top and bottom. The fourth swaps the finite sum and the gradient, and then uses the fact that probabilities sum to one — and the gradient of the constant 1 is zero. Replace sums with integrals for continuous actions and nothing changes. So subtracting b alters the variance and leaves the mean untouched. In statistics this is called a **control variate**.

The most useful baseline is the state value itself, $b(s_t) = V^\pi(s_t)$, which turns the weight into $G_t - V(s_t)$. That is a one-sample estimate of the **advantage** $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$: how much better this action turned out than what the policy would have gotten on average from that state. Better-than-average actions are pushed up, worse-than-average actions are pushed down, and it no longer matters whether the raw returns are all positive.

The **policy gradient theorem** (Sutton et al., 1999) states the general form, and it is the license for everything that follows in this book:

$$\nabla_\theta J(\theta) \propto \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) Q^\pi(s, a)]$$

IN PLAIN WORDS

The symbol $\propto$ means "equal up to a positive constant" — the direction is right, the scale is absorbed into your learning rate. d^π is the discounted state-visitation distribution: how often the current policy actually finds itself in each state, so $s \sim d^\pi$ means "states drawn from your own rollouts, not from anywhere you like". $a \sim \pi_\theta$ means the action was sampled by the current policy. $Q^\pi(s, a)$ is the expected return from taking action a in state s and then following π . The whole statement: the true gradient is an average, over states you visit and actions you take, of "make this action more likely" scaled by how good the action is.

By the baseline argument just proved, Q^π may be replaced by $A^\pi = Q^\pi - V^\pi$ without changing anything. The theorem's real message is that the exact gradient depends only on value quantities you can *learn to estimate*. That is the door into actor-critic. One caveat to plant now for Chapter 11: when the value estimate is imperfect, subtracting all of it can inject its errors into your gradient, and a damped baseline αV with $\alpha < 1$ is sometimes the better control variate.

2.4.5 A second worked example: one REINFORCE update by hand

Take the smallest interesting problem. One state, two actions, a softmax policy with logits $\theta = (\theta_1, \theta_2) = (0, 0)$, so $\pi_\theta(a^1) = \pi_\theta(a^2) = 0.5$. Action a^1 pays $r = 1$; action a^2 pays $r = 0$. Episodes are one step long, so $G_0 = r$.

For a softmax the score has a clean closed form. It is usually quoted rather than derived, but the derivation is one line of calculus and you should see it. The log-probability of action a^i is $\log \pi_\theta(a^i) = \theta_i - \log \sum_j e^{\theta_j}$. Differentiate that with respect to θ_j . The first term contributes 1 when $j = i$ and 0 otherwise. The second contributes $-e^{\theta_j} / \sum_k e^{\theta_k}$, which is exactly $-\pi_\theta(a^j)$. Put the two together:

$$\nabla_{\theta_j} \log \pi_\theta(a^i) = \mathbf{1}[i = j] - \pi_\theta(a^j),$$

where $\mathbf{1}[\cdot]$ is the indicator: 1 when the statement in brackets is true, 0 when it is false. At $\theta = (0, 0)$ both probabilities are 0.5, so the two score vectors are the ones computed below.

Use learning rate 1 throughout. *Case 1, no baseline, you sample a^1 .* The weight is $G_0 = 1$:

```
grad log pi(a1) = (1 - 0.5, 0 - 0.5) = ( 0.5, -0.5)
grad log pi(a2) = (0 - 0.5, 1 - 0.5) = (-0.5,  0.5)

update = 1 * 1 * ( 0.5, -0.5) = (0.5, -0.5)
theta  = (0, 0) + (0.5, -0.5) = (0.5, -0.5)
pi(a1) = e^0.5 / (e^0.5 + e^-0.5) = 1.6487 / (1.6487 + 0.6065)
       = 1.6487 / 2.2552 = 0.731
```

One lucky sample moved the policy from 50% to 73%. *Case 2, no baseline, you sample a^2 .* The weight is $G_0 = 0$, so the update is the zero vector and nothing happens. The bad action taught you nothing.

Now add the baseline $b = 0.5$, the policy's current expected reward. *Case 3* samples a^1 , weight $1 - 0.5 = +0.5$; *case 4* samples a^2 , weight $0 - 0.5 = -0.5$, with the score pointing the other way:

```
case 3: update = 1 * ( 0.5) * ( 0.5, -0.5) = (0.25, -0.25)
case 4: update = 1 * (-0.5) * (-0.5,  0.5) = (0.25, -0.25)

theta  = (0.25, -0.25)  # either way
pi(a1) = 1 / (1 + e^-0.5) = 1 / 1.6065 = 0.622
```

Stop and compare cases 3 and 4. They give the *same update*. Whichever action you happened to sample, the baseline version moves the policy to $\pi(a^1) = 0.622$. Now check the means. Without a baseline the update is $(0.5, -0.5)$ half the time and $(0, 0)$ half the time, averaging $(0.25, -0.25)$. With a baseline it is $(0.25, -0.25)$ every time. Identical expectation, and the variance has gone from nonzero to exactly zero. That is the mechanism of Figure 2.3, computed by hand instead of drawn. Do not read across the two too literally: this toy is small enough that the right baseline removes *all* the variance, which is the best case, whereas the figure shows the more typical outcome of a large reduction rather than a total one. And note what happened in case 4: the baseline extracted a useful update from a *failure*, which the naive estimator threw away.

WORDS YOU WILL HEAR

Score function — $\nabla_{\theta} \log \pi_{\theta}(a | s)$; also "log-derivative trick", "likelihood-ratio estimator". **Vanilla policy gradient (VPG)** — REINFORCE plus a learned value baseline; the thing PPO improves on. **Control variate** — the statistics name for a baseline: a zero-mean term subtracted to cut variance. **Unbiased** — correct on average over infinite samples, which says nothing about how noisy any one sample is. **Credit assignment** — deciding which of many actions caused an outcome. **On-policy** — the update is only valid for data generated by the current policy; **off-policy** methods reuse older data (Chapter 4).

2.5 Actor-critic in one page

The baseline $V^{\pi}(s)$ is not something you are handed. An **actor-critic** method makes it learnable, by training a second network to predict it. The **critic** $V_{\phi}(s)$ is that network — often just an extra output head on the policy's trunk — and it has its own parameter vector ϕ , kept deliberately separate from the policy's θ . Hold onto that separation; the pitfall box above and the `.detach()` in the code below are both about nothing else. The critic is trained by ordinary regression. You can fit it to the observed returns G_t , or you can **bootstrap** it — that is, train it toward its own prediction one step ahead, $r_t + \gamma V_{\phi}(s_{t+1})$, instead of waiting for the episode to finish. The **actor** is π_{θ} itself, updated by the policy gradient with an advantage estimate $\hat{A}_t$ in place of the raw return. Figure 2.4 shows how the pieces connect.

The two obvious advantage estimators bracket a spectrum. The **Monte Carlo** estimate — Monte Carlo meaning simply "use the actual observed return, no shortcuts" — is $\hat{A}_t = G_t - V(s_t)$. It uses the real return, so it is unbiased, but it carries all the noise of the actual episode. At the other end is the one-step **TD** residual,

$\hat{A}_t = \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$. TD stands for *temporal difference*: the gap between what the critic predicted for this state and what one real step plus the critic's prediction for the next state say it should have been. That estimator replaces most of the episode with the critic's guess, so it is far quieter, but it is biased by however wrong the critic currently is. GAE, in Chapter 3, interpolates smoothly between the two with a parameter λ , and that is the estimator you will actually use.

Two habits from practice are worth naming. Advantages are almost always normalized per batch to zero mean and unit standard deviation; technically that is a batch-computed baseline plus a rescaling — so, like any batch-computed baseline, mildly biased — and it exists to keep the effective step size stable across batches with different reward scales. And an **entropy bonus** — a small term in the actor loss rewarding the policy for staying stochastic — stops the policy collapsing to one deterministic action before it has explored. In Stable-Baselines3 that is `ent_coef`. One full iteration:

```
# one actor-critic iteration (on-policy)
batch = collect_rollouts(pi_theta, n_steps)      # fresh data, current policy
returns = discounted_returns(batch.rewards, gamma) # G_t
adv = returns - V_phi(batch.states)             # or TD / GAE (Chapter 3)
adv = (adv - adv.mean()) / (adv.std() + 1e-8)    # per-batch normalization
logp = log_prob(pi_theta, batch.states, batch.actions)
actor_loss = -(logp * adv.detach()).mean()      # detach: no grad into critic
critic_loss = ((V_phi(batch.states) - returns)**2).mean()
(actor_loss + 0.5 * critic_loss).backward(); opt.step()
```

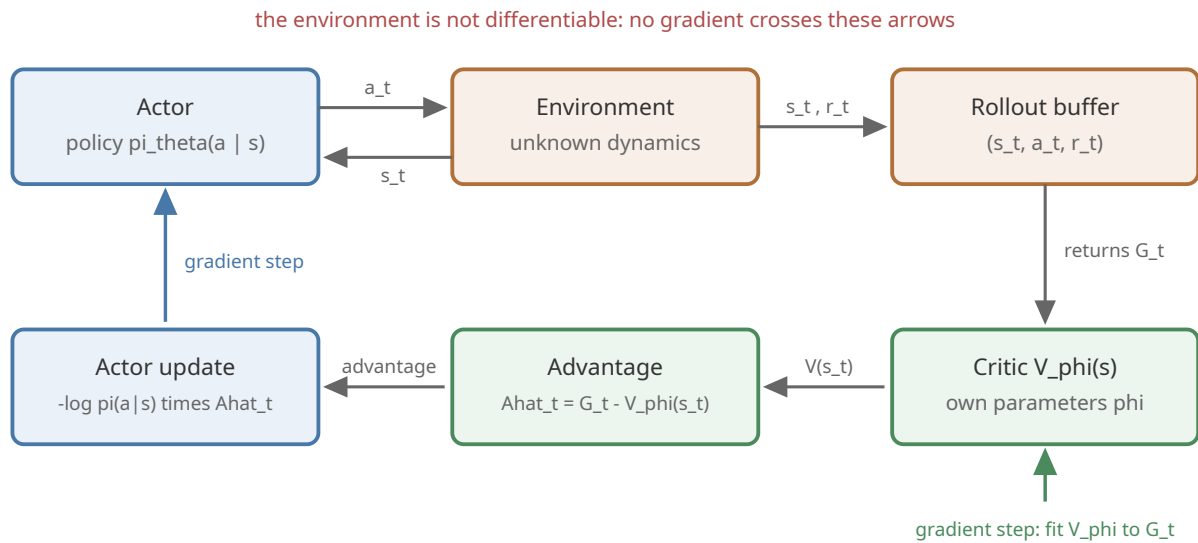


Figure 2.4 — The actor-critic loop, and the two places a gradient exists. Data flows clockwise: the actor acts, the world hands back a state and a reward, transitions are banked, the critic learns to predict returns, the difference becomes an advantage, and the advantage weights the actor's update. Both gradients live entirely inside your own code — the blue one, which is the only gradient that reaches the policy, and the green one, which is ordinary regression fitting the critic to observed returns. Neither ever crosses into the orange boxes; that is Section 2.1 drawn as a diagram.

Note the phrase *fresh data* in the first line of the pseudocode. The policy gradient theorem's expectation is taken under the *current* policy's own trajectory distribution. The moment you take one gradient step, the rollouts you already collected were generated by a different policy $\pi_{\theta_{old}}$, and reusing them naively biases the gradient. The principled correction is the importance ratio $\rho_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$, and Chapter 3 builds PPO around keeping that ratio near 1 so that a limited amount of reuse stays safe. This **on-policy** constraint — new data for every update — is why the whole family is so sample-hungry: cheap in a simulator running thousands of environments in parallel, brutal on one physical arm. Chapter 4 is the off-policy escape route, and it is the hinge of this book.

ON YOUR RIG

SB3's PPO on rl-rover is exactly this chapter's actor-critic plus Chapter 3's clip. `ActorCriticPolicy` is one object holding an actor (a diagonal Gaussian over wheel commands) and a critic (a scalar value head) — separate MLPs by default, a shared trunk only if `net_arch` says so. This chapter's score functions live in `common/policies.py`: `evaluate_actions()` calls `distribution.log_prob(actions)`, and `ppo.py` forms `ratio = th.exp(log_prob - old_log_prob)`. Put a breakpoint on that `log_prob` call and you are watching $\nabla_{\theta} \log \pi_{\theta}$ get assembled, one batch at a time.

2.6 The hidden requirement, and the bridge to Part III

Go back and look at every estimator in this chapter. REINFORCE, the baseline version, the policy gradient theorem, the actor loss in the pseudocode: all contain the same factor, $\nabla_{\theta} \log \pi_{\theta}(\mathbf{a} \mid \mathbf{s})$. Using any of them means you can evaluate the exact log-density of a sampled action and differentiate it with respect to the weights.

For classical deep RL policies that is trivial. A softmax over discrete actions is an explicit probability vector; take a log and index into it. A diagonal Gaussian — SB3's default for the rover's continuous wheel commands — has a closed-form log-density that fits in five lines:

```
# diagonal Gaussian policy: log pi(a|s) in closed form
mu, log_std = policy_net(obs)      # network emits mean and log sigma per dim
var = (2 * log_std).exp()          # sigma squared
logp = -0.5 * (((a - mu) ** 2) / var # squared-error term, per action dimension
              + 2 * log_std         # the log sigma normalizer
              + math.log(2 * math.pi)).sum(-1) # sum over action dimensions
logp.backward()                   # autograd now has grad_theta log pi
```

The requirement is met so cheaply, in every tutorial and every library, that it becomes invisible. Most people who have trained PPO for a year have never once thought of it as a requirement.

It stops being met in Chapter 5. Modern robot policies emit whole action **chunks** $\mathbf{a}_{t:t+H}$ — a short block of H future actions produced in one shot — by iteratively transforming random noise, and the network learns a denoiser (diffusion, Chapter 6) or a velocity field (flow matching, Chapter 7) rather than a density. No line of code in such a model computes $\log \pi_{\theta}(\mathbf{a}_{t:t+H} \mid \mathbf{o})$. The likelihood of a generated chunk is buried inside a multi-step integration path, and it has no closed form of the kind a Gaussian hands you for free.

Be careful how you state this, though, because the strong version — "the gradient cannot be computed at all" — is simply false, and you will meet papers that depend on its being false. The likelihood is not unavailable *in principle*. For a continuous-time flow you can recover it by integrating along the generation path, accumulating the divergence of the velocity field as you go; and there is a live line of work — you will see it under names like DPPO and Flow-GRPO — that sidesteps the integral entirely by treating the chain of denoising steps as its own small MDP and running PPO on *those* steps instead. What is true is that all of these are expensive, high-variance, and nothing like the free lunch a Gaussian gives you. In practice $\log \pi_{\theta}$ is off the table, and REINFORCE, actor-critic and PPO stall at the same wall for the same reason.

Tractability is also not the only obstacle, and arguably not the most important one. A policy-gradient update does not just push probability mass toward good actions; it pushes mass away from actions that scored badly. For a generative policy whose whole value is that it has learned a narrow manifold of physically coherent action chunks, pushing away is how you shove its predictions off that manifold and get incoherent motion out the other side. Chapter 9 puts the two worlds side by side, covers both failure modes, and works through which parts of your PPO intuition transfer and which do not.

What survives the crash is not the estimator but this chapter's central *quantity*: the advantage. Once you have $\hat{\mathbf{A}}$, you can weight training data by it (Chapter 4), condition the policy on it as an input (Chapter 10), engineer better

estimates of it (Chapter 11), and rank candidate action chunks by it at inference time (Chapter 12). Every one of those is a way of doing policy improvement without ever differentiating a log-probability. The estimator is disposable. The advantage is not. That sentence is the thesis of the rest of this book.

IN THE LEHOME SOLUTION

None of this chapter's estimators run on Larchenko's policy. $\pi_{0.5}$ generates action chunks by flow matching, so $\log \pi_{\theta}(\mathbf{a}_{t:t+H} \mid \mathbf{o})$ — the term every equation in this chapter differentiates — is not something you can cheaply evaluate, and PPO was off the table from the first day (Larchenko, 2026). The chapter's central quantity survives intact, though: his solution estimates advantages with a success-probability value head $\hat{p}_{succ}$, GAE, and a damped αV baseline (Chapter 11), then consumes those advantages through AWR weighting (Chapter 4) and RECAP conditioning (Chapter 10).

CHECKPOINT

- Explain to a colleague who has only done supervised learning why you cannot call `.backward()` on a sum of environment rewards, and why a differentiable simulator does not rescue you.
- Derive $\nabla_{\theta} J = \mathbb{E}[R(\tau) \nabla_{\theta} \log p_{\theta}(\tau)]$ from the identity $\nabla_{\theta} p = p \nabla_{\theta} \log p$, and say where the transition model drops out and why that makes the method model-free.
- State the REINFORCE estimator as an English instruction, and name what it does and does not require of your policy.
- Prove in one line that any action-independent baseline leaves the gradient unbiased, and explain why $\mathbf{b} = V(\mathbf{s})$ turns the weight into an advantage.
- Work the 100/101 example and the two-action bandit by hand, and use the $1/\sqrt{N}$ rule to turn a variance reduction into saved episodes on your SO-101.
- Point to the exact factor in every equation here that stops being available for a flow-matching policy, say why "unavailable in practice" is the accurate claim rather than "impossible in principle", and name the quantity that survives.

Where to go deeper. The original REINFORCE paper is Williams (1992), and it is short and readable. The policy gradient theorem with function approximation is Sutton et al. (1999); Sutton and Barto (2018) covers both with full derivations in its policy-gradient chapter and is the gentlest complete treatment in print. Schulman et al. (2016), the GAE paper, is the sharpest treatment of the bias-variance spectrum that Section 2.5 only sketched, and is the natural bridge into Chapter 3. Appendix A places each of these in a suggested reading order.

Chapter 3 — Trust regions, PPO, and advantage estimation

Chapter 2 left you holding an estimator: push up the log-probability of actions that turned out better than expected, push down the ones that turned out worse, and weight each push by the advantage $\hat{A}_t$. That is a direction, not a step size. Nothing in Chapter 2 told you how far to move before the estimate you built stops being trustworthy. That question separates a policy gradient that works from one that destroys itself in an afternoon.

You have already run PPO — on your differential-drive rover, through Stable-Baselines3, with a config you probably copied and tweaked. By the end of this chapter you should understand what every one of those knobs was doing, and why the TensorBoard curves looked the way they did.

One thing up front. PPO will not survive the jump to the policies of Part III: flow-matching vision-language-action policies, or **VLA**s — policies built on a pretrained vision-language model that output robot actions directly (Chapter 8), trained by flow matching, a generative recipe that turns noise into an action along a learned velocity field. Chapter 7 gives the technical reason PPO cannot be used on them; Chapter 9 makes the comparison explicit. The advantage machinery in Section 3.5, however, carries all the way to the LeHome capstone in Chapter 15. If you can only deeply learn one half of this chapter, learn the second half.

3.1 Why step size is the whole game

3.1.1 Supervised learning forgives; reinforcement learning does not

Think about a supervised training run — an image classifier. You take a gradient step that is far too large and the loss jumps. The next minibatch comes from the same fixed dataset, its gradient points back toward the good region, and within a few dozen steps you are roughly where you were. The dataset does not care what your model did.

On-policy reinforcement learning has no such object. The data *is* the policy. Every state in your next batch was reached because the current policy chose the actions that led there. A destructive update produces a worse policy, which collects worse data, which produces gradients computed on states the good policy would never have visited. The distribution of states you train on moves with every update, and it moves *because* of your updates.

An analogy. In a self-hosting compiler — one written in the language it compiles — you use version N of the binary to build version N+1. Introduce a subtle code-generation bug and N+1 is slightly wrong. N+1 then builds an N+2 that inherits the bug and adds its own. The corruption compounds because the tool is its own input, exactly as the policy is both the thing being improved and the source of its own training data.

Where the analogy breaks down. With a compiler you have a known-good binary in a bucket, and restoring it costs only the time since the bad commit. Restoring a policy checkpoint costs every environment step collected since — hours in simulation, days on real hardware. And the corruption is statistical, not logical: no diff, no stack trace, often no single step you can blame.

3.1.2 What a collapse actually looks like

The failure mode is stereotyped, so you can recognize it from the plots. In a healthy run `ep_rew_mean` climbs raggedly; entropy — a measure of how spread out the policy's action distribution still is, unpacked in Section 3.4 — decays gently; and the approximate KL divergence, a running measure of how much an update changed the policy's behavior (made precise in Section 3.3), sits in a narrow band. A collapse announces itself in four curves over two or three rollouts. The approximate KL spikes by an order of magnitude. Entropy either falls off a cliff toward zero or blows upward as the Gaussian's standard deviation diverges. The value loss — the critic's squared prediction error, Section 3.4 — explodes, because the critic's targets moved out from under it. And `ep_rew_mean` drops to the floor

and stays there. The KL spike usually comes *first*, by a rollout or two. That is why you monitor it: it is the earliest warning that an update is moving further than the data supports.

3.1.3 Why it is often unrecoverable

Why does the reward curve stay flat afterward instead of re-climbing? Suppose your rover had learned to approach the goal region and was collecting the occasional terminal reward. Those near-goal states were in its data, so the critic had learned something about them and the gradient had signal there. After the collapse the rover spins near the spawn point and never reaches the goal again. Those informative states are absent from every later batch.

So the gradient says nothing about them, the critic's estimates for them decay toward the network's default output, and the policy must rediscover the approach behavior by random exploration — often from a policy whose entropy has already collapsed, so it cannot explore.

This is why step size is not a detail you get to postpone. In on-policy RL, controlling how far each update moves the policy is the algorithm. The next three sections are three strategies for one goal: extract as much improvement as possible from a batch without moving so far that the batch stops describing the policy.

3.2 Reusing a batch: importance sampling from scratch

3.2.1 What we want, and the obvious thing to try

Collecting data is expensive. On the rover in simulation a 16,384-step batch takes seconds; on the SO-101 arm it takes minutes of real robot time and human supervision. The vanilla policy gradient from Chapter 2 is wasteful with it: the estimator is valid only at the exact parameters θ_{old} that collected the data, so you get one gradient step per batch and then throw the batch away.

The obvious thing to try: do ten gradient steps on the same batch. Why not? The data is right there.

Here is exactly how that fails. After the first step your parameters are no longer θ_{old} ; call them θ_1 . The policy gradient theorem says the gradient of J is an expectation over actions drawn from the *current* policy, and your stored actions were drawn from $\pi_{\theta_{old}}$, not π_{θ_1} . So the second step averages over the wrong distribution — not merely noisy but systematically wrong, and more wrong with each step.

Put that on the rover. The policy is a diagonal Gaussian over wheel commands. At one stored state its mean left-wheel command was **0.30** with standard deviation **0.10**, and the action stored there was **0.36** — an unremarkable sample, six tenths of a standard deviation out. Ten unconstrained gradient steps later, the mean for that state has drifted to **0.16**, so the stored action now sits two standard deviations into the tail: something the current policy would produce about once in forty tries. The gradient does not know that. It keeps pushing on the sample at full weight. Set `n_epochs=10` on `rl-rover` — ten passes over each stored batch — with clipping turned off, and you can watch the consequence: reward climbs for two or three rollouts, then falls to the floor and stays there.

3.2.2 The fix: reweight by how much more likely the new policy is

The repair is **importance sampling** — estimating an average under one distribution using samples drawn from a different one, by reweighting each sample by the ratio of the two probabilities. Define the **importance ratio**:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)},$$

where $\pi_{\theta_{old}}$ is the frozen snapshot of the policy that collected the batch, and π_θ is the policy as it stands right now, partway through the several passes we are about to take over that batch.

Put numbers on it. Suppose at state s_t the old policy assigned the action actually taken a probability of **0.20**, and the current policy assigns it **0.30**. Then $\rho_t = 0.30/0.20 = 1.5$: the new policy is one and a half times as likely to have

produced this sample, so it counts one and a half times as much in the average. Had the new policy dropped to **0.10**, then $\rho_t = 0.10/0.20 = 0.5$ and it counts half as much. If the policy has not moved, $\rho_t = 1$ and nothing changes.

The reason this is exactly right, not merely plausible, is one line of algebra. For any function f of the action,

$$\mathbb{E}_{a \sim \pi_{\theta_{old}}} \left[\frac{\pi_{\theta}(a)}{\pi_{\theta_{old}}(a)} f(a) \right] = \sum_a \pi_{\theta_{old}}(a) \frac{\pi_{\theta}(a)}{\pi_{\theta_{old}}(a)} f(a) = \sum_a \pi_{\theta}(a) f(a) = \mathbb{E}_{a \sim \pi_{\theta}}[f(a)].$$

(That display is written with sums, for a discrete action set. For the continuous actions your rover takes, replace $\sum_a$ with $\int da$ and read each π as a density rather than a probability; the cancellation is character-for-character identical.)

The $\pi_{\theta_{old}}$ cancels. Samples from the old policy, weighted by the ratio, give an unbiased average under the new policy. Note what this requires: the actual probability density of the action. In code, $\rho_t = \exp(\log \pi_{\theta}(a_t | s_t) - \log \pi_{\theta_{old}}(a_t | s_t))$, in log-space for numerical stability. For a Gaussian policy that log-probability is a closed form you can differentiate. Remember the requirement; Chapter 7 is where it stops being satisfiable.

3.2.3 The surrogate objective and the lie it tells

Plug the ratio into the objective. The result is a **surrogate objective** — a stand-in we optimize because the thing we actually care about is not directly computable from the data we have:

$$L(\theta) = \mathbb{E}_t [\rho_t(\theta) \hat{A}_t]$$

IN PLAIN WORDS

The left side, $L(\theta)$, is the score we push up by changing the policy parameters θ . The $\mathbb{E}_t$ means "average over all timesteps in the batch you collected". For one timestep: $\hat{A}_t$ estimates how much better than average the action taken at time t turned out to be — positive means better than the policy's own expectation, negative worse. $\rho_t(\theta)$ is how many times more likely the current policy is to take that same action than the old policy was. Multiplying them says: to raise the score, become more likely to take actions with positive $\hat{A}_t$ and less likely to take actions with negative $\hat{A}_t$.

This surrogate has a useful property: at $\theta = \theta_{old}$, where every $\rho_t = 1$, its gradient equals the true policy gradient of $J(\theta)$. Here is why, because it is worth seeing rather than believing. Differentiate the ratio, remembering that $\pi_{\theta_{old}}(a_t | s_t)$ is a stored number, not a function of θ : $\nabla_{\theta} \rho_t = \nabla_{\theta} \pi_{\theta}(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$. Evaluate that at $\theta = \theta_{old}$, where the denominator is the numerator's own probability, and apply the identity $\nabla_{\theta} \pi_{\theta} = \pi_{\theta} \nabla_{\theta} \log \pi_{\theta}$ from Chapter 2. The π_{θ} cancels, leaving $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$, so the surrogate's gradient is $\mathbb{E}_t [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t]$ — the score-function estimator you built in Chapter 2. PPO's first gradient step on a fresh batch and REINFORCE-with-a-baseline's step are the same step. Everything PPO adds is about the steps after it.

Ascending the surrogate therefore starts out exactly correct. But it is only a *local* approximation, and you should know precisely which part is a lie. The ratio corrects for the new policy choosing different *actions* at the states in your batch. It does nothing about the new policy visiting different *states*. The surrogate quietly assumes the state distribution is frozen at whatever $\pi_{\theta_{old}}$ produced.

Think of an A/B test on a web service. You ran variant B on 20% of traffic and now want to estimate what happens at 30%. Reweighting your logged sessions is reasonable. But if variant B also changes which users come back tomorrow, your reweighted logs describe a population that no longer exists. **Where the analogy breaks down:** in an A/B test that drift is slow and you can re-measure. In RL it happens inside one training iteration, and the next batch is already coming from the drifted policy.

So the further ρ_t strays from 1, the bigger the lie. Sections 3.3 and 3.4 are two mechanisms for ascending $L(\theta)$ while keeping ρ_t near 1.

3.3 TRPO in one section

3.3.1 The constrained problem

Trust Region Policy Optimization (Schulman et al., 2015) takes the honest route. If the surrogate is only trustworthy near θ_{old} , say so explicitly: maximize $L(\theta)$ subject to a hard constraint that the average **KL divergence** between $\pi_{\theta_{old}}$ and π_{θ} , taken over the states in the batch, stays below a small threshold such as **0.01**.

The region of parameter space where that constraint holds is the *trust region*: the neighborhood in which you trust your approximation. Inside it, improving the surrogate provably improves a lower bound on the true objective $J(\theta)$. That is a real guarantee, and more than PPO can claim.

3.3.2 KL divergence in plain words

KL divergence measures how different two distributions are, and the definition $D_{KL}(p \parallel q) = \mathbb{E}_{x \sim p}[\log p(x) - \log q(x)]$ is not illuminating on its own. Bind the symbols first: p is the old policy $\pi_{\theta_{old}}$, q is the new policy π_{θ} , and $\mathbb{E}_{x \sim p}$ says the average is taken over actions drawn the way the *old* policy would draw them. The useful intuition: *how surprised is the old policy by the new one?* If the two agree everywhere, every log-difference is zero and the KL is zero. If the new policy has taken mass *away* from actions the old one considered likely, those are precisely the actions you keep drawing; each one has **log p** large and **log q** small, so the log-differences are large and positive and the KL grows. Moving mass *onto* actions the old policy never took barely registers directly — you almost never draw those actions, so they almost never enter the average. It registers because that mass had to come from somewhere.

It is not symmetric — $D_{KL}(p \parallel q) \neq D_{KL}(q \parallel p)$ in general — and it is not a distance. For our purposes it is a scalar that reads zero when the policy has not moved and grows as it does, in units that reflect the shape of the distribution rather than the size of the parameter change. That distinction matters: two policies can have very different parameters and behave identically. KL measures behavior, not parameters.

3.3.3 Why almost nobody runs TRPO

The implementation is where it falls apart. Enforcing the constraint exactly requires a second-order object: the **Fisher information matrix**, which is the curvature of the KL at θ_{old} — a statement of how fast behavior changes per unit of parameter change. For a network with millions of parameters you cannot even store that matrix. So TRPO never forms it. It computes only the matrix's product with one vector at a time (a **Fisher-vector product**, obtainable from a pair of backward passes), solves for the step direction with **conjugate gradient** — an iterative linear solver that needs nothing but such products — and then **backtracks**, halving the proposed step repeatedly, until the KL constraint really holds.

That is three pieces of nontrivial numerical machinery around every update. It is slow, fiddly to get right, and interacts badly with architectures that share a trunk between policy and value function. TRPO's lasting contribution is the framing: a policy update as constrained optimization over a trust region. The practical tool arrived two years later with none of the machinery.

3.4 PPO: clip the surrogate

3.4.1 The clipped objective

Proximal Policy Optimization (Schulman et al., 2017) asks a different question: instead of forbidding large steps, what if we stop *paying* for them? If moving ρ_t beyond a small interval around 1 earns no extra objective value, plain gradient ascent has no reason to go there.

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

IN PLAIN WORDS

For each timestep in the batch, compute two numbers. The first is the plain surrogate term $\rho_t \hat{A}_t$ from Section 3.2, where $\hat{A}_t$ is the advantage estimate and ρ_t the importance ratio. The second is the same product, but with the ratio first squashed into the window $[1 - \epsilon, 1 + \epsilon]$: **clip** returns $1 - \epsilon$ if ρ_t is below it, $1 + \epsilon$ if above, ρ_t unchanged in between. The **clip parameter** ϵ , typically 0.1 to 0.3, sets the window's width. Then take **min** of the two — whichever is worse for us — and average over the batch. Preferring the smaller of an honest estimate and a capped one is what makes the objective refuse to reward large moves.

The **min** wrapped around the **clip** is the part everyone finds confusing at first, and the confusion is legitimate: two nested operations that look like they do the same job. They do not. Unpack it one sign of the advantage at a time, following Figure 3.1 as you go.

3.4.2 Case one: the advantage was positive

$\hat{A}_t > 0$ means the action turned out better than the policy expected, so we want to raise its probability — we want ρ_t above 1. As it grows, the unclipped term $\rho_t \hat{A}_t$ grows linearly, and the clipped term grows with it until ρ_t hits $1 + \epsilon$, where it freezes at $(1 + \epsilon) \hat{A}_t$. Past that point the clipped term is the smaller of the two, the **min** selects it, and the objective is flat.

Flat means zero gradient. Once the current policy assigns this action $1 + \epsilon$ times its old probability, this sample stops pushing at all. With $\epsilon = 0.2$: raise it by 20%, then stop. The sample has said what it has to say.

3.4.3 Case two: the advantage was negative

$\hat{A}_t < 0$ means the action was worse than expected, so we want ρ_t below 1. Because $\hat{A}_t$ is negative, shrinking ρ_t makes $\rho_t \hat{A}_t$ less negative, which improves the objective. That improvement stops at $\rho_t = 1 - \epsilon$, where the clipped term freezes at $(1 - \epsilon) \hat{A}_t$; below that the clipped term is the more negative of the two, the **min** selects it, and the gradient dies again.

Now the interesting half. What if ρ_t goes *up* instead — if an earlier minibatch update in this same epoch accidentally raised the probability of an action we know was bad? Then $\rho_t \hat{A}_t$ grows more and more negative with no cap, and the **min** selects that unclipped branch. The mistake is punished at full strength, with no ceiling.

3.4.4 The asymmetry is the entire point

Look at Figure 3.1 as a pair. The incentive to improve is capped; the penalty for getting worse is not. That asymmetry is what the **min** buys, and it is why the objective is called *pessimistic*. If you simply clipped ρ_t and multiplied, a sample whose probability had been wrongly pushed up would contribute nothing, and the mistake would go uncorrected.

Be clear about what this is not. It is an incentive structure, not a constraint. Nothing in L^{CLIP} makes it impossible to leave the window: a large learning rate, or too many passes over one batch, can carry θ far past $1 \pm \epsilon$, because a sample's gradient vanishes only once you are already outside. That is why you still monitor the KL.

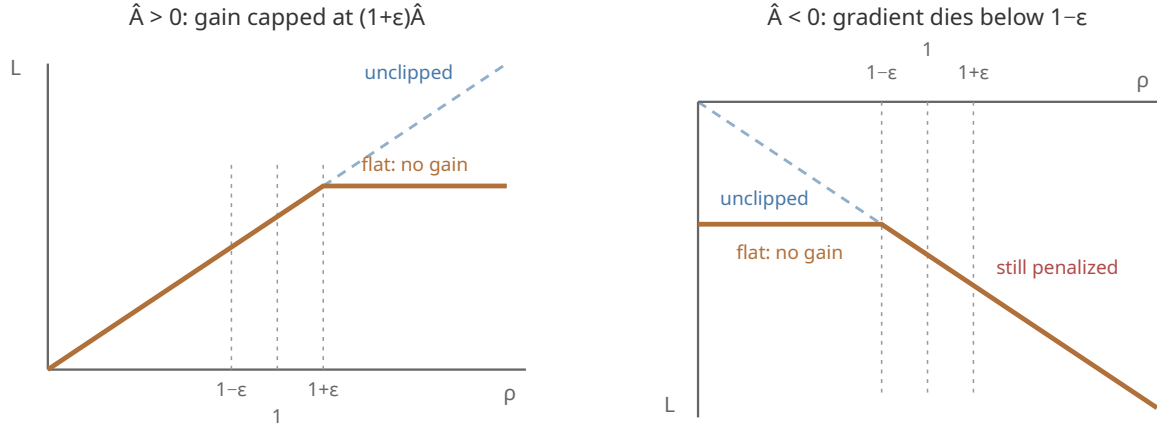


Figure 3.1 — The clipped surrogate for one sample, plotted against the importance ratio ρ . The panels are not mirror images: on the left the orange curve goes flat once the action's probability has risen by $1 + \epsilon$, while on the right it keeps falling as ρ grows. That is the pessimism — you stop being rewarded for pushing a good action harder, but you are never let off the hook for pushing a bad one up.

3.4.5 The full loss, and one iteration of code

Because the clip keeps updates near $\pi_{\theta_{old}}$, PPO can safely reuse a batch: collect a rollout of N steps from several parallel environments, then run several **epochs** — complete passes over the stored batch — of minibatch gradient descent before collecting fresh data. That reuse is where most of PPO's sample efficiency comes from. The policy term is not the whole loss, though:

$$L^{\text{PPO}}(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_v \mathbb{E}_t \left[(V_\phi(s_t) - \hat{G}_t)^2 \right] - c_e \mathbb{E}_t [\mathcal{H}(\pi_\theta(\cdot | s_t))]$$

IN PLAIN WORDS

This is what the optimizer minimizes, so signs flip relative to the objective. θ are the policy's parameters, ϕ the critic's. The first term is minus the clipped surrogate: minimizing it maximizes L^{CLIP} . The second is the **value loss** — squared error between the critic's prediction $V_\phi(s_t)$ and the observed return target $\hat{G}_t$ — which trains the critic the advantage estimator needs. The third is the **entropy bonus**: $\mathcal{H}$ is the entropy of the action distribution, high when the policy is spread out, near zero when nearly deterministic. It enters with a minus sign, so minimizing the loss *increases* entropy, paying the policy to stay random. c_v and c_e are scalar weights, commonly 0.5 and 0.01.

Where does $\hat{G}_t$ come from? Not from a second pass over the episode. Once Section 3.5 hands you $\hat{A}_t$, the target is just $\hat{G}_t = \hat{A}_t + V(s_t)$ — the critic's own estimate, corrected by the advantage you measured. That identity is what makes the critic loss and the advantage estimator one machine instead of two, and it is what fills the `returns` field below (and `rollout_buffer.returns` in Stable-Baselines3).

```

# One PPO iteration: collect once, then reuse the batch several times.
batch = collect_rollout(pi_old, n_envs=8, n_steps=2048) # 16384 steps total
adv = gae(batch.rewards, batch.values, gamma, lam) # Section 3.5
batch.returns = adv + batch.values # critic targets: G = A + V
batch.adv = (adv - adv.mean()) / (adv.std() + 1e-8) # per-batch normalization
for epoch in range(4): # 4 passes over the SAME data
    for mb in minibatches(batch, size=512):
        # ratio in log-space; mb.logp_old was stored during collection,
        # so no frozen copy of the old network is needed here
        rho = exp(logp(pi, mb.a) - mb.logp_old)
        # min() of unclipped and clipped: take the pessimistic branch
        l_pi = -min(rho * mb.adv,
                    clip(rho, 1-eps, 1+eps) * mb.adv).mean()
        l_v = (v(mb.s) - mb.returns).pow(2).mean() # critic regression
        loss = l_pi + 0.5 * l_v - 0.01 * entropy(pi, mb.s)
        loss.backward(); opt.step(); opt.zero_grad()
pi_old = pi # snapshot for the next iteration

```

PPO became the field's default for an unglamorous reason: it is a first-order method, roughly a dozen lines on top of an actor-critic, robust across a wide range of hyperparameters, and it scales by adding parallel environments. It gives up TRPO's guarantees and keeps most of its results.

But notice what it needs. Every line above depends on an exact, differentiable $\log \pi_{\theta}(a_t | s_t)$ for each sampled action. Chapter 7 shows that flow-matching VLAs cannot provide one, and Chapter 9 works through the consequences side by side with everything here. That single missing quantity is why the LeHome solution left PPO behind entirely.

WORDS YOU WILL HEAR

Clip fraction — the fraction of samples in a minibatch where ρ_t fell outside $[1 - \epsilon, 1 + \epsilon]$. **Approximate KL (approx_kl)** — a cheap sample estimate of the KL between old and new policy, used for monitoring and early stopping. **Explained variance** — $1 - \text{Var}(\hat{G} - V) / \text{Var}(\hat{G})$; near 1 means a useful critic, near or below 0 a useless one. **Adaptive KL penalty** — the other PPO variant from the same paper: subtract $\beta_{KL} D_{KL}$ from the objective you maximize, equivalently add $+\beta_{KL} D_{KL}$ to the loss you minimize, with β_{KL} retuned between iterations to hold the measured KL near a target. The subscript keeps it apart from the AWR temperature β later in the book. Rarely used today. **Epoch / minibatch** — one full pass over the stored rollout, and one slice of it.

3.5 Advantage estimation: from Monte Carlo to TD to GAE

The clipped objective consumes $\hat{A}_t$ and does nothing to check it. Whatever quality that estimate has, the learning inherits. So where does it come from? Two pure answers exist, and they fail in opposite directions.

3.5.1 Monte Carlo: unbiased and nearly unusable

The direct approach: run the episode to the end, add up the discounted rewards you actually received, subtract the baseline. That is $\hat{A}_t = G_t - V(s_t)$ with $G_t = \sum_k \gamma^k r_{t+k}$. Nothing is approximated — G_t is what happened. The estimate is unbiased.

Here is the concrete failure. On your rover, consider the first action of an episode, a small steering command. Two episodes start from nearly the same state and take nearly the same first action. In one, the randomized obstacle layout leaves a clear lane and the rover reaches the goal for $G_0 = +180$. In the other an obstacle sits square in the path, the rover detours, runs out of time, and $G_0 = -20$. The same action is scored at $+180$ in one sample and -20 in the other, and the difference has nothing to do with the action. G_t sums *every* random event between t and the end of the episode and charges all of it to a_t . The variance is enormous and grows with episode length.

3.5.2 TD: low variance and wrong

The opposite approach: do not wait. Take one real reward, then ask the critic what everything after is worth. That gives the **TD residual**, the one-step temporal-difference error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

In words: the immediate reward, plus the discounted value the critic assigns to where you landed, minus the value it assigned to where you started. If the transition went better than the critic expected, δ_t is positive. Only one step of real randomness enters, so variance is small and does not grow with the episode.

Here is its concrete failure. Early in training your critic is a barely-trained network outputting numbers near zero everywhere. Your rover takes the action that turns it toward the goal, moving from a state genuinely worth **0.9** to one genuinely worth **0.95**. The reward on this step is **0**, because reward is sparse. The critic reports $V(s_t) = 0.02$ and $V(s_{t+1}) = 0.01$. Then $\delta_t = 0 + 0.99 \times 0.01 - 0.02 = -0.01$: slightly negative. The estimator says a good action was mildly bad. That is not noise you can average away — run it a thousand times and the sign stays wrong, because the error lives in V , not in the sampling. That is bias.

A software analogy: MC times a request end-to-end with a stopwatch; TD reads a cost estimate off a cached model of how long each function takes. The stopwatch is honest, but every measurement is polluted by whatever else the machine was doing. The cost model is stable and, if stale, gives the same wrong number every time. **Where it breaks down:** a profiler's cost model is calibrated independently of the request you are timing, while the critic is trained on the very returns you are estimating, so its errors and your data's errors are correlated.

3.5.3 The dial between them

Between the extremes sit k -step estimators: sum k real rewards, then bootstrap with $\gamma^k V(s_{t+k})$. Larger k means more real signal and more variance; smaller k means more reliance on the critic and more bias. Picking a single k is a brittle decision. **Generalized Advantage Estimation** (Schulman et al., 2016) sidesteps it by blending all values of k at once with exponentially decaying weights:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

IN PLAIN WORDS

To score the action at time t , walk forward through the rest of the episode collecting TD residuals: δ_t from this step, δ_{t+1} from the next, and so on, where each δ is the reward r plus the discounted next value minus the current one. l counts how many steps ahead you have walked. Each residual is multiplied by $(\gamma \lambda)^l$, a number below 1 raised to the l -th power, so residuals further ahead count exponentially less. γ is the discount from the problem definition; λ , the **GAE parameter**, lies in $[0, 1]$ and sets how fast that decay happens. At $\lambda = 0$ only the first term survives: pure TD. At $\lambda = 1$ nothing decays beyond γ and the sum telescopes into $G_t - V(s_t)$: pure Monte Carlo. Everything between is a blend.

Figure 3.2 draws λ as the dial it is. In practice $\lambda \approx 0.95$ sits close to the MC end. Read it as: trust the real rewards for roughly the next twenty steps, then hand off to the critic. Note also that the implementation is one backward pass, not an infinite sum — accumulate $\hat{A}_t = \delta_t + \gamma \lambda \hat{A}_{t+1}$ from the end of the rollout to the start, which costs essentially nothing.

3.5.4 γ and λ are not the same knob

They look alike in the formula, they multiply together, and both are slightly below 1. Everything else about them differs, and conflating them is a common confusion.

γ belongs to the *problem definition*. It is part of the return G_t you decided to maximize back in Chapter 1, and it says how much the agent should care about a reward twenty seconds from now relative to one right now. Change γ and you change what the optimal policy is: a $\gamma = 0.9$ rover is a more impatient animal than a $\gamma = 0.999$ rover, and the two behave differently at convergence. Tune γ to the task's time horizon.

λ belongs to the *estimator only* and appears nowhere in the objective. It trades bias against variance in the numbers you feed the gradient, and with a perfect critic every λ estimates the same quantity with different noise; the optimal policy is unchanged. Tune λ to how much you trust your critic — young critic or sparse reward, push it up; well-fit critic and long noisy episodes, pull it down.

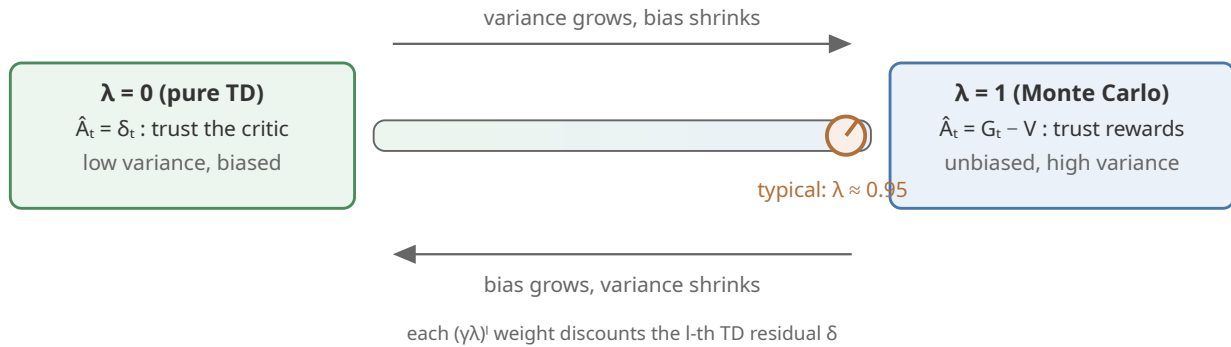


Figure 3.2 — GAE as a single dial between the two pure estimators. The thing to notice is that the endpoints are not two different algorithms but two settings of one formula: the same sum of weighted TD residuals gives pure bootstrapping at $\lambda = 0$ and the full Monte Carlo return at $\lambda = 1$. The usual operating point sits near the Monte Carlo end, not in the middle.

3.5.5 Worked example: three estimators, one three-step episode

Take a deliberately tiny episode so every number is checkable by hand. Three steps; $\gamma = 0.9$; rewards $r_0 = 0$, $r_1 = 0$, $r_2 = 1$ — success only at the end, the sparse-reward situation you face on a robot. The critic, mid-training and mediocre, reports $V(s_0) = 0.5$, $V(s_1) = 0.6$, $V(s_2) = 0.7$, and $V(s_3) = 0$ because s_3 is terminal.

First the three TD residuals, $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$, every multiplication shown:

$$\delta_0 = 0 + 0.9 \times 0.6 - 0.5 = 0 + 0.54 - 0.5 = 0.04$$

$$\delta_1 = 0 + 0.9 \times 0.7 - 0.6 = 0 + 0.63 - 0.6 = 0.03$$

$$\delta_2 = 1 + 0.9 \times 0 - 0.7 = 1 + 0 - 0.7 = 0.30$$

With $\lambda = 0$ the sum keeps only its first term, so the advantages are the residuals themselves: $\hat{A}_0 = 0.04$, $\hat{A}_1 = 0.03$, $\hat{A}_2 = 0.30$. Read what that says. The success at $t = 2$ gives a healthy signal for the last action, but the first action gets **0.04** — almost nothing. The estimator refuses to look past one step; it credits the first action only for the critic's opinion that s_1 beats s_0 , and that opinion is worth **0.04**.

With $\lambda = 0.95$ the decay factor is $\gamma\lambda = 0.9 \times 0.95 = 0.855$, and we accumulate backward from the end:

$$\begin{aligned} A_2 &= d_2 &&= 0.30 \\ A_1 &= d_1 + 0.855 * A_2 \\ &= 0.03 + 0.855 * 0.30 \\ &= 0.03 + 0.2565 &&= 0.2865 \quad \sim \quad 0.287 \\ A_0 &= d_0 + 0.855 * A_1 \\ &= 0.04 + 0.855 * 0.2865 \\ &= 0.04 + 0.24496 &&= 0.28496 \quad \sim \quad 0.285 \end{aligned}$$

Read this one too. The terminal success now reaches the first action almost undamped: **0.285** versus **0.04**, same action, same data, same critic. That is a factor of just over seven in the learning signal — $0.285/0.04 = 7.125$ — and

the only thing that changed was one number in the config file.

With $\lambda = 1$ the decay factor is $\gamma\lambda = 0.9 \times 1 = 0.9$, giving $\hat{A}_1 = 0.03 + 0.9 \times 0.30 = 0.03 + 0.27 = 0.30$ and $\hat{A}_0 = 0.04 + 0.9 \times 0.30 = 0.04 + 0.27 = 0.31$. Check that against the Monte Carlo definition: $G_0 = 0 + 0.9 \times 0 + 0.9^2 \times 1 = 0.81$, so $\hat{A}_0 = G_0 - V(s_0) = 0.81 - 0.5 = 0.31$. The two routes agree — the telescoping property in action.

So $\lambda = 0.95$ recovers **0.285** of the **0.31** full Monte Carlo gives, while still damping the far tail. On a three-step episode that damping buys little; on a 500-step episode it is the difference between a usable signal and noise. Figure 3.3 puts all three side by side.

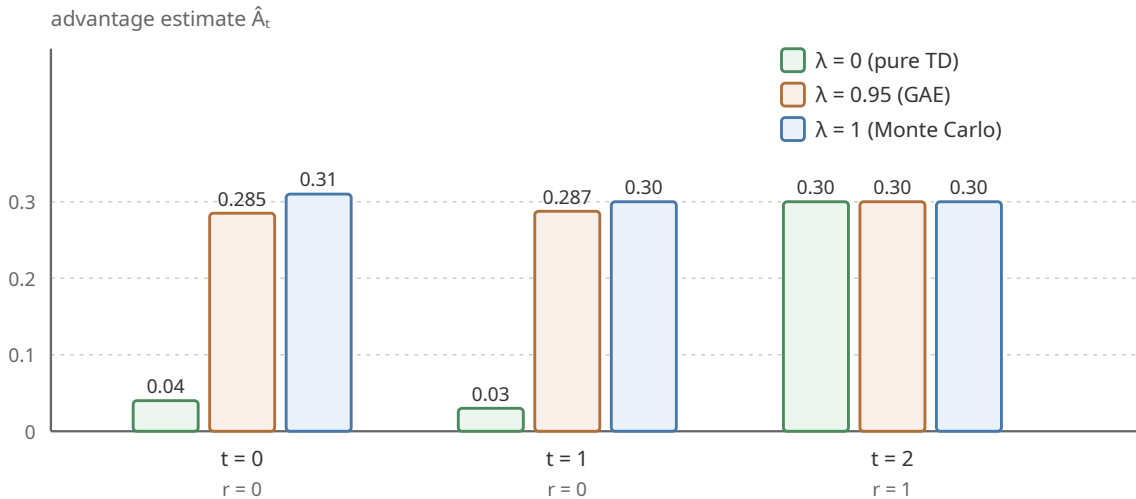


Figure 3.3 — The same three-step episode, the same rewards, the same critic, three settings of λ . Notice that at $t = 2$, where the reward actually arrives, all three estimators agree exactly — λ has nothing to spread. The difference appears at $t = 0$ and $t = 1$, where the green $\lambda = 0$ bars are almost invisible: pure bootstrapping stops the credit at the critic and never lets the terminal success travel back to the actions that made it possible.

3.6 PPO in practice on the rover

Everything so far has been mechanism. This section is what you actually do with it on rl-rover: a starting configuration, and the four plots that tell you whether the run is healthy.

3.6.1 A configuration to start from

On a task the size of rl-rover, a sane configuration is 8 parallel environments, 2048 steps each per rollout (a 16,384-step batch), 4 epochs of minibatch updates, $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.2$, and a budget of roughly 5M environment steps. Those are numbers to start from, not numbers to tune first. Two of them repay a second look. The 4 epochs are the reuse factor of Section 3.4: raise it and you extract more from each batch, but you also travel further from $\pi_{\theta_{old}}$ within one iteration. The $\lambda = 0.95$ is the dial of Section 3.5, sitting near the Monte Carlo end because your critic starts out untrained.

3.6.2 The four curves

Four TensorBoard curves tell you nearly everything. Read them as a set; their failure modes are joint.

Episode reward (ep_rew_mean) is the ground truth. Everything else exists to explain this curve. If it is going up, resist the urge to fix things.

Approximate KL and clip fraction tell you how far updates actually move the policy. Healthy is a KL in the **0.01–0.03** band with clip fraction around **0.1–0.3**. Clip fraction near zero means the clip never binds: either the learning

rate is too timid or learning has converged, and the reward curve tells you which. KL spiking with clip fraction above 0.4 is the danger sign of Section 3.1. Lower the learning rate, or cut the epochs per batch.

Value loss should fall early, then track changes in reward rather than sitting flat. If it stays large, check explained variance: a critic near zero there contributes bias and nothing else.

Entropy should decay slowly and smoothly. A cliff-drop in the first 10% of training means the policy went deterministic before it found anything worth being confident about; raise the entropy coefficient. Entropy that climbs steadily usually means a broken advantage signal.

ON YOUR RIG

In Stable-Baselines3 these knobs are `n_envs`, `n_steps`, `n_epochs`, `clip_range`, `gae_lambda`, `ent_coef`; the curves are `approx_kl`, `clip_fraction`, `entropy_loss`, `value_loss`, `explained_variance`. Re-run `rl-roller` and watch `clip_fraction` during the reward ramp — it rises exactly when the policy changes fastest, the most direct view of this chapter you will get. If a sim-to-real transfer got worse after a "better" run, check whether entropy collapsed early; overconfident policies randomize poorly and transfer poorly. Your `hil-serl` SO-101 setup uses an off-policy learner, so there is no `clip_fraction` there, but the critic diagnostics carry over.

PITFALLS

Two silent killers. (1) Skipping per-batch advantage normalization: raw $\hat{A}$ magnitudes drift over training, so the effective learning rate drifts with them. Normalize advantages to zero mean and unit variance per batch — but never normalize rewards mid-run with running statistics you later change, or the critic's targets shift underneath it. (2) Reward scaling: multiplying rewards by 100 multiplies the value loss by 10,000 and its gradients by 100. If the value loss dwarfs the policy loss, the shared trunk trains the critic and ignores the actor; rescale the rewards or lower the value coefficient c_v .

IN THE LEHOME SOLUTION

PPO never appears. A flow-matching VLA has no tractable $\log \pi_{\theta}(\mathbf{a} \mid \mathbf{o})$, so the ratio ρ_t simply cannot be computed (Chapter 7; Chapter 9 makes the comparison explicit). But Section 3.5 survives completely intact. Larchenko's advantage pipeline uses TD residuals, Monte Carlo returns, and GAE computed over checkpoint rewards, with the learned success-probability head $\hat{p}_{succ}$ standing in for V . Because that value estimate is imperfect, the full subtraction is replaced by a damped baseline αV , where the damping coefficient α sits below 1 and deliberately subtracts less than the whole value estimate (Chapter 11). The estimator outlived the algorithm that made it famous (Larchenko, 2026).

CHECKPOINT

- Explain why one oversized update can permanently damage an on-policy run, and name the TensorBoard signatures of a policy collapse.
- Compute the importance ratio ρ_t from two probabilities, and say which part of the surrogate objective is an approximation and why it degrades as ρ_t leaves 1.
- Show that at $\theta = \theta_{old}$ the surrogate's gradient is Chapter 2's score-function estimator.
- Walk through $\mathcal{L}^{CLIP}$ case by case for $\hat{A}_t > 0$ and $\hat{A}_t < 0$, and say why the objective is deliberately asymmetric.
- Write down the three terms of the full PPO loss and say what each is for.
- Compute GAE by hand from a short list of rewards and critic values, and say why γ changes the optimal policy while λ changes only the noise in your estimate of it.

Where to go deeper. The primary sources are short and unusually readable: Schulman et al. (2015) for TRPO, Schulman et al. (2016) for GAE, Schulman et al. (2017) for PPO. For the gap between the paper and code that actually trains, Engstrom et al. (2020) and Andrychowicz et al. (2021) dissect which implementation details in on-policy RL matter and which are folklore — read at least one before you tune anything. Appendix A places all three primary papers in a suggested reading order.

PART II

Learning from data you already have

Real robots make samples expensive. This part rebuilds RL as a weighted supervised problem — the form that survives everything that follows.

Chapter 4 — RL as weighted behavior cloning

Chapter 3 ended with PPO: a robust on-policy algorithm that improves a policy using data the policy itself just generated. This chapter takes the opposite road. It starts from supervised learning on demonstrations, asks how to turn that into *reinforcement* learning, and arrives at an update so simple it fits on one line: clone the data, but weight each sample by how good its action turned out to be.

That one line — advantage-weighted regression — is the load-bearing idea of this entire book. Not because it is the most powerful algorithm here; it is not. It is load-bearing because it is the form of RL that *survives* when the policy stops being a small Gaussian network and becomes a large generative model. Everything in Part III depends on this chapter, so it is worth going slowly.

Here is the route. We price out why on-policy learning is a bad fit for real hardware (4.1), try the obvious alternative and watch it break (4.2), retreat to plain behavior cloning, which cannot break that way but has a hard ceiling (4.3), and then fix the ceiling without giving up the safety: that fix is AWR (4.4). AWAC is a variation on the theme (4.5), and 4.6 states the property that makes the whole book hang together.

4.1 The robot-data reality

4.1.1 What we want, and the obvious thing to try

We want a policy on a physical arm that gets better with experience. You already know a method that does this and have personally made it work: PPO on your differential-drive rover. So the obvious move is to point it at the SO-101.

Before you do, price it out. PPO is **on-policy**, meaning it can only learn from data generated by the current policy (or something extremely close to it): each batch is collected, used for a handful of gradient epochs, and then discarded, because the importance ratio $\rho_t(\theta)$ only corrects for small policy drift. Suppose you budget a modest run of one million environment steps — small by simulation standards; your rover runs used more. Two rates show up in this chapter and both are real: you log teleoperation through the leader arm at 20 Hz, and you run the policy autonomously at 30 Hz. At the autonomous rate, an episode of 20 seconds is 600 steps, so one million steps is about 1,667 episodes.

Now add the part simulation does not have: the reset. After each episode a human has to re-flatten the cloth, or replace the cube, or untangle a cable. Call it 15 seconds, which is optimistic. Each episode therefore costs $20 + 15 = 35$ seconds of wall clock, and:

$$\frac{1,000,000}{600} \approx 1,667 \text{ episodes,} \quad 1,667 \times 35 \text{ s} = 58,345 \text{ s,} \quad \frac{58,345}{3600} \approx 16.2 \text{ hours.}$$

Sixteen hours for one run. Five seeds, the minimum anyone would accept in a paper, is 81 hours — two working weeks of a person standing next to a moving arm, and that is before anything on the arm wears out.

4.1.2 How it fails, and the fix

The deeper problem is not the hours. It is that on-policy learning *throws the hours away*. After the update, that motion is formally worthless for the next update: the policy has moved, so the data is off-distribution, and PPO's clipping machinery exists precisely to stop you from using it too hard. You are renting your data, not owning it.

Meanwhile, look at what is sitting on your disk. Your rl-rover PPO runs saved rollouts at every checkpoint. Your hilserl replay buffers hold hours of SO-101 experience: teleoperation demonstrations, Xbox-pad interventions (in your first working run, 98% of frames were interventions), and autonomous rollouts from a dozen policy versions. None of it came from your current policy. All of it is real physics, real cameras, real outcomes.

Learning about a policy π_θ from data generated by some *other* policy is called **off-policy** learning. Learning entirely from a fixed logged dataset, with no environment interaction at all during training, is **offline RL**. The policy that

actually generated the data — whatever it was — is the **behavior policy**, written π_β . Demonstrations are just the special case where π_β is a competent human holding the leader arm.

Why should this work at all? Because nothing about the data's origin makes its content less true. When your old checkpoint commanded a gripper close at a particular wrist angle and the cloth slipped, the slipping was real; physics does not care which network emitted the command. The hard part — and it is genuinely hard — is extracting that information without fooling yourself.

WORDS YOU WILL HEAR

Off-policy. Learning about one policy from data generated by another. **Offline RL** (older papers: *batch RL*). Training on a fixed dataset with zero environment interaction. **Behavior policy** π_β . Whatever produced the data: a human, a scripted controller, an old checkpoint, or a mixture. **Distributional shift.** The gap between the state-action distribution in your data and the one your new policy will actually visit; actions outside the data's coverage are called *OOD* (out-of-distribution). **Conservatism / pessimism.** The family of fixes — conservative Q-learning (CQL) and implicit Q-learning (IQL) — that deliberately push value estimates *down* on unseen actions, or refuse to evaluate them at all. **Temperature.** A scalar dividing something before an exponential, controlling how sharply the result concentrates.

4.2 Offline RL's failure mode

4.2.1 The obvious thing to try, again

You have the pile. The obvious move is to run a standard off-policy algorithm on it — a Q-learning-style actor-critic. Train a critic $Q(s, a)$ on the logged transitions by temporal-difference regression, then train the actor to output the action that maximizes the critic. No environment interaction needed. This is the structure of the algorithms in your hil-serl setup, minus the part where fresh data keeps arriving. It does not work, and the way it fails explains the design of every method in this chapter.

4.2.2 Exactly how it fails

Make it concrete. Your buffer contains SO-101 wrist-flex commands, and because a human generated them through a leader arm at 20 Hz, the per-step wrist delta is always somewhere in $[-10^\circ, +10^\circ]$. Nothing in the dataset has a delta of $+60^\circ$. Not one frame.

Now ask the critic what $Q(s, +60^\circ)$ is. The critic is a neural network. It will answer. It has no mechanism for saying "I have never seen that." It extrapolates smoothly, and the number it returns is whatever the network's shape happens to produce out there. Suppose the true returns in your task all lie in $[0, 1]$ (say, success probability), but the critic's surface happens to peak at $Q(s, +60^\circ) = 8.5$.

Now run the actor update. The actor's job is to find the action maximizing Q . It is a gradient-based optimizer with one objective: climb. It will find that peak, because the peak is exactly what it is searching for. This is the crucial asymmetry: extrapolation error is random in sign, but the optimizer *specifically hunts for the positive errors*. The policy walks toward $+60^\circ$, the arm slams the wrist over, and nothing in the training loop ever contradicts it, because offline there is no environment to send back the bad news.

It gets worse, because of **bootstrapping** — the fact that a value estimate is trained against a target built out of other value estimates rather than out of observed returns. The TD target for a logged transition is $r_t + \gamma \max_{a'} Q(s_{t+1}, a')$, where a' is any action you could take at the next state s_{t+1} , and that maximum is taken over the same inflated surface. So the fantasy value at $+60^\circ$ gets copied into the target for the *previous* state, and then the state before that. A few sweeps through the buffer and the whole value function has drifted upward, anchored to an action that never existed. Everyone who runs vanilla off-policy RL on static data sees this at least once: the loss curves look fine, the values look great, the robot is nonsense.

IN PLAIN WORDS

The critic takes a state $\mathbf{s}$ and an action $\mathbf{a}$ and returns a number: how much total future reward you expect if you take $\mathbf{a}$ here and then behave as usual. The dataset pins that number down only where data exists. Everywhere else it is a guess dressed up as a measurement. The actor is a search procedure over $\mathbf{a}$. Search plus unpinned guesses equals the search finding the guesses. Offline, no feedback loop closes.

A software analogy: a neural critic is like reading past the end of an array in C. There is no bounds check and no exception — you get a plausible-looking number back and the program keeps running. *Where the analogy breaks down:* an out-of-bounds read returns random garbage, so you usually notice fast. Neural extrapolation returns something smooth and confident, and because the optimizer selects for the highest such values, the errors you chase are systematically the flattering ones. The failure is quieter and better disguised than a memory bug.

4.2.3 The fix, and why it works

Every method that succeeds at offline RL applies the same medicine in a different flavor: **keep the policy close to the data**. Some add an explicit penalty on the divergence between π_θ and π_β . Conservative Q-learning (CQL) trains the critic to actively push values *down* on actions the policy proposes but the data does not contain. Implicit Q-learning (IQL) avoids ever evaluating Q at an action outside the dataset. It works for one reason: the queries you make are then queries the data can answer, so the asymmetry that generated the disaster has nothing left to exploit.

The family we build in this chapter enforces that constraint in the most literal way available. AWR, the method of 4.4, never evaluates the policy at an action of its own choosing at all — it only ever looks at actions physically present in the dataset, because it refuses to leave the supervised-learning template. (Its cousin AWAC, in 4.5, does reintroduce a critic and does query it at self-chosen actions; 4.5 explains what makes that survivable. Read 4.4 first.) The supervised-learning template is where we go next.

4.3 Behavior cloning as the base case**4.3.1 The obvious thing to try, one more time**

Behavior cloning (BC) treats control as ordinary supervised learning: collect observation–action pairs $(\mathbf{o}_i, \mathbf{a}_i)$ from a demonstrator and fit a network to predict the action from the observation. As an objective,

$$\max_{\theta} \sum_i \log \pi_{\theta}(\mathbf{a}_i | \mathbf{o}_i),$$

IN PLAIN WORDS

$\pi_{\theta}(\mathbf{a} | \mathbf{o})$ is the probability density the network, with parameters θ , assigns to action $\mathbf{a}$ after seeing observation $\mathbf{o}$ — on your rig, $\mathbf{o}$ is a pair of camera frames plus the joint positions. $\mathbf{a}_i$ and $\mathbf{o}_i$ are the action and observation recorded on frame i , and the sum runs over every frame you logged. The logarithm turns a product of per-frame probabilities into a sum you can minibatch. Maximizing that sum over θ means: make the recorded actions the ones the network would most likely have produced. And notice what is absent — there is no reward anywhere in this expression.

Or, for a deterministic policy that outputs one action rather than a distribution, minimize a mean-squared error between the predicted and recorded actions. That is the whole method. There is no reward, no value function, no exploration, no critic, no environment.

What BC inherits from supervised learning is exactly what makes deep learning pleasant: it is stable, the loss goes down most of the time, it scales with data and parameters, and it works with *any* architecture you can backpropagate through. Nothing in the recipe cares whether π_{θ} is a two-layer MLP or a three-billion-parameter transformer. Hold on to that sentence; it is the one this chapter is ultimately about.

4.3.2 Exactly how it falls short

Its ceiling is just as easy to state: *BC treats every action in the dataset as equally worth imitating*. It clones the demonstrator's mistakes with exactly the same fidelity as their successes.

Make that concrete. Say you record 40 teleop episodes of a grasp on the SO-101. In 12 of them you fumbled the approach and spent about three seconds scrabbling at the object before recovering. At 20 Hz that is 60 frames each, so those episodes contribute roughly 720 frames of scrabbling to a dataset with perhaps 8,000 frames near the grasp. Maximum likelihood does not know scrabbling was a mistake. It sees that from observations that look like "gripper near object," the recorded behavior is a clean grasp most of the time and a scrabble about 9% of the time, and it dutifully reproduces that mixture. You did not train a policy that grasps; you trained one that grasps with a 9% scrabble rate baked in.

A git analogy: plain BC is like building your release by merging every commit in the history with equal trust, including the ones later reverted. AWR, which we are about to derive, weights each commit by whether the release it shipped in passed its tests. *Where the analogy breaks down*: commits are independent and a merge is binary, but robot actions are sequential and correlated — one bad action changes every state you see afterwards, so "blame" spreads across a whole episode rather than sitting on one line. Advantage estimation (Chapter 3) exists to handle that spreading, and reward shaping (Chapter 11) to sharpen it.

4.3.3 What a fix has to preserve

BC has a second, separate problem: compounding error under covariate shift, where small deviations take the policy to states the demonstrator never visited and errors snowball. That thread is picked up in Chapter 13 (DAgger, HG-DAgger, HIL-SERL). Set it aside for now.

So here is the question that defines the rest of the chapter. **Can we keep the supervised loss — with all of its stability and architecture-independence — but make the data's quality matter?**

4.4 AWR derived in slow motion

This is the longest derivation in Parts I and II, and it is the one everything after it stands on, so we take it at walking pace. Here is the map. There are three moves. First, take the data's own action distribution and *tilt* it toward the actions that turned out well, while keeping it on a leash — a limit on how far from the data it may move. The tilt follows a rule we derive rather than guess. The rule has one knob, the temperature β ; 4.4.3 shows where it comes from. Second, *fit* the network to that tilted distribution. Third, notice that the two moves together collapse into something almost embarrassingly simple: plain behavior cloning with one extra number attached to each logged sample. Figure 4.1 is that map as a picture.

Two promises about how this section is written. First: almost nothing in it is new. Almost every step has a twin in Chapter 3 that you have already survived — the surrogate objective of 3.2.3, the trust region of 3.3.1, the KL recipe of 3.3.2, the importance-sampling line of 3.2.2 — and we name each twin as we reach it. The two genuinely new pieces are the Lagrangian, introduced patiently in 4.4.3, and the idea of fitting a network to a target distribution by KL, which 4.4.4 shows you have in fact been doing since 4.3 without the name. Second: one small problem is carried from 4.4.2 to the end — one state of one grasp on your SO-101, three hundred logged frames, three actions, three advantages — and every formula in the section gets evaluated on it with a calculator. If a formula stops making sense, put the toy's numbers in.

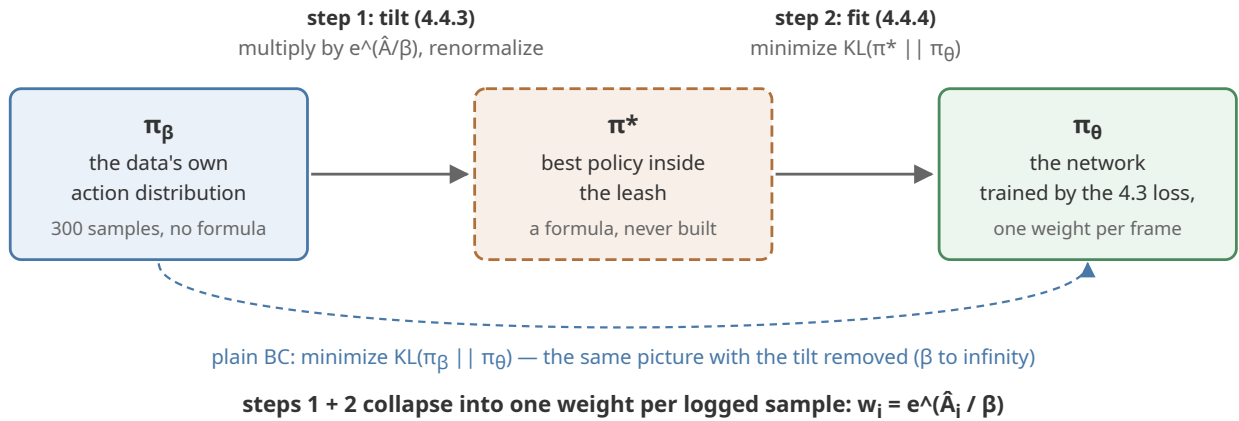


Figure 4.1 — The map of the derivation. Notice that π^* is never actually built; its box is dashed because it exists only as a formula, and step 2 turns "fit the network to π^* " into "fit it to the data you already have, each sample weighted," which is the bold line along the bottom. Plain BC is the same picture with the tilt removed: send $\beta \rightarrow \infty$ and every weight becomes 1.

4.4.1 The obvious thing to try: throw the bad data away

The simplest idea that could work: throw the bad data away. Compute a return for each episode, keep only the best — the top 10% by return, say — and behavior-clone that. This is a real method (filtered BC, or top- k % BC) and a respectable baseline you should run at least once.

Try it on the 40-episode grasp dataset of 4.3.2. Suppose the return penalizes wasted time (a discount factor or a per-step cost both do). Then the twelve scrabbling episodes score lowest. Put the cut just above them and you keep the twenty-eight clean episodes — 70% of the pool here, not the 10% above; the right fraction depends on the pool. The baked-in 9% scramble rate goes to zero. Notice, though, that you could only place that cut because 4.3.2 told you there were exactly twelve bad episodes. On a real pool nothing tells you, and the cut is a guess: the first complaint below. Filtered BC is crude in ways you can feel:

- The threshold is arbitrary. Why 10%? At 5% you have too little data to fit a big network; at 25% you are cloning mediocrity again. Nothing in the problem tells you where to cut.
- It is all-or-nothing. An episode just below the cut contributes nothing, even if it was nearly perfect and failed on one frame. An episode just above it counts in full, even if it succeeded by luck.
- It cannot express degrees. An episode twice as good as another gets the same treatment: included, at the same volume.
- It works at the wrong grain. Quality lives in frames, but the filter operates on episodes. Each of those twelve scrabbling grasps fumbled for three seconds and then recovered: it holds 60 bad frames and a few hundred good ones, and an episode-level cut keeps or discards them together.

What we want is the soft version: every sample stays, but each speaks with a volume proportional to how good it was. Notice the question that sentence hides. Proportional *how*? Linearly, so an action twice as good counts twice? Then what of a below-average action, whose advantage is negative — does it count negatively, pushing the network *away* as in Chapter 2's policy gradient, or a little but positively? There is a whole family of curves that turn "how good" into "how loud," and picking one by taste is a guess.

So we will not guess. We write down what we want as a constrained optimization problem, in the same shape as TRPO's in Section 3.3.1, and solve it; the curve falls out of the solution. To make that solution checkable rather than something to take on trust, the next subsection first sets up a problem small enough to solve on paper.

4.4.2 The constrained problem, on one state of one grasp

State the goal honestly, in words first. We want the policy with the highest expected advantage, subject to not straying far from the data. The second clause is the medicine of 4.2 written into the problem instead of bolted on afterward. Before formalizing either clause, meet the toy.

The SO-101's gripper is 3 cm above a cube and slightly to the left of it. Over many teleop sessions, exactly three different things were ever done from this position:

- $\mathbf{a}_1$: close the gripper now. The fingers close on air beside the cube; the grasp misses.
- $\mathbf{a}_2$: descend 1 cm. The offset is still there; sometimes the grasp lands, sometimes not. Average.
- $\mathbf{a}_3$: shift right 1 cm, then descend. The gripper centers on the cube and the grasp is clean.

Attach an advantage to each: $\hat{A}(\mathbf{a}_1) = -1$, $\hat{A}(\mathbf{a}_2) = 0$, $\hat{A}(\mathbf{a}_3) = +2$. As in Chapter 3, an advantage says how much better than usual an action turned out, judged against what typically happened from this state: $\mathbf{a}_2$ came out level with the baseline $V(\mathbf{s})$ — the value each estimate is measured against — $\mathbf{a}_1$ one unit below it, $\mathbf{a}_3$ two units above. They are estimates, made with Chapter 3's tools, which is why they wear hats.

Now the data. Three hundred logged frames sit at this state, one hundred of each action. So the behavior policy at this state, the distribution the data itself follows, is

$$\pi_{\beta}(\mathbf{a}_1 | \mathbf{s}) = \pi_{\beta}(\mathbf{a}_2 | \mathbf{s}) = \pi_{\beta}(\mathbf{a}_3 | \mathbf{s}) = \frac{1}{3}.$$

Formalize the first clause on the toy. The new policy π is three unknown numbers, $\pi(\mathbf{a}_1), \pi(\mathbf{a}_2), \pi(\mathbf{a}_3)$, non-negative and summing to one. Its expected advantage is

$$\mathbb{E}_{\mathbf{a} \sim \pi}[\hat{A}(\mathbf{a})] = \pi(\mathbf{a}_1) \cdot (-1) + \pi(\mathbf{a}_2) \cdot 0 + \pi(\mathbf{a}_3) \cdot 2.$$

Put the data's own distribution in and you get $-\frac{1}{3} + 0 + \frac{2}{3} = +\frac{1}{3}$: the score of cloning the data exactly as it is, which is what plain BC does. (A true advantage would average to zero over the data's own actions; ours average to $+\frac{1}{3}$ because they are estimates against an imperfect baseline. This does not matter, and 4.4.3 shows why in one sentence.)

The obvious way to push this up is to put all the mass on $\mathbf{a}_3$: $\pi = (0, 0, 1)$ scores **2**, the maximum possible. That is filtered BC in its purest form, and there are two reasons not to trust it. The first is noise: the **+2** is an estimate from one hundred frames, and betting the whole policy on it is betting the run on one number.

The second is worse. The unconstrained problem puts all its mass on whichever action has the largest advantage, and nothing says that action was ever logged. Give the toy a fourth action $\mathbf{a}_4$, the **+60°** wrist flex from 4.2, which no one ever performed. No logged return can score $\mathbf{a}_4$, because it was never taken; the only thing that can put a number on it is a critic asked to extrapolate, which is exactly what happened in 4.2. A critic asked about $\mathbf{a}_4$ will answer, and, as in 4.2, the answer is whatever flattering number its surface happens to hold out there; on the toy's scale call it **+8** — for an action that exists only in the critic's imagination. With nothing but the first clause, the answer is: always do the **+60°** flex. We need the second clause.

Before writing it, notice that the first clause is not new to you. In Section 3.2.3 you maximized the surrogate $L(\theta) = \mathbb{E}_t[\rho_t(\theta) \hat{A}_t]$, and the one-line algebra of 3.2.2 showed that, with ρ the importance ratio $\pi_{\theta}/\pi_{\theta_{old}}$, $\mathbb{E}_{\mathbf{a} \sim \pi_{\theta_{old}}}[\rho \hat{A}] = \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}}[\hat{A}]$: PPO's surrogate is, in words, the expected advantage of the old policy averaged over actions the new policy would choose. Our first clause is that sentence with $\pi_{\theta_{old}}$ renamed π_{β} . It tells the same lie 3.2.3 confessed to — it says nothing about the new policy later visiting different states — and, exactly as in Chapter 3, the lie grows with the distance from the policy that made the data. That is a third reason for a leash — noise and $\mathbf{a}_4$ were the first two — and it is the one Chapter 3 already gave you.

Now the second clause, and again you have seen it. TRPO in Section 3.3.1 took the honest route: since the surrogate is only trustworthy near the policy that generated the data, it constrained the KL divergence to that anchor policy to stay below a small threshold such as 0.01. Do the same thing here with two changes. The anchor is the behavior

policy π_β rather than the previous version of the policy, $\pi_{\theta_{old}}$, because π_β is who made the data. And the KL is written in the other direction — new policy on the left, data on the right; the paragraphs after the box explain why. Here the threshold is written δ :

$$\max_{\pi} \mathbb{E}_{a \sim \pi(\cdot|s)} [A^{\pi_\beta}(s, a)] \quad \text{s.t.} \quad D_{\text{KL}}(\pi(\cdot|s) \parallel \pi_\beta(\cdot|s)) \leq \delta.$$

IN PLAIN WORDS

Read the left half first. π is the new policy we are solving for. $\mathbb{E}_{a \sim \pi(\cdot|s)}[\cdot]$ means "sample an action from the new policy at state s , and average." $A^{\pi_\beta}(s, a)$ is the advantage of the behavior policy: how much better than average taking action a at s was, judged by what the data-generating policy went on to achieve; $\hat{A}$ is our estimate of it. So the left half says: put the mass on above-average actions — on the toy, $-\pi(a_1) + 2\pi(a_3)$. The right half is the leash. $D_{\text{KL}}(\pi \parallel \pi_\beta)$ measures how far the new distribution is from the data's, and δ is how far we allow it: the radius of the leash, in nats, defined below. Small δ : tight leash, timid update. Large δ : loose leash, and the extrapolation disaster of 4.2 returns.

One letter to watch: this δ , with no subscript, is the leash radius, not the TD residual δ_t of Chapter 3.

Read the KL the way Section 3.3.2 taught you: bind the two letters, then ask who is drawing the samples. In Chapter 3 you bound p to the old policy and q to the new one. Here p is the *new* policy π and q is the data's π_β — not TRPO's direction. The a_4 check below shows what this direction buys, and 4.4.3 says in one paragraph what the other direction would have let through. The recipe still works: the new policy draws the action; for each action drawn, compare how likely the new policy was to take it against how often the data took it; average the log of that ratio.

That average is measured in nats: a nat is the unit of a KL whose logarithm is natural, as every **log** in this chapter is. A KL of 1 nat means the log of the ratio averages to 1 over the actions the new policy takes. Undo the log and that says: a typical action is about e , roughly 2.7, times more likely under the new policy than under the data. (Typical in the sense that it is the logarithm that is averaged, so some actions more, some less.) On the toy, with $\pi_\beta = \frac{1}{3}$ everywhere,

$$D_{\text{KL}}(\pi \parallel \pi_\beta) = \sum_a \pi(a) \log \frac{\pi(a)}{\pi_\beta(a)} = \sum_a \pi(a) \log(3\pi(a)).$$

Two edge cases of the KL formula matter here. An action the *new* policy never takes contributes nothing: $0 \cdot \log 0$ is read as 0. An action the *data* never took, given any positive probability by the new policy, contributes $+\infty$: $x \cdot \log(x/0)$ with $x > 0$.

Put numbers in. At $\pi = \pi_\beta$ every ratio is 1, every log is 0, and the KL is 0: the leash is slack. At the **greedy** policy — all mass on the single best-scoring action, $\pi = (0, 0, 1)$ — the two actions it never takes contribute 0 and the only surviving term is $1 \cdot \log 3 \approx 1.10$ nats. Now bring back a_4 . The data never took it, so $\pi_\beta(a_4) = 0$. Give it any positive probability and its term is $\pi(a_4) \log(\pi(a_4)/0) = +\infty$: the constraint cannot be satisfied at any δ . This direction of KL reports an infinite leash length for inventing an action the data never took — compare 3.3.2, where in the other direction such a move "barely registers directly." That is what will make AWR structurally safe offline.

One question is left hanging: who chooses δ ? In TRPO you set 0.01 by hand. Here, you will find, nobody does.

4.4.3 Solving it by hand: why the answer is an exponential

Here is the pleasant surprise. TRPO's version of this problem has no closed-form solution — no explicit formula, only an iteration — which is why Section 3.3.3 needed Fisher-vector products, conjugate gradients, and backtracking, and why Chapter 3 ended up with PPO's clip instead. This problem *does* have one. The difference: in TRPO the unknown is θ , millions of network weights, and the KL as a function of them is a curved surface you can only probe locally. Here the unknown is the distribution itself — on the toy, three numbers, with no network in the problem — which is what **nonparametric** means; and the answer is an explicit formula rather than something you iterate toward, which is what **closed form** means. That is why the derivation fits in a few lines. This step is where most readers get lost, so we take it slowly.

Six moves. Price the constraint (the Lagrangian). Differentiate one probability at a time. Solve, and watch the exponential appear. Put the toy in at two temperatures. Try three variants — unequal data, the unlogged action, and a different leash — the last of which is the real answer to "why an exponential." Finally, put numbers on what β buys in leash.

The tool is the **Lagrangian**, and here is all it means. You have a thing to maximize and a rule you must obey. Constrained problems are awkward because the best point may sit on the edge of the allowed region, where derivatives are not zero, so "set the derivative to zero" finds nothing there. Instead of obeying the rule, *charge a price* for breaking it: subtract from the objective a price per unit of leash used, then maximize the resulting unconstrained expression. The price is the **Lagrange multiplier**, and for the right price the priced problem and the constrained one have the same answer (the end of this subsection shows why, once we have numbers to do it with).

An analogy from work: a cloud budget. A hard cap says "this team may use at most δ GPU-hours a month." A price says "each GPU-hour costs the team β dollars from its own budget." Under the price the team optimizes work minus cost, and for the right price it lands on exactly the usage the cap would have allowed; you steer usage by moving the price. *Where the analogy breaks down:* in the accounting version you fix the cap and search for the price that lands on it. AWR does the reverse: you fix the price β , accept whatever leash length comes out, and never search for the δ it corresponds to. The end of this subsection puts numbers on that exchange.

Write it out, on the toy first. Charge a price β per nat of KL and a second price λ per unit by which the probabilities fail to sum to one. (This λ lives only inside this derivation; it is not the GAE λ of Chapter 3.) Call the priced expression $\mathcal{L}$ (a script L, for Lagrangian; not Chapter 3's surrogate $L(\theta)$). With every $\pi_\beta(a) = 1/3$, each $\log(\pi(a)/\pi_\beta(a))$ is $\log(3\pi(a))$, as in the KL formula above:

$$\begin{aligned}\mathcal{L} = & -\pi(a_1) + 2\pi(a_3) \\ & - \beta \left[\pi(a_1) \log(3\pi(a_1)) + \pi(a_2) \log(3\pi(a_2)) + \pi(a_3) \log(3\pi(a_3)) - \delta \right] \\ & - \lambda [\pi(a_1) + \pi(a_2) + \pi(a_3) - 1].\end{aligned}$$

Three lines: the thing we want, minus the leash charge, minus the mass charge. For any list of actions the same expression is

$$\mathcal{L} = \sum_a \pi(a) \hat{A}(a) - \beta \left[\sum_a \pi(a) \log \frac{\pi(a)}{\pi_\beta(a)} - \delta \right] - \lambda \left[\sum_a \pi(a) - 1 \right].$$

For continuous actions the sums become integrals over a and each π is a density; nothing else changes.

Two remarks on what is in the bracket and what is not. The bracket is the constraint written as "leash used minus leash allowed", so it is zero exactly when the leash is taut. And we charge nothing for non-negativity: you will see in a moment that the solution respects it on its own, and the squared-distance contrast later in this subsection shows a leash whose solution does not.

Now "differentiate pointwise," the phrase papers use and rarely explain. It means: take the derivative with respect to *each unknown probability separately*, treating the others as constants, and set each to zero. On the toy that is three derivatives, one per unknown. Start with $\pi(a_3)$, the clean grasp, and go through the three terms of $\mathcal{L}$ that contain it.

The first, $2\pi(a_3)$, has derivative **2**. The last, $-\lambda\pi(a_3)$, has derivative $-\lambda$. The middle one is $-\beta$ times $\pi(a_3) \log(3\pi(a_3))$. Set the $-\beta$ aside for a moment and differentiate what it multiplies, which needs the product rule, the only piece of calculus in the whole derivation: $\frac{d}{dx} x \log x = 1 \cdot \log x + x \cdot \frac{1}{x} = \log x + 1$. Split the term as $\pi(a_3) \log \pi(a_3) + \pi(a_3) \log 3$. The $\log 3$ is a constant, so the second piece has derivative $\log 3$ and the first piece, by the product rule, $\log \pi(a_3) + 1$. The derivative of the whole split term is therefore $\log \pi(a_3) + 1 + \log 3 = \log(3\pi(a_3)) + 1$.

In the general symbols that $\log 3$ is $-\log \frac{1}{3} = -\log \pi_\beta(a_3)$, so the same line reads as follows (the curly ∂ is the partial-derivative sign: the derivative with respect to this one variable, every other variable held fixed — exactly what "pointwise" meant):

$$\frac{\partial}{\partial \pi(a_3)} \left[\pi(a_3) \log \frac{\pi(a_3)}{\pi_\beta(a_3)} \right] = \log \pi(a_3) - \log \pi_\beta(a_3) + 1 = \log \frac{\pi(a_3)}{\pi_\beta(a_3)} + 1.$$

On the toy that is $\log(3\pi(a_3)) + 1$, as we found.

Notice what did *not* appear: δ . It sits in a term with no $\pi(a_3)$ in it, so it vanished the moment we differentiated. Hold that thought. Put the $-\beta$ back in front of the middle piece, assemble the three pieces, and set the sum to zero:

$$2 - \beta \left[\log(3\pi(a_3)) + 1 \right] - \lambda = 0.$$

The equations for a_1 and a_2 are identical with -1 and 0 in place of 2 , and with the same λ , because there is only one mass constraint. Three equations of one shape, so solve the shape once. Write $\hat{A}(a)$ for the advantage and $\pi_\beta(a)$ for the data's share (the 3 above was $1/\pi_\beta(a_3)$), and write plain a for whichever of the three actions we are differentiating with respect to:

$$\hat{A}(a) - \beta \left[\log \frac{\pi(a)}{\pi_\beta(a)} + 1 \right] - \lambda = 0.$$

Now solve for $\pi(a)$, one move per line. Move the bracket across, divide by β , and subtract the 1 that was inside the bracket:

$$\log \frac{\pi(a)}{\pi_\beta(a)} = \frac{\hat{A}(a) - \lambda}{\beta} - 1.$$

Undo the logarithm by exponentiating both sides. This is the moment the exponential is born. The leash's derivative was a logarithm, and the only way to undo a logarithm is to exponentiate. Later in this subsection we swap the leash for a different one and watch a different curve come out. The exponential of a sum is a product of exponentials, so the right side splits into a factor that depends on the action and one that does not; then multiply through by $\pi_\beta(a)$:

$$\pi(a) = \pi_\beta(a) e^{\hat{A}(a)/\beta} e^{-\lambda/\beta - 1}.$$

Look at the last factor. It contains λ and β but not a : it is the same number for every action, and its only job is to make the probabilities sum to one, which is what λ was hired to do. So we never need λ itself: compute the first two factors for every action, add them up, and divide by the total. Call that total Z , so the last factor is simply $1/Z$: $e^{-\lambda/\beta - 1} = 1/Z$, and λ has done its job without ever being computed.

One consequence: add the same constant c to every $\hat{A}$ and every piece $\pi_\beta(a) e^{\hat{A}(a)/\beta}$ is multiplied by the same factor $e^{c/\beta}$, which Z removes. π^* does not care where the advantages' zero sits, which is why the $+\frac{1}{3}$ of 4.4.2 was harmless.

Now put the toy in, before writing anything in general symbols. At $\beta = 1$ the exponential factors are simply $e^{\hat{A}}$. Multiply each by its $\pi_\beta(a) = 1/3$ to get the three unnormalized pieces, add the pieces to get Z , and divide:

```
e^{-1} = 0.3679      e^{0} = 1.0000      e^{2} = 7.3891
pieces pi_beta(a) e^{A(a)}:
(1/3)(0.3679) = 0.1226   (1/3)(1.0000) = 0.3333   (1/3)(7.3891) = 2.4630
Z = (1/3)(0.3679 + 1.0000 + 7.3891) = (1/3)(8.7570) = 2.9190
pi*(a1) = 0.1226 / 2.9190 = 0.0420   -> 4.2%
pi*(a2) = 0.3333 / 2.9190 = 0.1142   -> 11.4%
pi*(a3) = 2.4630 / 2.9190 = 0.8438   -> 84.4%
check: 0.0420 + 0.1142 + 0.8438 = 1.0000
```

The data did the clean grasp one time in three; the best policy inside the leash does it 84 times in a hundred, keeps the average action at 11%, and still performs the miss 4% of the time.

That last number answers the question 4.4.1 left open. A below-average action is not pushed away, the way a negative advantage pushes in Chapter 2's policy gradient; $e^{-1} = 0.37$ is small but positive, so the miss is turned down, never turned off.

The objective rose from 0.3333 to $\mathbb{E}_{\pi^*}[\hat{A}] = 0.0420 \cdot (-1) + 0.1142 \cdot 0 + 0.8438 \cdot 2 = 1.6456$.

Is this π^* really the best point, or merely a place where the derivatives vanish? A stationary point is a point where every derivative is zero. It could in principle be a saddle, a peak along one direction and a valley along another. Here it is not. The expected advantage is a straight line in each probability; the KL is convex in them, because $x \log x$ curves upward, so a sum of such terms has a single valley. A straight line minus a single valley is a single peak. So this stationary point is the maximum, and there is no other.

Now raise β to 5. β divides the advantage before the exponential, which is exactly what 4.1's vocabulary box called a temperature. Raising it warms the tilt. Divide every advantage by 5 *before* exponentiating:

```
e^{-1/5} = e^{-0.2} = 0.8187      e^0 = 1.0000      e^{2/5} = e^{0.4} = 1.4918
sum = 0.8187 + 1.0000 + 1.4918 = 3.3105
Z = (1/3)(3.3105) = 1.1035; dividing (1/3)e^{A/5} by Z is the same as dividing e^{A/5} by 3.3105
pi*(a1) = 0.8187 / 3.3105 = 0.2473    -> 24.7%
pi*(a2) = 1.0000 / 3.3105 = 0.3021    -> 30.2%
pi*(a3) = 1.4918 / 3.3105 = 0.4506    -> 45.1%
E[A] under pi* = -0.2473 + 0 + 2(0.4506) = 0.6539
```

Same data, same advantages, a much gentler tilt: the clean grasp goes to 45% rather than 84%, and the objective to 0.65 rather than 1.65. You will meet these six percentages again in 4.4.5.

Why was one state enough? Peng et al. state the objective with an average over states rather than for a single state. But once the leash is priced with a single β , that price is shared by every state and nothing else joins the states together, so the derivative at any one state looks exactly like the one above. Each state carries its own sum-to-one constraint, so its own λ and its own Z . And a state's frequency in the data multiplies its objective term and its leash term equally, so it cancels from the condition that sets the derivative to zero. The solution therefore has this same shape at every state; their appendix has the derivation in the compressed form you just saw unpacked. Written once in general symbols, with the true advantage $A^{\pi\beta}$ in place of the estimate $\hat{A}$ we used on the toy — the algebra does not care which:

$$\pi^*(a | s) \propto \pi_\beta(a | s) \exp(A^{\pi\beta}(s, a)/\beta),$$

IN PLAIN WORDS

The best policy inside the leash is the data's own distribution, multiplied pointwise by an exponential factor, then renormalized. π^* is that best policy; the star means "the solution," not "a network." $\pi_\beta(a | s)$ is how often the data took action a at s . $A^{\pi\beta}(s, a)$ is the behavior policy's advantage from the first key equation; $\hat{A}$ is our estimate of it. $\exp(A^{\pi\beta}(s, a)/\beta)$ is a positive number: above 1 for an above-average action, below 1 for a below-average one. $\beta > 0$ is the **temperature**; formally it is the Lagrange multiplier of the KL constraint, the price per nat of leash, and in practice you tune it directly and never choose a δ . The $\propto$ means "up to the normalizing constant $Z(s)$ that makes the result sum to one." And if $\pi_\beta(a | s) = 0$, the data never took that action, so $\pi^*(a | s) = 0$ too. This tilt reweights; it never invents.

One notation note for reading papers: the letter used for the data-generating policy varies from paper to paper, and one common choice, π_{old} , is Chapter 3's letter for the previous iterate — which invites exactly the wrong reading. Whatever you see, read it as the policy the samples came from, the role π_β plays here. And Peng et al. write the leash radius as ϵ rather than δ — a leash radius, not PPO's clip parameter. The star in π^* means "the solution"; it is unrelated to the star in $\pi_{0.6}^*$, the Physical Intelligence model of Chapters 8 to 10, where it is part of a product name.

A physical analogy: think of a graphic equalizer on a mixing desk. The recording is fixed; you are only turning up and down the tracks already on it, and the temperature sets how far the faders may travel. *Where the analogy breaks down*: a real fader is bounded, usually to something like a 60 dB range, whereas **exp** is unbounded. One track can be turned up by a factor of three thousand, which is a genuine practical problem and the subject of this chapter's pitfall box.

Equal data for the three actions hides one thing the formula does. Suppose instead the logs were mostly misses: $\pi_\beta = (0.6, 0.3, 0.1)$, the clean grasp only one frame in ten. Keep $\beta = 1$:

```
unnormalized: 0.6 x 0.3679 = 0.2207    0.3 x 1.0000 = 0.3000    0.1 x 7.3891 = 0.7389
Z = 0.2207 + 0.3000 + 0.7389 = 1.2596
pi* = (0.2207, 0.3000, 0.7389) / 1.2596 = (0.1752, 0.2382, 0.5866)
```

The good action was 10% of the data and becomes 59% of π^* ; the miss falls from 60% to 18%. This is what AWR is for: it finds the best behavior latent in your pool, even when it is rare, and makes it the policy's usual behavior.

Now the variant that matters most for safety. Bring back a_4 , the $+60^\circ$ wrist flex: never logged, $\pi_\beta(a_4) = 0$, and carrying the critic's imagined advantage of about $+8$. Then $e^8 \approx 2981$, a factor that would swamp the other three — and $\pi^*(a_4) = 0 \times 2981 = 0$, with Z and the other three probabilities unchanged. Sit with that, because it is the whole reason this family is safe offline. The **support** of π_β is the set of actions the data gives any probability at all: the actions it ever took. a_4 is outside it. The tilt cannot move mass to an action outside the support, because zero times anything is zero. The failure mode of 4.2 is not being *penalized* here; it is structurally unreachable.

That safety has a price, and you should know it now. Because the tilt only moves mass among actions the data already contains, AWR is a reshuffling operator, not an exploration operator. You just watched it promote a one-in-ten action to the policy's default. It cannot invent a recovery motion nobody ever performed, so improvement is bounded by what the pool holds. That is why the LeHome loop spends most of its engineering on how rollouts are collected and which episodes enter the pool (Chapter 14) rather than on the policy update — and why exploration for this class of policy is still open.

You may still be asking the question this subsection promised to answer: why an *exponential*? What about the problem made it come out that way? The cleanest way to see it is to change the leash and watch the answer change. Suppose "close to the data" were measured by squared distance, $\sum_a (\pi(a) - \pi_\beta(a))^2 \leq \delta$, the first thing an engineer would reach for. The Lagrangian is the same one with the new leash in the bracket:

$$\mathcal{L}' = \sum_a \pi(a) \hat{A}(a) - \beta \left[\sum_a (\pi(a) - \pi_\beta(a))^2 - \delta \right] - \lambda \left[\sum_a \pi(a) - 1 \right].$$

Differentiate pointwise as before: the derivative of $(\pi(a) - \pi_\beta(a))^2$ with respect to $\pi(a)$ is $2(\pi(a) - \pi_\beta(a))$ by the chain rule, so setting the derivative to zero gives $\hat{A}(a) - 2\beta(\pi(a) - \pi_\beta(a)) - \lambda = 0$, and solving,

$$\pi(a) = \pi_\beta(a) + \frac{\hat{A}(a) - \lambda}{2\beta},$$

a *linear* tilt: add a slice proportional to the advantage. It looks harmless. Put the toy in at $\beta = 1$. The probabilities must sum to one: $\sum_a \pi(a) = \sum_a \pi_\beta(a) + \sum_a (\hat{A}(a) - \lambda)/2 = 1 + (\sum_a \hat{A}(a) - 3\lambda)/2$, which must equal 1, so $\lambda = \frac{1}{3} \sum_a \hat{A}(a) = \frac{1}{3}$. Then

```
pi(a1) = 1/3 + (-1 - 1/3)/2 = 1/3 - 2/3 = -1/3
pi(a2) = 1/3 + ( 0 - 1/3)/2 = 1/3 - 1/6 = 1/6
pi(a3) = 1/3 + ( 2 - 1/3)/2 = 1/3 + 5/6 = 7/6
```

A negative probability for the miss and a probability above one for the clean grasp. To rescue this you would have to bolt on a third constraint, $\pi(a) \geq 0$, and clip: an extra rule the KL leash never needs, because an exponential is positive on its own.

Now bring in a_4 , with $\pi_\beta(a_4) = 0$ and the imagined advantage of 8. The linear recipe gives $\pi(a_4) = 0 + (8 - \lambda)/2$, positive for any λ below 8. With a_4 in the sum, normalization sets $\lambda = (-1 + 0 + 2 + 8)/4 = 9/4$, so $\pi(a_4) = (8 - 9/4)/2 = 23/8$, about 2.9: a "probability" of nearly three for an action nobody ever performed. The squared-distance leash *invents the $+60^\circ$ action*.

Clipping the negative entries to zero does not save it either: carry the normalization through with the clip in place and the only consistent answer is $\pi = (0, 0, 0, 1)$, everything on the action nobody ever took. A linear tilt has no notion of "the data never did this."

So the leash is not cosmetic. KL is built out of logarithms, so its derivative is a logarithm. Setting that derivative to zero makes $\log(\pi/\pi_\beta)$ linear in the advantage. Undoing a logarithm gives an exponential, which is always positive and multiplies π_β , so it is zero exactly where the data is zero. The exponential is not a modeling choice; it is what "stay close to the data, measured in KL" turns into once you solve it.

Papers call the result an **exponential tilt** of π_β , or a Boltzmann or Gibbs distribution; the vocabulary box in 4.4.4 collects those names.

One last contrast, and then the question of δ . Had we written the leash in TRPO's direction, $D_{\text{KL}}(\pi_\beta \parallel \pi)$, the a_4 term would be $0 \cdot \log(0/\pi(a_4)) = 0$, whatever $\pi(a_4)$ is. That direction charges for mass drained *from* a_1, a_2, a_3 — 3.3.2's "it registers because that mass had to come from somewhere" — but not one nat for where the mass *lands*. So whatever mass the leash lets leave the logged actions is free to land on the $+60^\circ$ flex, and with an imagined advantage of $+8$ waiting there, that is exactly where the optimizer sends it. (Solve that version of the problem on the toy at $\beta = 1$ and 87% of the policy ends up on a_4 .) The direction 4.4.2 chose reads infinity for the same move. That is why the leash points the way it does.

Finally, who chooses δ ? Nobody, and now you can see the mechanism. δ vanished from the derivative, so the solution only knows the price β . But each price implies a leash length actually used — the KL between π^* and π_β — and you can compute it with the toy's KL formula $\sum_a \pi^*(a) \log(3\pi^*(a))$:

```
# ln = natural log, the same log as in every formula above
beta = 1:  0.0420 ln(0.1260) + 0.1142 ln(0.3426) + 0.8438 ln(2.5314)
          =  0.0420(-2.0715) + 0.1142(-1.0712) + 0.8438(0.9288)
          = -0.0870          - 0.1223          + 0.7837          = 0.574 nats
beta = 5:  0.2473 ln(0.7419) + 0.3021 ln(0.9063) + 0.4506 ln(1.3518)
          =  0.2473(-0.2985) + 0.3021(-0.0984) + 0.4506(0.3014)
          = -0.0738          - 0.0297          + 0.1358          = 0.032 nats
```

Those are two rows of a table you can extend in either direction. Push β up and the tilt fades toward plain BC; push it down and the tilt sharpens toward the greedy policy, whose KL of $\log 3$ you computed in 4.4.2:

beta	pi*(a1)	pi*(a2)	pi*(a3)	KL(pi* pi_beta)	KL(pi* pi_beta) = the leash this beta buys
inf	0.3333	0.3333	0.3333	0.000	(no tilt: plain BC)
100	0.3289	0.3322	0.3389	0.0001	
5	0.2473	0.3021	0.4506	0.032	
1	0.0420	0.1142	0.8438	0.574	
0.25	0.0000	0.0003	0.9997	1.096	
-> 0	0	0	1	1.099 = ln 3	(greedy: the leash cannot be longer)

So on this data, choosing $\beta = 5$ is the same decision as choosing a leash of radius $\delta \approx 0.03$, and $\beta = 1$ is the same as choosing $\delta \approx 0.57$. Why is it the same decision? Call the expected advantage a policy's *plain score* and the Lagrangian its *priced score*. For any distribution the mass term is zero, so the priced score is the plain score minus β times (KL used minus δ). Look at that bracket: a policy that uses less leash than δ is paid a credit, $\beta(\delta - \text{KL})$, on top of its plain score. Every policy inside the leash therefore has a priced score at least as large as its plain score. π^* has the highest priced score of any distribution: it is the single peak we found, a straight line minus a single valley. π^* also uses exactly δ of leash, so its credit is zero and its priced and plain scores are equal. Chain those: $\text{plain}(\pi) \leq \text{priced}(\pi) \leq \text{priced}(\pi^*) = \text{plain}(\pi^*)$. No policy inside the leash beats π^* on the plain score, which is what the constrained problem asked for.

Do not compare these radii with TRPO's 0.01 in 3.3.1: that is an average over states of a continuous Gaussian's KL, paid again at every update, and the scales have nothing to say to each other.

The multiplier is literally an exchange rate. Going from $\beta = 5$ to $\beta = 1$ bought **0.99** more expected advantage ($1.6456 - 0.6539$) for **0.54** more nats of leash ($0.574 - 0.032$): about 1.8 units of advantage per nat, averaged over that stretch.

At any single temperature the rate is exactly β : five units of advantage per nat at $\beta = 5$, one per nat at $\beta = 1$. That is a general fact about Lagrange multipliers, used here without proof. You can check it roughly on the table. Take two neighboring rows, compute the objective $-\pi^*(a_1) + 2\pi^*(a_3)$ for each, and divide its change by the change in KL; the answer always lands between the two rows' β s ($5 \rightarrow 1$ gives the 1.8 above; $100 \rightarrow 5$ gives about 9.5). That is what a Lagrange multiplier *is*: the rate at which the best achievable objective rises per unit of constraint.

Every temperature you set implies a leash radius you could compute the same way — one per state. At the same price a state with spread-out advantages uses more leash than a calm one: the toy's 0.57 nats at $\beta = 1$, against about 0.003 nats for a state whose three equally logged actions scored $\hat{A} = (-0.1, 0, +0.1)$. A single radius quoted for a whole dataset can only be an average of those. Nobody computes any of them, because the price is the more convenient dial: β enters the solution directly, while a chosen δ would have to be converted into its β by a search.

4.4.4 The projection step: from a formula to a network

There is one problem left, and it is not small. π^* is a formula, not a network. It is defined as "whatever π_β is, times an exponential," and we do not have π_β . On the toy we read it off by counting; for a camera image on the SO-101, the probability that the human would have sent this wrist delta given these pixels is not a number anyone possesses. We have samples of it — three hundred at the toy's state, one per logged frame on the real robot. So we need a network π_θ as close to π^* as its architecture allows, trained from samples alone, and able to generalize to states the formula has never seen. Papers call this the **projection** of π^* onto the network's family: the member of that family nearest to π^* . Nearest in what sense is the first decision; turning that choice into something you can compute from logged frames alone is the second. We take it in four steps.

Step one: you already know how to project onto a network, because plain BC is a projection. Section 4.3 trained BC by maximizing $\sum_i \log \pi_\theta(a_i | o_i)$. That objective has a standard name, **maximum likelihood**: pick θ to make the logged actions as probable as possible. Or, dividing by the number of frames N , which changes nothing about where the maximum is: maximize $\frac{1}{N} \sum_i \log \pi_\theta(a_i | o_i)$. Now fix one state, as 4.4.2 and 4.4.3 did — the toy's — and look only at its frames; the sum over states comes back near the end of this subsection. At one state the observation is fixed, so for the next few paragraphs write $\pi_\theta(a)$ for $\pi_\theta(a | o)$. An average over recorded samples estimates an expectation under whatever produced them, and π_β produced them, so BC is maximizing an estimate of $\mathbb{E}_{a \sim \pi_\beta}[\log \pi_\theta(a)]$. Add and subtract $\log \pi_\beta(a)$ inside that expectation, which changes nothing:

$$\mathbb{E}_{a \sim \pi_\beta}[\log \pi_\theta(a)] = \mathbb{E}_{a \sim \pi_\beta} \left[\log \frac{\pi_\theta(a)}{\pi_\beta(a)} \right] + \mathbb{E}_{a \sim \pi_\beta}[\log \pi_\beta(a)] = -D_{\text{KL}}(\pi_\beta \| \pi_\theta) + \mathbb{E}_{a \sim \pi_\beta}[\log \pi_\beta(a)].$$

The first piece is minus a KL, by the 3.3.2 definition with $p = \pi_\beta$ and $q = \pi_\theta$: $\log(\pi_\theta/\pi_\beta)$ is $-\log \pi_\beta + \log \pi_\theta$. The second piece has no θ in it; it is a property of the data alone, and no change to the network moves it. So maximizing BC's objective is the same as minimizing $D_{\text{KL}}(\pi_\beta \| \pi_\theta)$. Behavior cloning is the instruction: make the network's distribution as close as possible, in KL, to the data's distribution.

A KL is never negative and is zero only when the two distributions agree, so $-D_{\text{KL}} + \text{constant}$ peaks exactly at $\pi_\theta = \pi_\beta$. On the toy, BC's objective is $\frac{1}{3}[\log \pi_\theta(a_1) + \log \pi_\theta(a_2) + \log \pi_\theta(a_3)]$, maximized at $\pi_\theta = \pi_\beta$. AWR is the identical instruction with π^* in place of π_β :

$$\min_{\theta} D_{\text{KL}}(\pi^* \| \pi_\theta).$$

Same machinery, one target swapped — the lower and upper paths of Figure 4.1. On the toy, a network that could represent any three-number distribution would land exactly on $\pi^* = (0.042, 0.114, 0.844)$; in general it cannot, and the projection lands on the nearest thing it can.

Step two: the direction of the KL. The arguments have swapped relative to 4.4.2: there the thing we were solving for sat on the left of the KL, here π_θ sits on the right. KL is not symmetric (3.3.2), so this is a choice, not a typo; the 3.3.2 recipe says why. Ask who draws the samples. $D_{\text{KL}}(\pi^* \parallel \pi_\theta)$ averages over actions drawn from π^* , and step four shows that our logged actions can be reweighted into exactly that average. The other direction, $D_{\text{KL}}(\pi_\theta \parallel \pi^*)$, averages over actions drawn from the network. At each such action it needs $\log \pi^*(a) = \log \pi_\beta(a) + \hat{A}(a)/\beta - \log Z$. Two of those pieces we do not have. $\pi_\beta(a)$ is the behavior density at an action the network chose. $\hat{A}(a)$ is an advantage at an action the network chose — a critic query at a self-chosen action, the pattern that sent the wrist to $+60^\circ$ in 4.2.

Two names for what you just saw. Target on the left, model on the right is the **forward KL**, what maximum likelihood minimizes; the other direction is the **reverse KL**. By this naming, the leash of 4.4.2 was a reverse KL: $D_{\text{KL}}(\pi \parallel \pi_\beta)$ has the thing being solved for on the left. The projection here is a forward KL: the target π^* is on the left and the model π_θ on the right.

So the leash and the projection point in opposite directions, and both are right, because they do different jobs. The leash was chosen for what it charges an action the data never took: infinity, the $\mathbf{a}_4$ check of 4.4.2. The projection was chosen for who can draw the samples: the data supplies actions from π_β , and step four reweights them into π^* ; at the network's own draws we would need π_β and $\hat{A}$, which we do not have.

An exam analogy: forward KL grades the network on the questions in the answer key — the logged actions — and we have that key. Reverse KL grades the network on questions it wrote for itself, which needs an answer key for every possible question, including the $+60^\circ$ wrist flex nobody asked. *Where the analogy breaks down:* in a real exam both directions are possible and the choice is about fairness; here the reverse direction is not unfair but uncomputable.

Step three: drop the term without θ . Write the forward KL out in full, as we did for BC:

$$D_{\text{KL}}(\pi^* \parallel \pi_\theta) = \mathbb{E}_{a \sim \pi^*}[\log \pi^*(a \mid s)] - \mathbb{E}_{a \sim \pi^*}[\log \pi_\theta(a \mid s)].$$

Look at the two terms. The first contains no θ at all: it is the negative entropy of π^* (entropy: how spread out a distribution is; a constant here), a fixed number once π^* is fixed, and 4.4.3 fixed it. Nothing you do to the network changes it, so it contributes nothing to the gradient and you drop it. Minimizing the KL therefore reduces to one instruction: maximize $\mathbb{E}_{a \sim \pi^*}[\log \pi_\theta(a \mid s)]$, the expected log-likelihood the network assigns to actions drawn from π^* — the one quantity left to maximize.

Step four: the change of measure, which is the step that makes everything work. We cannot sample from π^* ; we can only sample from π_β , because that is what a dataset is. A **change of measure** rewrites an average under one distribution as an average under another by attaching a ratio of the two to each sample — and you have done one already. It is importance sampling, Section 3.2.2, where $\rho = \pi_\theta/\pi_{\theta_{\text{old}}}$ turned an average under $\pi_{\theta_{\text{old}}}$ into one under π_θ . Do the identical thing with π^* in the role of π_θ and π_β in the role of $\pi_{\theta_{\text{old}}}$, one equality per move:

$$\begin{aligned} \mathbb{E}_{a \sim \pi^*}[\log \pi_\theta(a)] &= \sum_a \pi^*(a) \log \pi_\theta(a) \\ &= \sum_a \pi_\beta(a) \frac{\pi^*(a)}{\pi_\beta(a)} \log \pi_\theta(a) \\ &= \sum_a \pi_\beta(a) \frac{e^{\hat{A}(a)/\beta}}{Z} \log \pi_\theta(a) \\ &= \mathbb{E}_{a \sim \pi_\beta} \left[\frac{e^{\hat{A}(a)/\beta}}{Z} \log \pi_\theta(a) \right]. \end{aligned}$$

Read it top to bottom. The first equality writes the expectation as a sum. The second multiplies and divides by $\pi_\beta(a)$. Do it only over actions the data contains; the other actions contribute nothing to the left side anyway, because π^* is zero there (4.4.3), so leaving them out changes nothing. The third substitutes the closed form $\pi^*(a) = \pi_\beta(a)e^{\hat{A}(a)/\beta}/Z$, and the two $\pi_\beta(a)$ cancel. The fourth reads the result as an expectation under π_β . It is

Section 3.2.2's line with the letters renamed and $f = \log \pi_\theta$. On the toy the first form puts weight **0.0420** on $\log \pi_\theta(a_1)$; the last form puts $\frac{1}{3} \times 0.3679/2.9190 = 0.0420$ on it. Same number, reached from the side we can actually sample.

Now the twist that 3.2.2 could not offer. Look at the ratio here: $\pi^*/\pi_\beta = e^{\hat{A}/\beta}/Z$. The behavior density canceled — the most important sentence in the section. Section 3.2.2 warned that the importance ratio requires the actual density of the action under both policies, and that Chapter 7 is where that stops being satisfiable. Here that requirement has vanished. The *weight* needs no density evaluation of anything — not π_β , not π^* , not the network's own.

$\hat{A}$ you compute before training. Z needs no density either: it is the average of $e^{\hat{A}/\beta}$ over the logged actions at that state, 2.919 on the toy at $\beta = 1$, and a few paragraphs from here it is dropped. The only density left anywhere in the expression sits outside the weight, in the *loss*: the ordinary BC term, $\log \pi_\theta$ at a logged action.

That single cancellation is why AWR needs no model of the behavior policy, and it is why Section 4.6 will be able to replace even $\log \pi_\theta$ with any supervised loss. When 4.6 says AWR never forms an importance ratio, this is what it means: w_i is a ratio only on paper. The densities that would make it one have already canceled, and what remains is a number you read off $\hat{A}_i$.

The last thing to see is that an average under π_β is exactly what a dataset average estimates. On the toy, for any function g of the action, $\sum_a \pi_\beta(a) g(a) = \frac{1}{300} \sum_{i=1}^{300} g(a_i)$, because each action appears in exactly the proportion π_β says. So

$$\mathbb{E}_{a \sim \pi^*} [\log \pi_\theta(a)] = \frac{1}{300} \sum_{i=1}^{300} \frac{e^{\hat{A}_i/\beta}}{Z} \log \pi_\theta(a_i).$$

On the toy this is exact, because the counts are exactly 100 each. On a real dataset the frames are a finite sample of π_β , so the same line holds only approximately, as an estimate — the same status as every sample average in Chapter 3. Either way the right side is a sum over the frames you have, each multiplied by its own number: at $\beta = 1$, **7.3891**/ Z for each of the hundred frames of a_3 , $1/Z$ for each frame of a_2 , **0.3679**/ Z for each frame of a_1 . Add up the weights before dividing by Z :

100 x 0.3679 =	36.79	share =	36.79 / 875.7 =	4.2%
100 x 1.0000 =	100.00	share =	100.00 / 875.7 =	11.4%
100 x 7.3891 =	738.91	share =	738.91 / 875.7 =	84.4%
total	= 875.70			

Those are the three probabilities of π^* from 4.4.3, reappearing as each action's share of the training signal. The projection lands the network on π^* not by building π^* but by letting each logged frame speak with the volume π^* would have given it. For continuous actions the sum over a becomes an integral and every π a density; the dataset average estimates that integral the same way, one weighted term per frame.

One more step, small but worth saying. Everything above was one state. Every other state in the data gets the same weighted average over its own frames, with its own $Z(s)$. Weight each state's average by how often that state appears in the logs and add them up. That weighting is not a choice: the objective is evaluated on the states the data visited — the frozen state distribution 3.2.3 confessed to, which 4.4.2 repeated — and it is the average over states that 4.4.3 mentioned. Each frame is one state-action pair and sits in exactly one per-state average, so the result is a single sum over every logged frame, divided by N . That is how the $\frac{1}{300} \sum$ at one state becomes the $\sum_i$ over the whole dataset in the key equation below; the $1/N$ is one constant for the entire sum, so it does not move the argmax. The $1/Z(s)$ is not one constant, and it is the next loose end.

That loose end first within a state. Written honestly, $\pi^*(a | s) = \pi_\beta(a | s) \exp(\hat{A}/\beta)/Z(s)$, where $Z(s)$ normalizes the distribution at state s . It depends on the state but not on the action, so within a state it scales every sample's weight by the same number: on the toy, dividing all three hundred weights by $Z = 2.919$ leaves the shares 4.2%, 11.4%, 84.4% exactly as before.

Across states Z does vary. Computing it exactly would mean averaging $e^{\hat{A}/\beta}$ over every action at every state. On the toy that average ran over three hundred frames; on a real robot a continuous state is visited once, holds one logged action, and offers no average to take. So nobody computes $Z(s)$, and every implementation drops the $1/Z$. Be clear that this is a simplification, and know which way it leans. A state whose logged advantages spread more widely, or sit higher on average, has a larger Z than a state whose actions all came out near average: at the toy's state $Z = 2.919$ at $\beta = 1$, while the calm state of 4.4.3, whose three equally logged actions scored $\hat{A} = (-0.1, 0, +0.1)$, has $Z = \frac{1}{3}(0.905 + 1.000 + 1.105) = 1.003$. Dropping the $1/Z$ therefore lets the varied state speak roughly three times louder ($2.919/1.003$, about 2.9), relative to the calm one, than the derivation said.

Every implementation accepts that, for two reasons. Standardizing $\hat{A}$ per batch puts all advantages on one scale, which keeps $Z(s)$ within a modest range across states instead of letting one state's Z run away to astronomical values. The variation that remains is a small distortion, tolerated in practice rather than proved harmless. And whatever uniform scale is left is indistinguishable from a change in the learning rate. That is why the weight in every implementation you will read is simply $\exp(\hat{A}_i/\beta)$, with no $1/Z(s)$ in sight.

One last shift, notational and deliberate. Everything in the derivation was written over states s , because that is where the MDP lives and where the advantage is defined; only the BC objective borrowed from 4.3 was already in θ . On a real robot you never see s ; you see an observation o — camera frames plus joint positions. From here on the policy is conditioned on o , and every statement about s is to be read with o substituted in. Together, the four steps plus the three housekeeping moves — sum over states, drop $1/Z$, read s as o — give weighted maximum likelihood on the data you already have. Maximum likelihood is 4.3's objective under the name step one gave it. Weighted means each frame's term is scaled by its own w_i :

$$\theta \leftarrow \arg \max_{\theta} \sum_i w_i \log \pi_{\theta}(a_i | o_i), \quad w_i = \exp(\hat{A}_i/\beta).$$

IN PLAIN WORDS

This is behavior cloning with one scalar multiplier per sample. θ is the network's weights, and $\arg \max_{\theta}$ says: find the weights that make the sum as large as possible. The sum runs over every logged frame i . $\log \pi_{\theta}(a_i | o_i)$ is the ordinary BC term: how likely the network thinks the recorded action a_i was, given the recorded observation o_i . w_i is a positive number attached to that sample and nothing else: the ratio π^*/π_{β} with Z absorbed, 0.37, 1.00, or 7.39 on the toy. $\hat{A}_i$ is that sample's estimated advantage, computed before training. β is the temperature: dividing by it before exponentiating sets how far the weights spread. As $\beta \rightarrow \infty$ every weight goes to 1: plain BC. As $\beta \rightarrow 0$ the largest advantage takes over: clone the winner.

This is **advantage-weighted regression** (AWR; Peng et al., 2019, generalizing reward-weighted regression, Peters & Schaal, 2007). Two housekeeping notes. First, the letter β does double duty in the standard notation: it is the temperature here, and it is also the subscript in π_{β} for the behavior policy. They are unrelated; everyone finds this mildly annoying and nobody has renamed them. Second, $\hat{A}_i$ comes from tools you already own from Chapter 3: a Monte Carlo return minus a learned value baseline, $\hat{A}_t = G_t - V(s_t)$, or GAE with parameters γ and λ (GAE's λ , not the Lagrange multiplier of 4.4.3). Nothing new is required.

WORDS YOU WILL HEAR

Lagrangian, Lagrange multiplier. A constrained problem rewritten with a price per unit of constraint used; the multiplier is that price. **Closed form.** An explicit formula for the solution, no iteration needed. **Nonparametric.** A distribution written out directly, not the weights of a network. **Forward vs. reverse KL.** Forward KL, $D_{\text{KL}}(\text{target} \parallel \text{model})$: what maximum likelihood minimizes; the model must cover everything the target does — which is why the single Gaussian of 5.2.3 puts its mean in the mug, between two swings. Reverse KL, $D_{\text{KL}}(\text{model} \parallel \text{target})$: content to sit on one peak and ignore the other. **Normalizing constant, partition function Z .** Whatever you divide by so the probabilities sum to one. **Change of measure, importance weight.** An average under one distribution rewritten as an average under another; the ratio each sample carries is the importance weight. **Exponential tilt, Boltzmann or Gibbs distribution.** A base distribution multiplied by $e^{(\cdot)/\beta}$ and renormalized; β is the temperature. **Effective sample size.** $(\sum_i w_i)^2 / \sum_i w_i^2$: how many equally weighted samples your weighted set is worth.

Figure 4.2 is the whole algorithm as one picture, and Figure 4.3 shows what the temperature actually does to a dataset.

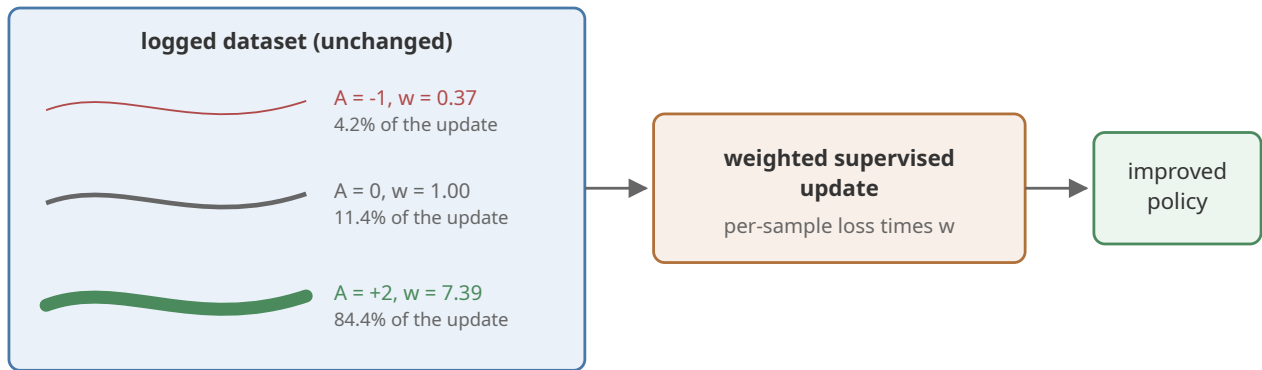


Figure 4.2 — AWR as a picture. Notice that the box on the left never changes: no episode is deleted and no new action is invented. Line thickness is the weight $w = e^{\hat{A}/\beta}$ at $\beta = 1$, and the percentages are each trajectory's share of the total weight. The failure still contributes a whisper of gradient, which is what keeps the policy anchored to data the robot can actually execute. One advantage per trajectory keeps the picture readable; in practice each frame carries its own $\hat{A}_t$.

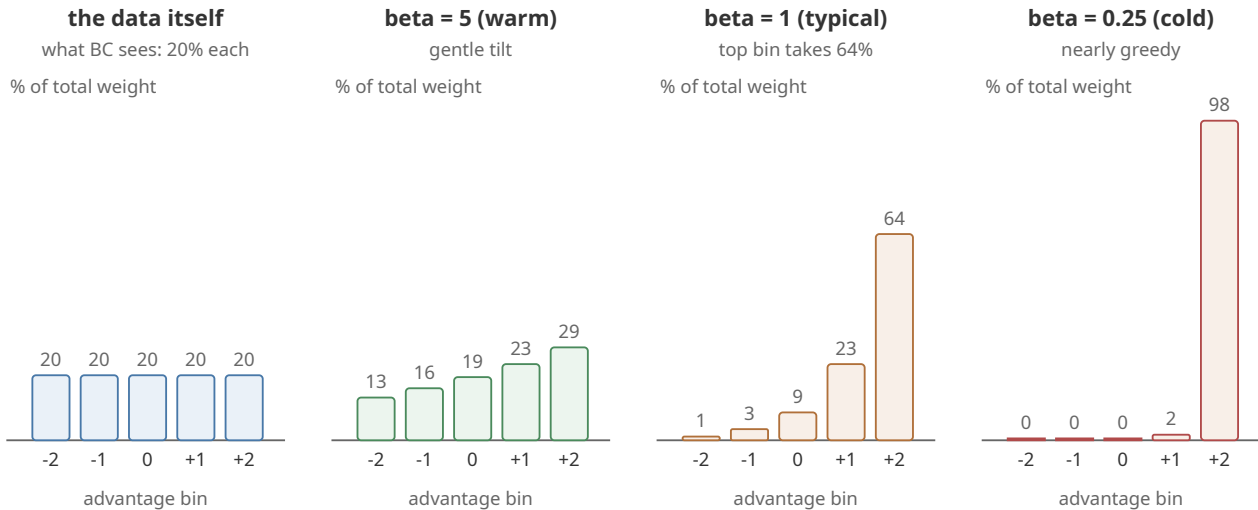


Figure 4.3 — The exponential tilt at three temperatures. The leftmost panel is a dataset with equal numbers of samples in five advantage bins; each later panel shows the same samples' share of the total weight after multiplying by $e^{\hat{A}/\beta}$, in percent. Notice that lowering β does not delete anything — it starves the lower bins. At $\beta = 0.25$ the top bin carries 98% of the update, so four fifths of your data has effectively stopped existing and you are one bad advantage estimate away from cloning noise.

4.4.5 Worked examples: making the temperature tangible

You have met the three numbers in this subsection already: $-1, 0, +2$ were the toy's advantages, and 0.37, 1.00, 7.39 their tilt factors at $\beta = 1$. What follows is the same arithmetic read from the training loop's side, with one change of scale: each advantage now belongs to a whole trajectory instead of one action at one state.

One clarification, because it decides what you build. The key equation weights per *sample* $i = (o_i, a_i)$ — that is, per frame. The examples below use one advantage per trajectory purely to keep the numbers checkable by hand. In a real implementation every frame carries its own $\hat{A}_t$ from GAE, and a trajectory-level weight is the special case where every frame in an episode is handed the episode's advantage. LeHome is per frame: frame i is drawn with probability proportional to $\exp(\text{clip}(\hat{A}_i, -2, 2))$.

Three logged trajectories with estimated advantages $\hat{A}_1 = -1, \hat{A}_2 = 0, \hat{A}_3 = +2$. Take $\beta = 1$ first, so the weight is just $e^{\hat{A}}$:

$$w_1 = e^{-1} \approx 0.37, \quad w_2 = e^0 = 1.00, \quad w_3 = e^2 \approx 7.39.$$

Now normalize, because what matters is each sample's *share* of the gradient, not its raw weight. Sum first, then divide, one line at a time:

```

sum    = 0.37 + 1.00 + 7.39 = 8.76
share1 = 0.37 / 8.76 = 0.042  -> 4.2%
share2 = 1.00 / 8.76 = 0.114  -> 11.4%
share3 = 7.39 / 8.76 = 0.844  -> 84.4%
check:  4.2 + 11.4 + 84.4 = 100.0%
```

Read that back into behavior: the update acts as if trajectory 3 had been duplicated about seven times, while trajectory 1 whispers at a third of its former volume. Nothing was deleted, and that residue is useful — it keeps the policy pinned to the data's support. These shares are the probabilities of π^* from 4.4.3 to the decimal, exactly as 4.4.4 promised.

Now the same three advantages with $\beta = 5$. Divide each advantage by 5 *before* exponentiating:

```

w1 = exp(-1/5) = exp(-0.2) = 0.819
w2 = exp( 0/5) = exp( 0.0) = 1.000
w3 = exp( 2/5) = exp( 0.4) = 1.492

sum      = 0.819 + 1.000 + 1.492 = 3.311
share1   = 0.819 / 3.311 = 0.247  -> 24.7%
share2   = 1.000 / 3.311 = 0.302  -> 30.2%
share3   = 1.492 / 3.311 = 0.451  -> 45.1%

```

Same data, same advantages, completely different training run. At $\beta = 1$ the best trajectory outweighs the worst by a factor of $7.39/0.37 \approx 20$; at $\beta = 5$ that ratio is $1.492/0.819 \approx 1.8$. The failure went from 4.2% of the update to nearly a quarter of it. In the language of 4.4.3, you tightened the leash from about 0.57 nats to about 0.03 by moving one knob. If your advantages are noisy — and on a real robot with a learned value function they are — the larger β is the conservative choice, at the price of learning more slowly.

Finally, watch the thing that will actually bite you in practice. Suppose a fourth trajectory got lucky and its estimated advantage came out at $\hat{A}_4 = +8$. At $\beta = 1$:

$$w_4 = e^8 \approx 2981, \quad \text{sum} = 0.37 + 1.00 + 7.39 + 2981 = 2989.76,$$

$$\text{share}_4 = \frac{2981}{2989.76} = 0.9971 \rightarrow 99.71\%, \quad \text{share}_3 = \frac{7.39}{2989.76} = 0.0025 \rightarrow 0.25\%.$$

Trajectory 3, which was carrying 84.4% of the update a moment ago, now carries a quarter of one percent. One sample — possibly one *lucky* sample — has become the entire training set. This is not a rare pathology; it is what exponentials do, and it is why the two defenses in the pitfall box are not optional.

Note the difference from the $+60^\circ$ action of 4.4.3: that one was never logged, so no advantage could give it weight. This trajectory *was* logged, so the tilt is entitled to amplify it, and only the scale you allow $\hat{A}$ stands between a noisy estimate and a collapsed run.

You met the same instinct in Chapter 3. PPO's ratio ρ_t is also unbounded, and Section 3.4 tamed it by clipping to $[1 - \epsilon, 1 + \epsilon]$. The clip at 20 in the code below is blunter: PPO's clip removes the incentive to move further (3.4.4 called it an incentive, not a constraint), while this one caps the weight outright.

```

# AWR: one training step, any supervised policy loss
batch = replay.sample()                # (o, a, A_hat) triples
A = (batch.A - A_mean) / (A_std + 1e-8) # normalize advantage scale
w = clip(exp(A / beta), max=20.0)       # tilt + clip
loss = (w * policy_loss(theta, batch.o, batch.a)).mean()
# policy_loss: -log pi_theta(a|o) for a Gaussian head, MSE for a
#               deterministic one, a flow-matching regression loss
#               for a generative one (Chapters 6 to 8)
theta = theta - lr * grad(loss)

```

ON YOUR RIG

Appendix B, rung E4, is this section on your own rl-rover logs: roll out a mediocre early checkpoint for about 500 episodes, then train one policy by plain regression onto its actions and one with $w_i = \exp(\hat{A}_i/\beta)$, $\hat{A}_i = G_i - V(s_i)$ from the checkpoint's own critic. Normalize $\hat{A}$ and clip w at 20. Then push β toward zero and watch the effective sample size collapse — dead data appearing on data you collected. Keep the weight histogram and be able to say what β does to its shape, as the rung asks. You cannot compute the leash radius of 4.4.3 here, because a continuous state holds one logged action rather than three hundred, but the histogram narrowing to a spike is the same leash lengthening, read from the weights instead.

PITFALLS

The exponential is unforgiving. Unnormalized advantages (scale 50 instead of 1) make every weight either 0 or an overflow — always standardize $\hat{A}$ per batch or per dataset before `exp`, and clip w , for example $w_i \leftarrow \min(w_i, 20)$. Too small a β creates *dead data*: the effective sample size collapses to a handful of trajectories, the policy overfits them, and it drifts off-support anyway. A cheap diagnostic is to log the effective sample size $(\sum_i w_i)^2 / \sum_i w_i^2$ each epoch. If it is a small fraction of your batch, or if most weights sit near zero, raise β : you are closer to greedy than you think. On the three-trajectory example above it is 1.38 at $\beta = 1$ and 2.81 at $\beta = 5$, out of three.

4.5 AWAC in one section

AWR as stated has a practical weakness: where does $\hat{A}_i$ come from? The plain answer is a Monte Carlo return minus a value baseline, which needs complete episodes and describes the *behavior* policy's performance from that state onward. That is noisy, it wastes partial trajectories, and it does not improve as your policy does.

AWAC (advantage-weighted actor-critic; Nair et al., 2020) keeps the weighted-regression policy update exactly as it is and changes only where the advantage comes from. It trains an action-value critic $Q(s, a)$ with off-policy temporal-difference backups over the whole buffer, and sets

$$\hat{A}(s, a) = Q(s, a) - \mathbb{E}_{a' \sim \pi_\theta} [Q(s, a')],$$

Name the pieces. a is the action actually recorded in the dataset at state s . a' is a *candidate* action — not read from the dataset, but drawn fresh from the current policy π_θ . Averaging $Q(s, a')$ over a handful of such draws is precisely an estimate of $V^{\pi_\theta}(s)$. So this is the same $A = Q - V$ you know from Chapter 3, with the baseline computed on the fly by sampling rather than by a second network. Note what it requires: the ability to *sample* from π_θ and to evaluate Q at those samples. It does not require the probability of any action. Keep that in mind for the next section.

If 4.2 stayed with you, that should worry you: querying a critic at actions the policy chose for itself is exactly the pattern that inflated the value surface and sent the wrist to $+60^\circ$. AWAC survives it because of what the actor update does. The actor is trained by weighted regression onto *logged* actions, which pins π_θ near π_β ; a policy pinned to the data's support draws its a' from inside that support, so the Q queries are ones the data can answer. But be clear about the status of that argument — the safety is conditional, not structural. Set β low enough, or fine-tune long enough with a stale critic, and AWAC drifts off support like anything else. Plain AWR cannot, because it never asks a critic anything.

Because both the critic and the actor update are fully off-policy, AWAC is built for the workflow real robots want: *pretrain offline* on demonstrations and logged data, then *fine-tune online*, still reusing everything in the replay buffer. Its selling point is the absence of the performance crash many methods suffer at the offline-to-online handoff: either their constraint is too rigid for the policy to move past the demonstrations once fresh data arrives, or their critic must effectively be relearned when the distribution shifts, and the policy degrades while it does.

ON YOUR RIG

Your HIL-SERL pipeline answers the same data-efficiency problem a different way: it is RLPD-family — off-policy Q-learning in the style of SAC (soft actor-critic: a Q-function plus an entropy-regularized Gaussian actor) over a replay buffer that mixes demonstrations, your pad interventions, and autonomous rollouts. That works because its small Gaussian action head has exact log-probabilities and cheap critic queries. AWR trades some of that per-sample efficiency for the one thing Q-learning-through-the-actor cannot offer: total indifference to the policy's architecture. Same problem, two off-policy answers — and Chapter 13 comes back to why the intervention data in that buffer deserves extra weight.

4.6 The property that matters

4.6.1 The shopping list

Now the hinge of this book. Look carefully at what the AWR update actually consumes:

- samples $(\mathbf{o}_i, \mathbf{a}_i)$ from a dataset;
- a supervised loss that scores how well the policy reproduces $\mathbf{a}_i$ from $\mathbf{o}_i$;
- one scalar weight w_i per sample.

That is the complete list. And here is what it never does: it never evaluates $\log \pi_\theta$ at an arbitrary action of its own choosing, and it never forms an importance ratio $\rho_t(\theta)$. Neither is a lucky accident — the whole point of the projection step in 4.4.4 was to turn every expectation into a sum over samples you already have.

The consequence is large. The $\log \pi_\theta(\mathbf{a}_i|\mathbf{o}_i)$ term in the key equation can be replaced by *whatever surrogate loss you already use to train this policy in a supervised way*: mean-squared error on actions for a deterministic head, a diffusion denoising loss (Chapter 6), a flow-matching velocity-regression loss (Chapter 7).

That substitution deserves better than a wave of the hand, because four chapters lean on it. For diffusion the case is solid: the denoising loss is a variational lower bound on the log-likelihood, so minimizing it is a principled stand-in for maximizing $\log \pi_\theta$, and weighting it by w_i weights a bound on the quantity you meant to weight. For flow matching the link is looser — the velocity-regression loss is not a likelihood bound in the same clean way, and the swap is a well-tested heuristic rather than a theorem. Be plain about the cost. Once you substitute, the weighted objective is no longer exactly the KL projection of π^* that 4.4.4 derived, and nobody can hand you a proof that it is. What is true, and what the empirical record supports, is that reweighting a regression loss toward high-advantage samples moves the fitted distribution the way you want. You are trading a guarantee for a policy class that can represent multimodal robot behavior at all. That trade is the premise of Part III, and it is worth making with your eyes open.

4.6.2 Why PPO cannot follow

PPO cannot make this move, and the reason is structural. Its objective is built from

$$\rho_t(\theta) = \frac{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}{\pi_{\theta_{old}}(\mathbf{a}_t|\mathbf{s}_t)},$$

which requires exact action likelihoods under two different networks. For a Gaussian head that is one line of code. Chapters 5 through 8 introduce policies for which it is not available at all. Diffusion and flow-matching policies produce an action by running an iterative sampling procedure, and expose no tractable $\pi_\theta(\mathbf{a}|\mathbf{o})$ at the end of it. Vision-language-action models (VLAs: a pretrained vision-language backbone with an action head bolted on, the subject of Chapter 8) are built on those samplers, so they inherit the problem. Action chunking is a separate matter — emitting $\mathbf{a}_{t:t+H}$ in one shot is not an iterative procedure and is not by itself what destroys the density. It compounds the difficulty from the other side: the object whose probability you would need is then a thirty-step trajectory rather than a single command. You can sample from any of these policies all day. You cannot ask them how likely a particular action was. PPO's central quantity is simply unavailable.

Those same policies are, however, *trained* by supervised regression. That is the whole reason they exist and scale. So here is the rule worth memorizing:

Any policy class you can train with supervised learning, you can improve with AWR. Weight the per-sample loss and you are done.

This is why weighted behavior cloning, not PPO, is what scales up to VLAs, and why the rest of this book looks the way it does. Chapter 9 lays the two worlds side by side; Chapter 10 shows how RECAP turns the advantage from a weight into a *conditioning input*, a further step down the same road. Figure 4.4 is the decision table.

	exact log-prob available	no tractable log-prob
data from this policy version only	PPO / TRPO — Chapter 3 Gaussian head, ratio is computable. Your rl-rover in simulation. Cheap samples assumed.	AWR anyway — Chapter 4 Fresh data does not restore a ratio. Score your own rollouts, reweight. PPO is unavailable even here.
data from a mixed pool of past versions	AWAC, RLPD / SAC — Ch. 4 Critic over the whole buffer. Your hil-serl SO-101 rig. Offline pretrain, online finetune.	AWR, then conditioning — Ch. 10 Weight the denoising or flow loss. Diffusion policies, VLAs; Ch. 6 to 8. RECAP; the LeHome flywheel.

the right-hand column is the whole of Part III

Figure 4.4 — Which family do you need? Two questions decide it: was this data collected by the policy version you are updating, or drawn from a pool of past versions; and can you evaluate the probability of a given action under your policy. Notice that the second question, not the first, is what separates PPO from everything else in this book — the top-right cell exists precisely because collecting fresh rollouts does not hand a flow-matching policy a usable importance ratio. LeHome sits bottom right: its pool holds roughly 12,500 rollout episodes from many checkpoints, decayed and resampled, which is off-policy by the definition in 4.1.

4.6.3 The same weight, applied at the sampler

One implementation note that seems trivial and is not. Multiplying the per-sample loss by w_i is the same in expectation as leaving the loss unweighted and instead *drawing* sample i with probability proportional to w_i . Both put the same emphasis on the same frames — but only up to one overall constant, the mean weight $\frac{1}{N} \sum_i w_i$, because loss weighting averages $w_i L_i$ over N uniform draws while sampling averages L_i over draws from $p_i = w_i / \sum_j w_j$. That constant rescales the gradient and is absorbed by the learning rate. Worth knowing on the day you switch an existing run from one form to the other and your effective step size quietly changes.

On a small MLP the difference does not matter. On a VLA it matters enormously: a frame you never draw is a frame you never decode, never move to the GPU, and never run a vision encoder on. With loss weighting you pay full price for a sample whose weight is 0.01; with sampling you skip it. That is the version Larchenko used.

```
# same update, weight moved into the sampler
p = softmax(clip(A_hat, -2.0, 2.0) / beta) # sampling law over frames
idx = multinomial(p, num_samples=batch_size) # low-weight frames never load
loss = policy_loss(theta, o[idx], a[idx]).mean() # plain, unweighted
# auxiliary heads (extra prediction outputs on the same backbone,
# Chapter 8) want the *original* data distribution, not the tilted one,
# so debias them with 1 / (N * p_i); the full form also divides by
# the episode length
```

IN THE LEHOME SOLUTION

AWR is one of the two pillars of the garment-folding solution (Larchenko, 2026); the other is RECAP (Chapter 10). The policy is derived from $\pi_{0.5}$ (a flow-matching VLA from Physical Intelligence, 2025a, dissected in Chapter 8), so it has no usable log-probabilities and improvement is exactly Section 4.6. He applies the weight at the sampler: frame i is drawn with probability proportional to $e^{\text{clip}(\hat{A}_i, -2, 2)}$, and the flow-matching loss itself is plain unweighted MSE. Failed episodes have their accumulated checkpoint rewards withdrawn (Chapter 11), pushing their advantages, and so their sampling probability, down. Advantages come from the policy's own success-probability head with a damped baseline, also Chapter 11.

CHECKPOINT

- Price out an on-policy run on real hardware in hours of human supervision, and explain why the data is discarded after the update.
- Explain, with a concrete action the dataset never contains, why naive offline Q-learning inflates its own value estimates — and why bootstrapping spreads the error.
- State behavior cloning's objective and its exact ceiling, and give an example of mixed-quality data where that ceiling bites.
- Derive AWR's shape from the KL-constrained problem: the closed-form exponential tilt, then the forward-KL projection onto the network as weighted maximum likelihood — including where the entropy term and $Z(\mathbf{s})$ went, and why the tilt can never reach an action the data does not contain, which is the source of both AWR's offline safety and its inability to explore — and why the tilt is an exponential at all: what a squared-distance leash would have produced on the toy instead.
- Compute weights and normalized shares from a list of advantages at two different temperatures, and say what $\beta \rightarrow 0$ and $\beta \rightarrow \infty$ each mean, and read a temperature as a price: name the leash radius $\beta = 1$ and $\beta = 5$ each buy on the toy of 4.4.2.
- State the rule: any policy class you can train with supervised learning, you can improve with AWR — explain why PPO's importance ratio blocks the same move, and say honestly what swapping a denoising or flow-matching loss in for $\log \pi_\theta$ costs in rigor.

Where to go deeper. The AWR paper (Peng et al., 2019) is short and readable, and its appendix contains, in compressed form, the derivation 4.4.3 unpacked at length; its ancestor is reward-weighted regression (Peters & Schaal, 2007). For the offline-pretrain-then-online-fine-tune story, read AWAC (Nair et al., 2020). Appendix A places both papers in the reading order for this book. RLPD (Ball et al., 2023) — the off-policy Q-learning alternative already running on your rig — and the conservatism-based offline methods this chapter deliberately routed around are optional context beyond the guide.



PART III

Generative policies: the new world

Why a Gaussian stopped being enough, what a generative model actually is, and how diffusion, flow matching, and vision-language-action models changed what a policy is — with PPO and VLA-RL finally placed side by side.

Chapter 5 — Why one Gaussian is not a policy

Everything you have built so far shares one hidden assumption, and this chapter is about breaking it. In your rover project, in the gridworlds, in the SAC-style policy HIL-SERL runs on your SO-101 today, the policy has the same shape: a network looks at the observation and produces the *center* of a bell curve, and the action is a random draw from it. That is not a detail of the library you used. It is a claim about the world — that at any moment there is one right thing to do, and everything else is a small perturbation of it.

For a rover chasing a waypoint that claim is true. For a pair of arms folding a shirt it is false in a way that destroys the policy rather than degrading it. This chapter's job is to make you feel that failure, then introduce — from zero — the class of models that fixes it: **generative models**, whose job is to produce samples from a complicated distribution rather than predict one number.

You will not learn the mechanics of diffusion or flow matching here; those are Chapters 6 and 7. What you get is the problem they solve, the vocabulary they are written in, and the one intuition — iterative refinement — both are built on. Everyone finds this confusing at first, so take it slowly.

5.1 What your policy has been until now

5.1.1 The shape of a Gaussian policy

Open the Stable-Baselines3 policy you trained on the rover. An MLP maps the observation $\mathbf{o}$ to a vector $\boldsymbol{\mu}_{\theta}(\mathbf{o})$, one entry per action dimension. Alongside it sits a much smaller parameter: a per-dimension standard deviation $\boldsymbol{\sigma}$, stored as $\log \boldsymbol{\sigma}$ and, by default, not a function of the observation. To act you draw a vector of independent standard normal numbers $\mathbf{z}$, one per action dimension, and return $\mathbf{a} = \boldsymbol{\mu}_{\theta}(\mathbf{o}) + \boldsymbol{\sigma} \odot \mathbf{z}$. The circle-dot $\odot$ means multiply entry by entry: one σ_j scaling one z_j , never mixing dimensions.

That is a **Gaussian policy**: $\pi_{\theta}(\mathbf{a} | \mathbf{o}) = \mathcal{N}(\boldsymbol{\mu}_{\theta}(\mathbf{o}), \boldsymbol{\Sigma})$, with $\boldsymbol{\Sigma}$ diagonal, built from the σ_j . PPO can use it because its log-density has a closed form you write on one line and differentiate for free.

Read $\boldsymbol{\Sigma}$ as the covariance matrix, and read *diagonal* as a real restriction rather than an implementation convenience. Diagonal means the policy adds independent noise to each action dimension. It has no way to say that joint 1 going high should come with joint 2 going low, because it draws every z_j on its own with no reference to the others. Remember that; it comes back in Section 5.4, where it is the reason one of the obvious repairs fails.

$$\log \pi_{\theta}(\mathbf{a} | \mathbf{o}) = \sum_{j=1}^d \left[-\frac{(a_j - \mu_{\theta,j}(\mathbf{o}))^2}{2\sigma_j^2} - \log \sigma_j - \frac{1}{2} \log 2\pi \right]$$

IN PLAIN WORDS

The left side is one number: how plausible this exact action $\mathbf{a}$ was under the current policy, on a log scale. On the right, d is the number of action dimensions and j indexes them, so the sum walks the action vector one entry at a time. a_j is the j -th number you sent to the robot; $\mu_{\theta,j}(\mathbf{o})$ is the network's preferred value for it given $\mathbf{o}$; σ_j is how far from that preference the policy will wander. The first term asks "how many σ -widths off-center did we land, squared" — further out, lower log-probability. The $-\log \sigma_j$ term is the price of vagueness, stopping the policy from shrinking σ to nothing for free. The last is a constant that cancels in every ratio.

5.1.2 What that looks like geometrically

Picture the space of possible actions as a plane, one axis per action dimension. A Gaussian puts down a single hill of probability with exactly one peak, at $\boldsymbol{\mu}_{\theta}(\mathbf{o})$; move away in any direction and the probability falls off smoothly and never comes back up. So it is **unimodal** — one most-likely value and no others — and the region it considers

plausible is one connected blob. There is no way to say "this action is good, that faraway action is also good, and everything between is terrible." A Gaussian cannot express a hole.

5.1.3 Why it was exactly right for the rover

None of this hurt you on rl-rover. At a given rover state — some heading error, some distance to the waypoint — there is essentially one sensible pair of wheel commands, and steering a little more or less is a graded change in quality. There is no second, equally good command far from the first. PPO's job was to slide $\mu_\theta(o)$ toward the right value and let σ shrink as confidence grew; the noise was never a claim about the world, it was your exploration mechanism.

The Gaussian also gave you three services free, all used constantly in Chapters 2 and 3: cheap *sampling*, cheap differentiable *evaluation* of $\log \pi_\theta(a | o)$, and reparametrized gradients — the trick of writing $a = \mu_\theta(o) + \sigma \odot z$ with all the randomness pushed into z , so a gradient can flow through the sampled action back into μ_θ and σ as if nothing random had happened. You already wrote that third one two paragraphs ago; it is worth knowing it has a name, because papers use the word without stopping to explain it. Hold onto the second. Section 5.7 is about losing it.

5.2 Where it breaks: multimodality

5.2.1 What we want, and the obvious thing to try

Now change the setting. You have a pile of teleoperated demonstrations and want a policy that reproduces them. Call the demonstrator's policy π_β — the distribution over actions a human actually produced at each observation. The obvious thing to try is **behavior cloning**: keep the Gaussian head, train $\mu_\theta(o)$ by minimizing squared error to the demonstrated action. With σ fixed that is exactly maximum-likelihood training of the Gaussian — the principled fit, not a hack.

That equivalence is load-bearing, so look back at the log-density a few paragraphs up and check it. With every σ_j held fixed, the only term on the right-hand side that depends on μ at all is $-(a_j - \mu_{\theta,j}(o))^2 / (2\sigma_j^2)$; the $-\log \sigma_j$ and the constant do not move when μ moves. Making the data as likely as possible under the Gaussian and making the squared error as small as possible are therefore the same optimization, term for term, differing only by a positive scale factor and a sign. **Maximum likelihood**, if the phrase is new to you, just means choosing the parameters that assign the highest probability to the data you actually collected. So the squared-error loss you have been using out of habit is not an arbitrary choice of distance — it is a distributional claim, and the rest of this section is about what that claim costs.

5.2.2 Exactly how and why it fails

Put a mug on the table, handle pointing away from the robot, and have a human teleoperate a grasp ten times. On roughly half the attempts the operator swings the gripper around the left of the mug; on the other half, around the right. Both work, and the operator is indifferent. No feature in the camera image predicts the choice, because the choice does not depend on the image.

So the demonstrated distribution $\pi_\beta(a | o)$ here has two **modes** — two separate peaks, with a valley between them where almost no demonstration lives. Fit a unimodal Gaussian to it by maximum likelihood and here is what you get.

$$\mu^*(o) = \arg \min_{\mu} \mathbb{E}_{a \sim \pi_\beta(\cdot | o)} [\|a - \mu\|^2] = \mathbb{E}_{a \sim \pi_\beta(\cdot | o)} [a]$$

IN PLAIN WORDS

Read it right to left. $\pi_\beta(\cdot \mid \mathbf{o})$ is the distribution of actions the human produced at this observation $\mathbf{o}$, and $\mathbf{a}$ is one such action, so $\mathbb{E}_{\mathbf{a} \sim \pi_\beta}[\mathbf{a}]$ is the plain average of all demonstrated actions there. The middle expression is what behavior cloning minimizes: the expected squared distance $\|\mathbf{a} - \boldsymbol{\mu}\|^2$ between a demonstrated action and the mean $\boldsymbol{\mu}$ your network outputs. $\boldsymbol{\mu}^*(\mathbf{o})$ is the $\boldsymbol{\mu}$ making that as small as possible. The equation says these are the same thing: under squared error, the best mean is the average of the data. That is the exact optimum, not poor optimization — more data only makes your network hit it more precisely.

5.2.3 Worked example: the mean lands in the mug

Now with real numbers. Reduce the grasp to a scalar: the approach angle $\mathbf{a}$, normalized so $\mathbf{a} = -1$ is "swing left", $\mathbf{a} = +1$ is "swing right", $\mathbf{a} = 0$ is "straight at the mug". Six demonstrations, three of each, with a little human variation.

demonstrated actions at observation $\mathbf{o}$:

left swings:	-1.2	-1.0	-0.8
right swings:	+0.8	+1.0	+1.2

The maximum-likelihood Gaussian has mean equal to the sample average:

$$\boldsymbol{\mu}^* = \frac{(-1.2) + (-1.0) + (-0.8) + (0.8) + (1.0) + (1.2)}{6} = \frac{0.0}{6} = 0.0$$

Now let the Gaussian fit its width too, rather than holding $\boldsymbol{\sigma}$ fixed the way the behavior-cloning setup above did. Both fits give the same $\boldsymbol{\mu}^*$, so nothing you just computed changes; you are simply asking for one more number. The maximum-likelihood $\boldsymbol{\sigma}^2$ is the average squared deviation from that mean:

$$\begin{aligned}\sigma^2 &= \frac{1.44 + 1.00 + 0.64 + 0.64 + 1.00 + 1.44}{6} = \frac{6.16}{6} = 1.0267 \\ \sigma &= \sqrt{1.0267} = 1.0132\end{aligned}$$

So the fitted policy is $\mathcal{N}(0.0, 1.0132^2)$. Its most likely action — what you get with exploration noise off, as at deployment — is $\mathbf{a} = 0.0$: straight into the mug, and the action *furthest* from every demonstration you collected, the nearest being **0.8** away.

Now ask how often the noisy policy is safe. Say the mug body occupies the band $|\mathbf{a}| < 0.6$, so any sample inside collides. Measure the edge of that band in standard deviations from the mean:

$$\frac{0.6 - 0.0}{1.0132} = 0.5922$$

Write Φ for the standard normal **cumulative distribution function** — the probability that a standard normal draw lands below a given point. Any normal table gives $\Phi(0.5922) \approx 0.7231$, and so does `scipy.stats.norm.cdf(0.5922)`. The mug band is symmetric about the mean, so the mass inside it is the mass below the upper edge minus the mass below the lower edge:

$$P(|\mathbf{a}| < 0.6) = \Phi(0.5922) - \Phi(-0.5922)$$

You only looked up one of those two numbers, and you do not need the other: a standard normal is symmetric about zero, so $\Phi(-x) = 1 - \Phi(x)$ for any x . The second term is therefore $1 - 0.7231$, which leaves

$$P(|\mathbf{a}| < 0.6) = 0.7231 - (1 - 0.7231) = 2 \times 0.7231 - 1 = 0.4462$$

About **45% of sampled actions crash into the mug**, and the mode of the policy is the worst action available — while the training loss sits at its global minimum. That is the part that catches people: the fit is perfect and the policy is unusable.

5.2.4 The same failure with two arms

The mug is a toy; bimanual manipulation makes this the default condition. When two arms fold a shirt, sometimes the left reaches across and leads while the right stabilizes the hem, and sometimes it is the reverse. Both are fine and both appear in the data. The average is "both arms half-leading", so both reach for the center of the table at once and collide over the garment.

Say the general form out loud, because the chapter turns on it: **mode averaging does not produce a mediocre action, it produces an action that appears in no demonstration at all.** A mediocre action would be recoverable. This one is off the **data manifold** — away from the thin region of action space where your demonstrations actually live — and off the manifold you have collected no evidence at all about what happens next, so the network's output there is extrapolation dressed up as a prediction (Figure 5.1).

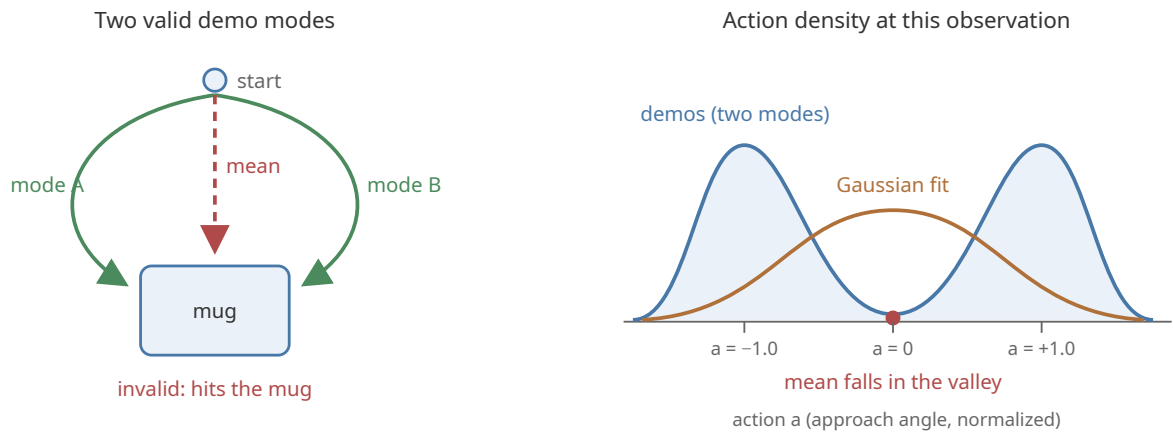


Figure 5.1 — Mode averaging, seen two ways. On the left, two demonstrated grasp trajectories go around opposite sides of the mug while the Gaussian mean (dashed red) drives straight into it. On the right, the same failure as densities: blue is the demonstrated distribution with its two peaks, orange the best-fitting single Gaussian, its mean at $a = 0$ where the data has no mass. Notice the fit is not sloppy — it is optimal, and that is the problem.

PITFALLS

Three things that look like fixes and are not. *Widening σ* does not help: a wider bell still peaks in the valley. *More demonstrations* makes it worse — more data means a more accurate mean, and the mean is the broken part. *Watching the loss* tells you nothing, because the squared-error minimizer is the conditional mean; the curve looks textbook while the robot fails. The diagnostic that works: plot the demonstrated actions at a fixed observation and count the clusters.

5.3 Action chunks, and why they make this worse

5.3.1 What a chunk is and why it exists

Modern imitation policies do not emit one action at a time. They emit an **action chunk** $a_{t:t+H}$ — the next H actions produced as a single output, then executed for some number of steps before the policy is queried again (Zhao et al., 2023). The π -family uses $H = 50$ at 50 Hz, one second of motion per query; the LeHome fine-tune uses $H = 30$ at 30 Hz, also one second. How many of those H actions you execute before re-querying is a separate knob, and it belongs to Chapter 12.

There are three reasons chunking won, and each is worth stating separately. There is also one cost, which the three benefits tend to hide, so it gets a paragraph of its own at the end.

Temporal consistency. A policy re-sampled fifty times a second can flip between mode A and mode B mid-reach: left, left, right, left. On a physical arm that dithering looks like a seizure. A chunked policy commits to one coherent motion.

Latency amortization. A 3-billion-parameter model does not run in the 33 ms budget of a 30 Hz loop. Suppose one query takes 100 ms. Querying every step is impossible. Query once and then execute all 30 actions and you spend 100 ms of compute per 1000 ms of motion — a 10% duty cycle, which one GPU holds comfortably. Execute fewer and the duty cycle rises in proportion: run 10 of the 30 before re-querying and you are querying three times as often, for 30% of the wall clock.

Fewer places for error to compound. Behavior cloning drifts because each small error moves you to a slightly unfamiliar state, where you make a larger error. Chunking cuts decision points per second by however many actions you execute per query — a factor of H if you run the whole chunk, less if you re-query early — and so cuts the chances for that loop to start.

What it costs. None of this is free, and the bill is paid in reactivity. Committing to a stretch of motion means not looking at the world for the duration of that stretch. If the grasp slips at step 4 of a 30-step commitment, or a human nudges the shirt, the policy plays out the remaining 26 actions anyway, because it has nothing newer to condition on. That is the open-loop cost, and it scales with exactly the same number that buys you the three benefits above. It is why real systems execute only part of each chunk before re-querying with a fresh observation — LeHome executes 3 to 5 of its 30 — and it is why Chapter 12 exists: choosing that execution length is choosing where to sit between smooth committed motion and fast reaction.

5.3.2 The arithmetic, and why it raises the stakes

A second, shorter worked example. The bimanual SO-101 has two 6-DOF arms, so $d = 12$ joint commands per timestep, with $H = 30$.

$$\text{chunk dimension} = H \times d = 30 \times 12 = 360$$

One decision is a single point in a 360-dimensional space. Now count modes. Suppose one second of folding contains three moments where two choices are equally good: which arm leads, which sleeve goes first, and whether the grasp is a pinch or a scoop. If the choices are independent, the number of distinct valid chunks is

$$2 \times 2 \times 2 = 2^3 = 8$$

Eight widely separated peaks in a 360-dimensional space, and the Gaussian's answer is their average, a point near none of them. Single-step multimodality gave you a bad action; chunk-level multimodality gives you a bad *second*.

File away one more consequence, because Section 5.7 and Chapter 9 both come back to it: inside a chunk there is no per-timestep action distribution. The model produced 360 numbers in one shot, so there is no $\pi_\theta(a_t | s_t)$ for the seventeenth action — it was never separately chosen.

5.4 What a generative model actually is

5.4.1 The obvious repairs, and where they stop

Try the two repairs any engineer thinks of first.

Repair one: a mixture of Gaussians. Output M means, widths, and mixing weights; sample a component, then a value. This genuinely fixes the mug, but you must pick M in advance, and in 360 dimensions where the mode count varies by observation, any fixed M is wrong somewhere.

Repair two: discretize. Chop each action dimension into bins and predict a categorical distribution per dimension — a classification problem, which you know how to train. Per dimension this is beautifully multimodal: joint 1 can put half its mass on "left" and half on "right". But the dimensions are predicted independently, so the model's distribution is their product, and a product that says half-left/half-right for both joints also puts a quarter of its mass on "joint 1 left, joint 2 right" — a chimera that appears in no demonstration.

That failure is instructive: what is missing is *coupling*. Once you commit to "left", every other number in the chunk must follow suit. Hold that thought — Sections 5.5 and 5.6 buy coupling in two different ways.

Chapter 6 opens by taking these same two repairs apart in more detail and adding a third, going autoregressive one dimension at a time. That repetition is deliberate. Here the repairs exist to isolate the word *coupling*, which is what Section 5.6 is about; there they exist to motivate diffusion specifically. If the arguments feel familiar when you meet them again in twenty pages, you have not missed a distinction — you are meant to already agree with them.

5.4.2 The definition

A **generative model** is a model whose output is a *sample* from a complicated distribution, produced by transforming a simple random draw. You already know how to build a network mapping an input to an output; a generative model also takes a random input — noise — arranged so that as the noise varies over its simple distribution, the output varies over the complicated one you want.

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{a} = T_{\theta}(\mathbf{z}, \mathbf{o}), \quad \mathbf{a} \sim \pi_{\theta}(\cdot | \mathbf{o})$$

IN PLAIN WORDS

Three steps of a recipe. First, draw $\mathbf{z}$: a vector of independent standard normal numbers, the same shape as the action chunk you want — 360 numbers for the bimanual SO-101. $\mathcal{N}(\mathbf{0}, \mathbf{I})$ means mean zero, unit variance, no correlations, the one distribution every library samples in a single line. Second, push $\mathbf{z}$ through T_{θ} , a network with parameters θ that also sees the observation $\mathbf{o}$, and call the output $\mathbf{a}$. The letter T stands for *transport*; the second framing in this section says why. Third — a definition, not a computation — the policy $\pi_{\theta}(\cdot | \mathbf{o})$ simply is whatever distribution $\mathbf{a}$ has when $\mathbf{z}$ is random. You never write a formula for π_{θ} ; you define it by saying how to draw from it. Section 5.1 went the other way round: there you wrote the density formula first and a way to sample fell out of it. Here you write the sampler first, and the density is whatever that sampler happens to produce. That is a real trade, and Section 5.7 is the bill.

You have already built one of these, in Section 5.1. The Gaussian policy draws $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and returns $\mathbf{a} = \mu_{\theta}(\mathbf{o}) + \sigma \odot \mathbf{z}$, which is exactly the recipe above with $T_{\theta}(\mathbf{z}, \mathbf{o}) = \mu_{\theta}(\mathbf{o}) + \sigma \odot \mathbf{z}$. So the difference between a Gaussian policy and a diffusion policy is not that one is generative and the other is not. Both are generative models. The Gaussian simply uses the simplest possible generator: stretch the noise ball along each axis, slide it to $\mu_{\theta}(\mathbf{o})$, stop. A ball stretched and slid is still a single connected ball, which is precisely why it cannot cover two separated blobs with a gap between them. Everything in Chapters 6 and 7 is about making T_{θ} richer than an affine map, and Section 5.6 is about why that turns out to be hard to do in one step.

5.4.3 Two ways to hold the idea in your head

Framing one: sampler versus density. There are two different things you might want from a probability model: to *draw* from it — give me a plausible action — or to *evaluate* it — here is an action, how likely was it? The Gaussian gave you both, which is why it never occurred to you these were separate abilities. They are. A generative model gives you the first and not, cheaply, the second. For control the first is what you need: acting means producing an action, not scoring one. Density evaluation was never for the robot; it was for the RL algorithm. That is the bill Section 5.7 hands you.

Framing two: transport. Picture a cloud of noise points, a fuzzy ball at the origin, and elsewhere in the same space the cloud of real demonstrated chunks — lumpy, several separated blobs. A generative model is a rule for *moving* the first cloud onto the second: every noise point is carried to a data-like point, and if the rule is right the moved cloud matches the data cloud, blobs and all. Nothing is left in the valley.

WORDS YOU WILL HEAR

Generative model — any model you can draw new samples from, as opposed to one that predicts a label or a number.

Latent — the random input $\mathbf{z}$; "latent space" is where it lives. **Mode** — a local peak of a distribution, one distinct way of doing the task; **unimodal** means one peak, **multimodal** several. **Mode collapse** — a sampler that only ever produces one mode. **Density** (or likelihood) — the number $\pi_{\theta}(\mathbf{a} \mid \mathbf{o})$; a model is "likelihood-free" when it can sample but not compute this. **Denoiser** — a network trained to remove noise from a corrupted input; Chapter 6 turns one into a sampler.

5.4.4 An analogy, and where it breaks

Think of two programs in a manufacturing pipeline. One is the **stamping machine**: press a button, a part comes out. The other is the **QA inspector**: hand it a part, it returns a score. Different interfaces, and you cannot in general build one from the other. The machine is the sampler; the inspector is the density. If your job is to make parts you need the machine, and being unable to score a part is no obstacle.

Where the analogy breaks. A real stamping machine's virtue is that every part is identical — variation is the defect. A generative model's virtue is the opposite: it must produce variation, in the proportions the data shows. Keep the interface half of the analogy; throw away the uniformity half.

5.5 The landscape in one page

Generative modeling is a large field and you do not need most of it. You need enough of the map that paper titles stop being opaque. Six families, and Figure 5.2 lays them out along the two axes that matter here: what each one learns from, and what each one costs you.

Variational autoencoders (VAEs) (Kingma & Welling, 2014). An encoder maps a data point to a small latent $\mathbf{z}$, a decoder maps $\mathbf{z}$ back, and the training signal is reconstruction error plus a penalty keeping the latent close to $\mathcal{N}(\mathbf{0}, \mathbf{I})$. Sampling is one cheap pass. The cost: the reconstruction term is a squared error, so samples come out blurred. You have met one already, since ACT, the action-chunking transformer, is a conditional VAE (Zhao et al., 2023). ACT works well in practice despite the blurring, because the chunk structure and a narrow per-task dataset do much of the disambiguating for it — which is why LeRobot still ships it and why you will see it succeed on your own bench. But the blurring is real, it is worst exactly where the demonstrations are most ambiguous, and it is why the field kept looking.

Generative adversarial networks (GANs) (Goodfellow et al., 2014). A generator turns noise into samples; a discriminator tries to tell real from generated; the generator's signal is "fool the discriminator." Samples are sharp, sampling is one pass. The cost is the adversary: a two-player game that oscillates and frequently collapses onto a single mode — the exact failure you were escaping.

Autoregressive / token models. Chop the action into discrete tokens, then predict them one at a time with cross-entropy, each conditioned on all previous ones. The signal is next-token prediction — the most reliable objective in modern machine learning — and because each token sees the earlier ones, Section 5.4's coupling problem disappears. The costs: a tokenizer that discretizes continuous actions without destroying precision, and sequential generation.

This branch is live, not historical. RT-2 (Brohan et al., 2023) put robot actions in a language model's vocabulary, $\pi_{0.5}$ uses the FAST action tokenizer throughout pre-training (Physical Intelligence, 2025a), and LeHome keeps FAST tokens in its prefix to shape the representation during training (Larchenko, 2026).

Energy-based / implicit models (Florence et al., 2021, for the robotics version). Learn a scalar $E_{\theta}(\mathbf{o}, \mathbf{a})$ — low energy means good action — and define the distribution as proportional to $\exp(-E_{\theta})$. Natively multimodal: put low energy on two separate regions and you are done. The cost lands on sampling, which needs either an optimizer or MCMC — Markov chain Monte Carlo: draw a sample by running a random walk that slowly settles into the distribution you want. Either way you are running an inner search loop for every single action, which is elegant on paper and awkward on a robot with a 33 ms budget.

Diffusion models (Ho et al., 2020; brought to manipulation by Chi et al., 2023). Corrupt data with noise at many levels, train a network to undo the corruption with a plain regression loss, then generate by starting from pure noise and denoising repeatedly. Training is stable supervised regression, and the sampler is genuinely multimodal on continuous outputs. The cost is many sequential network calls. Chapter 6 builds it from scratch.

Flow matching (Lipman et al., 2023). Same spirit, but along near-straight paths, which simplifies the target and lets far fewer steps suffice. Chapter 7 builds it; it is what the π -family and LeHome use.

family	what it learns from (training signal)	what it costs you
VAE	reconstruct data, keep latent Gaussian	samples come out averaged
GAN	fool a discriminator network	unstable; mode collapse
autoregressive	next-token prediction, cross-entropy	tokenizer; one pass per token
energy-based	score actions against negatives	sampling needs search
diffusion	denoise data at many noise levels	many steps (Chapter 6)
flow matching	regress a velocity toward data	a few steps (Chapter 7)

Figure 5.2 — The six families, by what they learn from and what that costs. Green marks the two robotics converged on; orange marks the other live branch, token models, which you meet inside $\pi_{0.5}$'s pre-training. Notice the deciding factor is the middle column, not sample quality: stable training signals won.

So why did robot learning converge on diffusion and flow matching? Training is ordinary supervised regression, multimodality is native with no fixed mode count, and they work directly on continuous chunks — no tokenizer, no precision lost to binning. Sample quality also degrades gracefully as you cut steps, giving you a real-time knob instead of a cliff.

ON YOUR RIG

LeRobot ships all of this as trainable policy classes: `act` (the conditional VAE), `diffusion`, and the flow-matching `pi0` ports, alongside the SAC-with-Gaussian policy HIL-SERL runs today. Two evenings. First, measure your own multimodality — but condition, or you measure the wrong thing. Use the first timestep of each SO-101 cube episode, where the arm is at home and the observation is near-identical across takes, or group episodes by matched cube pose. Scatter-plot the recorded action within that group: two clusters are two ways of doing one thing. Spread from the cube sitting somewhere different each take is initial-condition variation, not this chapter's multimodality. Second, train `diffusion` on those episodes and read its config for `horizon` and the denoising-step count.

5.6 Why iterative refinement

5.6.1 The obvious generative model, and why it struggles

Take the definition from Section 5.4 literally: build one network, feed it $\mathbf{z}$ and $\mathbf{o}$, have it output all 360 numbers of the chunk in one forward pass. Why did the field not stop there?

The trouble is what that single function must do. The noise $\mathbf{z}$ lives in a smooth, connected ball; the data lives in several separated blobs. So T_θ must tear the ball into pieces, mapping one region of noise space to blob A and a neighboring region to blob B with nothing in between — changing extremely fast across the boundary. Networks

trained by gradient descent are biased toward smooth functions, so asking one to be nearly discontinuous in 360 dimensions is asking against the grain.

The training signal makes it worse. Train T_θ by squared error and you are back in Section 5.2: the optimum averages. Train it against a discriminator and you have a GAN. A sharp, multimodal answer in one pass needs an adversary or a trick.

5.6.2 The fix: many small steps

The trick that won is **iterative refinement**. Instead of one big commitment the model makes K small edits: it starts at pure noise and, at each step, looks at where it is and nudges it toward being more data-like.

$$\mathbf{a}^{(0)} = \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{a}^{(k+1)} = \mathbf{a}^{(k)} + \Delta_\theta(\mathbf{a}^{(k)}, k, \mathbf{o}), \quad k = 0, 1, \dots, K-1$$

IN PLAIN WORDS

$\mathbf{a}^{(k)}$ is the candidate action chunk after k edits — a full 360-number chunk at every stage, starting as garbage and ending as a real motion. The parenthesized superscript counts refinement steps; it has nothing to do with the environment timestep t , and confusing the two is the most common beginner error here. $\mathbf{a}^{(0)} = \mathbf{z}$ says step zero is a fresh draw of Gaussian noise. Δ_θ is the network: it takes the current candidate, the step index k so it knows how far along it is, and the observation $\mathbf{o}$, and returns a small correction of the same shape. K is how many edits you make — ten, in the π -family — and the chunk you execute is $\mathbf{a}^{(K)}$.

5.6.3 Why the fix works

Four reasons, and the third is the one that people quite reasonably refuse to accept on faith.

Each step asks an easy question. The one-shot model had to answer "which of the eight valid chunks should this noise vector become?" The refinement model only answers "given where I am now, which direction is slightly more data-like?" Near any particular point that is usually unambiguous — if you have already drifted toward the left-arm-leads region, the local answer is "keep going left." A locally unimodal question has a well-defined average answer, so a plain regression loss is a correct training signal, and training stays stable.

The commitment is carried in the state. Section 5.4's coupling problem dissolves, because the model reads $\mathbf{a}^{(k)}$ at every step. Once the first few edits lean the chunk toward "left arm leads", every later edit sees that lean in its input and stays consistent.

What breaks the tie at the very first step. The paragraph above explains why later steps stay consistent once the chunk has already leaned one way. It does not explain what makes it lean in the first place, and that is the honest objection: at step zero the candidate is pure noise, so is the "slightly more data-like direction" there not just an average over all eight modes — the exact Section 5.2 failure you just learned to fear? The answer is that $\mathbf{z}$ is not a coin flip you are averaging over. It is 360 numbers, and the network reads every one of them. The question it is asked is never "what is the average of all valid chunks"; it is "starting from *this particular* noise vector, which way is more data-like?" Two draws are two different starting points in a 360-dimensional space, and different starting points already sit nearer different modes, so their conditional averages genuinely point in different directions. The noise carries information, and the model is conditioned on it from the first step to the last.

Now the caveat, because that argument is not airtight at the very beginning. Extremely close to step zero the noise really is close to uninformative about which mode is nearer, and the network's answer really is close to a blend of the directions toward all of them. What rescues it is that a blend of *directions* is a different object from a blend of *actions*: an early step that points broadly "toward the data" is correct for every mode at once, and it moves the candidate, and once the candidate has moved the modes are no longer symmetric around it. Working that out precisely takes a two-mode example with numbers in it, and Chapter 7 does exactly that — it is the right place for it, because flow

matching's training target is a velocity you can write down and average by hand. Take this reason on trust for one more chapter; you will get the arithmetic.

Coarse-to-fine comes for free. Early steps, on near-pure noise, can only decide which region of chunk space we head for; later steps fine-tune details. Neither behavior had to be designed; both fall out of feeding the candidate back in.

```
# (1) one shot: the network must commit immediately
mu = net(o)                                # (H, d) -- the whole chunk at once
a = mu + sigma * normal(0, 1, mu.shape)    # noise added AFTER the decision
# that noise cannot move `a` between modes; it only blurs the valley point

# (2) iterative refinement: K chances to change its mind
a = normal(0, 1, (H, d))                  # pure noise: 360 numbers, not a coin flip
for k in range(K):                         # K is small -- 10 in the pi-family
    a = a + delta_theta(a, k, o)           # reads the CURRENT a, so the draw steers it
return a                                   # the chunk a_{t:t+H}
```

Figure 5.3 puts the two side by side.

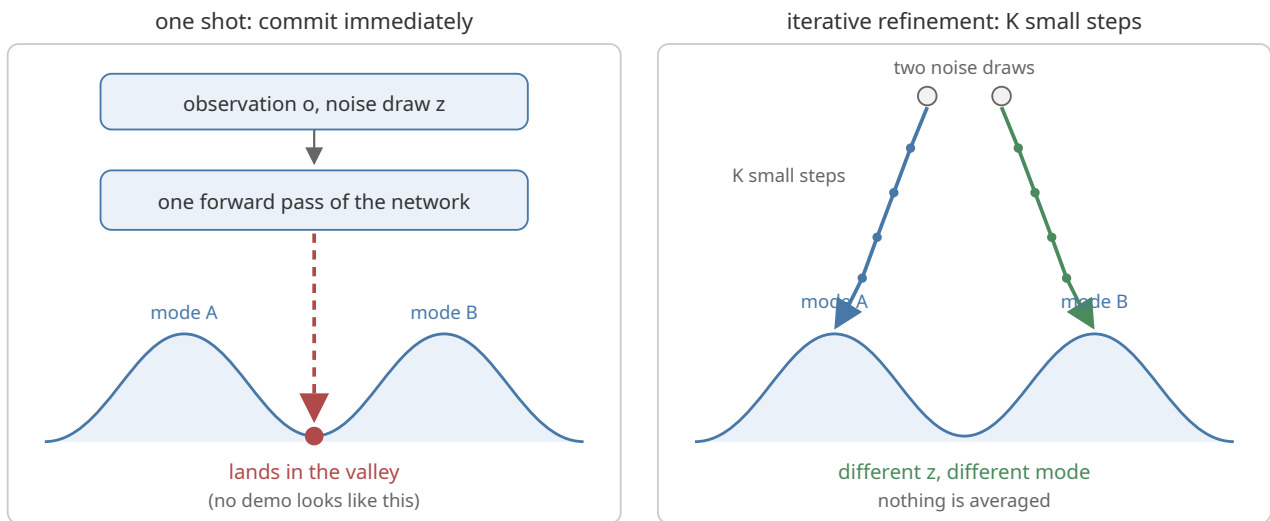


Figure 5.3 — One shot versus many steps, against the same two-mode action density. On the left, a single forward pass must answer immediately, and a squared-error signal drives it to the average: the red dot in the valley. On the right, each step re-reads the current candidate before nudging it, so the path bends toward whichever mode it has drifted into. The two draws end in different modes because every step is conditioned on the current candidate, and that candidate still carries the 360-number noise vector it started from — the tie-break argued just above, and calculated in Chapter 7.

5.6.4 Sculpting versus printing

Printing is one-shot: set the plate, press once, and whatever was on the plate is what you get. If the plate is wrong the print is wrong, and there is no intermediate state where you could have noticed. Sculpting is iterative: start from a rough block, take a small pass, step back and look, take the next pass based on that. Each pass is a small, local, low-risk decision, and the global shape emerges from the sequence.

Where it breaks, in two places. A sculptor works toward one statue they have in mind, while our sampler's whole purpose is that different starting blocks become different statues — randomness is a feature here and a flaw there. And sculpting only removes material, while a refinement step can move a candidate any direction, including undoing an earlier nudge.

5.7 The price you pay

Section 5.1 listed three services the Gaussian rendered: cheap sampling, cheap differentiable log-probability, reparametrized gradients. A generative policy keeps the first and loses the second, and that loss is why the second half of this book exists. The third survives only in a weakened form: the refinement loop is still a differentiable function of θ , so gradients can in principle be pushed back through it, but doing so means backpropagating through K network calls, and there is no log-density waiting at the end of it for a policy gradient to use. Sampling stays cheap. Scoring is what you lose.

For a Gaussian you get $\log \pi_{\theta}(a | o)$ by subtracting, squaring, dividing, and summing. Now ask the same of the sampler in Section 5.6: what is the log-probability of the chunk that came out of a ten-step refinement loop? A number exists — the loop is a definite procedure with a definite output distribution — but computing it means unwinding the whole loop and accounting for how each step stretched or squeezed the space around the sample. Chapter 9 works out how expensive that is and why the estimate's noise is fatal.

Then stack on the problem Section 5.3 planted. Even a perfect chunk-level log-probability is not the quantity PPO wants. The importance ratio $\rho_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$ is written per environment timestep, and inside a chunk there are no per-timestep decisions to take the ratio of. The clip at $1 \pm \epsilon$ is a statement about a ratio of densities. No cheap density, no ratio, no clip, no PPO.

The good news is one sentence, and it is why Chapter 4 was the hinge: the *training loss* of a refinement model is a per-sample regression, and regressions can be weighted and conditioned. Multiply a sample's loss by an advantage-derived weight and you get RL-flavored improvement without touching a density — AWR, applied to a generative policy. Or feed the advantage in as an extra input and ask the sampler for a good chunk — RECAP, in Chapter 10.

IN THE LEHOME SOLUTION

Larchenko's winning policy is exactly the model class this chapter argues toward: a flow-matching action expert emitting 30-step, 12-dimensional joint-delta chunks: one second of motion at the 30 Hz simulation rate, and the same 30 steps stretched over 1.5 s when the real round ran at 20 Hz (Larchenko, 2026). Multimodality is no theoretical concern there; folding a shirt has ties at every stage, starting with which arm leads. The tech report states the consequence plainly: PPO-style log-probability policy gradients "don't transfer cleanly to flow-matching VLAs," so improvement runs through weighted sampling or conditioning instead.

CHECKPOINT

- Explain why a Gaussian policy was right for your rover and wrong for a folding dataset, in terms of one peak versus several.
- Justify why fitting a unimodal model by maximum likelihood to bimodal data puts the mean in the valley, and reproduce the six-demo example: $\mu^* = 0$, $\sigma = 1.0132$, a 45% collision rate.
- Say what an action chunk is and why it exists, compute its dimension for a bimanual SO-101 at $H = 30$, and explain why chunking multiplies modes rather than reducing them.
- Define a generative model without using the word "diffusion", and explain the difference between sampling from a distribution and evaluating its density.
- Name the six families in Figure 5.2 with their training signals, and say why robotics converged on the two green ones.
- Explain iterative refinement in three sentences, including why many small steps let a plain regression loss produce a sharply multimodal sampler — and name what it costs you.

Where to go deeper. For the multimodality argument made empirically rather than rhetorically, Chi et al. (2023) opens with this comparison across manipulation benchmarks, and Florence et al. (2021) makes the same case from the energy-based side. Zhao et al. (2023) is the readable original for action chunking and doubles as a clean conditional VAE in robotics. Appendix A carries the Chi and Zhao papers with reading notes and says which sections to skip;

Florence is a short optional detour with no entry there, as are the VAE, GAN and RT-2 citations above. Chapters 6 and 7 build the two green rows of Figure 5.2 from nothing.

Chapter 6 — Diffusion policies from zero

Chapter 5 ended with a demand and no way to meet it. We need a policy that can represent a multimodal distribution over one-second action chunks — one that says "swing left around the mug *or* swing right, but never straight through it" — and we need samples from it thirty times a second on a robot. A Gaussian cannot do this. This chapter builds the first model class that could, from nothing.

You will find diffusion strange for about ten minutes and then obvious. Everyone does. The strangeness is that the model is defined by a process running *backwards* from a process you never train. Once that clicks, the rest is a regression and a for-loop.

Why this chapter, given that the π -family models and LeHome use flow matching rather than diffusion? Because Diffusion Policy (Chi et al., 2023) is the direct ancestor of all of them, and it established three ideas they inherit: diffuse the *action chunk* rather than a single action, condition on the observation without ever denoising it, and execute under a receding horizon — predict a long plan, run only its first part, re-plan from a fresh observation (Section 6.6). Chapter 7 is a cleaner parameterization of the same idea — and you cannot see that it is cleaner without having seen the thing it cleans up.

6.1 The idea in one page, no math

6.1.1 What we want, and the obvious thing to try

Here is the goal. At each moment in a demonstration there is an observation $\mathbf{o}$ (three camera images and twelve joint angles, say) and the chunk of actions the demonstrator produced next, $\mathbf{a}_{t:t+H}$. We want a model that, given a fresh $\mathbf{o}$, produces a *sample* from the distribution of chunks a good demonstrator might have produced. Not the average chunk. A sample.

The obvious thing to try is to have the network output the distribution's parameters directly — exactly what your PPO rover policy does, emitting a mean $\mu_\theta(\mathbf{o})$ so the policy is $\mathcal{N}(\mu_\theta(\mathbf{o}), \sigma^2)$. Chapter 5 showed why this fails on demonstration data: fitting a unimodal density to bimodal data parks the mean in the valley between the modes, producing an action that appears in no demonstration at all.

So make the output distribution richer. Chapter 5 worked through the two repairs any engineer thinks of first, and why each hurts: a **mixture of Gaussians**, whose component count is fixed in advance while the number of real modes changes frame to frame, and **discretization**, which either discards the correlations between action dimensions or needs a table with (bins)³⁶⁰ entries. Add the third that Chapter 5 left to this one: **autoregression**, principled but one sequential forward pass per dimension.

Here is what Chapter 5 did not say, and it motivates everything below. All three try to *write down* a complicated shape in 360 dimensions — a LeHome chunk is $H \times d_a = 30 \times 12 = 360$ numbers, d_a being the width of one action — in one shot, from one forward pass. That is a hard thing to ask of a network, and the question worth asking is whether we can avoid asking it.

6.1.2 The fix: learn a process, not a shape

Here is the move. Do not model the distribution; model a *process* that ends at the distribution. And start from destruction, because destruction is free. Take a clean action chunk. Add a small amount of Gaussian noise; the result is still recognizably the chunk, just slightly fuzzy. Add a little more. Repeat, say, a hundred times. By the end you hold something statistically indistinguishable from a fresh draw of `randn`. That procedure has no parameters, nothing about it is learned, and you could implement it in one line.

Now train a network to undo *one* step of it. Show the network a slightly-noised chunk and ask: what noise was added to get here? That is plain supervised regression with a known label, because you are the one who added the noise —

you have the answer key.

Finally, chain the undo steps. Start from pure Gaussian noise, apply the network, apply it again, and after a hundred applications you hold something that looks like it came from your dataset. Draw a different starting noise and you get a different chunk — sometimes the left-hand approach, sometimes the right-hand one, in proportion to how often each appeared in the demonstrations.

The cleverness is the decomposition. The global problem — sample from a multimodal distribution over 360 dimensions — is hard. Each local problem — this vector is slightly noisier than a real chunk, guess the noise — is easy, because the answer is close to the input. Diffusion trades one hard problem for a hundred easy ones, and networks are very good at easy regressions.

An analogy from your own work. You do not solve a least-squares problem in closed form; you run gradient descent, where each step is trivial and the difficulty lives in the number of steps. Diffusion does the same for *sampling*. Where it breaks: gradient descent lands on the same answer every time, diffusion somewhere different every time. The randomness is not a nuisance; it is the product.

An analogy from everyday physics. Drop ink into still water and film it. The ink spreads, the film ends in uniform gray, and the spreading needs no intelligence. Play the film backwards and gray resolves into a blob. A diffusion model is a network that plays that film backwards one frame at a time. Where it breaks: real ink diffusion is irreversible, since reversing it would require knowing where every particle came from. The network does not recover *the* original blob; it produces *a* plausible one. For a policy that is not a defect — it is the multimodality we asked for.

6.2 The forward process, precisely

Now the math, one symbol at a time. Here $\mathbf{x}$ stands for whatever vector we are generating; from Section 6.6 onward it is an action chunk, but nothing depends on that. Write $\mathbf{x}_0$ for a clean data point and $\mathbf{x}_k$ for that point after k rounds of noising, with $k \in \{1, \dots, N_{\text{diff}}\}$. One step of destruction is

A warning about direction before you read another equation. The diffusion literature numbers noise levels *upward from clean data*: $\mathbf{x}_0$ is the data point, $\mathbf{x}_{N_{\text{diff}}}$ is pure noise, and sampling therefore counts *down*. That is the reverse of Chapter 5's generic refinement sketch, where $\mathbf{a}^{(0)}$ was the noise draw and $\mathbf{a}^{(K)}$ the finished chunk. Flow matching (Chapter 7) picks a third convention: continuous $\mathbf{u} \in [0, 1]$ with $\mathbf{u} = 0$ at noise and $\mathbf{u} = 1$ at data. None of the three is more correct, and every paper assumes its own is obvious. Before reading anyone's equations, find the sentence that says which end is noise; getting it backwards makes every formula look subtly wrong.

$$q(\mathbf{x}_k | \mathbf{x}_{k-1}) = \mathcal{N}(\mathbf{x}_k; \sqrt{1 - \beta_k} \mathbf{x}_{k-1}, \beta_k \mathbf{I}).$$

Two letters carry the whole chapter, so fix them now. q is the **forward process**: fixed, known in closed form, never learned — the corruption you apply yourself. p_θ is the **reverse model**, arriving in Section 6.3: the network's attempt to undo q , and the only thing here with parameters in it. Approximating q run backwards by a p_θ run forwards is the entire chapter. Read $\mathcal{N}(\mathbf{x}_k; \mathbf{m}, \mathbf{S})$ as " $\mathbf{x}_k$ is drawn from a Gaussian with mean $\mathbf{m}$ and covariance $\mathbf{S}$ "; the semicolon separates the variable from the parameters, and the $\mathcal{N}(0, \mathbf{I})$ form below is the same object with the variable left implicit.

Read the equation as an instruction: to get $\mathbf{x}_k$, shrink $\mathbf{x}_{k-1}$ by the factor $\sqrt{1 - \beta_k}$, then add Gaussian noise with variance β_k in every coordinate. The numbers $\beta_1, \dots, \beta_{N_{\text{diff}}}$ are the **noise schedule**: a fixed list of small positive numbers you choose once, before training, and never learn.

Two borrowed letters, flagged once so they never confuse you again. The schedule's β_k and the $\alpha_k = 1 - \beta_k$ derived from it are local to the diffusion formalism and have nothing to do with the AWR temperature β of Chapter 4 or the baseline damping coefficient α of Chapter 11. The collision is inherited from the source papers, which were written by different communities; inside this chapter, β and α always carry a subscript k and always mean the schedule.

Why the shrink? Without it the variance would grow without bound. With it, the process is **variance-preserving**: if $\mathbf{x}_{k-1}$ has unit variance, then $\mathbf{x}_k$ has variance $(1 - \beta_k) \cdot 1 + \beta_k = 1$. Every noise level lives at the same scale, which is what lets one network handle all of them — and it is why you normalize actions to roughly unit scale before training. If your gripper command lives in $[0, 255]$ and your joint deltas in $[-0.01, 0.01]$, the process destroys them at wildly different rates.

6.2.1 The equation a practitioner actually uses

Applying that step k times in a row would be slow, and we would have to do it inside the training loop. We do not have to. Because a Gaussian pushed through a linear map plus more Gaussian noise is still Gaussian, the whole chain collapses. Define $\alpha_k := 1 - \beta_k$ and $\bar{\alpha}_k := \prod_{s=1}^k \alpha_s$. Then

$$\mathbf{x}_k = \sqrt{\bar{\alpha}_k} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, I).$$

IN PLAIN WORDS

A data point at noise level k is a weighted blend of two things: the clean data point $\mathbf{x}_0$, and one draw of standard Gaussian noise $\boldsymbol{\varepsilon}$ of the same shape. The weights are $\sqrt{\bar{\alpha}_k}$ on the data and $\sqrt{1 - \bar{\alpha}_k}$ on the noise, and $\bar{\alpha}_k$ — "alpha-bar", the running product of all the $1 - \beta_s$ up to level k — starts near 1 and slides to near 0 as k grows. So $\bar{\alpha}_k$ is an interpolation dial: at small k almost all data, at large k almost all noise. And this is why the equation matters: to build a training example at level k you never simulate the chain. Look up $\bar{\alpha}_k$ in a precomputed table, draw one $\boldsymbol{\varepsilon}$, mix.

To see the collapse, do two steps by hand. Substituting $\mathbf{x}_1 = \sqrt{\alpha_1} \mathbf{x}_0 + \sqrt{1 - \alpha_1} \boldsymbol{\varepsilon}_1$ into $\mathbf{x}_2 = \sqrt{\alpha_2} \mathbf{x}_1 + \sqrt{1 - \alpha_2} \boldsymbol{\varepsilon}_2$ gives $\mathbf{x}_2 = \sqrt{\alpha_1 \alpha_2} \mathbf{x}_0 + \sqrt{\alpha_2(1 - \alpha_1)} \boldsymbol{\varepsilon}_1 + \sqrt{1 - \alpha_2} \boldsymbol{\varepsilon}_2$. The last two terms are independent zero-mean Gaussians, so they fold into one draw whose variance is $\alpha_2(1 - \alpha_1) + (1 - \alpha_2) = 1 - \bar{\alpha}_2$. That is the claim at $k = 2$; induction does the rest.

6.2.2 A number you can hold on to

Ho et al. (2020) used $N_{\text{diff}} = 1000$ with a linear schedule from $\beta_1 = 10^{-4}$ to $\beta_{1000} = 0.02$. The running product gives:

$$\begin{aligned} \bar{\alpha}_{100} &= 0.8970, & \bar{\alpha}_{500} &= 0.0786, & \bar{\alpha}_{1000} &= 0.00004. \\ \sqrt{\bar{\alpha}_{100}} &= 0.9471, & \sqrt{\bar{\alpha}_{500}} &= 0.2803, & \sqrt{\bar{\alpha}_{1000}} &= 0.0064. \\ \sqrt{1 - \bar{\alpha}_{100}} &= 0.3209, & \sqrt{1 - \bar{\alpha}_{500}} &= 0.9599, & \sqrt{1 - \bar{\alpha}_{1000}} &= 1.0000. \end{aligned}$$

Take a normalized gripper position $\mathbf{x}_0 = 0.60$ and a drawn noise value $\boldsymbol{\varepsilon} = 1.20$. At level 100:

$$\mathbf{x}_{100} = 0.9471 \times 0.60 + 0.3209 \times 1.20 = 0.5683 + 0.3851 = 0.9533.$$

Ninety-five percent of the original amplitude survives — early steps are gentle. At level 500:

$$\mathbf{x}_{500} = 0.2803 \times 0.60 + 0.9599 \times 1.20 = 0.1682 + 1.1519 = 1.3201.$$

The data contributes 28% of the amplitude — recognizable only statistically. At level 1000 it contributes 0.6%: noise for all practical purposes, which is the point, because the last level must be indistinguishable from $\mathcal{N}(\mathbf{0}, I)$, where sampling starts. Figure 6.1 shows the same progression for a whole chunk.

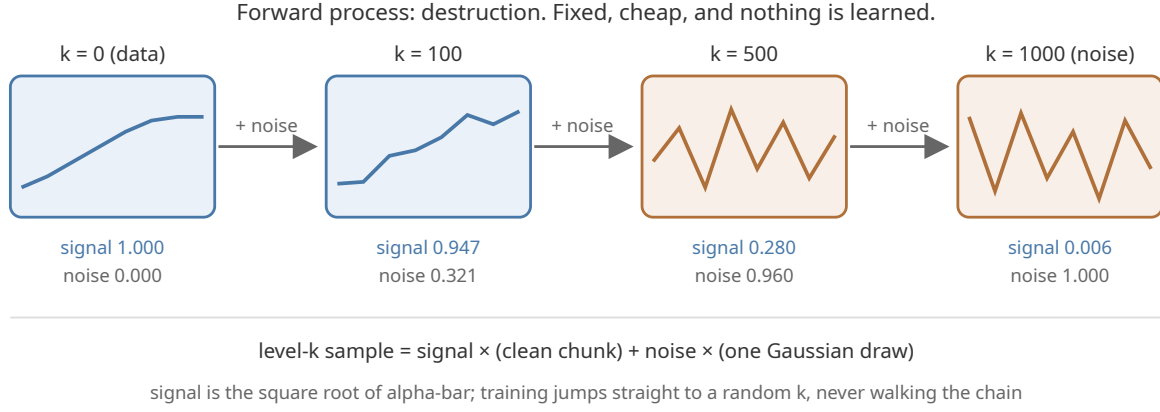


Figure 6.1 — One action chunk (a curve over the chunk's timesteps) at four levels of the standard 1000-step linear schedule. Notice how uneven the destruction is: at $k = 100$ the shape survives, and by $k = 500$ it is gone, so half the schedule is spent on samples carrying almost no information. That imbalance is what the cosine schedule fixes.

6.2.3 Schedules, and why the default is wrong for robots

Notice something uncomfortable in those numbers. The linear schedule's midpoint is already 96% noise. Tolerable at $N_{\text{diff}} = 1000$, fatal at the budgets robotics uses.

The **cosine schedule** (Nichol & Dhariwal, 2021) defines $\bar{\alpha}_k$ directly as a shifted cosine curve rather than defining β_k linearly, and it destroys information much more evenly. At $N_{\text{diff}} = 100$ — Diffusion Policy's setting — its midpoint $k = 50$ gives $\bar{\alpha}_{50} = 0.494$, so the mix is $0.703 x_0 + 0.711 \epsilon$: a genuine fifty-fifty blend, exactly where a midpoint should be. Chi et al. (2023) report this "Square Cosine Schedule" works best, and it is what LeRobot's diffusion config ships with — see the config listing in Section 6.6. Do not reach for the linear schedule just because it appeared in the original paper.

6.3 The reverse process and the training objective

6.3.1 What we would like, and why we cannot have it

Sampling means running the chain backwards: given x_k , produce x_{k-1} . The quantity we want is the true reverse conditional $q(x_{k-1} | x_k)$. By Bayes' rule it is proportional to $q(x_k | x_{k-1}) q(x_{k-1})$, and $q(x_{k-1})$ is *all your data noised to level $k-1$* — the very object we are trying to model. We are going in circles.

Here is the escape, the deep fact that makes diffusion work. If each β_k is small, each forward step barely moves the point, and the reverse of a barely-moving step is *also approximately Gaussian*. So we do not need the full shape of $q(x_{k-1} | x_k)$, only its mean and variance, and we posit the learned reverse model p_θ promised in Section 6.2 — a Gaussian of our own, aimed at the one we cannot compute:

$$p_\theta(x_{k-1} | x_k) = \mathcal{N}(x_{k-1}; \mu_\theta(x_k, k), \sigma_k^2 I)$$

and we let the network supply the mean while fixing the variance to a schedule-derived constant. Ho et al. (2020) tried both $\sigma_k^2 = \beta_k$ and $\sigma_k^2 = \tilde{\beta}_k = \frac{1-\bar{\alpha}_{k-1}}{1-\bar{\alpha}_k} \beta_k$ and reported "similar results" — not a choice worth agonizing over. Making each step tiny is what turned an intractable distribution into a Gaussian with an unknown mean: the same trade as before, seen from the other side.

6.3.2 Predicting the noise instead of the mean

We could have the network output μ_θ directly. Ho et al. found a reparameterization that trains far better. Rearranging the closed-form jump gives $x_0 = (x_k - \sqrt{1 - \bar{\alpha}_k} \epsilon) / \sqrt{\bar{\alpha}_k}$, so knowing ϵ means knowing x_0 , and a good guess at x_0

is enough to write down the reverse mean. Predicting ϵ and predicting the clean data are the same task in different coordinates, related by an affine map.

Why prefer ϵ ? You have an exact label for free, because you drew ϵ yourself. And ϵ has unit variance at every noise level by construction, so the target is pre-whitened and one loss scale works across all k , whereas an x_0 target varies enormously in difficulty with k . This convention is **ϵ -prediction** (epsilon-prediction), and almost every robotics implementation uses it.

The training objective is then embarrassingly simple:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{k, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_k} x_0 + \sqrt{1 - \bar{\alpha}_k} \epsilon, k) \right\|^2 \right]$$

IN PLAIN WORDS

The left side, $L_{\text{simple}}(\theta)$, is the number you minimize; θ is every weight in the denoiser, the only thing being changed. The right side says how to compute it. Pick a training example x_0 at random. Pick a noise level k uniformly from $\{1, \dots, N_{\text{diff}}\}$. Draw one Gaussian noise vector ϵ shaped like x_0 . Mix them with the two coefficients from Section 6.2; hand the result and the number k to the network ϵ_{θ} , which returns its guess at the noise; score that guess by squared error against the ϵ you actually drew. $\mathbb{E}$ means average the score over all the random choices — in practice, over a minibatch. The network is a denoiser graded on how well it identifies the corruption, and k is passed in because the answer depends on how much corruption there is.

One thing in the code below is a forward reference, so name it now rather than let it puzzle you. You will see an observation o passed to the network here and in every code block that follows. It is **conditioning**: an extra input the denoiser reads and never generates. It changes nothing about the loss, which is why o does not appear in L_{simple} above. Section 6.6 explains the three ways it gets wired in; until then, treat it as a fixed side input.

```
# one training step of a diffusion policy
o, x0 = batch                # o: observation; x0: clean action chunk, shape (B, H, d_a)
k      = randint(1, N_diff + 1, (B,)) # one noise level per sample, uniform over all levels
eps    = normal(0, 1, x0.shape)      # this is the regression label -- we drew it, so we know it
a_sig  = sqrt(abar[k]).view(B, 1, 1) # data coefficient; abar precomputed once from the schedule
a_noi  = sqrt(1 - abar[k]).view(B, 1, 1) # noise coefficient (NOT the sampler's sigma[k] below)
xk     = a_sig * x0 + a_noi * eps     # closed-form jump: the chain is never simulated
eps_hat = net(xk, k, o)              # k is conditioning too: sinusoidal embedding inside the net
loss    = ((eps - eps_hat) ** 2).mean() # plain MSE. That is all.
loss.backward(); opt.step(); opt.zero_grad()
ema.update(net)                 # keep an exponential moving average -- that is what you deploy
```

6.3.3 What this is not — and how it compares to PPO

Note what is *absent*, because the absences are why diffusion training is so reliable. No adversary, so no minimax game to balance. No likelihood and no partition function, so nothing to normalize. No sampling inside training, and no chain simulation.

Now hold this next to PPO, which you know from the inside. In PPO the loss estimates an expectation over trajectories that *the current policy generates*. The moment you update θ , the data distribution moves, your batch goes stale, and the importance ratio $\rho_t(\theta)$ drifts — which is why you need clipping, few epochs per batch, and a value function chasing a moving target. Every stability mechanism in Chapter 3 manages that feedback loop.

The diffusion loss has no feedback loop. The dataset is fixed, the noise levels are fixed, the targets are fixed. It is ordinary supervised regression and behaves like one: shuffle, use a large batch, decay the learning rate, walk away. Chi et al. (2023) report training stability as a headline property of diffusion policies, and this is the mechanical reason. The bill comes due in Chapter 9, when we try to improve such a policy with RL and find that the log-probability PPO needs is no longer available.

6.4 Sampling

6.4.1 The ancestral loop

Training gave us a network that identifies noise; sampling turns it into a generator. Start at $\mathbf{x}_{N_{\text{diff}}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, free to produce, and step down to $\mathbf{k} = 0$. This is **ancestral sampling**: walking the chain of conditionals from the last "ancestor" back to the data. The package you now have — that forward process, the ϵ -prediction loss, and this sampler — is what everyone calls **DDPM**, for *denoising diffusion probabilistic models*, after Ho et al. (2020). The name is all over the code below and the config files you will open.

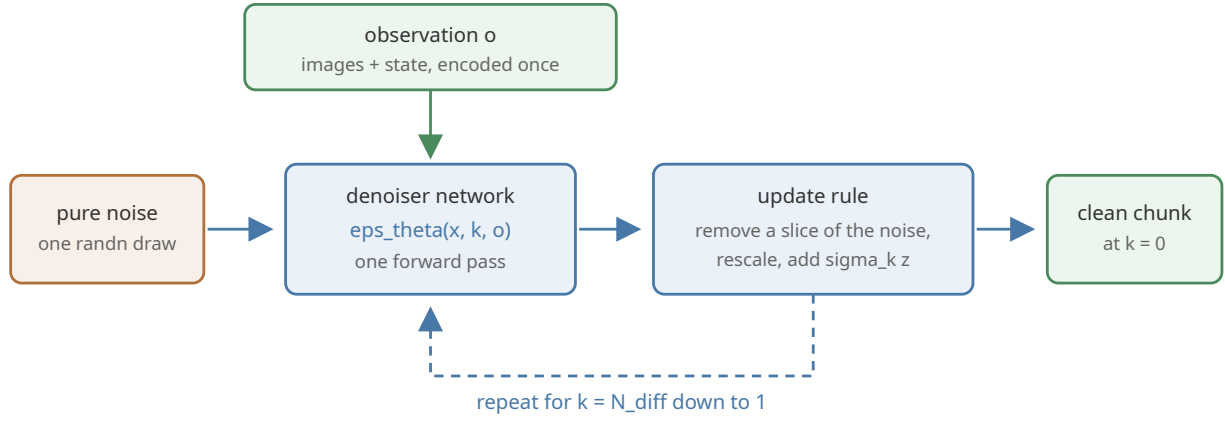
$$\mathbf{x}_{k-1} = \frac{1}{\sqrt{\alpha_k}} \left(\mathbf{x}_k - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k}} \epsilon_{\theta}(\mathbf{x}_k, \mathbf{k}) \right) + \sigma_k \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

IN PLAIN WORDS

Four operations. Ask the network what noise it sees in the current sample $\mathbf{x}_k$; that answer is $\epsilon_{\theta}(\mathbf{x}_k, \mathbf{k})$. Subtract a sized-down slice of it: ϵ_{θ} estimates *all* the noise in $\mathbf{x}_k$, and the coefficient $\frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}}$ (recall $1 - \alpha_k = \beta_k$) cuts that estimate to about this one step's worth. Take all of it and you would jump straight to $\hat{\mathbf{x}}_0$ — which at $\mathbf{k} = 1$ is exactly what happens, as the last step of the worked example shows. Divide by $\sqrt{\alpha_k}$ to undo the shrink this step applied; you now hold the mean of the reverse Gaussian. Add fresh noise $\sigma_k \mathbf{z}$, with $\mathbf{z}$ a new standard Gaussian and σ_k from the schedule — at $\mathbf{k} = 1$ set $\mathbf{z} = \mathbf{0}$ and stop. Here $\alpha_k = 1 - \beta_k$, $\bar{\alpha}_k$ is the running product from Section 6.2, and $\mathbf{x}_{k-1}$ is the cleaner sample you hand to the next iteration.

The last term deserves an explanation, because deleting it is a common instinct and it is wrong. We are not maximizing anything; we are *sampling*, and drawing from a Gaussian means taking its mean and adding a draw from its noise. Drop the noise and you are no longer sampling from $\mathbf{p}_{\theta}$: the samples come out over-smoothed. There is a correct deterministic sampler, coming shortly; it is just not "this update minus the noise term." Figure 6.2 shows the loop as a whole.

```
# DDPM ancestral sampling -- K = N_diff network evaluations
x = normal(0, 1, (H, d))          # start from pure noise, same shape as a chunk
for k in range(N_diff, 0, -1):
    eps_hat = net(x, k, o)         # o is FIXED: the observation is never denoised
    coef = beta[k] / sqrt(1 - abar[k]) # the slice of noise this step is responsible for
    mean = (x - coef * eps_hat) / sqrt(alpha[k])
    z = normal(0, 1, x.shape) if k > 1 else 0 # no noise injected on the final step
    x = mean + sigma[k] * z
return x                          # a clean chunk; execute its first rows on the
robot
```



the observation is encoded once and reused by every step; only the noisy chunk changes
cost of one action chunk = number of loop iterations, each one network evaluation

Figure 6.2 — The sampling loop. Notice where the cost is: the dashed feedback arrow is traversed once per denoising step, each traversal a full forward pass of the denoiser, while the observation encoder runs once. That asymmetry is why every latency trick here and in Chapter 12 targets the loop, not the encoder.

6.4.2 Worked example: one training step and a full three-step sample, by hand

Toy settings, so every number fits on a line: one scalar action (a normalized gripper command) and $N_{\text{diff}} = 3$ with an aggressive schedule $\beta = (0.1, 0.5, 0.9)$. Then $\alpha = (0.9, 0.5, 0.1)$, and the running products are $\bar{\alpha}_1 = 0.9$, $\bar{\alpha}_2 = 0.9 \times 0.5 = 0.45$, $\bar{\alpha}_3 = 0.45 \times 0.1 = 0.045$. Square roots:

$$\sqrt{\bar{\alpha}} = (0.9487, 0.6708, 0.2121), \quad \sqrt{1 - \bar{\alpha}} = (0.3162, 0.7416, 0.9772).$$

Suppose the demonstrations here are bimodal: half the time the demonstrator commands -0.5 (open), half the time $+0.6$ (closed) — the mug situation from Chapter 5, in one dimension.

One training step. Draw a demonstration $x_0 = 0.60$, a level $k = 2$, and a noise $\varepsilon = -0.80$. Build the input:

$$x_2 = 0.6708 \times 0.60 + 0.7416 \times (-0.80) = 0.4025 - 0.5933 = -0.1908.$$

Feed $(-0.1908, k=2, o)$ to the network; say it returns $\varepsilon_\theta = -0.55$. The loss is

$$(\varepsilon - \varepsilon_\theta)^2 = (-0.80 - (-0.55))^2 = (-0.25)^2 = 0.0625,$$

and the gradient with respect to the network's output is $-2(\varepsilon - \varepsilon_\theta) = -2 \times (-0.25) = +0.50$. A positive gradient means descent *decreases* the output, moving -0.55 toward -0.80 . That is the entire training signal.

A full sample, three steps. Use $\sigma_k^2 = \tilde{\beta}_k$, giving $\sigma_3 = 0.7199$, $\sigma_2 = 0.3015$, $\sigma_1 = 0$. Draw the starting point $x_3 = 1.300$ and the two injection noises $z_3 = 0.60$, $z_2 = 0.20$.

One convention first, because the direction of travel is what confuses people here. At each step we *posit the denoiser's output in clean-data coordinates* — its guess $\hat{x}_0$ at which gripper command this sample came from, the interpretable quantity — and then convert to noise coordinates with $\varepsilon_\theta = (x_k - \sqrt{\bar{\alpha}_k} \hat{x}_0) / \sqrt{1 - \bar{\alpha}_k}$, because ε_θ is what the update rule consumes. That is the $\hat{x}_0$ formula from Section 6.3 solved the other way around.

And we do not invent the $\hat{x}_0$ values, because for a two-mode dataset the ideal denoiser is Bayes' rule: weight each mode by how well it explains the sample. Under mode $x_0 = m$, the closed-form jump makes x_k Gaussian with mean $\sqrt{\bar{\alpha}_k} m$ and variance $1 - \bar{\alpha}_k$, so "how well it explains" is that density at the observed x_k , and $\hat{x}_0$ is the posterior-weighted average of the two modes. A trained network only approximates this; this is what it approximates. Full arithmetic for the first step, quoted for the other two.

Step 1, $k = 3$. We have $\sqrt{\bar{\alpha}_3} = 0.2121$, $1 - \bar{\alpha}_3 = 0.955$, and the sample $x_3 = 1.300$. Under the closed mode $x_0 = +0.6$ the predicted mean is $0.2121 \times 0.6 = 0.1273$, leaving a residual of $1.300 - 0.1273 = 1.1727$, so the

Gaussian exponent is $-1.1727^2/(2 \times 0.955) = -1.3752/1.910 = -0.7200$. Under the open mode $x_0 = -0.5$ the mean is -0.1061 , the residual is $1.300 + 0.1061 = 1.4061$, and the exponent is $-1.4061^2/1.910 = -1.9771/1.910 = -1.0351$. Only the difference of the two exponents matters, because the Gaussian's normalizing constant is the same for both and cancels:

$$\begin{aligned}\text{difference} &= -0.7200 - (-1.0351) = 0.3151, & e^{0.3151} &= 1.370, \\ \text{weights} &= \frac{1.370}{1.370 + 1} = 0.5781 \text{ on } +0.6, & \frac{1}{2.370} &= 0.4219 \text{ on } -0.5, \\ \hat{x}_0 &= 0.5781 \times 0.6 + 0.4219 \times (-0.5) = 0.3469 - 0.2110 = 0.136.\end{aligned}$$

The denoiser hedges — 58% closed, 42% open — and lands on **0.136**, a value that appears in no demonstration. That is fine here, because we are not done. In noise coordinates that guess is

$$\varepsilon_\theta = \frac{1.300 - 0.2121 \times 0.136}{0.9772} = \frac{1.300 - 0.0288}{0.9772} = \frac{1.2712}{0.9772} = 1.301,$$

and the update rule from the start of this section then gives:

$$\begin{aligned}\text{coefficient} &= \beta_3/\sqrt{1 - \bar{\alpha}_3} = 0.9/0.9772 = 0.9210, \\ x_3 - 0.9210 \varepsilon_\theta &= 1.300 - 0.9210 \times 1.301 = 1.300 - 1.198 = 0.102, \\ 0.102/\sqrt{\alpha_3} &= 0.102/0.3162 = 0.3226, \\ x_2 &= 0.3226 + \sigma_3 z_3 = 0.3226 + 0.7199 \times 0.60 = 0.3226 + 0.4319 = 0.7545.\end{aligned}$$

Step 2, $k = 2$. The sample now sits at **0.7545**, much closer to **+0.6** than to **-0.5**, and the same computation says so: with $\sqrt{\bar{\alpha}_2} = 0.6708$ and $1 - \bar{\alpha}_2 = 0.55$, the exponents come out -0.1127 for **+0.6** and -1.0799 for **-0.5**, a difference of **0.9672**, so $e^{0.9672} = 2.630$ and the weights are $2.630/3.630 = 0.7245$ and **0.2755**. The denoiser has started committing: $\hat{x}_0 = 0.7245 \times 0.6 + 0.2755 \times (-0.5) = 0.4347 - 0.1378 = 0.297$. Converting and updating:

$$\begin{aligned}\varepsilon_\theta &= \frac{0.7545 - 0.6708 \times 0.297}{0.7416} = \frac{0.7545 - 0.1992}{0.7416} = \frac{0.5553}{0.7416} = 0.7487, \\ \text{coefficient} &= 0.5/0.7416 = 0.6742, & 0.7545 - 0.6742 \times 0.7487 &= 0.7545 - 0.5048 = 0.2497, \\ 0.2497/\sqrt{\alpha_2} &= 0.2497/0.7071 = 0.3531, \\ x_1 &= 0.3531 + \sigma_2 z_2 = 0.3531 + 0.3015 \times 0.20 = 0.3531 + 0.0603 = 0.4134.\end{aligned}$$

Step 3, $k = 1$. With $\sqrt{\bar{\alpha}_1} = 0.9487$ and $1 - \bar{\alpha}_1 = 0.1$, the modes are now far apart relative to the remaining noise, and the arithmetic is emphatic about it: the exponents are -0.1214 and -3.9405 , a difference of **3.8191**, so $e^{3.8191} = 45.55$ and the weights are $45.55/46.55 = 0.9785$ and **0.0215**. The denoiser commits almost fully: $\hat{x}_0 = 0.9785 \times 0.6 + 0.0215 \times (-0.5) = 0.5871 - 0.0108 = 0.576$. Converting and updating, with $\sigma_1 = 0$ so no noise is injected:

$$\begin{aligned}\varepsilon_\theta &= \frac{0.4134 - 0.9487 \times 0.576}{0.3162} = \frac{0.4134 - 0.5464}{0.3162} = \frac{-0.1330}{0.3162} = -0.4206, \\ \text{coefficient} &= 0.1/0.3162 = 0.3163, & 0.4134 - 0.3163 \times (-0.4206) &= 0.4134 + 0.1330 = 0.5464, \\ x_0 &= 0.5464/\sqrt{\alpha_1} = 0.5464/0.9487 = 0.5760.\end{aligned}$$

The run went $1.300 \rightarrow 0.7545 \rightarrow 0.4134 \rightarrow 0.576$ and landed on the **+0.6** mode, not on the average **0.05** of the two. That is behavior a Gaussian policy could never produce; run it again with a different x_3 and you land on **-0.5**, roughly half the time.

Two things to extract. Watch the implied clean-data guesses: $0.136 \rightarrow 0.297 \rightarrow 0.576$. The denoiser hedges when the sample is noisy and commits as the noise clears; nobody programmed that coarse-to-fine mode selection, it falls out of training across noise levels. And the last step is special: because $\bar{\alpha}_1 = \alpha_1$, the update reduces algebraically to $x_0 = \hat{x}_0$ exactly, so the final denoising step is the conversion from noise prediction to clean-data prediction.

6.4.3 DDIM: the same network, far fewer steps

Ancestral sampling costs N_{diff} network evaluations — 100 forward passes per chunk at $N_{\text{diff}} = 100$. We need this cheaper.

DDIM (denoising diffusion implicit models; Song et al., 2020) — a sampler for the DDPM you just built, not a different model — is the standard answer, and its remarkable property is that it needs *no retraining*: the authors construct non-Markovian forward processes that share DDPM's training objective, so an existing checkpoint can simply be sampled differently. Mechanically it is two lines. Choose a subsequence of levels $\{k_1 < k_2 < \dots < k_K\} \subset \{1, \dots, N_{\text{diff}}\}$, typically $K = 10$ evenly spaced. At each chosen level, estimate the clean sample,

$$\hat{x}_0 = \frac{x_k - \sqrt{1 - \bar{\alpha}_k} \varepsilon_\theta(x_k, k)}{\sqrt{\bar{\alpha}_k}},$$

then re-noise it straight down to k' , the next level *below* k in your chosen subsequence — the subsequence was written in increasing order, so sampling walks it backwards:

$$x_{k'} = \sqrt{\bar{\alpha}_{k'}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{k'}} \varepsilon_\theta(x_k, k).$$

No fresh noise is injected, which makes the sampler deterministic: one starting noise vector maps to exactly one output. That is the $\eta = 0$ case, where η is DDIM's one knob — how much fresh noise to re-inject per step. At $\eta = 0$ you get the deterministic update above; at $\eta = 1$ the noise goes back in and you recover DDPM's ancestral update exactly, with everything in between available. Song et al. report comparable quality "10× to 50× faster in terms of wall-clock time."

Determinism is the under-appreciated half of the win for a controller, but be precise about what is deterministic. DDIM is deterministic *as a function of the starting noise vector* — call it x_{k_K} , the draw you make at the top of the subsequence. Fix x_{k_K} and the same observation returns bit-for-bit the same chunk. It is *not* deterministic as a function of the observation alone, and that is where the mental model usually goes wrong: draw a fresh x_{k_K} every control tick, as the pseudocode above and every default implementation do, and DDIM will return a different mode on two near-identical consecutive observations, judder and all.

What determinism buys you is a *handle*. Reuse one x_{k_K} across consecutive ticks and the arm commits to a single mode through the reach instead of re-rolling the dice thirty times a second; draw a fresh one per episode and the multimodality is still there, across *episodes*, where you wanted it. Chapter 7 reuses this handle for flow matching.

6.4.4 Why the step count is a control-systems problem

Do the arithmetic, because it explains a design choice you would otherwise find arbitrary. Chi et al. (2023) report that "using DDIM with 100 training iterations and 10 inference iterations enables 0.1s inference latency on a Nvidia 3080 GPU." Run that policy naively — one action per inference at 30 Hz — and your budget is $1/30 = 33$ ms per action against a cost of 100 ms. You are over by a factor of three, and no sampler cleverness closes a 3× gap.

Now run it the way Diffusion Policy does. Predict a chunk of $T_p = 16$ actions and execute the first $T_a = 8$. At a 10 Hz action rate the chunk spans 1.6 s and you execute 0.8 s of it before re-planning: 0.1 s of planning every 0.8 s, a 12.5% duty cycle, with the rest free for the encoder and the control loop. Same network, same sampler, problem gone. Write this down: **chunking, not sampler speed, is what makes iterative-refinement policies real-time.**

PITFALLS

Confusing k with t . The diffusion index k and the environment timestep t are unrelated axes. A full denoising run — all K steps — happens *between* two environment ticks and produces one chunk covering many values of t . Most codebases put sinusoidal embeddings on both axes; check which one yours indexes before debugging anything else.

Too few steps. Dropping K is the first thing everyone tries when latency hurts. Cut it far enough and samples get coarse and start landing *between* modes — the failure you adopted diffusion to avoid. Where that happens depends on your data, so measure it rather than trusting a number: sample one observation 50 times at $K = 10, 5$, and 2 , and look at the spread.

6.5 The score view, in one section

There is a second way to describe all of this, and you need it because half the literature and most of the variable names in diffusion codebases come from it.

The **score** of a density p is the gradient of its log, $\nabla_x \log p(x)$. Do not overthink it: at any point it is a vector pointing where the density increases fastest, its length saying how steeply — an uphill arrow toward where the data is dense. It does not involve the normalizing constant of p , because the log makes that constant additive and the gradient kills it, which is how score-based methods sidestep the partition function.

Knowing the score everywhere would be enough to sample: start anywhere, step uphill repeatedly, add a little jitter so you do not collapse onto the single highest peak. That is **Langevin dynamics**. The connection to what we built is exact algebra: since $x_k = \sqrt{\bar{\alpha}_k}x_0 + \sqrt{1 - \bar{\alpha}_k}\epsilon$, the optimal noise predictor satisfies

$$\nabla_{x_k} \log q(x_k) = - \frac{\epsilon_\theta(x_k, k)}{\sqrt{1 - \bar{\alpha}_k}}.$$

IN PLAIN WORDS

The left side is the score: the uphill direction of $q(x_k)$ — your dataset noised to level k — at the point x_k . The right side is the network's noise prediction $\epsilon_\theta(x_k, k)$, flipped in sign and divided by $\sqrt{1 - \bar{\alpha}_k}$, the noise coefficient from Section 6.2. The two are the same vector up to a negative scale. The minus sign: the noise you added is exactly what dragged the sample away from the data, so home is the opposite direction. A network trained to answer "which way did you get pushed?" has, unasked, learned "which way is home?" Hence papers use "noise model", "score model", and "denoiser" interchangeably, and hence the DDPM update — step along $-\epsilon_\theta$, then add noise — looks so much like Langevin dynamics.

One more idea completes the picture, and Figure 6.3 illustrates it. Song & Ermon (2019) pointed out that estimating the score of raw data is hopeless: real data sits on a thin sliver of the space, and everywhere else any gradient estimate is garbage. Their fix was to estimate the score of the data *blurred at many noise levels* and anneal from heavy blur to light. That annealing is what your sampling loop does walking k down to 1, and what you watched numerically above as the denoiser's guess moved from a hedge at **0.136** to a commitment at **0.576**.

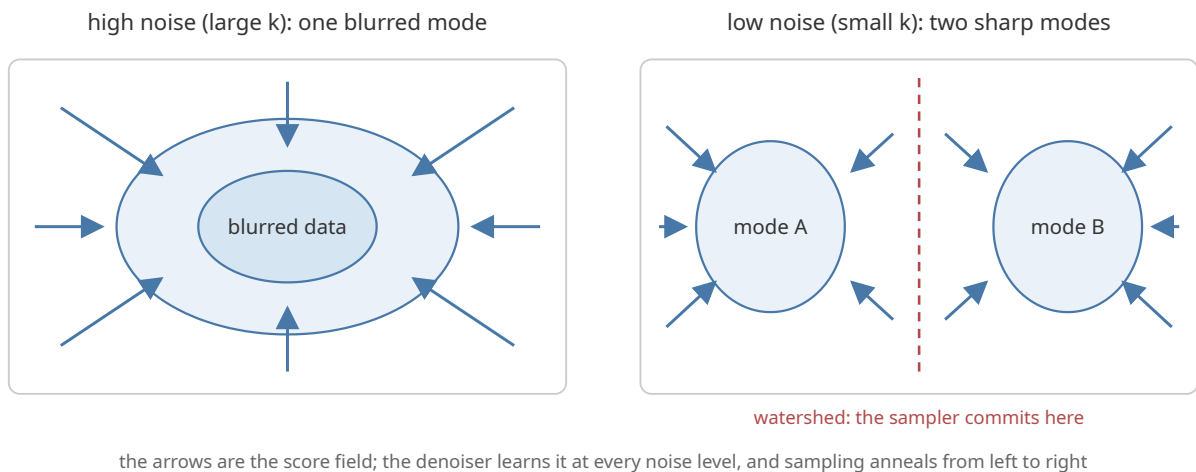


Figure 6.3 — Why sampling walks from high noise to low. On the left, heavy blur merges the two modes into one broad basin, so the uphill arrows are informative even far from the data. On the right the modes have separated and a red watershed appears: a sample that has drifted slightly left of center is pulled all the way into mode A. Commitment to a mode happens once, at the moment the basin splits.

WORDS YOU WILL HEAR

NFE — "number of function evaluations", denoiser forward passes per sample. Sampler papers report quality against NFE, not wall clock.

v-prediction — a third parameterization with target $v = \sqrt{\bar{\alpha}_k} \epsilon - \sqrt{1 - \bar{\alpha}_k} x_0$, blending ϵ - and x_0 -prediction. Better behaved at very low NFE; you meet it as a config flag.

VP and VE — variance-preserving (what we built) versus variance-exploding, where noise is added without shrinking the signal. Two coordinate systems for one family.

EMA weights — a moving average of the parameters, kept during training and used for all evaluation. Not optional here; raw weights sample noticeably worse.

6.6 From images to actions: Diffusion Policy

Everything so far was generic. Chi et al. (2023) turned it into a robot policy, and their adaptations are what you actually implement.

6.6.1 What gets diffused, and what does not

The first design decision is the one people most often get backwards. What is denoised is the **action chunk** — not pixels, not a latent image, not a single action. In their notation the policy takes the latest T_o steps of observation data O_t and predicts T_p future actions, of which T_a are executed. The diffusion variable is the whole matrix $A_t \in \mathbb{R}^{T_p \times d_a}$, where d_a is the action dimension: sixteen rows, one per future timestep, each row as wide as your action space — twelve numbers for the bimanual LeHome arms, seven for a single arm, two for the pushT task in the paper. The whole matrix is denoised jointly.

The observation is *conditioning*: encoded once, fed to the denoiser at every step, never noised or generated. The loss shows the asymmetry:

$$\mathcal{L} = \text{MSE}(\epsilon^k, \epsilon_\theta(O_t, A_t^0 + \epsilon^k, k)).$$

Read that against L_{simple} before you conclude you have misunderstood one of them. Chi et al. write the noisy input schematically as $A_t^0 + \epsilon^k$, superscript 0 meaning the clean chunk and superscript k the noise drawn for level k — both superscripts are noise levels, not timesteps. Expand ϵ^k with the schedule coefficients and you get exactly the mix $\sqrt{\bar{\alpha}_k} A_t^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon$ from Section 6.2. The only genuine change from L_{simple} is the extra input O_t . Two consequences. The encoder runs once per chunk while the denoiser runs K times, so compute is dominated by a small network called many times, not a large one called once. And the model has no obligation to predict future images — it is a policy, not a world model. Chapter 8 shows how LeHome recovers a slice of world-model benefit with auxiliary heads.

A notation bridge, because the field is not consistent: Diffusion Policy's T_p is this book's chunk horizon H , LeRobot's diffusion config calls it `horizon`, and π -family configs call it `chunk_size`.

6.6.2 How the observation gets in

"Conditioning" is a word that hides real mechanism, so here are the three ways it is done, cheapest first.

Concatenation. Flatten the observation into a vector and glue it onto the denoiser's input — appended to every action token, or stacked along the channel axis. One line, and it works. Its weakness is that in a deep network the signal washes out: modeling the marginal action distribution is easier than attending to extra channels.

FiLM. Feature-wise Linear Modulation (Perez et al., 2018) is the workhorse, and what Diffusion Policy uses for its CNN denoiser. Concatenate the observation embedding with the embedding of the diffusion step k and push the pair through a small MLP that outputs a per-channel scale vector and shift vector. Then after each convolution block's normalization apply $h \leftarrow s \odot h + b$, where h is the block's activation, $\odot$ is elementwise multiplication, and s, b are

that scale and shift. (Papers write these as γ and β ; the collision with the discount and the noise schedule means nothing.) Chi et al. apply this "to every convolution layer, channel-wise", so the observation sets the gain of every feature at every depth.

Cross-attention. For a transformer denoiser, the action tokens are queries and the observation tokens — image patches, per-camera features, language tokens — are keys and values. You need this when conditioning is spatial or sequential rather than one summary vector, and it is what VLAs use in Chapter 8.

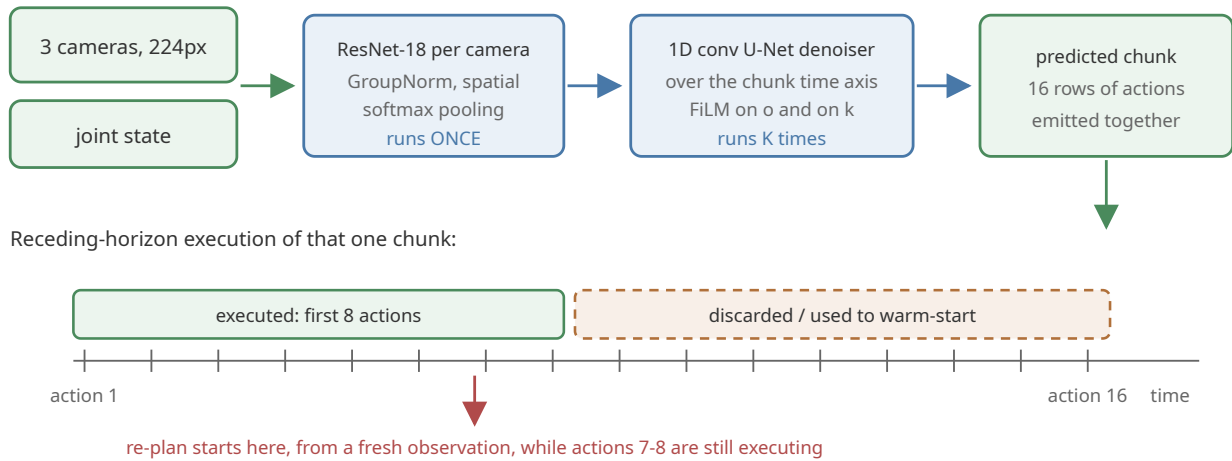
One detail traps everyone once. **The diffusion step k is conditioning too.** It goes through a sinusoidal embedding and an MLP, riding the same path as the observation. Omit it and the network cannot tell a nearly-clean sample from nearly-pure noise, so it learns an average correction and samples come out blurred. If your policy trains to a plausible loss and samples garbage, check this first.

6.6.3 Receding-horizon execution

The chunk is predicted, but not all of it is used. Diffusion Policy re-plans continuously and executes only the first T_a actions, warm-starting the next inference with the previous prediction. Chi et al. found "the action horizon of 8 steps to be optimal for most tasks that we tested", with $T_o = 2$ and $T_p = 16$ in the released configs.

Why not the whole chunk? The tail of a long open-loop plan is stale; the world has moved. Why not one step? Latency, as computed in Section 6.4, and temporal consistency: a policy re-sampled every tick can flip modes mid-reach, while a chunked one commits to a coherent motion. Figure 6.4 puts the pipeline together.

Executing a fixed fraction and re-planning is precisely **model-predictive control**, and if you have written an MPC loop the structure will feel familiar. Where it breaks: MPC optimizes at run time against an explicit dynamics model and cost function, and can say why it chose what it chose. The diffusion policy runs no optimization and holds no model; it samples from a distribution fitted offline. You get MPC's robustness to staleness, none of its interpretability.



At 10 Hz the chunk spans 1.6 s, you execute 0.8 s of it, and the 0.1 s of planning fits inside that window.

Figure 6.4 — The full pipeline, with the timing that makes it work. Compare the two "runs" labels in the top row: the expensive vision encoder is evaluated once per chunk, the small denoiser K times, so latency scales with K rather than image resolution. Below, re-planning starts before the executed segment ends — the robot never waits on the GPU.

6.6.4 Architectures, and what it bought

Two denoiser variants are offered: a 1D temporal convolutional U-Net over the action-time axis, FiLM-conditioned, and a transformer with cross-attention to the observation tokens. Their guidance is direct: "we recommend starting with the CNN-based diffusion policy implementation as the first attempt at a new task", reserving the transformer for "task complexity or high-rate action changes", and noting they "found the transformer to be more sensitive to hyperparameters".

The vision encoder is a per-camera ResNet-18 trained end-to-end with the policy, with two substitutions worth copying: GroupNorm instead of BatchNorm, which interacts badly with EMA weights, and spatial-softmax pooling instead of global average pooling, because where a feature is matters more than whether it is there.

The payoff across 12 tasks and 4 benchmarks was "an average improvement of 46.9%" over the prior state of the art, for Chapter 5's reason: diffusion "can express arbitrary normalizable distributions", multimodal ones included, while Gaussian policies average over modes. Two smaller findings that will save you a week: "position-control action space consistently outperforms" velocity control, and the square cosine schedule works best.

```
# LeRobot diffusion policy: the fields that matter (check names against your version)
n_obs_steps      = 2      # T_o -- how many past frames the encoder sees
horizon          = 16     # T_p -- chunk length that gets denoised (pi-family: "chunk_size")
n_action_steps   = 8      # T_a -- how many of those rows actually reach the motors
num_train_timesteps = 100 # N_diff -- levels used to BUILD training batches
beta_schedule     = "squaredcos_cap_v2" # the cosine schedule; do not use "linear" at N=100
prediction_type   = "epsilon" # eps-prediction, as in Section 6.3
noise_scheduler_type = "DDIM" # swap DDPM -> DDIM to decouple sampling from training
num_inference_steps = 10   # K -- denoiser calls per chunk. Latency scales with THIS, not N_diff.
```

ON YOUR RIG

You trained this policy on your SO-101 cube episodes in Chapter 5; here you take it apart. `num_train_timesteps` and `num_inference_steps` are different knobs — the first shapes training batches, the second is your latency bill — so leave training at 100 and swap the sampler from DDPM to DDIM at 10. Time both, and confirm the success rate holds. Then sample one held-out observation 30 times, drawing a fresh noise vector for each of the 30 (that is what produces the clusters), and plot the chunks: identical chunks mean a unimodal fit, distinct clusters mean Chapter 5's multimodality on your own data.

IN THE LEHOME SOLUTION

LeHome's policy contains no diffusion — it is a Gemma-300M flow-matching action expert emitting 30-step chunks with 10 Euler steps (Larchenko, 2026). Yet nearly every structural choice here survives into it: the chunk is the generated object, the observation conditions without being generated, iterative refinement produces the sample, execution is receding-horizon. Even fine detail transfers — Diffusion Policy warm-starts each inference with the previous chunk, and LeHome's "soft inpainting" between chunks is the same idea, upgraded into a tuned inference-time knob (Chapter 12). Read Chapter 7 as a change of parameterization, not of plan.

6.7 Why we do not stop here

Diffusion policies work. They are also, once you have built one, more complicated than they need to be. Three complaints, in increasing order of importance.

Bookkeeping. Count what you must get right: a β schedule, its derived α and $\bar{\alpha}$ tables, a choice of σ_k , two samplers with a subsequence rule for the second, and an ϵ -versus- x_0 -versus- v flag. None of it is hard, all of it is surface area for bugs, and none of it is about robots.

Curved paths. Look at Figure 6.1 again as geometry. The coefficients $\sqrt{\alpha_k}$ and $\sqrt{1 - \alpha_k}$ move at very different rates at different k , so the trajectory from data to noise bends. Cutting from 100 steps to 10 approximates that curve with 10 straight segments, with error proportional to how much it bends. This is the real reason few-step sampling degrades; on a straight path a coarse integrator would be nearly exact.

Stochasticity you did not ask for. The sampler injects fresh noise at every step, and DDIM removes it at no cost in quality — a sign that the noise was never load-bearing in the first place. Be careful what you conclude from that,

though. The other source of randomness, the arbitrary pairing of a noise vector with a data point, is not a diffusion problem: flow matching inherits it unchanged, and Chapter 7 says so plainly rather than pretending otherwise.

Now the wish: a straight-line path from noise to data at constant speed, one loss and one integrator with no schedule tables, ten steps that are accurate because the path they approximate is straight by construction rather than because the schedule was tuned. That is flow matching, and it is Chapter 7 — a smaller idea than diffusion, which is why the π -family and LeHome use it. Chapter 8 then puts a vision-language model in front of it, Chapter 9 confronts what all this does to PPO, and Chapter 12 returns to the inference-time knobs the receding horizon opened here.

CHECKPOINT

- Explain without notation why trading one hard sampling problem for many easy denoising regressions is a good deal, and why destruction needs no learning.
- Write the closed-form jump $\mathbf{x}_k = \sqrt{\bar{\alpha}_k}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_k}\boldsymbol{\epsilon}$ from memory, name every symbol, and say why training never simulates the chain.
- Implement $\mathcal{L}_{\text{simple}}$ and the ancestral loop, and say what each coefficient in the update undoes.
- Relate noise prediction to the score, and use that to explain why sampling walks from high noise to low.
- Describe what Diffusion Policy diffuses, how the observation conditions the denoiser, and why k must be conditioning too.
- Compute whether a chunked policy fits a real-time budget, and explain why chunking rather than sampler speed is what makes it fit.

Where to go deeper. Read Ho et al. (2020) for the forward process, $\mathcal{L}_{\text{simple}}$, and Algorithms 1 and 2 — the two boxes you will implement; skip the ELBO derivation on a first pass, since none of it appears in working code. Then Song et al. (2020) for DDIM and its non-Markovian construction. Chi et al. (2023) is the one to read closely and repeatedly, especially the ablations on action horizon, action space, and architecture. Appendix A gives the full reading order.

Chapter 7 — Flow matching

Chapter 6 gave you a policy that can represent more than one right answer. It also gave you a pile of machinery: a noise schedule $\beta_1 \dots \beta_{N_{diff}}$, tables of α_k and $\bar{\alpha}_k$, a sampling variance σ_k , a linear-versus-cosine decision, and a second sampler (DDIM) bolted on because the first was too slow for a robot.

This chapter is a reformulation that keeps every benefit and throws most of that machinery away. It is called **flow matching**: instead of a noisy, curved, many-step journey, you connect noise to data by a straight line and learn how fast to walk along it. It is what π_0 , $\pi_{0.5}$, and $\pi_{0.6}^*$ use, and by the end you will be able to write the training step and the sampler from memory, in about eight lines.

The chapter also delivers the bad news this book has been walking toward since Chapter 2. For this model class, the quantity every policy-gradient algorithm is built on — $\log \pi_\theta(a | s)$ — is not merely expensive. It is unavailable. Seeing exactly why, and exactly what survives, is the hinge that Chapters 9 through 12 turn on.

7.1 The same goal, a straighter road

Diffusion in three sentences. A fixed forward process destroys a clean action chunk by adding Gaussian noise in many small steps, until only noise remains. A network learns to predict the noise added at a given level, which is the same as learning one small step back toward the data. Generation starts at pure noise and runs that one-step-back network repeatedly until a clean chunk falls out.

7.1.1 What we actually want

What we want is narrow and practical: turn noise into a valid action chunk in as few **network evaluations** — forward passes through the model — as possible, because each costs wall-clock time and a robot has a control budget. Diffusion Policy reports 0.1 s of inference latency on an RTX 3080 using 10 DDIM steps (Chi et al., 2023). Ten evaluations fit a receding-horizon schedule. A hundred do not.

7.1.2 The obvious thing to try, and exactly how it fails

The obvious thing to try is to keep the trained DDPM and run fewer steps: 10 big ones instead of 1000 small ones. This fails, and it is worth being precise about why. The DDPM update is a step along a *curve* through action space. Its direction depends on the current noise level: at high noise the field pulls broadly toward the whole data distribution, at low noise sharply toward one specific mode, and in between the direction rotates continuously. Taking a step means approximating a curved arc by a straight chord. That error grows with the square of the step size, so a step 100 times bigger is roughly 10,000 times worse per step while saving only a factor of 100 in count. In Chapter 6's linear schedule $\sqrt{\bar{\alpha}_k}$ moves from 0.9471 at $k = 100$ to 0.2803 at $k = 500$; one jump across that gap asks the network to be right about a trajectory it never traversed.

Chapter 6's answer was DDIM (Song et al., 2020): a non-Markovian sampler sharing DDPM's training objective, so you can skip steps on an already-trained checkpoint, reported as 10× to 50× faster in wall clock. A real and widely used fix — but look at its shape. You trained one process and sample with a different one, on the strength of a non-obvious equivalence, and that is one more thing in the codebase that can be configured wrong.

7.1.3 The fix, and why it works

Flow matching asks a different question: instead of patching the sampler so it survives a curved path, why not make the path straight? Take a noise sample z and a data sample a . Connect them with a straight segment, traverse it at constant speed, and train the network to predict that speed — that *velocity* — at every point along the line. There is no schedule, because there is nothing to schedule — Chapter 6's β_k and α_k simply do not appear here, and with them goes the collision with Chapter 4's temperature β and Chapter 11's damping α . No noise is injected during sampling,

because the path is deterministic; no second sampler is needed, because plain Euler integration is the right tool. Figure 7.1 shows the whole arrangement at once.

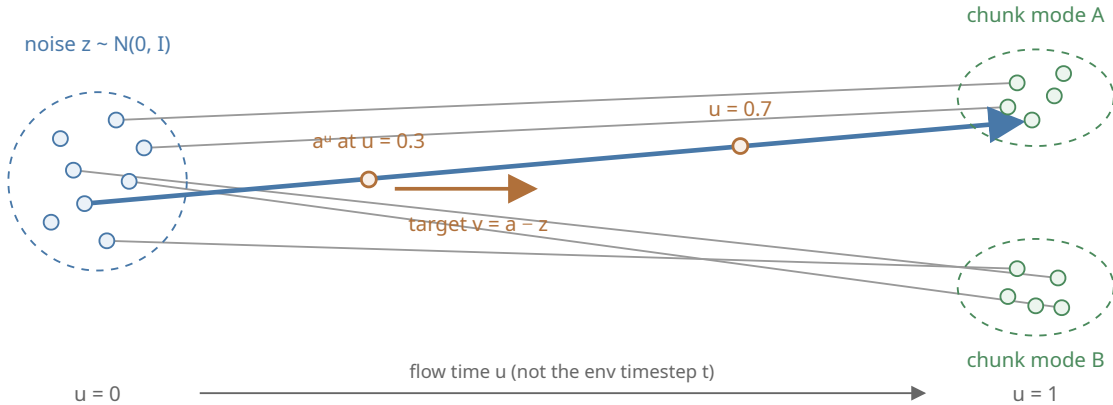


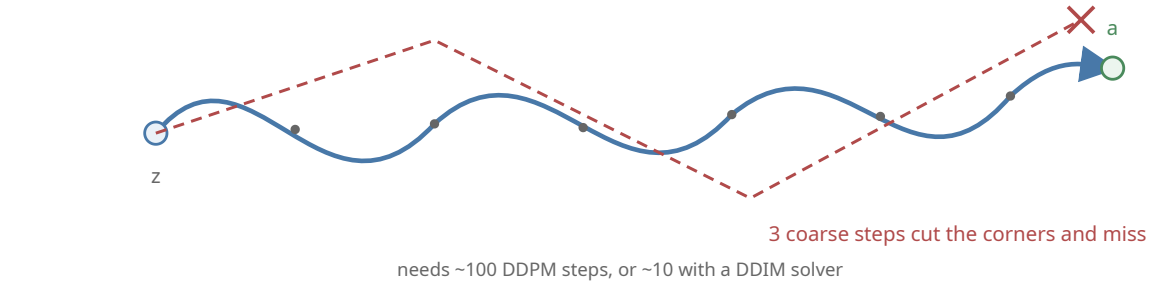
Figure 7.1 — Flow matching as transport. Each noise sample on the left joins one action chunk on the right by a straight segment, and different draws land in different modes — nothing is averaged in action space. The highlighted path marks the interpolant a^u at $u = 0.3$ and $u = 0.7$; the target $a - z$ (derived in Section 7.3) is the same vector at both, because it is constant along the line. Watch the crossings near the left: that is where the learned field must average.

Why does that help? Because a first-order integrator is *exact* on a straight line traversed at constant speed. If you sit at position $\mathbf{p}$ moving at constant velocity $\mathbf{v}$, then after time Δ you are at $\mathbf{p} + \Delta\mathbf{v}$ — not approximately, exactly, whether Δ is one thousandth or the whole interval. All the step-size error that punished the coarse DDPM sampler came from curvature, and curvature is what we removed. Figure 7.2 draws this side by side.

An honest caveat, up front. The straightness is exact for each individual training pair $(\mathbf{z}, \mathbf{a})$, and only approximate for the field the network actually learns, because that field averages over many pairs whose lines cross. Section 7.3 explains this, and it is why practitioners use $K = 10$ steps rather than one.

An analogy, with its limit. Driving to a town 60 km away. On a straight highway you set the cruise control, glance at the map ten times, and arrive. On a mountain road of switchbacks you must steer continuously; glance ten times and you end in a ditch. Diffusion's probability path is the switchbacks; flow matching's is the highway. *Where it breaks down:* a real highway exists before you drive on it, so staying on it is a control problem. Here there is no road — the road is the learned field itself, and if the network is wrong, the straight path you imagined is not the path you travel.

Diffusion: curved path, noise injected at every step



Flow matching: straight path, no noise during sampling

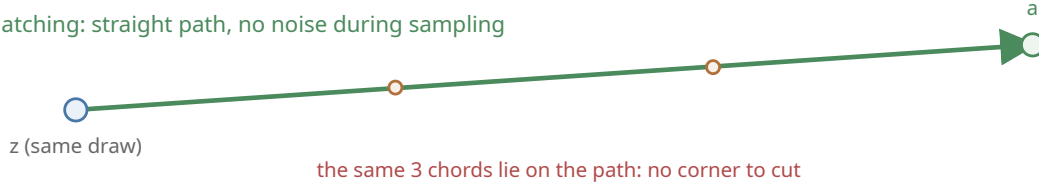


Figure 7.2 — The same two endpoints, two paths between them. On top the diffusion trajectory bends, so the red dashed three-step approximation cuts corners and lands away from the target (the X). On the bottom the path is straight, so the same three chords sit exactly on it and arrive. This picture is the whole argument for flow matching.

7.2 Flow time, and why it is not the environment timestep

A warning before any equations: this is the most common confusion in the material, and everyone hits it. Flow matching introduces a new variable, the **flow time** $u \in [0, 1]$: a continuous progress coordinate measuring how far generation has gotten from noise toward data. At $u = 0$ you are at pure noise; at $u = 1$ at a data point. It has nothing whatsoever to do with the environment timestep t .

Papers write flow time as τ (the π_0 paper) or, disastrously, as t — in the generative-modeling literature there is no other "time" competing for the letter. In this book u is always the flow time and t is always the environment timestep, without exception.

Hold the two apart like this. The environment timestep t indexes the MDP: at t the robot is in state s_t , takes action a_t , gets reward r_t , advances. It is real, irreversible time. The flow time u indexes a computation that runs to completion *inside a single control decision*: to produce the chunk $a_{t:t+H}$ at one timestep t , you sweep u from 0 to 1, then throw u away and execute. The MDP never sees u ; the ODE never sees t . Chapter 6's index k was the same kind of private axis, just discrete and running from high noise down to low.

7.2.1 The interpolant

Given a noise draw $z \sim \mathcal{N}(0, I)$ shaped like the action chunk and a real chunk a from the demonstrations, the **interpolant** is the point you get by mixing them in proportion u :

$$a^u = (1 - u)z + ua.$$

A weighted average whose weights sum to one. At $u = 0$ it is exactly z ; at $u = 1$ exactly a ; in between it slides linearly. Compare Chapter 6's closed-form noising, $a_k = \sqrt{\bar{\alpha}_k} a_0 + \sqrt{1 - \bar{\alpha}_k} \varepsilon$, whose coefficients are square roots of schedule quantities and do *not* sum to one. Flow matching's are $1 - u$ and u . There is nothing to look up.

Make it concrete with a scalar. Suppose the demonstrated action is $a = 0.8$ (one normalized joint command) and the drawn noise is $z = -0.5$. Then:

$$\begin{aligned} a^{0.0} &= 1.0(-0.5) + 0.0(0.8) = -0.500 \\ a^{0.3} &= 0.7(-0.5) + 0.3(0.8) = -0.350 + 0.240 = -0.110 \\ a^{0.5} &= 0.5(-0.5) + 0.5(0.8) = -0.250 + 0.400 = 0.150 \end{aligned}$$

$$a^{0.7} = 0.3(-0.5) + 0.7(0.8) = -0.150 + 0.560 = 0.410$$

$$a^{1.0} = 0.0(-0.5) + 1.0(0.8) = 0.800$$

Notice the spacing. From $u = 0$ to $u = 0.3$ the value moved by $-0.110 - (-0.500) = 0.390$; from $u = 0.3$ to $u = 0.5$, a gap of 0.2, it moved by $0.150 - (-0.110) = 0.260$. Both are the gap in u times **1.3**. The speed along this line is constant and equals **1.3**. Keep that number.

7.3 The velocity field and the CFM loss

The speed is not a coincidence. Differentiate the interpolant with respect to flow time:

$$\frac{da^u}{du} = \frac{d}{du} [(1-u)z + ua] = -z + a = a - z.$$

In the scalar example $a - z = 0.8 - (-0.5) = 1.3$, exactly the speed we measured. Crucially, this derivative does not depend on u : along one training pair's line the velocity is a *constant vector*. That is the whole design.

So we train a network — the **velocity field** $v_\theta(a^u, u, o)$, which takes the current point, the current flow time, and the observation, and returns a vector shaped like the action chunk — to output that constant. The loss is plain mean-squared error, called the **conditional flow matching** (CFM) loss because the target is conditioned on one specific data sample a rather than on the whole distribution:

$$\mathcal{L}(\theta) = \mathbb{E}_{(o,a) \sim \mathcal{D}, z \sim \mathcal{N}(0,I), u \sim \mathcal{U}[0,1]} \|v_\theta(a^u, u, o) - (a - z)\|^2, \quad a^u = (1-u)z + ua.$$

IN PLAIN WORDS

Draw a real observation-and-chunk pair (o, a) from your demonstration dataset $\mathcal{D}$; draw a noise chunk z from a standard Gaussian, same shape as a ; draw a flow time u uniformly between 0 and 1. Mix noise and data in proportion u to get the point a^u . Ask the network, given that mixed point, that flow time, and that observation, to output the vector $a - z$ — the straight-line direction and speed from the noise you drew to the data you drew. The double bars are squared Euclidean distance summed over all $H \times d$ numbers in the chunk; $\mathbb{E}$ says average this over the whole random draw. That is it.

There is no likelihood, no adversary, no schedule.

7.3.1 The objection you should be raising

A careful reader asks this immediately, and it is the right question. Many pairs (z, a) pass through the same point at the same flow time, each demanding a different velocity, and the network can output only one vector per input. So what does it learn?

Make the collision concrete. The pair $z = -0.5, a = 0.8$ sits at $a^{0.5} = 0.150$ and demands velocity **1.3**. A different pair, $z = 0.4, a = -0.1$, sits at $0.5(0.4) + 0.5(-0.1) = 0.150$ — the same point — and demands $-0.1 - 0.4 = -0.5$. Two examples, identical network input, targets **1.3** and -0.5 .

Mean-squared error has a known answer here: its minimizer is the conditional average of the targets. If both pairs are equally likely, the network learns $(1.3 + (-0.5))/2 = 0.4$ at that input. In general,

$$v_\theta^*(x, u, o) = \mathbb{E}[a - z \mid a^u = x, u, o].$$

Your alarm should now be ringing, because *averaging* is precisely the disease Chapter 5 said generative models were supposed to cure. A Gaussian policy fails because it averages two valid grasps into a collision. Are we not doing the same thing?

No. The Gaussian averages the *actions* and emits the average; flow matching averages the *velocities at intermediate points* and then integrates. At small u the sample is mostly noise, the bundles of paths heading toward different modes genuinely overlap, and the averaged velocity is a broad "head toward the data" push that is correct for all of them. As u grows, the paths heading to mode A and those heading to mode B separate in action space. The network's input now

tells it which bundle it is in, so the conditional average there is no longer a compromise: it is the velocity of the one bundle that reaches that point. The commitment to a mode happens gradually, and by $u = 1$ it is total. Look back at Figure 7.1: interpolants crowd and cross near the noise end, and separate into two clean mode-bundles near the data end.

The theorem behind this — three lines of calculus in the source papers — says the averaged field is exactly the field that transports the standard Gaussian onto the data distribution (Lipman et al., 2023; Liu et al., 2023). Not approximately: with the exact conditional expectation, integrated exactly, samples would be distributed exactly as the data is. Every practical error comes from an imperfect network and a coarse integrator, not from the recipe.

One consequence deserves naming, the honest footnote to Section 7.1: because the bundles cross, the *averaged* field is not perfectly straight even though every conditional path is. That is why $K = 1$ fails in practice and $K = 10$ works. The **rectified flow** line of work (Liu et al., 2023) attacks exactly this: re-pair each generated a with the z that produced it and retrain, which removes most crossings and lets you cut K further.

7.4 Sampling: an ODE and a for-loop

Training taught the network a velocity at every point and flow time. Sampling is: start at noise and follow it. That is an **ordinary differential equation** (ODE) — an equation giving a quantity's rate of change as a function of its current value, solved by stepping forward:

$$\frac{da}{du} = v_\theta(a, u, o), \quad a(0) = z \sim \mathcal{N}(0, I) \quad \implies \quad a^{u+1/K} = a^u + \frac{1}{K} v_\theta(a^u, u, o).$$

IN PLAIN WORDS

The left equation says: the rate at which the sample changes as flow time advances is whatever the network says the velocity is right there, starting from a fresh noise draw z . The right equation is how you solve it on a computer — an **Euler step**: ask for the velocity where you are, multiply by the step size $1/K$, add it to your current point, advance u by $1/K$, repeat. K is the number of steps, so K network evaluations per chunk; the π -family uses $K = 10$, which π_0 writes as $\delta = 0.1$. After K steps you stand at $u = 1$, on the emitted chunk.

```
# --- one training step (batch of 1, for clarity) ---
o, a = batch                # a: action chunk, shape (H, d)
z = normal(0, 1, a.shape)   # noise, same shape as the chunk
u = uniform(0, 1)           # FLOW time -- not the env timestep t
a_u = (1 - u) * z + u * a   # straight-line interpolant
target = a - z              # constant along this pair's line
loss = mse(v_theta(a_u, u, o), target)
loss.backward(); opt.step()

# --- sampling: K Euler steps of the ODE ---
a = normal(0, 1, (H, d))    # start at u = 0: pure noise
for k in range(K):          # K = 10 in the pi-family
    a = a + (1 / K) * v_theta(a, k / K, o) # one Euler step
return a                   # the chunk a_{t:t+H}, ready to execute
```

Compare Chapter 6's sampler, which needed α_k , $\bar{\alpha}_k$, σ_k , and a fresh noise draw inside the loop. Here: one division, one addition, no noise in the loop.

7.4.1 Worked example, part one: a training step by hand

A scalar action, so you can check every number. The demonstration action is $a = 0.8$; we draw noise $z = -0.5$ and flow time $u = 0.3$.

$$a^u = (1 - 0.3)(-0.5) + (0.3)(0.8) = 0.7 \times (-0.5) + 0.3 \times 0.8$$

$$= -0.350 + 0.240 = -0.110$$

$$\text{target} = a - z = 0.8 - (-0.5) = 1.300$$

Suppose the half-trained network outputs $v_\theta(-0.11, 0.3, o) = 0.9$. The loss for this sample is

$$\mathcal{L} = (0.9 - 1.3)^2 = (-0.4)^2 = 0.160$$

and its gradient with respect to the network's output is

$$\frac{\partial \mathcal{L}}{\partial v} = 2(0.9 - 1.3) = 2 \times (-0.4) = -0.8,$$

a negative number, which backpropagation turns into an update pushing the output *up*, toward 1.3. Nothing here is special to flow matching — it is the regression loss you would write for any supervised problem, which is the entire point.

7.4.2 Worked example, part two: sampling in three Euler steps

Now generate. With observation o , we draw a fresh noise value that happens to be $z = -0.5$ again, and use $K = 3$ so the arithmetic stays visible; the step size is $1/3 = 0.3333$. The trained network returns these velocities, which differ slightly from step to step because the marginal field is only approximately straight:

```
start:   u = 0.0000   a = -0.500           (this is z)

step 1:  v_theta(-0.500, 0.0000, o) = 1.20
         a <- -0.500 + (1/3)(1.20) = -0.500 + 0.400 = -0.100
         u <- 0.3333

step 2:  v_theta(-0.100, 0.3333, o) = 1.35
         a <- -0.100 + (1/3)(1.35) = -0.100 + 0.450 = 0.350
         u <- 0.6667

step 3:  v_theta(0.350, 0.6667, o) = 1.29
         a <- 0.350 + (1/3)(1.29) = 0.350 + 0.430 = 0.780
         u <- 1.0000

emit:    a = 0.780           (the demonstrated action was 0.800)
```

Two things to notice. First, had the field been exactly constant at 1.3, the answer would have been $-0.5 + 1.3 = 0.800$ in *any* number of steps, including one; the 0.02 discrepancy is entirely due to the velocity varying along the path. Second, one step alone would have given $-0.5 + 1 \times 1.20 = 0.700$, an error of 0.08 rather than 0.02. A handful of steps buys a large accuracy gain with sharply diminishing returns after — which is why $K = 10$ is standard and $K = 100$ is not.

7.4.3 Determinism, and why it matters later

After the initial draw of z the sampling loop contains no randomness. Same o , same z , same chunk, bit for bit. All of the policy's stochasticity lives in that one draw — which is worth more than it looks. "Sample N candidate chunks" is precisely "draw N values of z and integrate each," with no hidden coupling between candidates — which is what makes best-of- N selection in Chapter 12 clean to implement. It also means you can deliberately *reuse* a z across consecutive control ticks to keep the motion coherent, instead of letting a fresh draw flip the policy to a different mode mid-reach. Chapter 12's real-time chunking and inpainting tricks lean on exactly this handle.

A second analogy, with its limit. If you have written a game or a physics simulation, this loop is familiar: `position += velocity * dt`, a fixed number of iterations. The velocity field is a pure function of position, progress, and context; the solver is a `for` loop with no state of its own. *Where it breaks down:* in a physics engine the velocity comes from real dynamics, and too large a `dt` makes the simulation explode. Nothing explodes here. Too

large a step just lands you slightly off the data manifold: a slightly wrong chunk, not a catastrophe. The failure mode is quiet degradation, which is worse than a crash because it never shows up in a log file.

ON YOUR RIG

LeRobot ships both families, so you can hold them side by side: the `diffusion` policy from Chapter 6 and the flow-matching `pi0` ports from this one. Train the diffusion policy on a handful of SO-101 teleop episodes, then read both configs and find the three knobs this chapter and the last one named: chunk length H , integration/denoising steps K , and how many actions execute before the policy is re-queried. A good evening's experiment: hold everything fixed and sweep the flow policy's K over 1, 2, 5, 10, 20 in open loop, and watch where the emitted trajectory stops changing. That number is your real inference budget.

7.5 Diffusion versus flow matching, honestly

You will meet both, often in the same repository. The comparison, dimension by dimension:

Training target. Diffusion regresses the *noise* ϵ added at a schedule-determined level (Ho et al., 2020); flow matching regresses a *velocity*, $\mathbf{a} - \mathbf{z}$. Close relatives — both point, up to scaling, away from the data — but the flow-matching target is constant along its path while the diffusion target's meaning shifts with $\bar{\alpha}_k$. Note that π_0 writes its interpolant with the opposite orientation, $\mathbf{A}^\tau = \tau\mathbf{A} + (1 - \tau)\epsilon$, so $\tau = 1$ means data (Black et al., 2024). Conventions genuinely differ between papers; when you read code, find the interpolant line first and derive everything else from it.

Path shape. Diffusion's path is curved and variance-preserving, its shape dictated by the schedule. Flow matching's is a straight line at constant speed. This is the substantive difference; everything else follows from it.

Sampling cost. Plain DDPM ancestral sampling uses roughly 100 to 1000 network evaluations; DDIM cuts that by a reported 10× to 50× in wall-clock time (Song et al., 2020). Flow matching uses forward Euler at $K = 10$, which π_0 states as "10 integration steps." Both land near 10 evaluations — but flow matching gets there with its own sampler, and diffusion by switching samplers.

Implementation complexity. Flow matching wins clearly, which is not a small thing when you are debugging a policy at 2 a.m. No β schedule, no $\alpha/\bar{\alpha}$ tables, no σ_k , no separate DDIM path. Its one real tunable is the distribution you draw $\mathbf{u}$ from, and π_0 does tune it: rather than uniform, a beta distribution emphasizing lower (noisier) flow times, $\text{Beta}(\frac{s-\mathbf{u}}{s}; 1.5, 1)$ with $s = 0.999$.

Empirical quality. Roughly a wash in the regimes robotics cares about. Diffusion Policy reported an average 46.9% improvement over prior state of the art across 12 tasks and 4 benchmarks using DDPM/DDIM (Chi et al., 2023); the π -family reports comparable results with flow matching. Nobody has shown one family to be fundamentally more expressive, and there is a reason: in DDIM's deterministic setting, diffusion sampling is an ODE solve on the probability-flow ODE. They are cousins, not rivals; flow matching is the cleaner parameterization of the same idea.

Who uses what. Diffusion Policy and its descendants use DDPM/DDIM. Modern VLAs use flow matching: π_0 is explicitly a "vision-language-action flow model" with $H = 50$ at up to 50 Hz, chosen for precision and multimodal capability at high control rates (Black et al., 2024). $\pi_{0.5}$ keeps the flow-matching action expert at 10 denoising steps per chunk (Physical Intelligence, 2025a); $\pi_{0.6}^*$ enlarges it to 860M parameters (Physical Intelligence, 2025b). The honest reading: the VLA world adopted flow matching as much for implementation simplicity and few-step accuracy as for raw sample quality.

WORDS YOU WILL HEAR

Continuous normalizing flow (CNF) — the model class this belongs to: a distribution defined by pushing noise through the solution of a learned ODE. Flow matching *trains* a CNF without ever computing its likelihood. **Probability-flow ODE** — the deterministic ODE whose solutions share a diffusion process's marginals; the bridge between the two families. **NFE (number of function evaluations)** — the standard sampler cost metric: forward passes per sample. **$K = 10$** means 10 NFEs. **Rectified flow / reflow** — retraining on model-generated $(\mathbf{z}, \mathbf{a})$ pairs to straighten the marginal field. **OT-CFM** — pairing $\mathbf{z}$ with $\mathbf{a}$ inside a minibatch by optimal transport instead of at random, another way to reduce crossings. **Hutchinson trace estimator** — the cheap randomized approximation to the trace of a Jacobian.

7.6 Why log-probability dies here

Now the pivot. PPO (Schulman et al., 2017) needs the **importance ratio** $\rho_t(\theta) = \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) / \pi_{\theta_{old}}(\mathbf{a}_t | \mathbf{s}_t)$ for every sampled action, differentiable in θ , so it can clip at $1 \pm \epsilon$; plain REINFORCE from Chapter 2 needs $\nabla_\theta \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)$. For a Gaussian that is a closed-form expression you evaluate in one line. So: what is $\log \pi_\theta(\mathbf{a} | \mathbf{o})$ for a flow policy?

It exists — worth saying, because it is tempting to assume otherwise. The integration is, in the continuous limit, an invertible map from $\mathbf{z}$ to $\mathbf{a}$, and the change-of-variables formula for continuous flows (Chen et al., 2018) gives it explicitly:

$$\log \pi_\theta(\mathbf{a} | \mathbf{o}) = \log \mathcal{N}(\mathbf{z}; \mathbf{0}, I) - \int_0^1 \nabla \cdot \mathbf{v}_\theta(\mathbf{a}(u), u, \mathbf{o}) du,$$

IN PLAIN WORDS

The left side is the log-probability that this policy assigns to the chunk $\mathbf{a}$ it just emitted, given observation $\mathbf{o}$. The first term on the right is the log-density of a standard Gaussian evaluated at $\mathbf{z}$ — the noise draw that produced $\mathbf{a}$, recovered by integrating the ODE *backward* from $\mathbf{a}$. That term is easy. The second term is the correction for how much the flow stretched or squashed volume along the way: $\nabla \cdot \mathbf{v}_\theta$, the **divergence** of the velocity field, is the sum of the partial derivative of each output component with respect to its own input component — positive where the field spreads samples apart, negative where it pulls them together. $\mathbf{a}(u)$ is the trajectory of the learned ODE with $\mathbf{a}(1) = \mathbf{a}$ and $\mathbf{z} = \mathbf{a}(0)$. The integral runs along that ODE path — not along the straight training interpolant $\mathbf{a}^u$, which is a different curve entirely.

7.6.1 Counting the cost, slowly

The formula is correct and unusable. Count why, one factor at a time, with LeHome's numbers: a bimanual SO-101 chunk with $H = 30$ and $d = 12$ is a point in $30 \times 12 = 360$ dimensions. Figure 7.3 lays out the same accounting visually.

First: the divergence is a trace. $\nabla \cdot \mathbf{v}_\theta$ is the trace of the Jacobian of $\mathbf{v}_\theta$ with respect to its 360-dimensional input — a 360×360 matrix, of which you need the 360 diagonal entries. Automatic differentiation gives one row per backward pass, so exact evaluation costs 360 backward passes, for one point on one path.

Second: it is an integral, not a value. You need the divergence at every point along the ODE path — in practice, at every integration step: $360 \times 10 = 3,600$ passes for one log-probability.

Third: PPO needs it repeatedly. $\log \pi_{\theta_{old}}$ is cacheable once at rollout time; $\log \pi_\theta$ changes every gradient step. With PPO's standard 4 epochs, that is one cached evaluation plus four fresh ones: $5 \times 3,600 = 18,000$ passes per chunk. For a minibatch of 256, $18,000 \times 256 = 4,608,000$ — about 4.6 million network passes per update, against $5 \times 256 = 1,280$ for a Gaussian policy. Roughly 3,600 times more compute for the same update.

Fourth: it must be differentiable through θ . PPO backpropagates through the ratio, so you are differentiating a quantity itself built from derivatives: second-order terms, and a graph spanning the entire integration.

Fifth: the cheap version is noisy. The standard escape is the Hutchinson trace estimator, a randomized estimate costing a handful of passes instead of 360. That makes the cost bearable — but consider what PPO does with the result. It forms a log-ratio, the difference of two such integrals, and clips at $\epsilon \approx 0.2$, while the estimator's standard deviation is comfortably larger than 0.2. The clip — the entire trust-region mechanism, the reason PPO is stable at all — would be zeroing gradients on estimator noise rather than on any real policy change. Coarse $K = 10$ integration then adds a bias that does not average away.

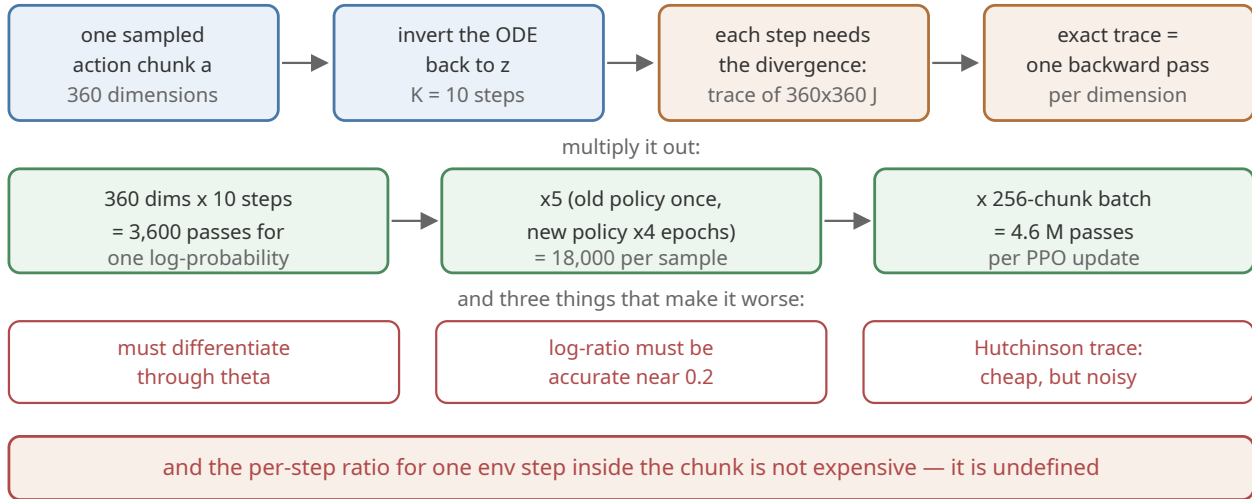


Figure 7.3 — The anatomy of one log-probability, for a LeHome-sized chunk. Read the top row left to right for where the work comes from, then the green row for the multiplication: roughly 4.6 million network passes per PPO update against about 1,280 for a Gaussian policy. The red boxes say why paying that bill would not give you a usable clip; the banner says why no amount of compute helps.

7.6.2 The factor that ends the discussion

Suppose you accepted all five costs. A sixth problem remains that no amount of compute fixes. PPO's surrogate is written in *per-timestep* ratios ρ_t : one state, one action, one ratio, one advantage. But the policy emits a chunk covering H environment steps as one indivisible sample. There is no $\pi_\theta(a_t | s_t)$ for the seventh action inside a chunk, because the model never defined a distribution over that action alone — only over the whole 360-dimensional object. The per-step ratios are not expensive. They are undefined.

One more analogy, with its limit. For a Gaussian policy the log-probability is like reading a field off a struct. For a flow policy it is like being handed the output of a long ETL pipeline and asked for the exact probability of the input record: run the pipeline backward, instrumenting every stage for how much it compressed or expanded the data. *Where it breaks down:* an ETL pipeline is discrete and finite, so its reverse pass is in principle exact. Here the forward map is invertible only in the continuous limit, so even the "exact" answer approximates something your 10-step sampler never computes.

The conclusion, stated plainly: log-probability-based RL on flow policies is a dead end. It is why the LeHome solution contains no PPO at all, and it is the observation Chapter 9 builds its entire side-by-side comparison on.

PITFALLS

Two classics, both of which cost people weeks. **First**, conflating flow time u with the environment timestep t . The ODE integration is an inner loop that runs to completion *inside one* control decision; u never appears in the MDP, is never discounted, and has no reward attached. If you find yourself writing γ^u , stop. **Second**, hoping to get log-probs "approximately" — a Hutchinson estimate, an ELBO surrogate, a per-step Gaussian proxy — and feed them to PPO anyway. Each individual estimate will look plausible in a debugger. The estimator noise is far larger than the ϵ -sized signal the clip needs, so the update direction is garbage even when nothing looks broken.

7.7 What survives

Here is the asymmetry that saves the enterprise, as a slogan: *densities are gone, but the loss is still a regression, and regressions can be weighted and conditioned.*

The CFM loss is an expectation of a per-sample squared error. Nothing in it evaluates a density, and nothing couples samples. So take Chapter 4's advantage-weighted regression weight $w_i = \exp(\hat{A}_i/\beta)$, attach it to sample i , and multiply:

$$\mathcal{L} = \mathbb{E} \left[w_i \|v_\theta(a_i^{u_i}, u_i, o_i) - (a_i - z_i)\|^2 \right]$$

IN PLAIN WORDS

The index i runs over samples in the batch; each carries its own observation o_i , recorded chunk a_i , noise draw z_i , and flow time u_i . The squared-norm term is exactly the CFM loss of Section 7.3, unchanged. The new factor $w_i = \exp(\hat{A}_i/\beta)$ is one positive number computed from that sample's estimated advantage $\hat{A}_i$, with temperature β setting how sharply good samples are favored. A chunk that beat the baseline gets $w_i > 1$ and pulls the velocity field harder toward itself; one that underperformed gets $w_i < 1$ and barely moves it. Weighted behavior cloning, where "cloning" means learning to denoise.

That is a full reinforcement-learning algorithm running on a generative policy, with no density in it. The second route is conditioning rather than weighting: append extra input to the model — in $\pi_{0.6}^*$, a literal text token reading "Advantage: positive" or "Advantage: negative" — train on all the data with its true label, and at inference ask for the positive one. That is RECAP, and Chapter 10 is devoted to it.

Notice the pattern, because it holds for the rest of the book: every technique from here on touches the regression loss or the sampler, never the density. Advantage engineering in Chapter 11 changes what $\hat{A}_i$ is; guidance and best-of- N in Chapter 12 change how you integrate the ODE and which z you keep; human interventions in Chapter 13 change which samples enter the batch and with what weight. Density evaluation: intractable. Weighted or conditioned regression: trivial.

IN THE LEHOME SOLUTION

LeHome's policy is a $\pi_{0.5}$ fine-tune for bimanual SO-101 shirt folding, and its action expert is exactly the flow-matching model of this chapter: $K = 10$ Euler steps producing $H = 30$ chunks at 30 Hz in simulation, 20 Hz in the real round (Larchenko, 2026). Chunks are 360-dimensional, which is the number Section 7.6's cost accounting used. PPO was ruled out for precisely the reasons above, and improvement runs entirely through the two survivors: AWR-style weighting of the CFM loss (Chapter 4) and RECAP-style advantage conditioning (Chapter 10). Nothing in the LeHome pipeline ever evaluates a log-probability.

CHECKPOINT

- Explain why a coarse Euler solve fails on a curved path but is nearly exact on a straight one, and why that motivates the straight-line interpolant.
- Write the interpolant $\mathbf{a}^u = (1 - u)\mathbf{z} + u\mathbf{a}$ and the CFM loss from memory, derive the target $\mathbf{a} - \mathbf{z}$, and name every symbol — including what u is and is not.
- Answer the crossing-paths objection: what the MSE minimizer converges to, and why averaging velocities is not the failure a Gaussian commits by averaging grasps.
- Implement the sampler, and explain why generation is deterministic given $\mathbf{z}$ and why Chapter 12 cares.
- Compare diffusion and flow matching on target, path shape, NFEs, and implementation complexity, and say which the π -family uses and why.
- Walk someone through the log-probability cost accounting, and finish with the argument that per-step ratios inside a chunk are undefined, not merely expensive.

Where to go deeper. Lipman et al. (2023) is the flow-matching paper and is unusually readable; go there for the conditional-versus-marginal argument Section 7.3 summarized in prose. Liu et al. (2023) gives the rectified-flow view and the reflow procedure. Chen et al. (2018) is the source of the divergence-integral formula, and skimming it makes Section 7.6's cost concrete rather than asserted. Black et al. (2024) shows flow matching wired into a VLA action expert, which is where Chapter 8 picks up. Appendix A sequences Lipman et al. and Black et al. with reading notes; the rectified-flow and neural-ODE papers are optional context beyond the guide.

Chapter 8 — Vision-language-action models

Chapter 7 gave you a policy that can represent anything: a flow-matching model that turns Gaussian noise into a one-second chunk of joint commands, with no averaging between modes. That solved the *representation* problem. It did nothing for the problem that decides whether a robot works in a house it has never entered.

That problem is knowledge. A policy trained on your demonstrations knows what your demonstrations contain and nothing else. This chapter bolts it onto a model that has already read the internet. You will meet the architecture in pieces — what a vision-language model is, how tokens, attention and caches work, how actions attach, what π_0 and $\pi_{0.5}$ do — then one idea that matters as much: the **trunk**, the shared stack of weights every output hangs off, can predict how the episode is going as well as what to do next. That is a value function for free, and Chapters 10 through 12 all spend it.

8.1 What a VLA is for

What we want. A policy that folds a shirt it has never seen, in a room it has never seen, when told "put the laundry away" in words the training set never used. Not a policy that folds *your* gray t-shirt on *your* table under *your* lamp. Generalization along all three axes: new objects, new scenes, new phrasings.

The obvious thing to try. Collect more demonstrations and train a bigger policy from scratch. Your rover instincts say more data, more capacity, more randomization. Take the flow-matching policy of Chapter 7, randomly initialize its vision encoder and trunk, and train it on 500 teleoperated episodes of shirt folding.

Exactly how it fails. Suppose your 500 episodes are all gray cotton t-shirts. The vision encoder learns whatever distinguishes one gray shirt configuration from another: fabric shading, the collar shadow, sleeve edges against the table. Nothing forced it to learn a general notion of "garment". Now put a red striped polo on the table. The stripes are edge patterns the encoder has never seen, so its features land somewhere arbitrary — not near "shirt lying flat", just somewhere. The expert reads them and emits an arbitrary chunk: the arm reaches for a stripe boundary as if it were a sleeve edge.

Now the language axis. Your policy has no language input, or at best a task index. Ask it for "the sock, not the shirt" and there is no mechanism by which "sock" could mean anything: that embedding was initialized randomly and updated only by episodes where nobody said "sock". A bigger network does not fix this. The information is absent from the data.

There is a measured version of this failure. $\pi_{0.5}$ was trained with progressively more home environments in the mixture — 3, then 12, 22, 53, 82, 104 locations — and evaluated on following instructions about objects it had not been trained on. At the small end of that sweep, success was around 20%; at the large end, around 60% (Physical Intelligence, 2025a). The small end is roughly where your 500 episodes live.

The fix. Start from a **vision-language model (VLM)** — a transformer pre-trained on hundreds of millions of web images paired with text, for tasks like captioning ("a red striped polo shirt on a wooden table"), visual question answering, and object localization. Then attach an action-producing module, train it on robot data, and keep training on web data alongside. That combination is a **vision-language-action model (VLA)**.

Why the fix works. The captioning objective forced the pre-trained encoder to build features in which "striped polo" and "gray t-shirt" sit near each other and far from "ceramic mug", because a model that confused them would caption badly and be penalized. Those features exist before your first teleoperated episode, so your robot data only has to learn the mapping from features to motor commands. And "sock" already has a meaning wired to visual appearance.

Be precise about the division of labor, though, because this is where people over-claim. The VLM supplies semantics and robustness; it supplies no motor skill whatsoever, since nothing on the internet tells you the joint velocities that flatten a sleeve. What it shrinks is not how much robot data you need, but how narrow that data may be.

Before the pieces, the whole. Figure 8.1 is the thing this chapter builds. Look at it now without trying to understand it, and come back after each of the next four sections, when one more of its boxes has acquired a meaning.

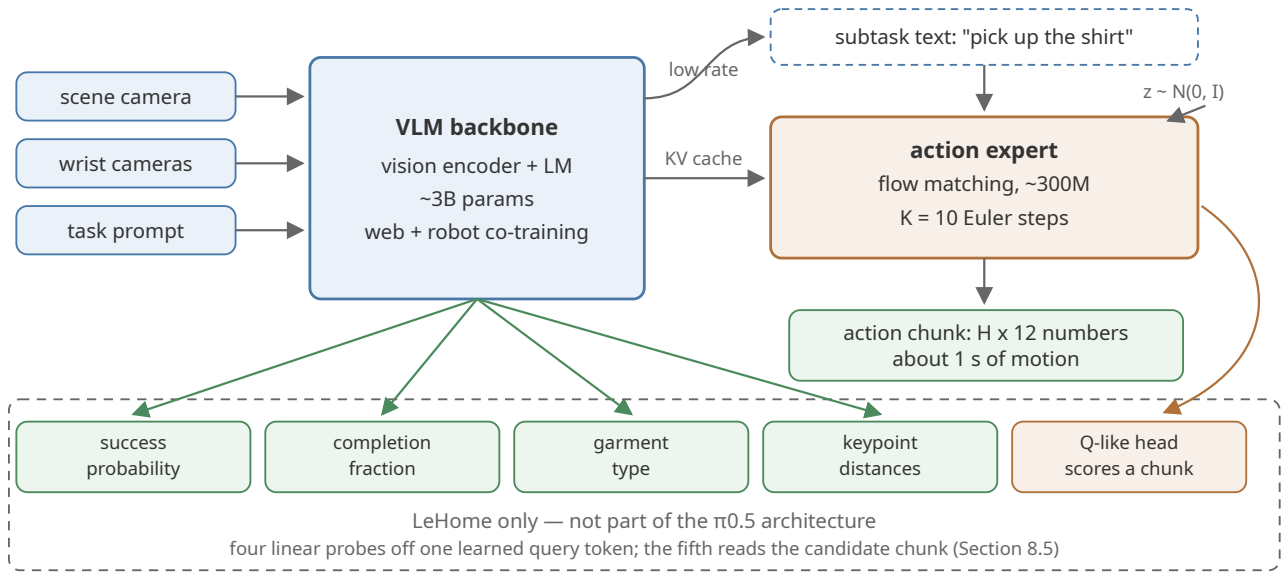


Figure 8.1 — The whole architecture on one page. Read the spine first: cameras and prompt enter a ~3B VLM backbone, which writes a subtask in text at low rate and hands its KV cache to a ~300M action expert; the expert integrates the flow from noise z and emits a chunk ($\pi_{0.5}$: $H = 50$ at 50 Hz; LeHome: $H = 30$ at 30 Hz). Then the heads: four green ones seeing the current frame only, and an orange one on the expert's tail seeing a candidate chunk — a Q to the green heads' V . One attribution, because the picture invites the wrong reading: the spine is $\pi_0/\pi_{0.5}$, but all five heads are LeHome's addition, which is why they sit inside the dashed box, and $\pi_{0.6}^*$ uses a separate critic instead (Section 8.5).

WORDS YOU WILL HEAR

Prefix / suffix. The part of the token sequence processed once and cached (images, state, prompt) versus the part re-processed on every denoising step (the action tokens). **Linear probe.** A single matrix mapping a frozen-or-shared feature vector to a prediction; "probe" signals that it adds no capacity of its own. **Stop-gradient** (written $\text{sg}(\cdot)$, $\text{detach}()$ in PyTorch): use a value in the forward pass but block gradients from flowing back through it. **Embodiment.** A specific robot body; *cross-embodiment* data mixes bodies with different arms and action spaces. **Knowledge insulation.** Stopping action-expert gradients from reaching the backbone so it keeps its web knowledge.

8.2 A VLM, for someone who has never used one

Build the mental model from where you already are. Your PPO rover policy was a multilayer perceptron: a fixed-length vector of floats in (lidar ranges, heading error, speed), two hidden layers, a mean and a log-standard-deviation out. Fixed input size, fixed output size, one shot.

The obvious extension, and why it stalls. To handle a camera you would bolt on a small convolutional network, flatten its output, concatenate that with the proprioceptive vector and feed the MLP. Most from-scratch robot policies do exactly this. But it gives you no way to accept a variable-length sentence, no way to let image features and words interact before the final layer, and no way to inherit pre-trained weights, because the world's vision-language knowledge is stored in transformers.

8.2.1 Tokens

A **token** is one element of a sequence, a vector of fixed width d (2048 for the Gemma-2B backbone the π -family uses). Everything becomes tokens. An image goes through a **vision encoder** — a separately pre-trained network, SigLIP-So400m/14 in π_0 and its descendants (Zhai et al., 2023) — which cuts a 224×224 image into 14×14 -pixel patches ($224/14 = 16$ per side, $16 \times 16 = 256$ total) and maps each patch to one vector, so one camera frame is 256

tokens. Text goes through a **subword tokenizer**, a fixed table of a few tens of thousands of common character sequences in which a frequent word is one entry and a rare one is spelled from two or three pieces; an embedding table then turns each entry into a vector, and "pick up the shirt" becomes perhaps five tokens. Joint state becomes tokens too: LeHome bins each of the 12 joint values into one of 256 levels and looks the level up in an embedding table (Larchenko, 2026). Once everything is a sequence of d -dimensional vectors, one architecture processes all of it.

8.2.2 Attention

A transformer layer's job is to let every token gather information from other tokens. It gives each token three roles. From token i 's vector it computes, by learned linear maps, a *query* q_i ("what am I looking for?"), a *key* k_i ("what do I offer?"), and a *value* v_i ("what do I hand over if chosen?"). Stack these as matrices Q, K, V with one row per token, and the layer computes (Vaswani et al., 2017):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

IN PLAIN WORDS

Every token asks every other token "how relevant are you to me?" The dot product of my query row q_i with your key row k_j is that relevance score, and $QK^\top$ is the whole table of scores, one row per asking token, one column per answering token. Dividing by $\sqrt{d_k}$, the square root of the key width, keeps those dot products from growing large merely because the vectors are long, which would saturate the next step. **softmax** turns each row into weights that are positive and sum to one — a mixing recipe. Multiplying by V applies it: my output is an average of everyone's value vectors, weighted by how relevant I judged them.

That is the entire mechanism. A transformer is this operation, plus a per-token MLP, plus normalization, repeated L times (18 layers for Gemma-2B). You can work with these models without knowing positional encodings or normalization placement; you cannot skip the idea that every token's output is a learned weighted average over other tokens' values.

8.2.3 KV cache and prefix

Notice what the equation implies about reuse. Token j 's contribution to the layer is captured entirely by its key k_j and value v_j . If j is an image patch and the image has not changed, k_j and v_j do not change either, no matter how many new tokens ask about them. So compute them once and store them. That store is the **KV cache**: for every layer, the keys and values of every token already processed.

The **prefix** is the part you process once and cache: images, robot state, prompt. The **suffix** is what comes after and gets re-processed. This is valid only if the attention mask is directional, meaning prefix tokens never attend to suffix tokens. If they could, adding a suffix token would change the prefix outputs and the cache would be stale.

IN PLAIN WORDS — WHY THE CACHE MATTERS HERE

The action expert of Section 8.3 runs $K = 10$ times per control decision. Without a cache, each of those ten passes would re-encode three camera images through a 3B-parameter backbone: ten full vision passes per decision, several decisions per second. With the cache, the images are encoded once per decision, and the ten passes run only the small expert over its own short suffix, reading stored keys and values for everything before it. That one engineering fact is what makes a 3B model usable for closed-loop control at all. Section 8.7 does the arithmetic.

It also buys the two tricks Chapter 12 spends. **Classifier-free guidance** runs the expert twice on the same prefix, once with the conditioning and once without, then extrapolates between the two predicted velocities to sharpen the result. **Best-of- N** samples N candidate chunks and keeps the one a scoring head rates highest. Both multiply expert passes and neither touches the prefix, so both ride on one cached encoding of the images.

An analogy, with its limit. The KV cache is memoization: a pure function of the prefix, computed once, reused by every later caller. The analogy is exact about *why* it is legal — the cached result cannot depend on the later calls — and about the speedup. Where it breaks: memoized results usually persist, keyed by their inputs, whereas this cache is thrown away the instant the camera frame changes, which is every control decision. There is no cross-decision reuse, so do not budget memory as if the cache accumulates.

8.2.4 How it produces output

Everything so far describes a sequence going in, and nothing describes anything coming out. After the last layer, every token has an output vector of width d ; take the one belonging to the last token. A learned output matrix maps its 2048 numbers to one score per entry in the **vocabulary**, the fixed list of subword pieces the tokenizer can produce, and softmax turns those scores into a probability distribution. You sample one token from it.

That is one token. To get a second, append it and run again. That loop — sample, append, run again — is what **autoregressive** means, and training it by cross-entropy against the token that actually came next is what **next-token prediction** means. Here the cache pays off: the second pass does not re-encode the image tokens and the words already written, since their keys and values are stored, so only the new token is encoded and appended. Each extra token costs one token's worth of arithmetic instead of a whole sequence's.

8.3 Bolting actions onto a language model

What we want. The same transformer that consumed images and text should produce 30 timesteps of 12 joint deltas: 360 continuous numbers, at 30 Hz control, from a multimodal distribution (Chapter 5).

The first obvious thing: make actions into words. A language model already emits discrete tokens from a vocabulary. So discretize actions and emit them the same way. This is the RT-2 recipe (Brohan et al., 2023): bin each action dimension into, say, 256 levels, assign each bin a token, and predict the tokens autoregressively with the ordinary next-token-prediction loss.

Where the naive version fails. Count the tokens. A chunk of $H = 30$ steps with 12 dimensions is $30 \times 12 = 360$ tokens, and autoregressive decoding means one forward pass per token. With the KV cache each pass is cheap in arithmetic: one token against a stored cache. What you cannot cache away is that the passes are strictly sequential, because token 360 cannot start until token 359 exists. At a realistic 5 to 10 ms per decode step, one chunk takes 2 to 4 seconds against the 167 ms budget Section 8.7 derives. That is the wall, and it is latency, not FLOPs. The **FAST tokenizer** (Pertsch et al., 2025) attacks the count: a discrete cosine transform along time concentrates smooth motion into a few low-frequency coefficients, quantized and entropy-coded, giving tens of tokens instead of hundreds. That makes training practical, but not high-rate closed-loop decoding, and quantization still puts a floor under action precision.

The second approach: a continuous action expert. Attach a separate, smaller set of transformer weights — the **action expert** — that takes H noisy action tokens as its suffix, attends to the backbone's cached prefix, and outputs the flow-matching velocity field $v_\theta(a^u, u, o)$ of Chapter 7. Name those symbols again, because one is the collision this book most wants you to avoid. u is the *flow time*, an internal axis running 0 to 1 inside a single control decision, with no relation to the environment timestep t . a^u is the partially denoised chunk at that flow time, pure noise at $u = 0$ and the chunk you execute at $u = 1$. o is the current observation. All H action tokens come out in parallel each pass, so you need $K = 10$ passes, not 360, and the flow handles multimodality as Chapter 7 described.

What each buys. Discrete tokens buy one objective for everything, since captioning, subtask text and actions all become next-token prediction. The continuous expert buys speed and precision at inference, at the cost of weights that inherit nothing from pre-training.

So $\pi_{0.5}$ uses both, at different stages. This detail confuses everyone reading the paper for the first time, so read it slowly. During pre-training — 280k gradient steps — there is no action expert at all; actions are FAST tokens and the

loss is pure next-token prediction. During post-training — 80k steps — the flow-matching expert is added from scratch and the loss becomes a weighted sum of both objectives (Physical Intelligence, 2025a):

$$\mathcal{L}_{\text{post}}(\theta) = \mathcal{L}_{\text{NTP}}(\theta) + \alpha \mathcal{L}_{\text{FM}}(\theta), \quad \alpha = 10.$$

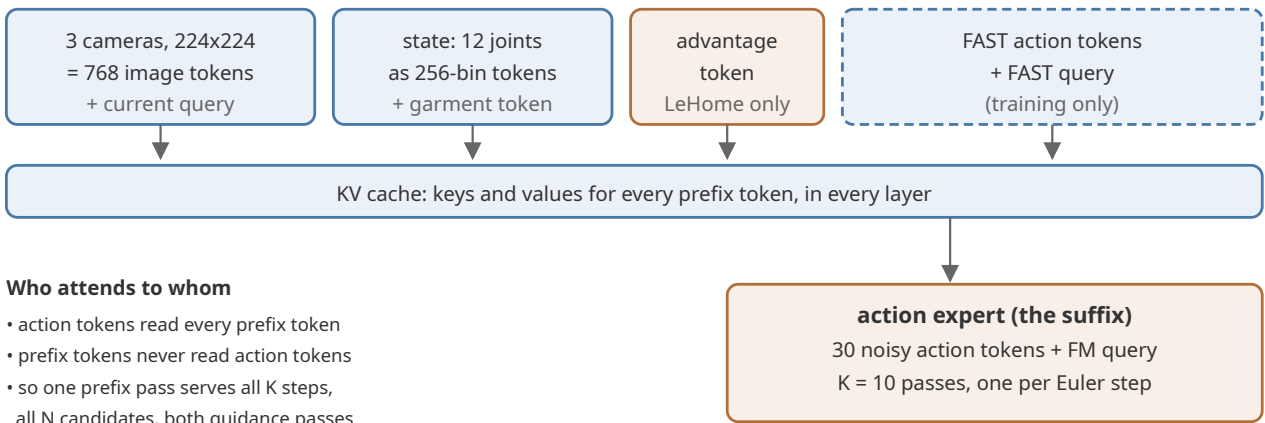
IN PLAIN WORDS

The total post-training loss has two parts. $\mathcal{L}_{\text{NTP}}$ is next-token prediction: the cross-entropy of the backbone's predicted token against the true next token, over web text, subtask annotations and FAST-encoded actions. $\mathcal{L}_{\text{FM}}$ is the flow-matching regression of Chapter 7, the squared error between the expert's predicted velocity and the target $\mathbf{a} - \mathbf{z}$. The coefficient $\alpha = 10$ weights the flow term ten times as heavily, because that is the term producing the actions you execute; the token term is kept on to hold the backbone's representation in place. θ covers backbone and expert together: the flow-matching gradients do reach the backbone here.

Two warnings about letters. This α is the $\pi_{0.5}$ paper's loss weight; the α of Chapter 11 is a different quantity, the damping coefficient on the value baseline. And the λ weights of Section 8.5 are per-head loss coefficients, unrelated to GAE's λ . Both clashes are inherited from the source papers.

The LeHome solution keeps a related trick: FAST action tokens sit in its prefix as a *training-only* representation shaper, and are stripped out before the action expert runs at inference (Larchenko, 2026). Figure 8.2 shows the resulting sequence layout.

Prefix — one backbone pass per control decision, then cached



At inference the dashed group is stripped; in LeHome there is no text prompt at all.

Figure 8.2 — The token sequence, in groups. Notice that the expensive part, 768 image tokens through a 3B backbone, sits entirely in the prefix and is computed once, while the part that runs ten times is a 31-token suffix through a 300M expert. The directional mask is not a detail: it is what makes the cache reusable, and therefore what makes Chapter 12 cheap. Two prefix entries are LeHome's own rather than $\pi_{0.5}$'s — the garment token and the advantage token of Chapter 10.

8.4 π_0 and $\pi_{0.5}$, concretely

π_0 (Black et al., 2024) set the template. A PaliGemma backbone of about 3B parameters — roughly a 400M SigLIP vision encoder feeding a ~2B Gemma language transformer — plus an action expert of about 300M parameters that attends to the backbone's KV cache and is trained by flow matching to emit 50-step chunks for 50 Hz control, one second of motion per query.

$\pi_{0.5}$ (Physical Intelligence, 2025a) keeps that skeleton and changes the recipe, which is where the generalization comes from. Its numbers, worth memorizing because Chapter 15 asks you to reproduce a descendant of them:

- **Pre-training, 280k gradient steps**, everything as discrete tokens ($\alpha = 0$; no action expert yet). The mixture: about 400 hours of mobile-manipulator data from roughly 100 homes; static-arm data from more homes; cross-embodiment lab data including Open X-Embodiment; annotated subtask descriptions for all robot data; and web data — captioning, visual question answering, object localization.
- **The statistic that explains the model**: 97.6% of first-phase training examples are *not* mobile-manipulator household data. That household data is a 2.4% slice of what the model learned from.
- **Post-training, 80k steps**: mobile-manipulator and multi-environment data filtered to successful, short-enough episodes; subtask annotations; verbal-instruction data where expert users speak step-by-step commands; web data retained; lab data dropped. The flow-matching expert is added from scratch and the hybrid loss above is used.

What we want from language. A robot you can command in one sentence — "clean up the kitchen" — that works out the rest for itself.

The obvious thing to try. Condition the action expert on that sentence directly, and ask it for the next second of motion.

Exactly how it fails. The chunk must now encode a plan and a motion at once. Stand the robot at a counter holding a plate, a mug and a sponge. "Clean up the kitchen" is satisfied by picking up any of the three, so the demonstrations hold quite different chunks for near-identical observations, differing at the level of *intent* rather than motor detail. That is the multimodality of Chapter 5 one level up, and it bites harder, because the modes are half a meter apart rather than a centimeter. A flow model can represent the mixture, but nothing holds it to one branch across decisions: the arm commits to the plate now and the sponge five decisions later. And you have nowhere to look, because the choice existed only as a direction in a 2048-dimensional vector. Meanwhile the expert must learn task decomposition from motor data, the narrowest data you own.

The fix: hierarchical inference. One model runs at two rates. First the backbone autoregressively writes a subtask *in plain text*, using the loop of Section 8.2 and all four cameras: $\hat{\ell} \sim \pi_{\theta}(\cdot \mid \mathbf{o}_t, \ell)$, where ℓ is the overall command you gave the robot ("clean up the kitchen"), $\mathbf{o}_t$ is the current observation, and $\hat{\ell}$ is the subtask the model writes for itself ("pick up the plate"). Then the action expert, conditioned on that subtask and on the forward and wrist views, integrates $K = 10$ Euler steps and emits $\mathbf{a}_{t:t+H} \sim \pi_{\theta}(\cdot \mid \mathbf{o}_t, \hat{\ell})$, far more often than the high-level step runs.

Why it works. Conditioning on $\hat{\ell}$ removes the intent-level multimodality: given "pick up the plate", the remaining spread over chunks is motor detail, which is what flow matching is good at. The decomposition is learned from annotated subtask text, cheap to label and trained by the same next-token loss as web data, rather than from joint trajectories. And $\hat{\ell}$ is a string, so the plan is something you can print, log, evaluate or overwrite. Figure 8.1 shows the arrangement; the curved arrow into the dashed box is this low-rate path.

Why co-training keeps semantics from washing out. Here is the failure it prevents. Take a pre-trained VLM and fine-tune it for 80k steps on shirt-folding episodes alone. Every gradient step pulls every weight toward the loss on that narrow distribution. The features that distinguished "sock" from "spatula" were useful for captioning and are useless for this loss, so nothing preserves them: they drift and get overwritten, the standard catastrophic-forgetting story. You would still measure excellent shirt folding, which is why this is easy to miss until you test out of distribution.

The fix is to keep web data in the mixture during post-training, so each minibatch contains both kinds of example. The averaged gradient then carries a term that punishes any weight change which breaks captioning: directions that improve folding at the cost of semantics are opposed, and only directions that do both survive.

Knowledge insulation, and which recipe it belongs to. A follow-up recipe from the same group goes further. *Knowledge insulation* blocks the action expert's gradients from reaching the backbone entirely, so the backbone trains on discrete-token losses alone and motor learning cannot damage the language model at all. It is not part of the $\pi_{0.5}$ post-training above; it is a later variant, carried into $\pi_{0.6}^*$ (Physical Intelligence, 2025b). Keep the three cases apart. In plain $\pi_{0.5}$ post-training — the $\alpha = 10$ equation — flow-matching gradients do reach the backbone, which is why θ

covers both. Knowledge insulation severs that path. LeHome uses neither it nor any stop-gradient into the trunk, and Section 8.5 depends on that.

What the recipe bought. $\pi_{0.5}$ cleans kitchens and bedrooms in homes never seen in training, over runs of 10–15 minutes, with individual multi-stage tasks taking 2–5 minutes; it was evaluated in three real kitchens and three real bedrooms, ten trials per task. Success rises smoothly as training locations go from 3 to 104, and at the top of that curve it matches a control model trained directly on the test homes: co-training closed the in-domain gap. The language-following numbers of Section 8.1 come from the same sweep. The practical consequence, which Chapter 15 depends on: fine-tune a pre-trained VLA, do not train one from scratch.

8.5 Auxiliary heads: the policy is its own value function

What we want. Everything in Chapters 10 through 12 needs a value estimate. Advantages need $V(\mathbf{s})$ (Chapter 11). RECAP's advantage conditioning needs advantages (Chapter 10). Best-of- N chunk selection needs a score for a candidate chunk (Chapter 12). Without a value function, none of it runs.

The obvious thing to try. Train a critic, as PPO does. You have done this: in Stable-Baselines3 the actor and critic shared one feature extractor and the value head was two extra layers, essentially free. Scaled up, that becomes the $\pi_{0.6}^*$ design: a separate 670M-parameter Gemma 3 VLM trained as a **distributional critic**, meaning it predicts a probability over a grid of possible values rather than regressing a single number, which trains more stably. It predicts $p_\phi(V \mid \mathbf{o}_t, \ell)$ over $B = 201$ value bins, with cross-entropy against the empirical **return-to-go** $G_t = \sum_k \gamma^k r_{t+k}$, computed from the episode that actually happened, this frame to the end (Physical Intelligence, 2025b). Two symbols: ϕ is the critic's own parameters, written apart from the policy's θ because it is a separate model, and ℓ is the language command of Section 8.4. It works — and that critic is one component of RECAP, the method that roughly halves failure rate on the hardest tasks; Chapter 10 takes the whole method apart.

Where it hurts on one workstation. Count what a separate critic costs. A second checkpoint to store, load and version. A second forward pass over the same three camera images every time you want a value, competing with the policy for the GPU during rollouts. And it must learn from scratch a representation of "how is this fold going" — the representation the policy already built, from the same data, only with less of it, since the critic sees observations and returns alone. You pay twice for one perception system. Figure 8.3 makes the duplication visible.

8.5.1 The four current-frame heads

The fix. Do not build a second model. Add one learned **query token** to the backbone's input — a single trainable vector that participates in attention like any other token — and hang small heads off its output. Each head is a **linear probe**: one matrix mapping that token's 2048-dimensional output to a handful of numbers, plus an activation. In the LeHome solution the current-frame heads are (Larchenko, 2026):

- **Success probability $\hat{p}_{succ}$:** sigmoid output, binary cross-entropy against whether the episode eventually succeeded, trained on RL rollouts. Two refinements that matter: labels are smoothed toward the per-garment mean success rate (coefficient 0.05), and the last 20 frames of successful episodes get 20× the BCE weight, because the end of an episode is where the prediction must be sharp.
- **Completion fraction:** sigmoid, MSE against the target t/T — how far through the episode this frame is — trained on successful episodes only.
- **Garment type:** a 4-way softmax over long tops, short tops, long pants, shorts. Above 99% accurate late in training, which matters because the evaluator never tells the policy which garment it is facing.
- **Keypoint distances:** a raw linear head emitting 21 numbers. Unpack this one, because those numbers are the task's own definition of success. Every LeHome garment carries labelled **keypoints** — collar, cuffs, hems, waistband — and the success checker is a conjunction of pairwise conditions on them: certain pairs must come within a threshold, others must stay beyond one. So $\mathbf{dist}_i$ is the current distance for condition i , $\mathbf{threshold}_i$ is that condition's pass radius, and $d^{(i)} = \mathbf{dist}_i / \mathbf{threshold}_i$ falls below 1 exactly when condition i is met. The official

checker uses 5 conditions for tops and 4 for pants; for his own dense reward Larchenko expanded long pants from 4 conditions to 7, so his head covers 5 for long tops, 5 for short tops, 7 for long pants and 4 for shorts — $5 + 5 + 7 + 4 = 21$. The head emits every garment's slice at once, the others masked out as NaN. When later chapters say "five conditions for tops, four for pants" they mean the official checker; the 21 here is his expanded set. These targets exist only in simulation, where privileged state is available.

One design detail is worth more than it looks. The query token sits inside the *image* group of the attention layout, not the state group, so every head must read its answer out of pixels alone: no proprioceptive shortcut, no copying the garment-type input token. That keeps the probes honest and makes them useful as a training signal for the trunk.

8.5.2 The action-conditional head

The fifth head is different in kind. A separate query token sits at the *tail of the action expert's suffix*, where it can attend to the partially denoised action chunk. Because it sees a candidate action, it is a *Q*-like head rather than a *V*-like one. It predicts three things about the world after that chunk has been executed: the keypoint distance ratios *H* steps in the future, the completion fraction at that point, and a success residual

$$\Delta_{succ} = y - \text{sg}(\hat{p}_{succ}),$$

where *y* is the episode's actual binary outcome and *sg* is the stop-gradient. **Stop-gradient in plain words:** in the forward pass use $\hat{p}_{succ}$ as it is; in the backward pass treat it as a constant, so no gradient reaches the success head through it. Without that, the *Q*-like head could cut its own loss by pushing $\hat{p}_{succ}$ around, corrupting the *V*-like estimate Chapter 11 depends on. With it, the residual head can only improve by learning what this *particular chunk* adds to the baseline the observation already implies. In PyTorch it is one `.detach()`, and forgetting it is a bug you will not notice for days.

8.5.3 One loss, several jobs

All of this is one weighted sum:

$$\begin{aligned} \mathcal{L}(\theta) = & \mathcal{L}_{\text{FM}} + \lambda_{succ} \mathcal{L}_{succ} + \lambda_{comp} \mathcal{L}_{comp} + \lambda_{gar} \mathcal{L}_{gar} + \lambda_{kp} \mathcal{L}_{kp} \\ & + \lambda_{Q,succ} \mathcal{L}_{Q,succ} + \lambda_{Q,comp} \mathcal{L}_{Q,comp} + \lambda_{Q,kp} \mathcal{L}_{Q,kp}. \end{aligned}$$

IN PLAIN WORDS

One network, one optimizer, one backward pass, several jobs. $\mathcal{L}_{\text{FM}}$ is the flow-matching action loss at weight 1.0: producing good actions is the job. The next four terms are the current-frame heads — success, completion, garment type, keypoints — and the last three are the action-conditional head's targets, which is what the *Q* subscript marks. Each λ says how much the network may care about that side job relative to actions. LeHome's simulation values: $\lambda_{succ} = 0.05$, $\lambda_{comp} = \lambda_{gar} = \lambda_{kp} = 0.02$, $\lambda_{Q,succ} = 0.1$, $\lambda_{Q,comp} = 0.05$, $\lambda_{Q,kp} = 0.02$. All small, deliberately: the heads *shape* the shared features rather than dominating training. Gradients do reach the backbone — no stop-gradient into the trunk.

8.5.4 What it buys

Four payoffs, in order of how much they matter. First, the value function sees what the policy sees, updates when the policy updates, and ships in the same checkpoint at near-zero extra cost; Chapter 11 builds its estimate as $V(s_t) = \hat{p}_{succ} - R_{acc}$, where R_{acc} is the reward already banked earlier in the episode by the checkpoint scheme of Chapter 11 — expected total minus already collected leaves what is still ahead, which is what a value is. Second, the auxiliary losses shape the trunk: predicting keypoint distances forces geometric task state into the features the action expert reads, which tends to help the actions too. Third, the keypoint head is a cheap world model — instead of future pixels it predicts the handful of numbers the reward depends on. Fourth, the *Q*-like head can score candidate chunks at inference, which makes best-of-*N* possible in Chapter 12 with no extra model; the current-frame heads cannot, since they see only the observation and would score every candidate alike.

The full inventory, for when you read the report. Five heads carry every idea in this book, and those are the five taught here. The tech report's list is slightly longer: alongside the success head sit checkpoint and time-to-completion (TTC) probes, minor siblings trained the same way, and there are two future-prediction queries rather than one — a training-only query on the FAST token stream, stripped before the action expert ever runs, plus the flow-matching Q -like query described above. Chapter 14's deployment recipe names all of them, because stripping a policy for real hardware means deciding what to do with each.

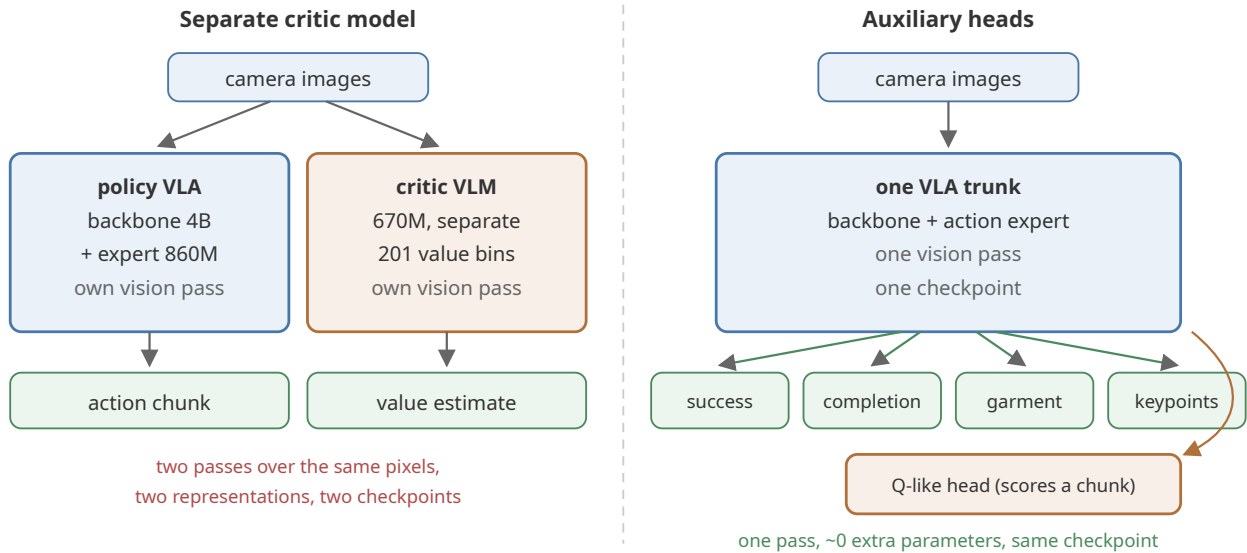


Figure 8.3 — The same value function, two ways. On the left the critic is a whole second model that re-encodes the same frames and must rediscover the representation the policy already has; on the right the value estimate is a linear map off a token the policy computes anyway. The left design keeps one advantage the figure cannot show — a critic independent of the policy cannot be corrupted by policy-loss gradients — which is why a well-resourced lab may still prefer it.

```
# heads hanging off one shared trunk (schematic)
h_img = trunk_out[query_idx]          # 1 x 2048, the learned query token
p_succ = sigmoid(W_succ @ h_img)      # V-like: P(episode succeeds)
comp = sigmoid(W_comp @ h_img)       # progress t/T, successes only
gtype = softmax(W_gar @ h_img)       # 4 garment classes
kp = W_kp @ h_img                    # 21 distance ratios, NaN-masked

h_act = expert_out[fm_query_idx]      # tail of the action expert suffix
d_succ = W_qs @ h_act                 # residual on top of p_succ
kp_fut = W_qk @ h_act                 # the same 21 ratios, H steps ahead
comp_f = sigmoid(W_qc @ h_act)        # completion fraction, H steps ahead
target = y - p_succ.detach()          # stop-grad: protects the V-like head

loss = loss_fm + 0.05*loss_succ + 0.02*loss_comp \
      + 0.02*loss_gar + 0.02*loss_kp \
      + 0.1*mse(d_succ, target) + 0.05*loss_q_comp + 0.02*loss_q_kp
```

PITFALLS

Three that cost real days. (1) Forgetting the stop-gradient: the residual head drags $\hat{p}_{succ}$ toward whatever makes its own loss small, and your Chapter 11 advantages quietly degrade while every training curve still looks fine. (2) Head weights too high: at $\lambda \approx 1$ the trunk optimizes for prediction instead of action, and success rate drops with no obvious culprit. (3) Assuming a head that predicts well is useful downstream — LeHome's Q -like head had roughly zero correlation with outcomes yet still improved best-of- N , because it caught rare catastrophic chunks, not average quality.

8.6 What this costs

Parameters first. $\pi_{0.5}$ -class: about 3B backbone plus about 300M expert, call it 3.3B. $\pi_{0.6}^*$: a Gemma 3 4B backbone, an 860M action expert, and the 670M critic on top — over 5.5B across two checkpoints.

Now memory, with the arithmetic visible. In bfloat16 weights cost 2 bytes each, so 3.3B parameters are $3.3 \times 10^9 \times 2 = 6.6$ GB. That is inference. Training with Adam is a different order. Count the bytes held per trainable parameter in a normal mixed-precision setup: fp32 master weight 4, fp32 gradient 4, Adam's first moment 4, Adam's second moment 4, and $4 + 4 + 4 + 4 = 16$ bytes. So $3.3 \times 10^9 \times 16 = 53$ GB before a single activation. Add activations and you are asking for an 80 GB card at minimum. Larchenko trained on one H200: batch 192, bf16, frozen vision encoder, cosine learning rate 1e-4 to 1e-5, a 20k-step behavior-cloning warm-up, about 300k steps in total.

On a 24 GB consumer GPU, full fine-tuning of a 3B VLA is not available. Here is what is. Freeze the vision encoder, which LeHome does anyway, and freeze most or all of the backbone, training the expert plus heads. Use **LoRA** adapters: instead of updating a weight matrix, train a small low-rank matrix alongside the frozen one, so optimizer state covers a few million parameters rather than billions. Turn on **gradient checkpointing**: discard most activations on the forward pass and recompute them on the backward pass, trading roughly 30% more compute for a large memory saving. Cut the batch size, then recover the effective batch with **gradient accumulation**: run several small batches, sum their gradients, take one optimizer step at the end, and the update behaves like a large batch while peak memory stays small. Inference is comfortable — 6.6 GB of weights plus a KV cache of tens of megabytes.

Which is the real argument of this section. Nobody outside a well-funded lab reproduces 280k steps over that corpus, and nobody has to: the checkpoints are released, so the expensive 97.6% is already paid for. Your job is the last 80k steps.

ON YOUR RIG

LeRobot ships ports of both π_0 and $\pi_{0.5}$; your `h1l-serl` checkout already has them under the policies package, next to `act` and `diffusion`. Before fine-tuning anything, spend an evening reading one config: chunk size H , `n_action_steps` (actions executed before re-query — the n_e of Chapter 12), denoising steps K , the tokenizer, the image resize. Then load a pre-trained checkpoint and run inference on a few frames of your SO-101 teleop dataset — not for good actions, but to see the shapes move and to time one query on your GPU. That number sets your control rate.

8.7 One inference call, end to end

Now trace a single control decision through the LeHome-configured model, with shapes and numbers at every stage. Assume three RGB cameras at 224×224 (one overhead, one per wrist), a 12-joint bimanual SO-101, $H = 30$ at 30 Hz, $K = 10$.

Stage 1: tokenize the observation. Each image is cut into 14×14 patches: $224/14 = 16$ per side, $16 \times 16 = 256$ tokens per image. Three cameras give $3 \times 256 = 768$ image tokens. Add the learned current-query token: 769. The 12 joint values are each discretized into one of 256 bins, giving 12 state tokens: 781. Add the garment-type token: 782. Add the advantage token of Chapter 10: 783. (A $\pi_{0.5}$ -style prompt such as "clean up the kitchen" would add roughly five more; LeHome uses no language at all.)

Stage 2: the prefix pass. All 783 tokens go through the backbone once. Estimating compute with the usual rule of about 2 FLOPs per parameter per token:

```

vision encoder: 2 x 0.4e9 params x 768 tokens = 0.61 TFLOP
Gemma-2B trunk: 2 x 2.0e9 params x 783 tokens = 3.13 TFLOP
prefix total                                     = 3.74 TFLOP

```

```

KV cache (bf16, 18 layers, one 256-wide KV head per layer):
  2 (K and V) x 18 x 783 x 256 x 2 bytes = 14.4 MB
  (with full 2048-wide K and V it would be 8x = 115 MB)

```

Stage 3: the high-level step, if you have one. $\pi_{0.5}$ would now decode a subtask such as "pick up the plate" — five or so tokens, autoregressively, at roughly 1 Hz rather than every decision. LeHome skips this stage entirely: no language input, no subtask text.

Stage 4: integrate the flow. The suffix is 30 noisy action tokens (one per chunk timestep, each a 12-dimensional action projected to the expert's width) plus the FM query token: 31 tokens. Start from $z \sim \mathcal{N}(0, I)$ of shape 30×12 and take ten Euler steps at $u = 0, 0.1, \dots, 0.9$ — u is the flow time, an axis internal to this one decision, not the environment clock — each step being $a \leftarrow a + \frac{1}{10} v_\theta(a, u, o)$:

```

one expert pass: 2 x 0.3e9 params x 31 tokens = 0.019 TFLOP
K = 10 steps:    10 x 0.019                  = 0.19 TFLOP
share of the decision: 0.19 / 3.74           = 5.1 %

```

```

with CFG (2 passes) and best-of-N = 3 candidates:
  6 x 0.19 = 1.14 TFLOP; total 3.74 + 1.14 = 4.88 TFLOP
  versus 3.74 + 0.19 = 3.93 without       = +24 %

```

That 5.1% is the number to remember — but remember it as a *FLOP* share, not a time share. At batch size one, which is what a robot runs, arithmetic is not the bottleneck: the denoising loop is ten short sequential passes, each too small to saturate the GPU and each paying kernel-launch and memory-bandwidth overhead that the FLOP count does not see. So the loop costs more wall-clock than 5.1% suggests. Chapter 12 measures the real budget, and there the six-fold expert loop that guidance and candidate selection require comes to roughly three quarters of a decision's latency (Section 12.3.4). The FLOP arithmetic tells you the expert is cheap to *train* around; the latency budget tells you what it costs to *run*.

Stage 5: execute and re-query. The output is $30 \times 12 = 360$ numbers — joint deltas for the next second. LeHome executes only the first n_e of them: 5 for tops, 3 for pants. At 30 Hz, $n_e = 5$ is $5/30 = 0.167$ s of motion, so the model is re-queried at $30/5 = 6$ Hz. Each decision therefore has a 167 ms budget. LeHome deploys with guidance and candidate selection on, so a decision costs the 4.88 TFLOP computed above, a sustained $6 \times 4.88 \approx 29$ TFLOP/s; turn both off and you drop to the 3.93 TFLOP floor, $6 \times 3.93 \approx 24$ TFLOP/s. Either way you are inside a modern GPU's bfloat16 throughput, provided you get decent utilization at batch size one, which is its own engineering problem.

Why throw away 25 of the 30 actions? Because the chunk was planned from an observation that is now stale, and its later actions are the most speculative. Executing a short prefix of a long plan gives you the temporal consistency of chunking with the reactivity of frequent replanning. Chapter 12 treats n_e as a tunable knob.

IN THE LEHOME SOLUTION

The stack is exactly Figure 8.1, sized down: a frozen SigLIP-So400m/14 encoder, a Gemma-2B prefix transformer, and a Gemma-300M flow-matching expert emitting 30×12 joint-delta chunks at 30 Hz in simulation (20 Hz in the real round), from three 224×224 cameras, with *no language input at all* — the VLM earns its place through visual representation, not instruction following. FAST tokens live in the prefix as a training-only shaper. All five heads are present, and the garment-type head does real work, since the evaluator never reveals the garment (Larchenko, 2026).

A second analogy, with its limit. Think of the auxiliary heads as instrumentation on a service you already run: metrics scraped from data structures that exist anyway, versus standing up a parallel service that re-parses every

request to compute the same metrics. That captures the cost argument precisely. It breaks on one point, the important one: real instrumentation is read-only, whereas these heads emit gradients that change the trunk's weights. That is a feature — payoff number two — but it means a badly weighted head is not a harmless dashboard. It is a second objective quietly competing with your policy.

CHECKPOINT

- Explain what a VLM contributes to a robot policy and what it does not — semantics and robustness yes, motor skill no — with the $\pi_{0.5}$ numbers supporting both halves.
- Define token, attention, KV cache and prefix from memory, describe how the model generates a token at a time, and say why the directional mask is what makes the cache legal.
- Describe the two ways actions attach to a language model, why serial token decoding misses a 167 ms budget, and why $\pi_{0.5}$ uses tokens in pre-training and a flow expert in post-training.
- Explain why co-training on retained web data prevents catastrophic forgetting, in terms of what the averaged gradient penalizes, and where knowledge insulation differs.
- List the five auxiliary heads with their targets, say why the query token sits in the image group, and explain what the stop-gradient in $\Delta_{succ} = y - \text{sg}(\hat{p}_{succ})$ protects.
- Estimate the compute split of one inference call, and state what fraction the denoising loop consumes — the fact Chapter 12 spends.

Where to go deeper. Read Black et al. (2024) for π_0 and the action-expert design, then Physical Intelligence (2025a) for the $\pi_{0.5}$ co-training recipe; the 97.6% statistic and the scaling curve are the parts to sit with. Pertsch et al. (2025) explains FAST tokenization properly, and Brohan et al. (2023) is the original case for discrete action tokens. For the auxiliary heads as an engineering artifact, Larchenko (2026) Section 5 lists every head, target and loss weight. Chapter 9 next puts PPO and VLA-RL side by side.

Chapter 9 — PPO versus VLA-RL: the two worlds side by side

9.1 Why this chapter exists

You have trained PPO and it worked. You wrote a Gymnasium environment for a differential-drive rover, let Stable-Baselines3 run for a few million steps, watched the return curve climb, added domain randomization, and transferred the policy to a 3D-printed robot. That means you know something people usually only read about: what it feels like when an on-policy algorithm is learning, when it is stuck, and when it has quietly diverged.

Then you open a modern manipulation paper — $\pi_{0.5}$ (Physical Intelligence, 2025a), $\pi_{0.6}^*$ and RECAP, its recipe for reinforcement learning from experience and corrections (Physical Intelligence, 2025b), the LeHome solution (Larchenko, 2026) — and almost nothing you recognize is there. No clipped surrogate. No importance ratio. No entropy bonus. No rollout buffer flushed after four epochs. Instead: a flow-matching action expert, a replay pool — a persistent store of past episodes that the trainer draws from again and again — with a decay factor on the old ones, a success head that doubles as a critic, an advantage token, a bandit tuning guidance at inference. It is easy to conclude you have been learning the wrong thing.

You have not. But the two worlds really are different, and the difference is not cosmetic — it is forced by the policy class and by the cost of a sample. This chapter puts them side by side, arrow for arrow and symbol for symbol, so that when you read a paper you know which world you are standing in, what the authors could not have done, and why they did what they did instead.

WORDS YOU WILL HEAR

On-policy — the update uses only data generated by the policy being updated, right now. **Off-policy** — it uses data from some other policy: an old checkpoint, a human, a script. **Staleness** — how many checkpoints old a sample is; stale means more off-policy. **KL divergence** — a single number measuring how far one probability distribution sits from another; a trust region is a cap on it. **GRPO** — group-relative policy optimization: a policy-gradient method that drops the critic and scores each rollout against the average return of a group of sibling rollouts from the same start. **Classifier-free guidance** — generating twice, once with the conditioning and once without, then extrapolating away from the unconditioned result to sharpen the sample toward the condition.

9.2 The two loops, drawn

Start with the shapes of the two training loops, because almost every other difference follows from them. Figure 9.1 draws both. Take them one arrow at a time.

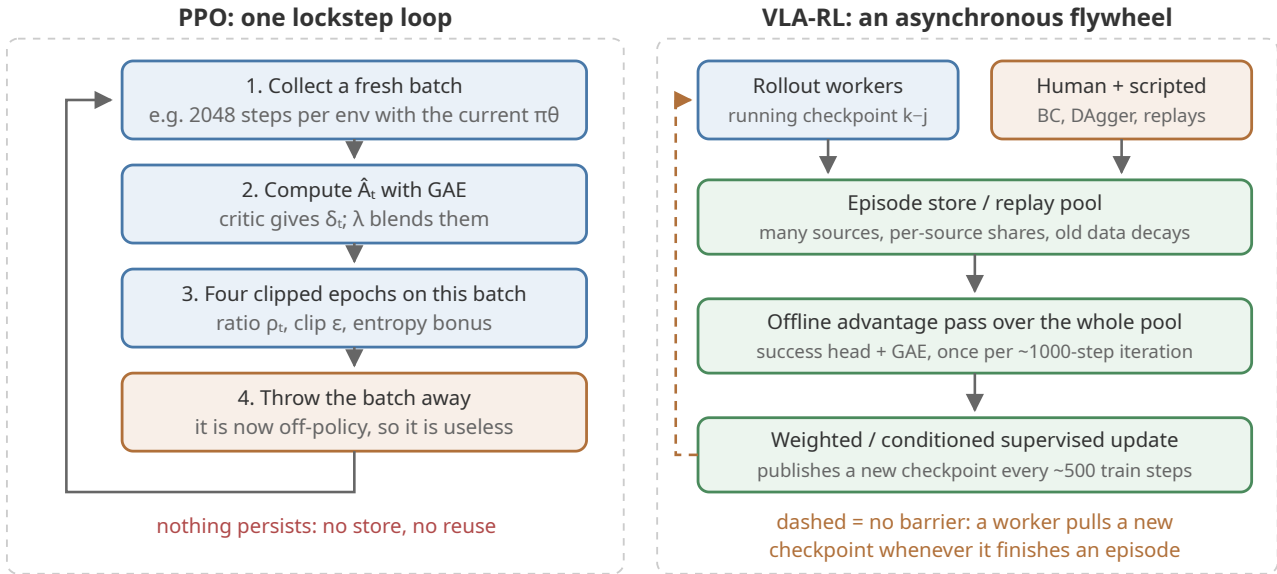


Figure 9.1 — The same four jobs (collect, evaluate, update, repeat) arranged two ways. On the left every arrow is a hard dependency: nothing starts until the previous box finishes, and the orange box destroys the data. On the right the store decouples everything, so the dashed return arrow is allowed to be late — a worker several checkpoints behind still produces useful episodes.

9.2.1 Walking the PPO loop

Arrow one: you run the current policy π_θ for a fixed number of steps and store the transitions, which belong to π_θ and to nothing else. Arrow two: you sweep backwards through the batch computing TD residuals $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ and folding them into $\hat{A}_t$ with the GAE parameter λ . Arrow three: several gradient epochs over that same batch, during which the policy drifts away from the one that collected the data — exactly what the clip parameter ϵ exists to bound. Arrow four: you drop the batch on the floor, because after a few epochs $\rho_t(\theta)$ is no longer a small correction, and the surrogate objective — the function you actually differentiate, standing in for the expected return, which samples alone give you no usable gradient of — has stopped being a faithful proxy.

The critical property is that this loop is a *barrier*: collection cannot overlap the update, because the update changes the policy that collection needs. In your rover project that was invisible, because a simulator step cost microseconds and the loop spun thousands of times per minute.

9.2.2 Walking the VLA-RL loop

Now the right panel. Arrow one: several rollout workers run *some* checkpoint — not necessarily the newest — and append finished episodes to a store on disk. Arrow two: human demonstrations, replayed episodes and DAgger corrections — dataset aggregation, meaning you run the current policy, a human corrects it while it runs, and the corrected frames join the pool (Chapter 13) — flow into the same store from a different pipeline. Arrow three: before each training iteration, a job walks the whole pool and computes an advantage for every frame from the policy's own value-like head. Arrow four: the trainer samples with probabilities that depend on those advantages and takes ordinary supervised steps. Arrow five, dashed: a new checkpoint is published, and workers pick it up whenever they finish an episode.

Nobody waits for anybody. In LeHome an Isaac Sim episode takes about 30 seconds, while the trainer publishes a checkpoint roughly every 40 minutes on a single H200 (Larchenko, 2026). Locked together, those two rates would idle the GPU for most of the run. PPO's loop is a pipeline in which each stage invalidates the previous stage's output; this one is a producer/consumer system around a durable dataset. Once you have that store, every other design decision changes.

9.3 Component by component

Now the machinery, piece by piece. For each: what PPO does, what VLA-RL does, and — the part that matters — why the change was forced rather than chosen.

9.3.1 (a) The policy class

PPO's policy is a small network emitting a mean and a log standard deviation; the action is one draw from that Gaussian at one timestep. It is cheap, it has a closed-form density, and it is unimodal. VLA-RL's policy is a vision-language backbone plus a flow-matching action expert that generates a whole chunk $\mathbf{a}_{t:t+H}$ — in LeHome, 30 timesteps of 12 joint deltas at once — by integrating a learned velocity field $\mathbf{v}_\theta(\mathbf{a}^u, \mathbf{u}, \mathbf{o})$ from noise over flow time $\mathbf{u} \in [0, 1]$ in K steps.

Why forced: Chapter 5 showed that a single Gaussian cannot represent "go left around the obstacle or right around it, but never straight through," and manipulation is full of such choices. But the machinery of Chapters 6 and 7 buys that expressiveness by giving up the one thing PPO needs most.

9.3.2 (b) What the update actually consumes

This is the hinge of the whole chapter, so slow down here. PPO's objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right], \quad \rho_t(\theta) = \frac{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t | \mathbf{s}_t)}.$$

IN PLAIN WORDS

The left side, $L^{\text{CLIP}}(\theta)$, is the clipped surrogate objective: the score we push upward by gradient ascent. $\mathbb{E}_t$ averages over every timestep in the batch. $\rho_t(\theta)$ is how much more likely the policy we are building now is to have taken the action actually taken than the policy that collected it: 1.0 unchanged, 1.4 forty percent more likely. $\hat{A}_t$ is how much better than average that action turned out, so the product says "make good actions more likely, in proportion to how good they were." **clip** flattens the ratio outside $1 \pm \epsilon$ (typically $\epsilon = 0.2$, SB3's default), and **min** takes the smaller product, so once the policy has moved far enough in the direction this sample asked for — a good action already more than $1 + \epsilon$ times as likely, a bad one already below $1 - \epsilon$ — the sample stops pulling. The reverse case is deliberately not capped: if the ratio runs the wrong way, the sample keeps pulling at full strength.

Every quantity there is one line of code for a Gaussian. So the obvious thing to try is to port it: keep the loop, swap the Gaussian head for the flow-matching expert, compute ρ_t , go. Here is exactly why that fails.

Start with what a flow policy is. It is a sampler, not a formula. You hand it an observation, it draws noise $\mathbf{z}$, integrates the velocity field for K steps, and gives back a chunk. What you cannot do is hand it a chunk and ask "how probable was that?" — the number $\pi_\theta(\mathbf{a} | \mathbf{o})$ is never computed anywhere in that process. The distinction is familiar from your own code: a torch distribution object gives you both `sample()` and `log_prob()`, and PPO leans on the second every epoch. A flow policy ships only the first, and the missing one is not a gap in the library — for this class of model it is a hard integral.

The integral can be written down, and Chapter 7 §7.6 prices it in full; Figure 7.3 is the anatomy. The short version: recovering $\log \pi_\theta(\mathbf{a} | \mathbf{o})$ from a flow means integrating the divergence of the velocity field along the generation path, which costs about 3,600 backward passes through the action expert per log-probability — for one sample, at one gradient step, against one forward pass for the supervised alternative. The cheap approximation makes it worse rather than better: a stochastic trace estimate leaves noise in log space that the ratio exponentiates, and a wobble of ± 0.7 moves ρ_t between 0.5 and 2.0 while PPO's whole clip window at $\epsilon = 0.2$ is 0.8 to 1.2. The noise would not perturb the objective; it would choose it.

What matters for this comparison is the shape of the requirement, not the size of the bill. PPO's update *consumes* a density: it must evaluate how probable a specific already-chosen action was, under two different parameter settings, differentiably. That is a strong demand, and a Gaussian meets it only because its density happens to have a closed form. Weighted imitation consumes something far weaker — a recorded action, a loss that can reproduce it, and one scalar per sample. Every architectural difference in the rest of this section follows from that gap. A policy class chosen for the quality of its *samples* cannot also be asked for its *densities*, and once you accept that, you stop trying to compute a ratio and start asking which data to imitate and how loudly.

Now suppose the exact log-density were free. A second barrier remains, and it is not about compute. Half of any policy gradient is push-away: when $\hat{A}_t$ is negative, the update lowers the probability of the action just produced. For a Gaussian that is harmless — the mean slides, the distribution stays a distribution. For a flow policy, the chunks that make sense occupy a thin, curved manifold inside those 360 dimensions: joint-delta sequences that are smooth, reachable and physically coherent. Most directions that lower one chunk's probability point off that manifold, so the update produces not a different sensible motion but an incoherent one, and the damage shows up in everything the network generates afterwards. Hence (Larchenko, 2026) gives two reasons, not one: log-prob policy gradients do not transfer cleanly, *and* push-away updates shove predictions off the narrow action manifold. Weighted imitation never takes such a step, because every gradient it takes says "reproduce this recorded chunk."

So the ratio is off the table, and with it the clipped objective, the log-prob KL penalty, the entropy bonus, and GRPO-style objectives too (group-relative policy optimization scores a rollout against a group of sibling rollouts instead of a critic, but it still needs $\log \pi_\theta$). What is left is something the flow policy is extremely good at: being told what to imitate. The Chapter 4 objective — advantage-weighted regression, or AWR — is

$$\max_{\theta} \sum_i w_i \log \pi_\theta(a_i | o_i), \quad w_i = \exp(\hat{A}_i / \beta).$$

IN PLAIN WORDS

Sum over samples i drawn from whatever data you have. $\log \pi_\theta(a_i | o_i)$ is the ordinary behavior-cloning term: "make the action a_i that was taken in observation o_i more likely." w_i is a non-negative number scaling how hard you pull on that sample. $\hat{A}_i$ is the same advantage estimate as before; β is a temperature, small β sharpening the exponential so only the best samples matter, large β flattening it toward plain cloning. Notice what is gone: no old policy, no ratio, no second network to evaluate. Notice also what is still sitting there: $\log \pi_\theta$, the density under the network we are training — the very thing just declared unaffordable. Resolving that is the next four paragraphs.

So look hard at what this objective asks for. It must make the recorded chunk a_i more likely under the current network, and pull harder when w_i is large. Read literally, that means computing $\log \pi_\theta(a_i | o_i)$ — the quantity we just spent four paragraphs proving unaffordable. It looks like we have walked in a circle.

We have not, and the reason is the load-bearing step of this whole family of methods. Gradient descent never uses the *value* of a loss term; it uses the gradient. The optimizer does not care what $\log \pi_\theta(a_i | o_i)$ equals. It needs only a direction in weight space that raises the likelihood of a_i given o_i , scaled by w_i . A ratio is a different animal: ρ_t is a number that gets divided, clipped and compared against $1 \pm \epsilon$, none of which is possible without the value. Ratios need values; likelihood terms need only directions. That asymmetry is why one of these objectives survives the move to flow matching and the other does not.

And a direction that raises the likelihood of a_i is exactly what flow matching hands you. Its loss asks the network, at every point along the path from noise to action, to predict the velocity that carries noise toward *this* chunk given *this* observation, so descending it makes the network more likely to generate a_i from o_i . So we substitute: wherever AWR calls for a step that raises $\log \pi_\theta(a_i | o_i)$ in proportion to w_i , take a step that lowers the flow-matching loss on (o_i, a_i) with that same w_i .

Two consequences. First, the sign flips: AWR *maximizes* a log-likelihood, flow matching *minimizes* a squared error, and raising one while lowering the other moves the weights the same way — so **max** becomes **min**, and w_i multiplies a squared error instead of a log-probability. Second, be honest about the claim. The flow-matching loss is not equal to $-\log \pi_\theta(a_i | o_i)$, and nothing above proves it is; it is a *surrogate*, a different function whose gradient points the same way. The equality is never needed, because the number was never needed — only the direction. Here is that loss, from Chapter 7:

$$\min_{\theta} L(\theta) = \mathbb{E}_{i, u, z} \left[w_i \|v_\theta(a_i^u, u, o_i, c_i) - (a_i - z)\|^2 \right], \quad a_i^u = (1 - u)z + u a_i.$$

IN PLAIN WORDS

Pick a sample i from the pool. Draw a flow time u uniformly in $[0, 1]$ and a noise chunk z from a standard Gaussian. a_i is the action chunk that was executed; a_i^u is the straight-line blend between pure noise and that chunk, a fraction u along. v_θ is the network's guess at the direction that moves from the blend toward the real chunk, given the observation o_i and the conditioning c_i — typically an advantage token, an input that tells the model during training how well this chunk turned out, so that at inference you can ask it for a good one (Chapter 10). The target $a_i - z$ is the true direction of that line, the loss is the squared error between guess and target averaged over samples, flow times and noise draws, and the AWR weight w_i scales each sample's contribution. This is supervised learning; the only trace of RL is *which* samples get drawn, how heavily each counts, and what c_i says about them.

Figure 9.2 lays the two ingredient lists side by side. The crossed-out box on the right is the whole content of this section.

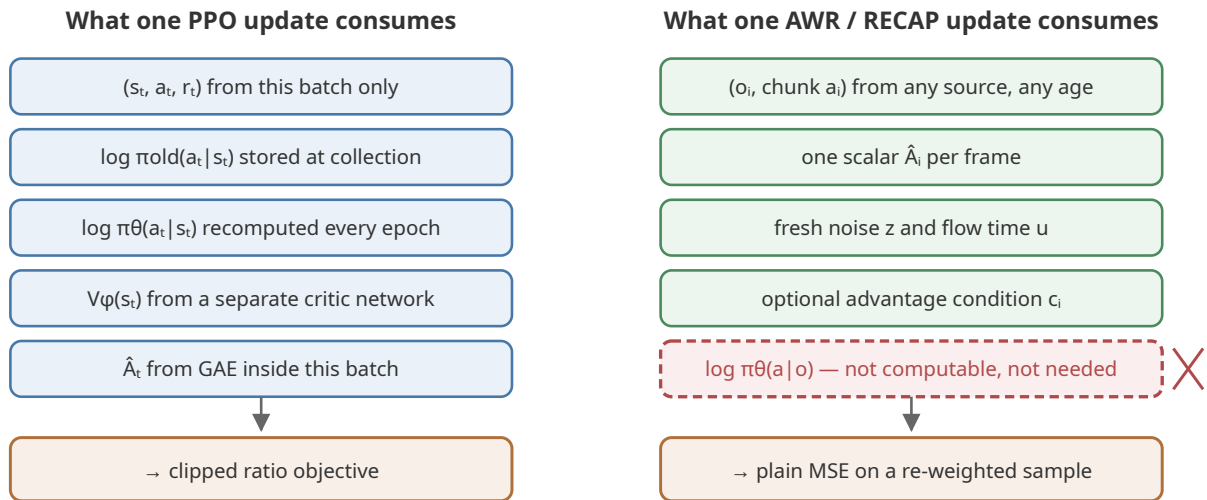


Figure 9.2 — The ingredient lists for one gradient step. Notice that three of PPO's five ingredients are probabilities of actions, and the right column has none. The red crossed-out box is the whole story of this chapter: a flow policy cannot cheaply report how likely its own action was, so every method needing that number is unavailable and the advantage must enter elsewhere.

9.3.3 (c) Where the data comes from

PPO consumes fresh on-policy data and nothing else; a batch from three updates ago is not suboptimal, it is invalid input to the surrogate. VLA-RL consumes a mixed pool: rollouts from dozens of past checkpoints, teleoperation, DAgger corrections, curriculum-selected episodes, replays from saved physics snapshots. Why forced: when an episode costs 30 seconds of simulator time or two minutes of a human's attention, discarding it after one update is unaffordable. The pool is not a compromise; it becomes the primary object you engineer (Chapter 14).

9.3.4 (d) Staying close to something

Both worlds know an unconstrained update destroys a policy, but they constrain different things. PPO keeps the policy near *its own previous version*, using the clip on ρ_t as a cheap stand-in for a KL trust region — a cap on how far one update may move the policy, measured as a KL divergence in probability space (Schulman et al., 2015; Schulman et al., 2017). Weighted BC keeps it near *the data*. Every gradient it takes has the form "reproduce this recorded chunk, harder or softer," so all such an update can do is redistribute probability mass among the actions the policy already produces with non-negligible probability; it never pushes mass toward a region the policy currently treats as essentially impossible. The temperature β is the leash length, and in the derivation it comes from a KL constraint to the behavior policy π_β (Peng et al., 2019; Nair et al., 2020).

An analogy, and its limit. PPO's trust region is a leash tied to where you stood a moment ago: any direction, just not far. Weighted BC's constraint is a fence around the ground the policy can already reach: anywhere inside, instantly, but nothing outside. There is real room inside that fence: a flow policy interpolates, which is exactly why sampling temperature, guidance and best-of- N change anything at all. Limited exploration here means narrow, not zero. Where the analogy breaks: the fence moves. Your own rollouts keep entering the pool, so the enclosed ground creeps outward into whatever the policy has recently managed to do — which is why these methods drift rather than jump, and why LeHome needed deliberate checkpoint rollbacks to escape a local optimum the fence had grown around.

9.3.5 (e) The value function

PPO trains a separate critic $V_\phi(s)$, usually a twin of the policy trunk, by squared-error regression to returns; ϕ is a second set of weights, updated alongside θ but never shared with it. VLA-RL cannot afford a second three-billion-parameter model and does not need one: Chapter 8 showed that a linear head on the policy's own query token predicts $\hat{p}_{succ}$, the probability that the episode ends in success.

A value then follows by subtraction, $V(s_t) = \hat{p}_{succ} - R_{acc}$, and that identity deserves a moment because it is not obvious. Under LeHome's scoring, once the withdrawal rule for failed episodes is applied, an episode's total reward is effectively binary, so $\hat{p}_{succ}$ — the head's probability that this episode ends in success — is its expected total reward. R_{acc} is the part of that total already banked at the checkpoints passed so far, and a value is by definition the reward still ahead. Expected total minus already collected leaves what remains: that is the whole derivation, and Chapter 11 uses the damped version of it. Why forced: cost — with a bonus, since a head sharing the backbone sees exactly the representation the policy acts on, so critic and actor cannot disagree about what they are looking at.

9.3.6 (f) The advantage estimate

PPO computes $\hat{A}_t$ with GAE inside the current batch, using γ and λ , normalized per minibatch. VLA-RL computes GAE too, with identical mechanics — the single largest piece of your PPO knowledge that survives intact — but four things around it change. First, it runs offline, over the entire pool, before each training iteration, instead of inline over one fresh batch. Second, the baseline comes from the success head rather than a critic, and it is multiplied by a damping coefficient α that shrinks it toward zero, because a baseline that is itself noisy adds variance instead of removing it. Third, a completion head supplies potential-based shaping: extra per-step reward built from a function of the state, in a form that provably cannot change which policy is optimal. Fourth, as an episode ages its per-frame value predictions — made by a checkpoint since replaced — become less trustworthy, so the estimate fades toward a GRPO-style group anchor, which compares whole episodes against the average outcome of a group rather than trusting any per-frame number. Why forced: staleness, in a form PPO never has to think about. Chapter 11 is the full treatment.

9.3.7 (g) Exploration

PPO explores by sampling from the Gaussian and by an entropy bonus that pays the policy to keep its standard deviation up. In VLA-RL there is no entropy term to add, and as 9.3(d) said, the update can only shift mass among

behaviors the policy already produces. Exploration therefore moves to the data pipeline — DAgger corrections, hard-state mining (deliberately collecting more episodes from the states where the policy currently fails), replays with heavier augmentation — and to inference: noise temperature on the initial $\mathbf{z}$, the classifier-free guidance scale $\mathbf{g}$ (generate once with the conditioning and once without, then extrapolate away from the unconditioned direction by a factor $\mathbf{g}$, which sharpens the sample toward the condition), and best-of- $\mathbf{N}$ scored by a learned head (Chapter 12). Why forced: the same missing density. The cost is real: this family explores weakly, and every serious write-up says so.

9.3.8 (h) Synchronization

PPO is a barrier; VLA-RL is a set of processes sharing a filesystem. In LeHome, trainer, rollout workers and DAgger station communicate only through a Hugging Face Hub repository, with no synchronization primitives at all. Why forced: heterogeneous hardware and timescales. Adding a worker machine should not require touching the trainer, and here it does not.

9.3.9 (i) Where the improvement actually comes from

This is the deepest difference and the easiest to miss. PPO improves the policy by ascending a gradient; the mechanism is optimization. Weighted BC and conditioning improve it by *changing which data the supervised loss sees* and then *querying the resulting model well*; the mechanism is re-weighting, re-labeling and clever sampling. Hence the knobs in these papers: per-source shares, decay factors, sampling priorities, curricula, guidance, best-of- $\mathbf{N}$, execution length — data and inference knobs, not optimizer knobs.

A second analogy, and its limit. PPO is like tuning a live service by shipping a change, measuring it against fresh traffic, and discarding the measurement the moment you ship again. VLA-RL is like curating a training corpus in version control: keep everything, label each example with how well it went, oversample the good ones, let old entries decay, retrain. Where it breaks: a corpus is static, but this one is written by the model's own past checkpoints, so a bug you introduce today can poison the pool for weeks.

9.3.10 A worked example: one transition, two updates

Take a single scalar action — say a gripper aperture delta. The pre-update Gaussian policy has mean $\mu_{old} = 0.10$ and standard deviation $\sigma = 0.20$. The action taken was $\mathbf{a}_t = 0.34$, and GAE says $\hat{\mathbf{A}}_t = +1.2$. One gradient step later the mean has moved to $\mu_{new} = 0.16$, σ unchanged. Here is the PPO arithmetic, every line:

```
z_old = (0.34 - 0.10)/0.20 = 1.20          # standardized action, old policy
log_pi_old = -0.5*(1.20^2) - ln(0.20) - 0.5*ln(2*pi)
            = -0.720 + 1.6094 - 0.9189 = -0.0295

z_new = (0.34 - 0.16)/0.20 = 0.90          # standardized action, new policy
log_pi_new = -0.5*(0.90^2) - ln(0.20) - 0.5*ln(2*pi)
            = -0.405 + 1.6094 - 0.9189 = +0.2855

rho = exp(0.2855 - (-0.0295)) = exp(0.3150) = 1.3703
clip(rho, 0.8, 1.2) = 1.2000              # eps = 0.2
unclipped term = 1.3703 * 1.2 = 1.6443
clipped term = 1.2000 * 1.2 = 1.4400
objective = min(1.6443, 1.4400) = 1.4400 -> the CLIPPED branch wins
                                     -> gradient from this sample is zero
```

The mechanism is visible: the policy has already moved more than $1 + \epsilon$ in the direction this sample was asking for, so PPO switches the sample off. (Had $\hat{\mathbf{A}}_t$ been negative with the same ratio, **min** would have picked the unclipped branch and the sample would still be pulling.) Every number above needed $\log \pi$.

Now the other world, after one preliminary. The equation above puts $\mathbf{w}_i$ on the loss; the code below puts it in the sampler. Those are the same thing in expectation — drawing a sample twice as often and multiplying its loss by two produce the same average gradient — and the sampler version is strictly cheaper on a robot pool, because a frame that

is never drawn is never decoded, never loaded, and never pushed through a three-billion-parameter backbone. So the arithmetic below computes weights and then reads them as sampling probabilities.

Take a pool of four frames with advantages $+1.2, +0.3, -0.4, -1.5$ and $\beta = 0.5$, and sample proportionally to $w_i = \exp(\hat{A}_i/\beta)$:

```
w1 = exp(+1.2/0.5) = exp(+2.4) = 11.0232
w2 = exp(+0.3/0.5) = exp(+0.6) = 1.8221
w3 = exp(-0.4/0.5) = exp(-0.8) = 0.4493
w4 = exp(-1.5/0.5) = exp(-3.0) = 0.0498
sum = 13.3444

p1 = 11.0232/13.3444 = 0.8261    p2 = 1.8221/13.3444 = 0.1365
p3 = 0.4493/13.3444 = 0.0337    p4 = 0.0498/13.3444 = 0.0037
```

Frame 1 is drawn 83 percent of the time; the badly-failing frame 4 once in 270 draws. LeHome does not tune β at all — it fixes $\beta = 1$ and clips $\hat{A}_i$ into $[-2, 2]$ before exponentiating, so the heaviest frame can be at most $e^4 \approx 55$ times the lightest, which is what stops one lucky episode from owning every batch.

Say frame 1 is drawn. It is the same transition as the PPO example above: the same gripper-aperture action $a_i = 0.34$ in the same observation $\mathbf{o}_i$ — camera images and joint states, which the network consumes but which never show up in the arithmetic — which is why its advantage is again $+1.2$. Real chunks are 360 numbers wide; a single scalar keeps this readable, and nothing about the loss changes with dimension except that the squared error is summed over coordinates. With flow time $u = 0.3$ and noise draw $z = -0.85$:

$$a^u = (1 - 0.3)(-0.85) + (0.3)(0.34) = -0.5950 + 0.1020 = -0.4930, \quad a - z = 0.34 - (-0.85) = 1.1900.$$

If the network predicts $v_\theta(a^u, u, \mathbf{o}) = 0.95$, the loss is $(0.95 - 1.19)^2 = (-0.24)^2 = 0.0576$. That is the entire update: a squared error on a sample chosen because its advantage was high. No ratio, no clip, no old policy, no critic pass.

```
# --- PPO: the update needs the old policy's log-probs ---
logp_old = batch.logp                                # frozen, stored at collection time
for epoch in range(10):                              # several passes over the SAME batch
    logp = policy.log_prob(batch.obs, batch.act)      # recomputed every epoch
    ratio = torch.exp(logp - logp_old)               # this is rho_t
    loss = -torch.min(ratio * batch.adv,
                      torch.clamp(ratio, 1-eps, 1+eps) * batch.adv).mean()
    loss = loss + vf_coef * v_loss - ent_coef * entropy # critic + exploration
batch.discard()                                       # now off-policy: unusable

# --- VLA-RL: the update never asks how likely the action was ---
idx = sample(pool, p=softmax(clip(pool.adv, -2, 2))) # advantage enters HERE
o, a = pool.obs[idx], pool.chunk[idx]               # a is a whole H-step chunk
u = torch.rand(len(idx))                            # flow time in [0,1]
z = torch.randn_like(a)                             # fresh noise, same shape
a_u = (1-u)[: , None, None] * z + u[: , None, None] * a # straight-line interpolant
v = model.velocity(a_u, u, o, cond=adv_token[idx])   # advantage also conditions
loss = ((v - (a - z)) ** 2).mean()                  # unweighted: w_i lives in the sampler
```

9.3.11 The comparison as a reference table

Component	PPO	VLA-RL (AWR / RECAP)
Policy class	Gaussian over one action	Flow-matching generator over a chunk $\mathbf{a}_{t:t+H}$
Update input	ρ_t , needs $\log \pi_\theta$ and $\log \pi_{\theta_{old}}$	Regression target plus one scalar $\hat{A}_t$
Data	Fresh on-policy batch, then discarded	Persistent mixed pool: rollouts, BC, DAgger, replays
Staying close	Clip ϵ / KL to $\pi_{\theta_{old}}$	Stay inside the data; leash length is β
Value function	Separate critic V_ϕ	Auxiliary head on the policy trunk, $V = \hat{p}_{succ} - R_{acc}$
Advantage	GAE inside the batch, per-minibatch normalized	GAE offline over the pool, damped baseline, group anchor
Exploration	Gaussian noise plus entropy bonus	Temperature, guidance $\mathbf{g}$, best-of- N , human corrections
Synchronization	Lockstep barrier	Fully asynchronous through a shared store
Source of improvement	Gradient ascent on a surrogate objective	Re-weighting the data, then querying the model well

PITFALLS

The characteristic error here is importing PPO reflexes into a weighted-BC pipeline, where they are not wrong so much as meaningless. There is no ratio, so tuning ϵ or adding KL early-stopping does nothing. There is no closed-form entropy, so an entropy bonus has nothing to attach to — the diversity knob is the sampling temperature at inference. Per-minibatch advantage normalization is actively harmful when advantages come from an offline pass over a pool: use a global scale from unbiased rollouts, or a batch of pure failures will manufacture positive advantages out of nothing. And "run more environment steps" is usually the weakest lever you have.

9.4 What transfers unchanged

Now the reassuring part: your PPO experience is the vocabulary the new world is written in.

The MDP framing. States, observations, actions, transitions, episodes, discounting: identical. When LeHome sets $\gamma = 0.999$, you know why a 600-frame horizon demands a number that close to one.

Returns, values, advantages. G_t , $V^\pi(s)$, $Q^\pi(s, a)$ and $A^\pi = Q^\pi - V^\pi$ mean exactly what they meant in Chapter 1. Chapter 8's future-prediction head is "Q-like" because it conditions on the actions being generated; the success head is "V-like" because it does not. And "how much better than the policy's own average was this?" is the same question in both worlds. Only the consumer changes: a multiplier on a log-probability gradient in one, a sampling probability and a conditioning token in the other.

GAE mechanics and baselines. The backwards recursion over δ_t , the bias-variance dial on λ , the bootstrap at episode boundaries — unchanged, right down to the code; the loop just runs offline over a pool instead of inline over a batch. And Chapter 2's proof that subtracting any state-dependent baseline leaves the gradient unbiased while shrinking its variance is exactly what licenses the damped baseline $\alpha(\hat{p}_{succ} - R_{acc})$ and the group anchor of Chapter 11.

Curve-reading. Watching for value-loss explosions, distrusting a success curve that climbs while the value head saturates, noticing a run that has collapsed to one behavior — these instincts transfer completely, and matter more here, because in an asynchronous pipeline a bad checkpoint keeps poisoning the pool long after a lockstep loop would have shown you the problem.

9.5 What does not transfer

And the honest list of what to put down.

The ratio, the clipped objective, and per-step log-probabilities. Gone, for the reason worked through in 9.3(b), and with them every method whose derivation needs $\log \pi_\theta(a | s)$: vanilla policy gradients, PPO, GRPO, most maximum-entropy RL (the family, SAC among them, that adds an entropy term to the reward itself). If a paper here mentions clipping, look twice — it is almost certainly clipping the *advantage* into a range like $[-2, 2]$ before exponentiating it, which is a different operation entirely.

The entropy bonus as the exploration knob. To get more diverse behavior from a flow policy you raise the noise temperature, lower the guidance scale g , or add demonstrations of the behavior you want. You do not add a term to the loss.

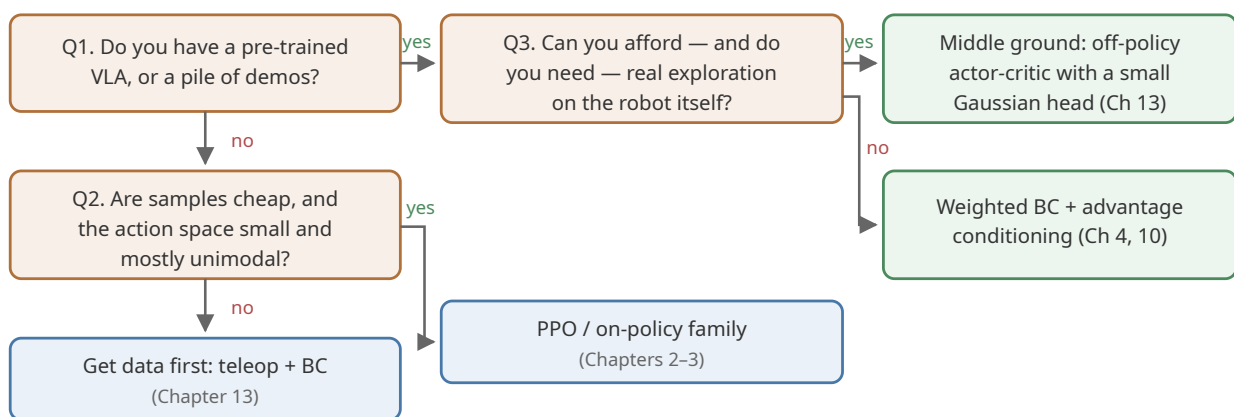
Lockstep batches, and the instinct that environment steps are the main lever. Three checkpoints are in flight at once, and every episode carries value predictions from whichever one collected it. In PPO more steps almost always helps; here the marginal 500 rollouts are usually worth less than fixing per-source sampling shares, adding 50 targeted DAgger corrections, or re-tuning execution length. That is the hardest habit to unlearn, because it was correct for years.

ON YOUR RIG

Your three projects sit at three different points on this comparison, which is a lucky position to be in. **rl-rover** is the pure left-hand column: small action space, cheap simulator, Gaussian head, lockstep PPO, entropy bonus and all. **hil-serl** on the SO-101 is the middle ground of 9.6: off-policy actor-critic with a small Gaussian head, so log-probs exist and Q-learning is on the table, but data lives in a buffer and a human supplies exploration — your 98 percent intervention run is that dial turned all the way up. A LeRobot flow-matching policy on the same arm would be the right-hand column: same hardware, entirely different training loop.

9.6 When to use which

Neither family is universally better. The decision is usually made for you by two facts: what a sample costs, and whether you have a prior worth keeping. Figure 9.3 puts the questions in order.



Sample cost and the existence of a prior decide this, not algorithmic taste.

Figure 9.3 — A decision order, not a ranking. There are four blue and green landing places: collect data first, the PPO family, the off-policy middle ground, and weighted BC with advantage conditioning; the orange boxes are the questions that route you to one of them. Notice the first question asks what you already have rather than which algorithm you prefer, because a pre-trained VLA or a demonstration set is an asset on-policy methods cannot exploit. The middle-ground box is where a physical arm with a human at the controls usually lands.

Reach for the PPO family when the spaces are small, the simulator is fast, there is no pre-trained prior to preserve, and you need the policy to discover behavior nobody has demonstrated — locomotion gaits, a rover learning to drive, game agents. PPO's exploration is real exploration.

Reach for the weighted-BC and conditioning family when samples are expensive, demonstrations exist or can be produced, the action distribution is genuinely multimodal, and you start from a pre-trained VLA whose broad competence you must not destroy. Real-world manipulation is almost always in this box. What you accept: limited exploration, in exchange for stability and the use of every episode you have ever collected.

The middle ground is real and you already run it. HIL-SERL keeps a small Gaussian head, so log-probabilities still exist and off-policy actor-critic and Q-learning remain available — and Q-learning genuinely propagates value backwards through a buffer in a way weighted BC does not. What it gives up is the multimodal chunk generator and the pre-trained visual-language prior; exploration comes from a human on an Xbox pad. For one task on one arm it is often the fastest path to a working policy (Chapter 13).

9.7 One task, seen through both lenses

Make it concrete. The task: fold a towel lying flat on a table with a bimanual arm setup, judged by a geometric check on keypoint positions at the end of the episode.

9.7.1 The PPO attempt

You would write a Gymnasium environment around a physics simulator: a 12-dimensional joint-velocity action space, a Gaussian over those 12 numbers at 30 Hz, 2048 steps per batch, four epochs, $\epsilon = 0.2$. You would fight the reward first, because binary success at the end of a 600-step episode is a needle in a haystack, so you shape it into per-step terms for corner proximity, flatness and grasp contact.

Then it gets stuck, in three ways. Exploration: random joint-space noise essentially never grasps a cloth corner, so your shaping terms are the only thing keeping the run alive, and the policy finds whatever degenerate behavior maximizes them — dragging the towel into a heap that scores well on flatness is the classic. Multimodality: folding left-first and right-first are both correct, and a Gaussian averages them into a motion that grabs neither. Sample cost: a deformable-cloth step is not microseconds, and PPO's appetite runs to tens of millions of them.

9.7.2 The VLA-RL attempt

You start from a pre-trained VLA for which grasping a cloth corner is behavior it can already produce — unreliably, but it produces it — and fine-tune on a few hundred demonstrations. Now the flywheel: workers collect episodes, most failing; a success head trained on those binary outcomes gives $\hat{p}_{succ}$ and hence a value; a completion head gives shaping; GAE over the pool gives per-frame advantages. Training draws frames with probability proportional to $\exp(\hat{A}_i/\beta)$, so good episodes dominate the gradient. Multimodality is a non-issue: the flow model represents both fold orders, and best-of- N picks between them per chunk.

Then *this* gets stuck, also in three ways. Exploration: if the policy never once succeeds on a garment type, the advantage signal is flat and there is nothing to re-weight, so you must go collect demonstrations — the algorithm will not invent the behavior. Staleness: predictions attached to two-week-old episodes came from a much worse checkpoint, and unless you decay or re-anchor them they drag the advantage estimate. Local optima: the fence closes around a mediocre strategy, and LeHome's answer was to roll back to an earlier checkpoint and re-diversify, three times in the online round alone.

IN THE LEHOME SOLUTION

Every row of the right-hand column is visible in the winning run (Larchenko, 2026). No log-probability appears anywhere: the advantage enters through the sampler as $P(i) \propto e^{\text{clip}(\hat{A}_i, -2, 2)}$ — the same w_i at $\beta = 1$, moved off the loss — leaving the flow-matching loss a plain unweighted MSE, plus a RECAP-style advantage token conditioning generation. The value is the success head, $V = \hat{p}_{\text{succ}} - R_{\text{acc}}$, damped by $\alpha = 0.5$. Trainer, rollout workers and the DAgger station share only a Hugging Face repository. The final pool held roughly 12,500 rollout episodes and 4.3 million frames, old datasets decaying by 0.98 per iteration. Exploration lived at inference: guidance g around 7 to 9, best-of- N with $N = 2$ or 3.

CHECKPOINT

- Draw both loops from memory and say, arrow by arrow, which are hard dependencies and which are asynchronous handoffs — and why PPO's loop cannot be made asynchronous.
- Explain why $\rho_t(\theta)$ is unavailable for a flow-matching policy, roughly what computing it would cost, and why the $\log \pi_\theta$ sitting inside AWR is not the same problem: a ratio needs the number, a likelihood term needs only the gradient.
- Given an advantage and a temperature β , compute an AWR weight and a set of sampling probabilities, and say what the same advantage would have done inside PPO's clipped objective.
- List what transfers from your PPO knowledge (MDP, returns, advantages, GAE, baselines, normalization discipline) and what does not (ratios, clipping, entropy bonuses, lockstep batches).
- Place a new task on Figure 9.3 and defend the choice by sample cost and the existence of a prior, not preference.
- Name three failure modes specific to each family on a concrete task such as towel folding.

Where to go deeper. Re-read the PPO paper (Schulman et al., 2017) and the AWR paper (Peng et al., 2019) back to back in one sitting; the contrast is far sharper with the two objectives on adjacent screens. Then read sections 2.2 and 2.3 of the LeHome report (Larchenko, 2026), which is where the argument in 9.3(b) comes from — that log-prob policy gradients do not transfer to flow-matching VLAs and that push-away updates leave the action manifold — and the RECAP section of $\pi_{0.6}^*$ (Physical Intelligence, 2025b) for the advantage-conditioning recipe itself and the systematic-restart practice. Chapters 10 through 12 build out the right-hand column; Chapter 13 covers the middle ground you already run.

Chapter 10 — Learning by conditioning: RECAP

Chapter 9 left you in an awkward place. The generative policies of Chapters 6 through 8 are the ones that actually work on real robots, and the PPO machinery you already know does not attach to them cleanly: no tractable log-probability, so no importance ratio $\rho_t(\theta)$, so no clipped surrogate. Chapter 4 gave you one escape hatch — advantage-weighted regression, which improves a policy by reweighting a supervised loss and never needs a ratio at all. This chapter gives you the other escape hatch, the one that powers Physical Intelligence's $\pi_{0.6}^*$ and Larchenko's LeHome solution.

It is a strange idea the first time you meet it, and almost embarrassingly simple once it lands. We are going to stop trying to make the *loss* prefer good actions, and instead make the *model* aware of which actions were good — then simply ask it, at inference time, for the good ones. Let us get there slowly.

10.1 The idea, motivated slowly

10.1.1 What we want

Here is the situation. You have a buffer of robot data: chunks $\mathbf{a}_{t:t+H}$ paired with the observations $\mathbf{o}_t$ they were taken from. The data is of mixed quality. Some of it came from demonstrations, some from your current policy's own rollouts, some of those rollouts succeeded and some failed. You have a critic that can score each chunk with an advantage estimate $\hat{\mathbf{A}}_i$ — Chapter 11 builds that estimator; for now assume it exists and is roughly trustworthy.

What you want is a new policy that behaves like the good parts of that buffer and not like the bad parts. That is the whole goal. Everything below is about how to get there without a policy gradient.

10.1.2 The obvious thing to try, and exactly how it hurts

The obvious thing is Chapter 4's answer: clone everything, but louder where the advantage is high. Advantage-weighted regression assigns each sample the weight $w_i = \exp(\hat{\mathbf{A}}_i/\beta)$, with β a temperature you choose, and minimizes the weighted behavior-cloning loss (Peng et al., 2019). It is correct, it is simple, and it works. But it has a cost that is easy to miss because it is hidden inside an exponential.

Let us make it concrete. Take a tiny buffer of six chunks with advantages $+1.8, +0.6, +0.1, -0.2, -0.9, +0.3$ — these same six numbers will follow us through the whole chapter. Set the temperature $\beta = 0.5$, a common choice. Compute every weight and then normalize, so we can read each chunk's share of the total gradient signal:

```
A-hat:      +1.8    +0.6    +0.1    -0.2    -0.9    +0.3

w = exp(A / 0.5) = exp(2A):
exp(+3.6) = 36.598    exp(+1.2) = 3.3201    exp(+0.2) = 1.2214
exp(-0.4) = 0.6703    exp(-1.8) = 0.1653    exp(+0.6) = 1.8221

sum of weights = 36.598 + 3.3201 + 1.2214 + 0.6703 + 0.1653 + 1.8221
                = 43.797

share = w / sum:
chunk 1  36.598 / 43.797 = 0.836    -> 83.6% of the gradient
chunk 2   3.3201 / 43.797 = 0.076    -> 7.6%
chunk 3   1.2214 / 43.797 = 0.028    -> 2.8%
chunk 4   0.6703 / 43.797 = 0.015    -> 1.5%
chunk 5   0.1653 / 43.797 = 0.004    -> 0.4%
chunk 6   1.8221 / 43.797 = 0.042    -> 4.2%
```

Look at what happened. One chunk out of six absorbed 83.6% of the training signal. The worst chunk, the one with $\hat{\mathbf{A}} = -0.9$, got 0.4% — it is present in the dataset, it came from an episode that cost you thirty seconds of real robot

time, and it is contributing essentially nothing. You collected a failure and then threw it away.

Before we build anything on that number, be clear about what it is a property of. The 83.6% is a property of the *temperature*, not of AWR. β controls how sharply the exponential separates good from bad, and it is a knob you turn. Run the same six advantages at $\beta = 1$ and the top chunk's share drops to 52.4%, while the worst chunk climbs from 0.4% to 3.5%. At $\beta = 2$ the top chunk takes 32.5% and the worst takes 8.4% — a buffer that is being used, not a buffer that has quietly become one sample.

Production implementations blunt the tail further, in two ways this chapter's toy calculation leaves out. Peng et al. (2019) cap each weight at a maximum value — 20 in their experiments — which on these six numbers alone pulls the top share from 83.6% down to 73.5%. Larchenko clips $\hat{A}$ into $[-2, 2]$ before exponentiating, so that one wildly overconfident critic estimate cannot swallow the batch (Larchenko, 2026). So do not walk away believing that AWR always throws 83.6% of its attention at one sample. It throws as much as you let it.

That is the first cost, and notice that it is a tunable one: **at a low temperature, failures are effectively discarded**, even though failures carry real information. A failed fold shows you what a stalled grasp looks like, which sleeve slips, where the policy dithers instead of committing. At $\beta = 0.5$ almost none of that reaches the model. Raise the temperature and more of it does — but a higher temperature also flattens the very preference you introduced the weights to express, so you are trading one thing you want for another.

The second cost is subtler and, I think, the more important one. **Weighting is invisible to the model.** The network never learns that chunk 1 was good and chunk 5 was bad. It only sees chunk 1 more often, or with a bigger multiplier on its loss. There is no representation inside the network of the concept "this was a good action." The quality information exists only in the training loop, and it evaporates the moment training ends.

Now notice which of those two costs survives tuning, because that is the one this chapter is really about. The first cost is a dial setting. Pick a gentler temperature, cap the weights, clip the advantages, and the failures come back into the gradient. The second cost does not move at all. No temperature, no cap, and no clip makes the weight visible to the network, because the weight lives in the optimizer's arithmetic and never enters the forward pass at any temperature whatsoever. That is the durable defect.

Say that plainly, because it also settles a question you would otherwise carry for the rest of the chapter: we are not about to demolish weighting. Weighting works. Section 10.4.2 will recommend running it *alongside* what follows, and that is not a contradiction of anything above — the fix in this chapter supplies exactly the thing weighting structurally cannot supply, which is a representation inside the model of what "good" means.

10.1.3 The flip: annotate instead of reweight

Here is the fix. Do not change how loudly each sample speaks. Change what each sample says.

Attach the advantage to the input. Train an **advantage-conditioned policy** — a policy that takes an extra input c describing how good the demonstrated action was — written $\pi_\theta(a_{t:t+H} \mid o_t, c)$. The label c can be a binary indicator, a small set of bins, or a learned token; the binary indicator is the workhorse. Now training is plain supervised learning on *all* the data at equal weight. Successes teach the model what $c = +$ behavior looks like. Failures teach it what $c = -$ behavior looks like. Each sharpens the other by contrast: knowing what a bad fold looks like makes the boundary of a good fold crisper. Nothing is thrown away (Figure 10.1).

An analogy from your day job. Imagine you are building a model to suggest code patches, and you have a corpus of ten thousand pull requests, each labeled by a reviewer as "approved" or "rejected." Two strategies. Strategy A: delete every rejected PR and train on what remains. Strategy B: keep everything, feed the reviewer's verdict in as an input token, and at suggestion time ask the model for an approved-style patch. Strategy B has strictly more to learn from — it can see the near-misses, the off-by-ones, the patches that were almost right. Strategy A never sees a bug.

Where the analogy breaks. Reviewer verdicts are given by humans who understood the code. Your c labels come from a learned critic that may be miscalibrated, may drift as it trains, and produces a continuous number you had to threshold somewhere. A wrong reviewer label is rare; a wrong advantage label is routine. That is why Section 10.5

spends so much effort on *where* to put the threshold, and why the binary form — which only asks the critic for a ranking, not a magnitude — is more robust than it looks.

10.1.4 The objective

The standard construction labels each chunk with an indicator thresholded at ϵ_A , and trains the conditioned policy jointly with an unconditioned one:

$$c_i = \mathbb{1}(\hat{A}_i > \epsilon_A), \quad \min_{\theta} \mathbb{E}_{(o_i, a_i, c_i) \sim D} \left[-\log \pi_{\theta}(a_i | o_i) - \eta \log \pi_{\theta}(a_i | o_i, c_i) \right]$$

IN PLAIN WORDS

$\hat{A}_i$ is the advantage your critic assigned to sample i , and ϵ_A is a threshold you pick. $\mathbb{1}(\cdot)$ is the indicator function: 1 when the statement inside it is true, 0 otherwise. So c_i is just "was this chunk better than the threshold, yes or no." In the second line, $\min_{\theta}$ says adjust the policy parameters θ to make what follows small, and $\mathbb{E}_{(o_i, a_i, c_i) \sim D}$ says average over triples drawn from your dataset D : the observation o_i , the chunk $a_i = a_{t:t+H}$ that was actually taken, and its label c_i . The term $-\log \pi_{\theta}(a_i | o_i)$ is ordinary behavior cloning. The term $-\log \pi_{\theta}(a_i | o_i, c_i)$ is the same thing with the label also fed in, weighted by the mixing constant η .

Two notes on those symbols before we go on. ϵ_A is a threshold on advantages, and it has nothing to do with the PPO clip parameter ϵ of Chapter 3 — which is why it carries a subscript. The two collide constantly in the literature and it is worth keeping them apart in your head from the start. And η sets how much you care about the conditioned branch relative to the unconditioned one; turn it to zero and you are doing plain behavior cloning with an unused input.

One network, two jobs, both learned from the same data. That phrasing should immediately raise a question, and if you have never built a conditional generative model it is the question that stalls people right here: the first term asks for $\pi_{\theta}(a_i | o_i)$, with no c in it at all — so how does a single network evaluate that, when c is one of its inputs? It cannot simply leave the input out; a network has a fixed input shape.

The answer is that the c slot always exists in the architecture. There is a place for it in every forward pass, and the unconditioned branch is obtained by putting a dedicated neutral value in that slot — a **null token**, a value whose meaning is "no label given." Read that again, because it is the piece of the trick that never gets written down: "no label" is itself a label. It is a specific vector the network sees thousands of times during training, and it learns a distribution to associate with it, exactly as it does for $c = +$ and $c = -$. Two lines of Section 10.5.1's code block do this concretely — build the conditioning token from c for most samples, substitute the null token for the rest — and it may help to glance ahead at them now.

Now the step that is usually left as intuition and should not be: *why* does asking for $c = +$ give you better actions? The reason is what maximum likelihood converges to. Minimizing $-\log \pi_{\theta}(a | o, c)$ over your dataset drives the model toward the empirical conditional distribution of a given the pair (o, c) . That is what minimizing a negative log-likelihood does, and it is the only thing it does. Now ask what that empirical distribution is when you plug in $c = +$. It is, by construction, the distribution of the actions in your buffer that were taken at observations like o and carried an above-threshold label. So $\pi_{\theta}(a | o, c = +)$ is the fit to the good subset of your data. Not an approximation of it, not a nudge in its direction: it is the fit to that subset, because the split has already been performed by the conditioning variable instead of by a weight. AWR approximates that same split softly, with an exponential multiplier on the loss. Conditioning makes it exactly, by partitioning the input space instead — which costs nothing and keeps every sample.

There is a corollary worth holding onto, because the chapter's main failure mode falls straight out of it. The network will use c only to the extent that c carries information about a that o does not already carry. Suppose it does not — suppose that at some observation, knowing o already determines the label. Then the empirical distribution of a given $(o, c = +)$ and the one given $(o, c = -)$ are the same distribution there, maximum likelihood fits them identically, and the network has no reason to represent c at all. That is conditioning collapse. You will meet it by name in Section

10.2 and in the pitfall box, and when you do, notice that it is not a separate phenomenon bolted onto the method: it is this same maximum-likelihood fact read from the other side.

At inference you fix $c = +$ and sample. Mechanically nothing about generation changes, and it is worth being explicit about that rather than leaving you to infer it. If the policy is a flow policy — Section 10.2 — you run exactly the K -step integration of Chapter 7: start from a fresh noise draw z , and step the learned velocity field from $u = 0$ to $u = 1$, with the single difference that the c slot holds the positive token at every one of those K steps. You do not sample a label, you do not average over labels, and at no point do you evaluate a probability. You are asking the model a counterfactual question: “in this state, act the way the above-threshold data acted.” By the argument just given, the $c = +$ distribution is the fit to the above-threshold subset, so it concentrates on the better modes of the data (Figure 10.2). Read that sentence again, because it is the hinge of the chapter: **improvement happens at query time, not in the loss**. The loss is pure maximum likelihood. It has no idea it is doing reinforcement learning.

This trick has a name: **upside-down RL** — so called because it inverts the usual arrangement in which rewards drive the gradient. Here the reward information is an input, and the gradient is plain supervised regression. You have turned reinforcement learning into supervised learning plus a well-chosen question.

The honest caveat is exactly AWR's caveat. Conditioning redistributes probability mass *within the support of the data*. It selects; it does not explore. If no chunk in your buffer ever grabbed the collar correctly, no amount of conditioning will invent that grasp. To keep improving you re-collect rollouts with the improved policy, re-score them, and iterate — which is precisely why Chapter 14's asynchronous loop exists.

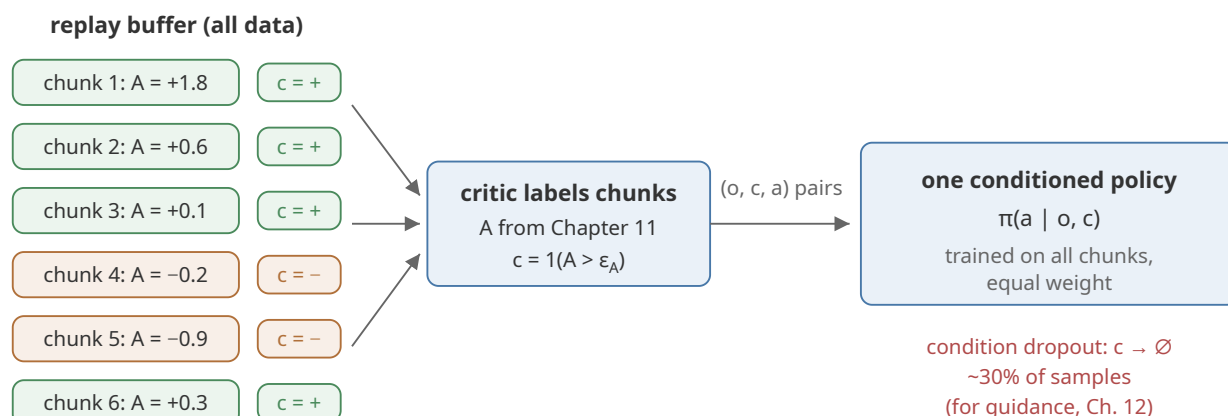


Figure 10.1 — Training by conditioning. Notice that every chunk in the buffer, successes and failures alike, flows into the same policy along the same arrow — nothing is filtered and nothing is downweighted. The advantage label is an input, not a weight, which is the entire structural difference from Chapter 4. The red annotation at the bottom is the detail everyone forgets on their first attempt, and the $\emptyset$ in it means “no label” — the null token that stands in for c on those samples; Section 10.5 explains why it matters.

WORDS YOU WILL HEAR

Conditioning — giving a generative model an extra input that describes the sample it should produce, then choosing that input at generation time. **Upside-down RL** — the family of methods that feed reward information in as input rather than using it in the gradient. **Return-to-go G_t** — the sum of rewards from time t to the end of the episode; what Decision Transformers condition on. **Percentile threshold** — a cut point defined by rank rather than by value, so it moves as the critic's scale drifts. **Condition dropout** — replacing the label with a neutral **null token** on a fraction of training samples, so the same network also learns the unconditioned distribution. **AdaRMS** — adaptive RMS normalization, where a conditioning vector modulates each layer's normalization gain.

10.2 Why this works so well for flow policies

Everything above would apply to any policy class. But conditioning is not merely *compatible* with flow-matching and diffusion policies — it is the natural fit, and it is worth being precise about why.

Recall the wall from Chapter 9. A flow-matching policy defines its action distribution implicitly, by integrating a learned velocity field from noise to action. There is no closed-form $\pi_\theta(\mathbf{a} \mid \mathbf{o})$ you can evaluate. That kills the importance ratio $\rho_t(\theta) = \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t) / \pi_{\theta_{old}}(\mathbf{a}_t \mid \mathbf{s}_t)$, and with it PPO's clipped objective. It also kills the more general family of updates that push probability mass *away* from bad actions, because "away" has no well-defined direction on a manifold you can only sample from — in practice those updates shove predictions off the narrow manifold of coherent action chunks, and the policy starts emitting motions that are not merely wrong but physically incoherent.

Conditioning asks nothing of the loss. It does not need a ratio, a probability, or a direction to push away from. The label $\mathbf{c}$ simply joins $\mathbf{o}$ as another input to the velocity field, which becomes $\mathbf{v}_\theta(\mathbf{a}^u, \mathbf{u}, \mathbf{o}, \mathbf{c})$. The regression target is untouched:

$$\mathcal{L}(\theta) = \mathbb{E}_{(\mathbf{o}, \mathbf{a}, \mathbf{c}) \sim D, \mathbf{u} \sim \mathcal{U}[0,1], \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left\| \mathbf{v}_\theta(\mathbf{a}^u, \mathbf{u}, \mathbf{o}, \mathbf{c}) - (\mathbf{a} - \mathbf{z}) \right\|^2, \quad \mathbf{a}^u = (1 - \mathbf{u})\mathbf{z} + \mathbf{u}\mathbf{a}$$

IN PLAIN WORDS

$\mathcal{L}(\theta)$ is the number you minimize by adjusting the network weights θ . The expectation $\mathbb{E}$ averages over three independent draws: a training triple $(\mathbf{o}, \mathbf{a}, \mathbf{c})$ from your dataset D , being the observation, the recorded action chunk, and its advantage label; a flow time $\mathbf{u}$ drawn uniformly on $[0, 1]$, saying how far along the noise-to-action path you are; and a noise sample $\mathbf{z}$ from a standard Gaussian $\mathcal{N}(\mathbf{0}, \mathbf{I})$, shaped like the chunk. The interpolant $\mathbf{a}^u = (1 - \mathbf{u})\mathbf{z} + \mathbf{u}\mathbf{a}$ is the straight-line blend of pure noise at $\mathbf{u} = \mathbf{0}$ and the real action at $\mathbf{u} = \mathbf{1}$. The network $\mathbf{v}_\theta$ sees that blend, $\mathbf{u}$, $\mathbf{o}$, and now $\mathbf{c}$, and outputs a velocity. The target $\mathbf{a} - \mathbf{z}$ is that line's constant velocity, and the double bars squared are ordinary squared error.

That is the conditional flow-matching loss of Chapter 7 with exactly one thing added, and the addition is worth staring at for a moment in the argument list of $\mathbf{v}_\theta$. The label $\mathbf{c}$ has simply joined $\mathbf{o}$ among the things the velocity field is allowed to look at. It is conditioned on, not optimized against. Nothing here multiplies a term by $\hat{\mathbf{A}}_i$, sorts the samples, or skips any of them. No term of this loss knows or cares that $\mathbf{c}$ encodes reward.

That last sentence deserves a careful qualification, because it is easy to overclaim. The joint objective of Section 10.1 was written with $-\log \pi_\theta$ terms, and I just told you a flow policy has no tractable log-probability. Both are true. For a flow-matching policy, each $-\log \pi_\theta$ term in that objective *stands in for* the tractable surrogate you already train — the flow-matching regression above. The correct statement of the difficulty is therefore narrow: intractable likelihoods rule out **ratio-based gradients**, which need to evaluate and divide two probabilities. They do not rule out **maximum-likelihood-style surrogates**, which only need a training signal that pulls the model toward observed data. Conditioning lives entirely in the second category. That is the whole reason it transplants onto a flow policy without a fight.

A second analogy, and its limit. Think of AWR and RECAP as two ways of shipping a behavior change. AWR trains a *different function*: same signature, different training distribution, and the only way to get a different behavior is to retrain. RECAP adds a *parameter* to the same function: one function, and you pick the behavior at call time by passing $\mathbf{c}$. Anyone who has replaced a forked code path with a flag will recognize the appeal. Where the analogy breaks: a flag in code is honored exactly, by construction. A conditioning input is honored only to the extent the network found it useful during training. If the observation already predicts the label — if all your failures happen to involve the striped shirt — the network will read the shirt and ignore the flag, and flipping $\mathbf{c}$ at inference will do nothing at all. That failure has a name, conditioning collapse, and it is in the pitfall box below.

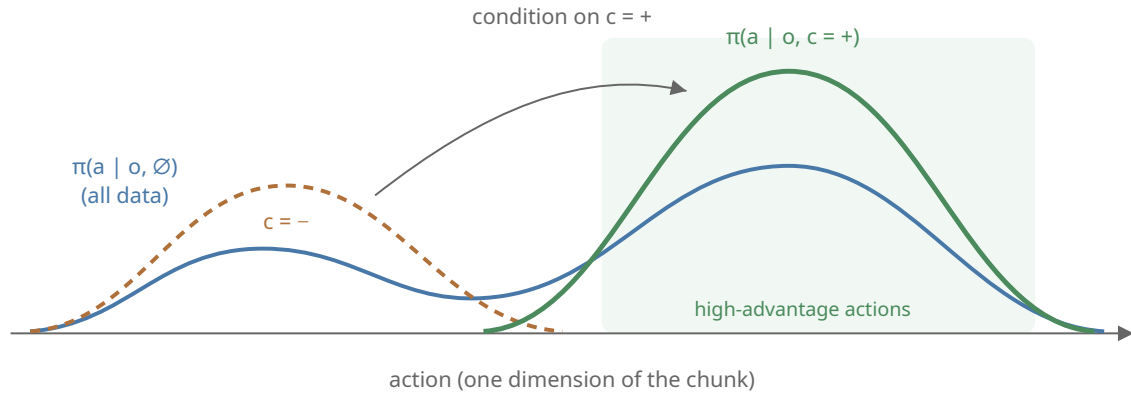


Figure 10.2 — Inference by conditioning. The blue curve is what the policy produces with no label: the whole data mixture, including the left-hand mode that came from failures. Querying $c = +$ selects the right-hand mode instead — the model has not learned a new action, it has learned which of its existing modes to prefer. The gap between the green and orange curves is the quantity classifier-free guidance will amplify in Chapter 12.

10.3 Where this comes from: $\pi_{0.6}^*$ and RECAP

Advantage conditioning is the backbone of Physical Intelligence's $\pi_{0.6}^*$ (Physical Intelligence, 2025b). Their recipe has an acronym worth memorizing because it tells you the whole story: **RECAP**, for *RL with Experience and Corrections via Advantage-conditioned Policies*. Experience means autonomous rollouts. Corrections mean human interventions. Advantage-conditioned policies mean the mechanism of this chapter.

10.3.1 The model and the critic

The policy is the $\pi_{0.5}$ architecture of Chapter 8 (Physical Intelligence, 2025a) scaled up: a Gemma 3 4B vision-language backbone with an 860M-parameter flow-matching action expert. The only architectural addition is the advantage indicator, and they insert it in the most literal way imaginable — as a text token in the input sequence, reading “Advantage: positive” or “Advantage: negative.” The VLM already knows how to read text; this costs nothing to implement.

The value function is a separate, smaller model: a 670M-parameter Gemma 3 VLM trained as a **distributional critic** — one that predicts a full probability distribution over possible values rather than a single number. It outputs a categorical distribution $p_\phi(\mathbf{V} \mid \mathbf{o}_t, \ell)$ over 201 discretized value bins, where ℓ is the task command, and is trained with a cross-entropy loss against the Monte Carlo return-to-go of each episode. Two details are worth pausing on. First, there is no bootstrapping: the target is the actual sum of rewards observed to the end of the episode — the paper’s formulation applies no discounting, so $\gamma = 1$ there — rather than $r_t + \gamma V(s_{t+1})$. That trades variance for the removal of an entire class of instability. Second, the critic is a separate model, not a head on the policy — a design choice that Chapter 11 will contrast with Larchenko’s opposite choice.

Advantages derived from this critic are binarized with a per-task threshold ϵ_A — the paper writes ϵ_ℓ , indexing by task — set at the **30th percentile** of the critic’s outputs for that task. Setting the cut by percentile rather than by an absolute value keeps the fraction of negative labels roughly stable even as the critic’s numerical scale drifts during training, which it will.

One notational caution here, because it will bite you at the keyboard. The paper phrases that cut in terms of the critic’s *value* outputs for the task; everywhere else in this book, ϵ_A is a percentile of the *advantage* distribution, which is what the worked example and the code in Section 10.5 compute. Those are two different arrays with different supports and different spreads, and a percentile of one is not a percentile of the other. Pick one and stay in it. This book picks advantages, so whenever you see ϵ_A in these pages, the percentile is being taken over $\hat{\mathbf{A}}$ values.

10.3.2 Three data streams, one dataset

Three sources feed a single per-task dataset, and each gets its label a different way.

Demonstrations are assumed good and forced to $c = +$ — but only during the task-specific fine-tuning iterations. There, no critic call and no threshold: a human did it, so it counts as above average by fiat. In the offline-RL pre-training stage described below it works the other way. The demos are scored by the critic and labeled against the per-task threshold like everything else, which is what those estimated thresholds are for. That distinction matters more than it looks. If demonstrations were forced positive from the very beginning, pre-training would contain no negative labels at all, the $c = -$ conditional would never be fit, and the model would arrive at fine-tuning with a conditioning input it had never once seen vary.

On-policy autonomous rollouts get labels computed from the critic. This is where the failures enter and earn their keep — the whole reason the conditioning machinery exists.

Expert teleoperated corrections — a human taking over mid-episode when the policy is about to fail, which is exactly the takeover pattern you already run in HIL-SERL (Chapter 13) — are also forced to $c = +$. The expert's fix is presumed advantageous.

The pipeline then runs in stages. Offline-RL pre-training first: train the value function on the multi-task demonstration corpus, estimate the per-task thresholds, and train the policy with advantage conditioning on all the demos. Then a repeatable task-specific loop: fine-tune the critic on task data, fine-tune the policy from the pre-trained checkpoint, deploy autonomously with an expert monitoring and correcting, fold the new rollouts and corrections back in, repeat. If that loop shape feels familiar, it should — it is DAGger's loop — deploy the policy, collect the states it actually visits, have an expert label those states, retrain, repeat (Chapter 13) — with a critic bolted on, and Chapter 14 turns it asynchronous.

10.3.3 Does it actually work

The results justify the machinery. On the hardest tasks — diverse laundry folding and making espresso drinks on a professional machine — RECAP more than doubles task throughput, measured as successful completions per hour, and roughly halves the failure rate. Most tasks land in the 90%-plus success range. The endurance numbers are the ones that stick with me: a 13-hour uninterrupted autonomous espresso run, and doubled box-assembly throughput after two iterations of the loop. These are not benchmark numbers; they are shift-length numbers.

10.4 Family resemblances

10.4.1 Return-conditioned policies, and why advantage is better

Conditioning on an outcome and then querying the good outcome is not new. It is the upside-down RL and Decision Transformer lineage (Srivastava et al., 2019; Kumar et al., 2019; Chen et al., 2021), which conditioned on the return-to-go G_t — you feed the model "this trajectory eventually scored 87," and at test time you ask for 95 and hope.

RECAP's refinement is to condition on *advantage* instead of return, and the reason is worth spelling out with an example. Suppose your policy is folding a shirt. Chunk P is an unremarkable nudge of a sleeve that was already lying flat; the garment was easy, the episode succeeded, so the return-to-go attached to chunk P is high. Chunk Q is a difficult recovery from a badly tangled sleeve; the garment was hard, the episode barely succeeded, so the return-to-go attached to chunk Q is lower. Condition on return and you have just taught the model that the lazy nudge is the good behavior and the skilled recovery is the mediocre one. You have graded the situation, not the action.

Advantage fixes this because it is *state-relative*. The true advantage $A^\pi = Q^\pi - V^\pi$ subtracts off how well you would have expected to do from this state anyway, and $\hat{A}$ is your critic's estimate of that quantity — an estimate, with all the error that implies, which is why the paragraph after this one cares so much about ranks rather than magnitudes. A positive advantage label does not mean "this went well." It means "this went better than typical *from here*." Chunk

Q's recovery beats the baseline for a tangled sleeve by a mile; chunk P's nudge barely beats the baseline for a flat sleeve. The labels come out right. Binarizing at a percentile adds a second layer of robustness on top: you are only asking the critic to rank chunks, not to get their magnitudes right, so a critic that is systematically off by a constant still produces perfect labels.

10.4.2 AWR versus RECAP — and why you might want both

Put the two side by side on the same six chunks and the difference is stark (Figure 10.3). Weighting **moves the training distribution** and leaves the query fixed. Conditioning **keeps the training distribution** and moves the query. Weighting biases what the model practices; conditioning hands you a control knob at inference.

They are complementary, and nothing stops you from doing both. You can sample data with probability proportional to $\exp(\hat{A}_i/\beta)$ and attach an advantage label to whatever gets sampled. Larchenko's LeHome solution does exactly this: AWR implemented through the data sampler rather than the loss (Chapter 4), with RECAP-style conditioning layered on top (Larchenko, 2026). The sampler makes sure the model spends its gradient steps on behavior worth imitating; the conditioning makes sure it still knows the difference.

10.4.3 The guidance connection

One more consequence, stated here and cashed out later. Because c is just an input, you can also *drop* it: during training, replace c with a neutral null token on some fraction of samples, so the same network learns both a conditioned and an unconditioned velocity field. At inference you can then blend the two — and, more interestingly, extrapolate past the conditioned one, amplifying whatever separates high-advantage behavior from average behavior. This is classifier-free guidance (Ho and Salimans, 2022), transplanted from image diffusion, and it turns policy improvement into a dial you can turn at inference time without retraining anything.

Chapter 12 derives the guided velocity field and treats the guidance scale g as an inference-time knob, so I will not write the equation here. What matters for *this* chapter is the prerequisite, and it is set now, at training time: $\pi_{0.6}^*$ gets its unconditioned branch from the two-term joint objective and anchors its extrapolation at the negative conditional rather than a null token, describing the effect as steering toward actions the model learned were improvements. Either way, the rule is absolute. No condition dropout during training, no guidance later.

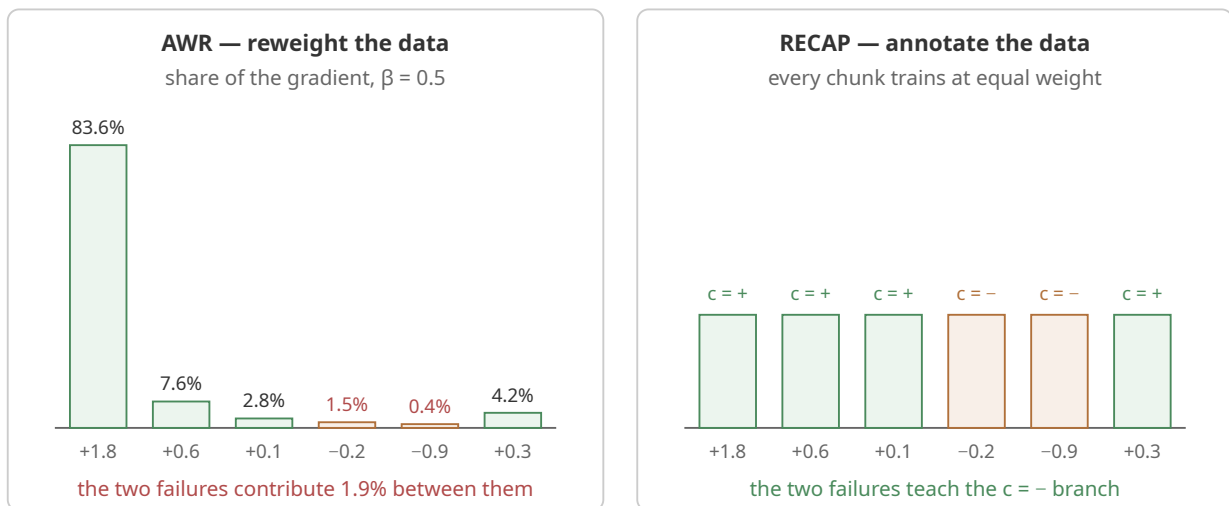


Figure 10.3 — The same six chunks, two mechanisms. On the left, AWR's exponential weights collapse the effective training distribution onto a single chunk; the two failures are visible only as slivers. On the right, all six bars are the same height because conditioning changes nothing about how often a sample is used — the ranking information has moved from the bar height into the label above it. Both panels use the same six advantages: the left-hand shares are computed in Section 10.1.2, the right-hand labels in Section 10.5.

10.5 Practical mechanics

10.5.1 Four steps

One: score every chunk in the buffer. Each stored chunk needs an $\hat{A}_i$ from your critic. Chapter 11 builds that estimator properly — success-probability heads, damped baselines αV , GAE over shaped rewards; here we take it as given. The operational detail that matters: recompute the labels whenever the critic improves. Larchenko recomputes advantages over the entire rollout pool before every training iteration, which is cheap compared to collecting the rollouts and keeps stale labels from poisoning the conditioning.

Two: discretize. A binary indicator thresholded at a fixed percentile per task is the robust default. Finer bins carry more information in principle, but each bin sees less data, and at inference you only ever query the top one, so the extra resolution buys you little.

Three: make sure a second, contrasting conditional exists. This step is usually stated as "choose a dropout rate," and that framing quietly misleads, so let us state the requirement instead. Guidance in Chapter 12 works by taking the difference between two conditionals and pushing past it. Training therefore has to leave you holding two of them: the one you will deploy, $c = +$, and something to measure it against. What you need is the contrast. The rate is an implementation detail of one particular way of getting it.

There are two ways, and they are structurally different rather than two settings of the same thing. The first is an *always-on auxiliary loss term*. The η -weighted two-term objective of Section 10.1.4 evaluates both $-\log \pi_\theta(a \mid o)$ and $-\log \pi_\theta(a \mid o, c)$ on every single sample, so the unconditioned branch trains on 100% of the data and there is no dropout rate anywhere in the scheme. The second is *stochastic replacement*: on each sample, with some probability, discard the label and feed the null token in its place. That one does have a rate, and 10% to 50% is the usual range. It is what the code block below implements, and it is what Larchenko's gate does with a probability that depends on the advantage.

Which conditional a system then anchors its guidance on is a further, separate choice, and the two systems in this chapter make it differently. A null-token setup extrapolates away from the unconditioned branch. $\pi_{0.6}^*$ extrapolates away from the *negative* conditional $c = -$ instead (Section 10.4.3), which is why its always-on unconditioned term is not the thing unlocking its guidance. Either anchor works, because either one gives you a second distribution to take a difference against. Having no second conditional at all does not.

Four: audit for leakage. The label must supply information the observation does not already carry, and both labels must appear across all visually distinct slices of the data. This is the step everyone skips and then spends a week debugging; see the pitfall box.

```
# This is the DROPOUT variant: one conditioned branch plus a null token on
# some draws. The two-term joint loss of 10.1.4 is the alternative, not the
# same thing -- it runs both branches on every sample and has no rate at all.

# once per training iteration, after the critic has been refreshed
A = critic.advantage(buffer)          # one scalar per stored chunk
eps_A = np.percentile(A, 30)          # over ADVANTAGES, per TASK
c = (A > eps_A).astype(int)           # 1 = positive, 0 = negative

for obs, act, ci in loader(buffer):    # equal weight: no reweighting here
    keep = rand(len(ci)) > 0.30       # 30% of draws see no label at all
    tok = where(keep, cond_token[ci], null_token)
    u = rand(len(act)).reshape(-1, 1, 1) # (B,1,1): broadcasts against
    z = randn_like(act)                # act's (B, H, action_dim)
    a_u = (1 - u) * z + u * act        # the interpolant of Chapter 7
    loss = mse(v_theta(a_u, u, obs, tok), act - z)
    loss.backward()

# at inference: the same K-step integration as Chapter 7, c pinned positive
a = integrate(v_theta, steps=K, obs=obs, cond=cond_token[1])
```

10.5.2 A worked example, every step shown

Here is the whole machine on the toy buffer of six chunks — the same six drawn in Figures 10.1 and 10.3. Sort the advantages, find the 30th percentile by linear interpolation the way NumPy does it, and label:

```
A-hat:    +1.8  +0.6  +0.1  -0.2  -0.9  +0.3
sorted:    -0.9  -0.2  +0.1  +0.3  +0.6  +1.8
           [0]  [1]  [2]  [3]  [4]  [5]

30th percentile:
index = 0.30 * (6 - 1) = 1.5      -> halfway between [1] and [2]
eps_A = -0.2 + 0.5 * ( 0.1 - (-0.2) )
       = -0.2 + 0.5 * 0.3
       = -0.2 + 0.15
       = -0.05

labels (c = 1 if A > -0.05):
        +      +      +      -      -      +      (4 positive, 2 negative)

train:    (o1,+)->a1    (o2,+)->a2    (o3,+)->a3
          (o4,-)->a4    (o5,-)->a5    (o6,+)->a6
          + dropout copies (oi, null)->ai on ~30% of draws

test:     sample a ~ pi(. | o, c=+)    # never c=-, never null
```

Walk through what just happened. The threshold landed at **−0.05**, which is *below zero* — so chunk 3, with a barely-positive $\hat{A} = +0.1$, got a $+$ label, and a mildly negative chunk would have too. The percentile decides, not the sign. Two of six chunks got $-$, which is 33%, close to the 30% we asked for; with six samples you cannot hit it exactly. All six chunks contribute gradient at equal weight. The two failures are not wasted: they are the only thing teaching the $c = -$ conditional, and without them the $c = +$ query has nothing to be better *than*.

Now move the threshold to the median and watch the labels change. The interpolation index becomes

$$0.50 \times (6 - 1) = 2.5,$$

which is halfway between the sorted entries **+0.1** and **+0.3**, so

$$\epsilon_A = 0.1 + 0.5 \times (0.3 - 0.1) = 0.1 + 0.1 = +0.20,$$

and the six labels, in the original order, become

$+, +, -, -, -, +$ (3 positive, 3 negative).

Chunk 3 flipped. Nothing about chunk 3 changed — same observation, same action, same advantage of $+0.1$. Its label is not a property of the chunk; it is a property of the chunk's rank within its task. That is worth internalizing, because it explains why the percentile must be computed *per task* and over a large pool. Compute it per minibatch and every batch redefines "good," which is a fine way to train a model that has learned nothing.

Which threshold is better? The median gives you a cleaner contrast and more negative examples to learn from; the 30th percentile gives you more positive data to imitate. $\pi_{0.6}^*$ chose 30, which is the conservative direction: when in doubt, more of your data should be teaching the branch you actually deploy.

IN THE LEHOME SOLUTION

Larchenko implements the conditioning not as text bins but as a single gated learned **advantage token** living in its own **attention group** — a block of tokens the attention mask treats as one unit, so this token can be masked in or out independently of the rest of the sequence. A continuous advantage vector also enters through **AdaRMS**, modulating each layer's normalization gain. The gate doubles as dropout: chunks with $\hat{A} < 0$ always get the neutral token, while positive chunks are masked stochastically, the probability of going neutral ramping from 0.5 at $\hat{A} = 0$ down to 0.1 at $\hat{A} \geq 2$ (Larchenko, 2026).

Notice the elegance of that gating scheme. It is not a fixed dropout rate; it is a rate that depends on how good the chunk was. A chunk with $\hat{A} = 0$ is a coin flip away from being labeled at all, so the model learns to treat the positive token as a claim about *strongly* positive behavior rather than merely above-threshold behavior. That sharpens the gap between conditionals — and the size of that gap is exactly what determines how far you can safely extrapolate with guidance later. A sharp gap is what the stochastic masking bought him: with a genuinely separated pair of conditionals, his bandit — the online knob-tuning search of Chapter 12, which tries different guidance scales during deployment and keeps whichever ones pay off — drove the guidance scale g to a startling 7 to 9 (Larchenko, 2026).

Two of his choices depart from the defaults I gave you a page ago, and I would rather name them than leave you wondering whether to copy the recommendation or the exemplar. First, he gates on the *sign* of $\hat{A}$, cutting at zero, where step two and the worked example told you to cut at a percentile that may well land below zero. He can do that because his advantages are already normalized and clipped into $[-2, 2]$, which makes zero a stable, meaningful point on a fixed scale rather than a number that wanders as the critic trains. The percentile rule exists to protect you when you have no such guarantee; when you do have one, a fixed cut is simpler and behaves identically.

Second, his scheme is neither the plain binary indicator of step two nor the finer bins that step warned against. The token itself stays binary. What varies continuously is the *probability* of showing it, plus the advantage vector flowing through AdaRMS. That is a genuinely different place to put the magnitude information, and it is the better one: coarse where the critic is unreliable, graded where gradations are cheap. The token stays coarse, which is what makes it robust to a wrong label. The training signal stays graded, which is what makes the contrast between conditionals sharp.

PITFALLS

Two silent failures. *Conditioning collapse*: if the labels correlate with something visible — one garment type mostly fails, or the demonstrations all come from one camera setup — the network infers the label from pixels and learns to ignore the token entirely. Flipping c at inference then changes nothing, and you will not notice because the policy still works. Defend with per-task percentile thresholds, and check that both labels occur inside every visually distinct slice of the data. *Skipping condition dropout*: with no trained unconditioned or negative branch there is nothing to contrast against, and Chapter 12's guidance is dead on arrival — a retraining-scale mistake, not a config change.

ON YOUR RIG

Your HIL-SERL setup already produces RECAP's raw material: action chunks, a binary success signal from the reward classifier, and takeover corrections from the Xbox pad. The minimal SO-101 experiment: label every chunk of a successful episode $c = +$ and every chunk of a failed one $c = -$; force $c = +$ on intervention frames; feed the label as one extra input to your BC policy with 30% dropout; deploy conditioned on $c = +$. Then run the honest test — deploy conditioned on $c = -$ and confirm the policy visibly gets worse. If it does not, you have conditioning collapse. That is RECAP minus the critic; Chapter 11 upgrades the labels to real advantages.

CHECKPOINT

- Explain, with the exponential-weight arithmetic in front of you, why AWR at a sharp temperature effectively discards low-advantage data, why raising β or capping the weights fixes that, and why the second cost — the weight being invisible to the model — survives any such tuning.
- State the advantage-conditioned objective, name every symbol in it, and say what η and ϵ_A each control — including why ϵ_A has nothing to do with PPO's ϵ .
- Explain why conditioning transplants onto a flow-matching policy when PPO does not: intractable likelihoods rule out ratio-based gradients, not maximum-likelihood-style surrogates.
- Describe RECAP's three data streams and how each one gets its label, and say what $\pi_{0.6}^*$'s 670M distributional critic predicts and against what target.
- Argue why conditioning on advantage beats conditioning on return-to-go, using a concrete easy-state-versus-hard-state example — then compute a percentile threshold on a small buffer by hand, label the chunks, and predict what changes if the percentile moves.
- Say what a second, contrasting conditional is for, name the two ways of getting one, and say what specifically becomes impossible if you have neither.

Where to go deeper. The primary source is the $\pi_{0.6}^*$ paper (Physical Intelligence, 2025b), which is short on ceremony and unusually precise about the critic, the thresholds, and the data streams — read the method sections covering those three things before you implement anything. Ho and Salimans (2022) covers the guidance mathematics that Chapter 12 picks up, and Larchenko (2026), Section 4.3, documents the gated-token variant used throughout this book's capstone in Chapter 15. Appendix A sequences all three in reading order. The Decision Transformer (Chen et al., 2021) and upside-down RL (Srivastava et al., 2019) are optional context, useful mainly for seeing clearly what advantage conditioning fixes.



PART IV

Making it work

The engineering that decides whether any of it trains: reward and advantage design, inference-time gains, the human in the loop, and the simulator.

Chapter 11 — Reward and advantage engineering

Every algorithm in this book eventually consumes one number. PPO clips around it (Chapter 3). Advantage-weighted regression exponentiates it into a sample weight (Chapter 4). RECAP thresholds it into a conditioning token (Chapter 10). That number is the advantage estimate $\hat{A}_t$ — "how much better than average was this action, in this state." Chapters 3 through 10 assumed it exists. This chapter builds it, for a task that hands you exactly one bit of ground truth per episode, several hundred timesteps after the actions that decided the outcome.

Nothing here is deep mathematics. It is bookkeeping, arithmetic, and one theorem, arranged so the resulting scalar is dense in time, low in variance, robust to a stale critic, and still faithful to the task. Five moves get you there: reward densification, potential-based shaping, a value function you get for free, a damped baseline, and a staleness-proof anchor.

11.1 The reward problem

11.1.1 What the environment actually tells you

LeHome defines success the way most real robot tasks do: as a **predicate** — a yes/no test — on the final state. Every garment carries labeled **keypoints**, marked points on the mesh such as the cuffs, the shoulder seams, the hem. The checker measures distances between specific pairs and declares success when a conjunction — an AND, all at once — of pairwise distance conditions holds. **Proximity pairs** must end *within* a threshold: a cuff close to its target on the torso, which is what "the sleeve is folded in" means geometrically. **Spread pairs** must stay *beyond* one, which stops the checker accepting a garment wadded into a ball. Five conditions for tops, four for pants (Larchenko, 2026) — that is the official checker; for his own dense reward Larchenko expanded long pants to seven conditions, which is where Chapter 8's 21-output keypoint head gets its width. So the truthful reward is

$$r_t = 0 \quad \text{for all } t < T, \quad r_T = \mathbb{1}[\text{success}]$$

where T is the final timestep and $\mathbb{1}[\cdot]$ is the indicator function, 1 when the statement inside is true and 0 otherwise. One bit, at the end. That is all the environment owes you.

11.1.2 The obvious thing to try, and exactly how it fails

The obvious thing is to just use it — not a silly idea, since it is what you did on rl-rover and what HIL-SERL does on your SO-101 today. Here is why it fails on a 600-step folding task, in two ways that compound.

First, it is sparse. An untrained policy essentially never folds a garment — not "rarely," but essentially never, because success needs four or five geometric conditions to hold at once after a specific sequence of bimanual grasps. So almost every episode returns $G = 0$. Chapter 2 showed that the policy gradient is built from *differences* in return between trajectories; when all returns are the same number, every difference is zero and so is the gradient. You can burn a week of simulator time and learn nothing while the loss curve looks healthy. Figure 1.2 is a picture of this.

Second, it is long-horizon. An episode runs roughly 600 steps at 30 Hz, about twenty seconds. Suppose one does succeed. That single bit must be apportioned across 600 action choices, and Monte Carlo credit assignment cannot tell which mattered: the grasp at step 140 that lifted the sleeve cleanly and the twitch at step 502 that did nothing get identical credit. Chapter 3 quantified this — Monte Carlo return variance grows with the horizon, because all 600 stochastic choices dump their noise into the same scalar.

So **densification** — moving reward earlier in time, closer to the actions that caused it — is not a refinement here. Without it there is no signal to start from.

11.1.3 Four routes to a denser reward

Four standard routes, ordered by how far each strays from the objective you care about — that distance is what eventually bites you.

- **Hand-shaped rewards.** Write dense terms yourself: distance to a goal pose, action smoothness, a penalty for dropping the garment. Cheap and fully under your control. But every hand-written term is a chance to quietly redefine the task. Section 11.3 gives the one provably safe form.
- **Learned reward models.** Fit a network to predict reward from pixels or preference comparisons. Flexible, and often the only option when success is hard to program. But the policy optimizes the model's *errors* as eagerly as its signal — **reward hacking**, where the agent finds inputs on which the reward model is confidently wrong — so it needs constant re-fitting.
- **Success classifiers.** Automate only the final judgment: a binary classifier on images answering "did this succeed," as in HIL-SERL (Luo et al., 2024). Faithful, because you approximate the real predicate and nothing else. But still sparse in time — you automated the human, not the signal.
- **Task-criteria checkpoints.** Decompose the evaluator's own conditions into partial milestones, paying out when one is first reached. The most grounded route: you invent no new metric, only reveal partial progress on the real one. The cost is access — it needs the checker's internal state, which is **privileged information**, data available to training but not to the deployed policy's sensors. Normally that means simulation only (Chapter 14).

Larchenko (2026) took the fourth route, plus Section 11.3's shaping. The LeHome simulator ships a hand-shaped "niceness" reward, route one, ready to use. He ignored it — a small act of discipline the rest of this chapter justifies.

ON YOUR RIG

Today *you* are the reward function: your button press at the end of a HIL-SERL episode is r_T , and nothing else in the loop knows whether the task went well. Step one is route three, HIL-SERL's own recipe — label a few hundred wrist and scene frames success/failure, train a small binary classifier, keep the button as an override for false positives. Do keep it; classifiers trained on 300 images fail in embarrassing ways. Route four is closed on hardware: it needs privileged garment state your SO-101 cannot observe, which is why checkpoint rewards live in simulation.

11.2 Checkpoint rewards from the task's own criteria

11.2.1 Decomposing a conjunction

Route four generalizes far beyond garments, because programmatic success checkers are almost always conjunctions — *A and B and C* — and a conjunction decomposes into an ordering of partial satisfactions.

For a long-sleeved top the milestones follow the physical order of the fold: first "sleeves folded onto the body," a subset of the proximity conditions, then "body folded in half," the rest. So award a **checkpoint reward** of 0.5 the first time the sleeve conditions hold, and the remaining 0.5 at full success. Cumulative reward then reaches exactly 1.0 on precisely the episodes the official checker accepts. Shorts get no mid checkpoint — one fold has no halfway point — while tops and long pants do. In the LeHome solution that first 0.5 is not a lump; it is granted *gradually*, in proportion to the reduction of the primary proximity distance. Which should make you uneasy.

11.2.2 First exploit: farming a progress bonus

If the policy earns reward whenever the cuff gets closer, what stops it dragging the sleeve halfway in, letting it slip back, and dragging it in again? Nothing. That is a reward pump, and a policy-gradient method will find it within a few thousand episodes. It will never fold anything; it will oscillate forever, collecting on every inbound swing.

The fix is a **running-min** rule: track the minimum distance achieved so far this episode, and pay only when the current distance sets a *new* minimum. Sliding back out earns nothing, and re-approaching earns nothing until the

sleeve goes strictly further than ever. Progress is paid for at most once, so cyclic motion cannot farm it — the pump is sealed by construction, not by tuning.

11.2.3 Second exploit: bank and thrash

Checkpoints create a second exploit the running-min rule does not touch. A policy can bank the 0.5 by half-folding cleanly, then thrash the garment into a heap for the remaining 400 steps. Episode return: 0.5. The true objective says 0.

The fix is **reward withdrawal**. If the episode ends in failure, take everything back: emit negative reward spread uniformly over the frames after the cumulative-reward peak, summing to exactly what was granted (Figure 11.1). Let t_{peak} be the *first* frame at which cumulative reward reaches its maximum — equivalently, the frame on which the last reward was granted, since nothing is paid out after it. Frames $t_{\text{peak}} + 1$ through T are the ones that share the clawback, and there are $T - t_{\text{peak}}$ of them:

$$r_t \leftarrow r_t - \frac{\sum_k r_k}{T - t_{\text{peak}}} \quad \text{for } t > t_{\text{peak}}, \text{ failed episodes only} \quad \implies \quad \sum_{t=0}^T r_t = \mathbb{1}[\text{success}]$$

IN PLAIN WORDS

On the left, r_t is the reward at frame t ; t_{peak} is the first frame at which the running total reaches its highest value, which is the same as the frame on which the last reward was granted; T is the last frame; $\sum_k r_k$ is everything granted over the episode. Divide that total by the number of frames after the peak and subtract it from each — a uniform clawback. On the right is the consequence: the rewards sum to 1 if the checker says success and 0 if it does not. An episode's *total* reward is still identically the original binary objective. You have not redefined the task, added a term, or introduced a coefficient to tune. What changed is only *when within the episode* the credit appears.

Concretely: a failure scores 0 whether or not it passed a checkpoint, so bank-and-thrash pays nothing, while *inside* a failed episode the frames that reached the milestone now carry positive reward and the frames after the disaster carry negative reward.

An analogy from your day job. Withdrawal is a database transaction rollback: you write rows during the transaction, and at commit time either they stick or every write is reversed and the ledger balances as if nothing happened. *Where it breaks down:* a real rollback is invisible afterward. Here the writes are the entire point — the learner sees the intermediate rewards, and their *timing* survives the rollback even though their sum does not.

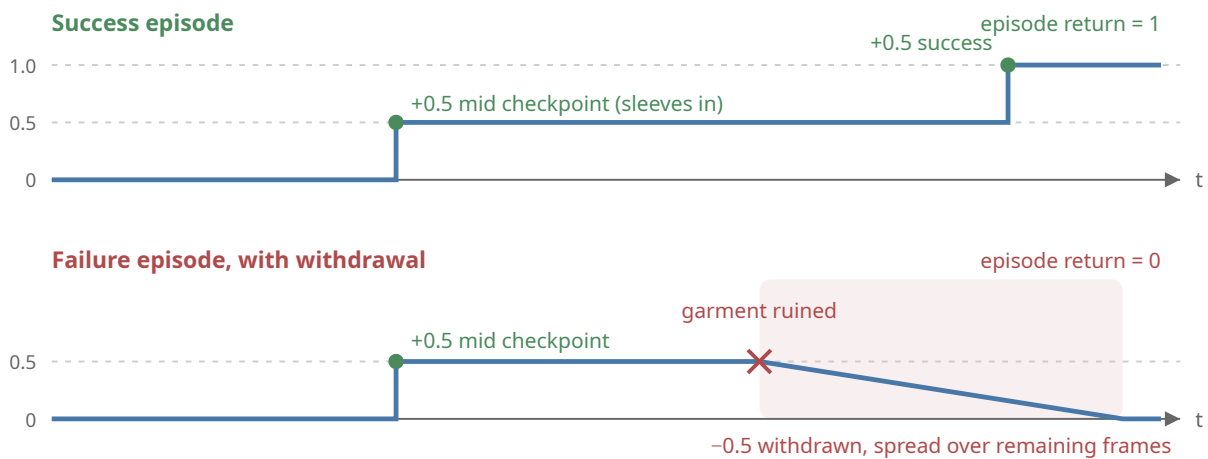


Figure 11.1 — Cumulative reward R_{acc} over two episodes. On top, checkpoints split the terminal bit into two milestones. Below, the checkpoint is granted, the garment is later ruined, and the banked 0.5 is withdrawn. Notice the bottom curve ends at exactly zero, yet the bump in the middle still tells the estimator which frames were the good ones.

```
# offline reward post-processing, per episode (before advantages)
r = zeros(T) # array indices 0..T-1, so "after peak" is T-peak-1
frames
if mid_checkpoint_hit: # tops and long pants; shorts have no mid checkpoint
    r[first_hit(mid_checkpoint)] += 0.5
if success:
    r[success_frame] += 1.0 - r.sum() # top cumulative reward up to exactly 1.0
else:
    peak = argmax(cumsum(r)) # first frame at the reward peak
    r[peak+1:] -= r.sum() / max(T - peak - 1, 1) # withdrawal: sum(r) -> 0
R_acc = cumsum(r) - r # reward banked strictly before step t
V_hat = alpha * (p_succ_pred - R_acc) # Secs. 11.4-11.5
A_hat = gae(r, V_hat, gamma=0.999, lam=0.99) # Ch. 3
```

11.3 Potential-based shaping done right

11.3.1 What we want, and the trap

Checkpoints gave you two payouts in 600 steps. You want more. The tempting move is a dense bonus $F(s, a, s')$ on every transition, nudging the policy toward states that look like progress. When is that safe?

The trap is Section 11.2.2's, in general form: any bonus that pays for *being closer* to something can be pumped by moving toward it and away again. Ng, Harada, and Russell's canonical example is a bicycle agent rewarded for reducing distance to a goal — it rode small circles near the start, harvesting the inbound arc of every loop, and never went anywhere (Ng et al., 1999). Your version is a gripper shuttling a cuff back and forth.

11.3.2 The theorem

They answered the safety question exactly: the shaped MDP has the same optimal policies as the original — for *any* dynamics and *any* underlying reward — if and only if F is **potential-based**:

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s)$$

IN PLAIN WORDS

$F(s, a, s')$ is the bonus you hand out for a transition from state s , taking action a , landing in s' . Φ (read "phi") is a **potential function**: any function you invent that assigns one number to each state — "how promising does this state look." γ is the usual discount factor. The equation says the only safe bonus is the discounted difference between how promising the new state looks and how promising the old one looked. If Φ went up you get paid; if it went down you pay. Crucially Φ may depend on the state only — not the action, not the timestep, not anything remembered from earlier. The "if and only if" makes this a theorem, not a recipe: any bonus *not* of this form can change the optimal policy in some MDP.

11.3.3 Why it works, step by step

Take any trajectory $s_0, s_1, \dots, s_T$ and add up the discounted shaping rewards along it, exactly as the return formula does:

$$\sum_{t=0}^{T-1} \gamma^t [\gamma \Phi(s_{t+1}) - \Phi(s_t)] = \sum_{t=0}^{T-1} [\gamma^{t+1} \Phi(s_{t+1}) - \gamma^t \Phi(s_t)]$$

Write out the terms. The first bracket gives $\gamma \Phi(s_1) - \Phi(s_0)$; the second gives $\gamma^2 \Phi(s_2) - \gamma \Phi(s_1)$, whose second half cancels the first bracket's first half. The same happens at every interior state: each $\gamma^t \Phi(s_t)$ appears once positive and once negative. Everything in the middle collapses and only the ends survive, leaving $\gamma^T \Phi(s_T) - \Phi(s_0)$.

Read what that means. The total shaping any trajectory can collect depends *only* on where it started and where it ended — not on the route, the length, or how many times it wobbled. The undiscounted potential difference around a closed loop is zero, which is why the pump is impossible. With γ slightly below 1 it is better than zero, from your point of view: a loop that leaves $\mathbf{s}_0$ and comes back after n steps collects $\gamma^n \Phi(\mathbf{s}_0) - \Phi(\mathbf{s}_0) = \Phi(\mathbf{s}_0)(\gamma^n - 1)$, which is a small net *penalty* whenever $\Phi(\mathbf{s}_0)$ is positive. Circling either pays nothing or costs you. It never pays.

An analogy from the physical world. Φ is elevation and the shaping reward is gravitational potential energy: walk any route across a landscape and the net energy gained depends only on your start and end altitudes. Hikers cannot invent energy by walking in circles. *Where it breaks down:* gravity is exactly conservative, whereas γ tilts the field, and the cancellation above assumes $\Phi(\mathbf{s}_T) = 0$ at terminal states or that γ^T is negligible. Here it is not: $0.999^{600} \approx 0.55$. The rigorous argument does not need that assumption — it is the uniform value shift $V'(\mathbf{s}) = V(\mathbf{s}) - \Phi(\mathbf{s})$, which lowers every state's value equally and leaves every **arg max** untouched — but set Φ to zero on terminal states anyway.

11.3.4 Choosing a potential

The ideal choice is $\Phi \approx V^*$, the optimal value function — but if you had V^* you would not need any of this, so Φ is a rough proxy in practice. Here is the theorem's real gift: a *bad* potential wastes your shaping budget and can slow learning, but it can never corrupt what "optimal" means. The worst case is wasted time, not a broken task. Very few knobs in RL have that property.

The practical Φ in the LeHome solution is the **completion head** from Chapter 8: a linear probe on the policy's backbone, trained only on successful episodes to regress the elapsed fraction t/T . Having seen nothing but successes, it encodes "what fraction of a *good* fold is done." Set $\Phi(\mathbf{s}) = \alpha_c \bar{C}(\mathbf{s})$ with $\alpha_c = 0.5$, where $\bar{C}$ is the smoothed head output.

That word "smoothed" hides a snag, and it is worth naming rather than stepping over. $\bar{C}$ is an exponential moving average of the head's per-frame predictions (Section 11.4.2), so it is formally a function of the frame's *history*, not of $\mathbf{s}$ alone — and the theorem you just proved was about a Φ that reads the state and nothing else. Applied literally, the chapter's own test rejects the chapter's own potential. There are two honest ways to close the gap. The clean one is to fold the smoother's running value into the state definition: it is one more number your pipeline already carries per frame, and once it is part of $\mathbf{s}$ the potential is state-only again by construction. The practical one is to notice that at coefficient 0.2 the smoothed value trails the raw head by a fraction of a single step's change, which is tiny next to the shaping magnitude $\alpha_c = 0.5$ — so the invariance holds to a good approximation, and the residue is far below the noise you are fighting. Either answer is fine. Silently ignoring the question is not.

This channel earns its keep on *saturated* tasks. Once shorts fold at 93%, $\hat{p}_{succ}$ is pinned near 1 and checkpoint rewards almost always pay, so the advantages of Sections 11.4 and 11.5 collapse toward zero and learning stalls. The completion potential still separates fast, clean progress from dawdling, so a gradient survives — and being potential-based, it runs at full strength in Section 11.6's blend.

Be precise about what "full strength, no risk" buys you, because it is a narrower claim than it sounds. The theorem says the shaping cannot change *which policy is optimal* in the MDP. It does not say the machinery downstream of $\hat{A}$ is unaffected. Chapter 4 feeds $\hat{A}$ through $\exp(\hat{A}_i/\beta)$ and Chapter 10 through a threshold, both nonlinear, and at any finite β adding a shaping term genuinely moves sampling priorities and conditioning labels around. That is the whole reason you added it. So read the guarantee as "no risk of redefining the task," not "no effect" — the shaping changes what your optimizer looks at first, which is exactly what you wanted, while leaving the definition of success untouched, which is what you were afraid of losing.

PITFALLS

A raw progress bonus — "+k whenever the cuff gets closer" — is *not* potential-based, because it depends on the transition rather than a fixed function of state, and it is exploitable exactly as the bicycle was. Section 11.2.2's running-min rule patches one instance; it is no general defense. The test: can you write your bonus as $\gamma\Phi(s') - \Phi(s)$ for a Φ that reads only the current state? If not, assume it changes the optimal policy.

11.4 The value function for free

11.4.1 The obvious cost, and how to avoid it

GAE (Chapter 3) needs a value estimate $V(s)$ at every frame. The standard answer is a critic network: a second head, a second loss, a second set of hyperparameters, and a real slice of your compute. On rl-rover that was free — the critic was a three-layer MLP. On a Gemma-2B backbone it is not.

You do not have to pay it. Chapter 8 already put a **success head** on the model: a linear probe on the policy's backbone, trained with cross-entropy against the eventual outcome, outputting $\hat{p}_{succ}(s)$, the predicted probability that this episode ends in success given the current frame. It is already a value function, up to one bookkeeping term.

$$V(s_t) = \hat{p}_{succ}(s_t) - R_{acc,t}, \quad R_{acc,t} = \sum_{k < t} r_k$$

IN PLAIN WORDS

The left side is the value of the current state — the total reward you expect from here to the end. $\hat{p}_{succ}(s_t)$ is the success head's output at this frame, between 0 and 1. $R_{acc,t}$ is the **accumulated reward**: the sum of every reward already paid strictly before the current step, which under the checkpoint scheme is 0, or 0.5, or something in between. The t in the subscript is doing real work — this quantity changes as the episode runs, growing by r_t at every step, so it must be read at the same frame as $V(s_t)$. This chapter keeps that subscript because the timestep is the point here; elsewhere in the book, where only one frame is under discussion, the same quantity is written R_{acc} . So: your expected future reward is your probability of eventually succeeding, minus whatever the episode already paid you. The subtraction is there because the total is capped at 1 — if half the credit is banked, only half remains to collect.

11.4.2 Verify it yourself, by cases

Worth checking rather than believing. Set $\gamma \approx 1$ and enumerate the two possible futures. **If the episode succeeds**, the scheme tops the cumulative total up to exactly 1.0 at the success frame; it has already paid $R_{acc,t}$, so the rewards still to come sum to $1 - R_{acc,t}$. **If it fails**, withdrawal claws back everything granted, so the episode total is 0 and what remains sums to $-R_{acc,t}$. Those are the only outcomes, and the head gives their probabilities:

$$\mathbb{E}[G_t | s_t] = \hat{p}_{succ}(1 - R_{acc,t}) + (1 - \hat{p}_{succ})(-R_{acc,t}) = \hat{p}_{succ} - \hat{p}_{succ}R_{acc,t} - R_{acc,t} + \hat{p}_{succ}R_{acc,t} = \hat{p}_{succ} - R_{acc,t}$$

The two $\hat{p}_{succ}R_{acc,t}$ terms cancel and the claim falls out. Note what $R_{acc,t}$ is *not*: not learned, not predicted, carrying no error. It is a running sum known exactly. All the uncertainty lives in $\hat{p}_{succ}$, which is the honest place for it. From V you get $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ and then $\hat{A}_t$ with Chapter 3's GAE machinery — LeHome uses $\gamma = 0.999$, $\lambda = 0.99$, computed offline over all stored rollouts before each iteration.

Two practical touches appear in no equation. Smooth $\hat{p}_{succ}$ with an exponential moving average (coefficient 0.2), because per-frame probe predictions jitter and jittery GAE differences are advantage noise. And interpolate the last thirty frames toward the known outcome: on a stored episode you already hold the answer, so do not let a noisy head disagree with it.

11.5 The damped baseline

11.5.1 Chapter 2's promise, and its fine print

Chapter 2 taught that subtracting a baseline $V(s)$ from the return leaves the policy gradient unbiased and reduces its variance. True. The fine print: *reduces* is guaranteed only for a *good* baseline. A wrong value estimate can add variance instead — and in the pipelines of Chapters 4 and 10, where $\hat{A}$ passes through $\exp(\hat{A}_i/\beta)$, clipping, and sampling priorities, it adds bias too, because those transformations are nonlinear in $\hat{A}$. Your value head is a linear probe trained on a moving target. It is decent. It is not good.

11.5.2 The A/B testing view

The cleanest way to see what to do comes from online experimentation. In case you have never run an experiment platform: you want to know whether a new checkout page raises revenue per user, so you split users into two arms and compare mean revenue. Spending varies enormously — most users spend nothing, one spends four thousand dollars — so the effect is buried in noise. But you have a covariate: each user's spend *before* the experiment, which correlates with their spend during it and, fixed before randomization, cannot have been affected by the treatment. **CUPED** subtracts it (Deng et al., 2013). Instead of analyzing the outcome Y directly, you analyze $Y - \theta(X - \bar{X})$, where X is the user's pre-experiment spend, $\bar{X}$ is the average of X over everyone, and θ is a fitted *coefficient*, not 1. Both details matter. The centering by $\bar{X}$ is what keeps the adjusted quantity an unbiased estimate of $\mathbb{E}[Y]$, since the subtracted term has mean zero. The fitted θ is what makes the variance actually drop.

A baseline is the same object: a **control variate**, a correlated quantity you subtract to cut variance without biasing the estimate you actually care about. Be careful about which estimate that is. In the A/B case it is the mean of Y , which the centering protects. In the RL case it is the policy *gradient*, not the mean of the advantage — subtracting αV does shift the average advantage by $\alpha \mathbb{E}[V]$, and the gradient survives anyway, for a reason spelled out just below the equation. So subtract αV rather than V , with a **damping coefficient** α , and expand:

$$\text{Var}(G - \alpha V) = \text{Var}(G) - 2\alpha \text{Cov}(G, V) + \alpha^2 \text{Var}(V), \quad \alpha^* = \frac{\text{Cov}(G, V)}{\text{Var}(V)} = \rho_{GV} \frac{\sigma_G}{\sigma_V}$$

IN PLAIN WORDS

G is the return actually observed for a frame; V is what the value head predicted for it; α is how much of that prediction you subtract. The left side is the variance of the resulting advantage estimate. On the right, $\text{Var}(G)$ is the noise you started with, $\text{Cov}(G, V)$ measures how much G and V move together, $\text{Var}(V)$ how much the prediction bounces. The middle term is the payoff, linear in α ; the last is the cost, quadratic. That is an upward parabola with one minimum, found by setting the derivative to zero: α^* . Here σ_G, σ_V are the standard deviations of G and V , and ρ_{GV} their correlation — unrelated to Chapter 3's ratio $\rho_t(\theta)$, hence the subscript.

Settle one worry before going further, because otherwise it will nag at you for the rest of the chapter. Section 11.5.1 opened by promising that subtracting a baseline leaves the policy gradient unbiased. Does scaling that baseline by $\alpha = 0.5$ break the promise? No, and the reason is worth seeing once. The unbiasedness argument never needed the subtracted quantity to be the *right* value function. It only needed it to be a function of the state and not of the action, because then

$$\mathbb{E}_{a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) b(s)] = b(s) \nabla_\theta \sum_a \pi_\theta(a|s) = b(s) \nabla_\theta 1 = 0$$

for any such b . And $\alpha V(s)$ is a function of the state alone for every constant α — scaling a state-only quantity leaves it state-only. So α is a pure variance knob. It cannot tilt the gradient, whatever value you give it, including 0 and including 5. What it does change is the variance, which is the whole point, and the average size of $\hat{A}$, which matters

only where something nonlinear reads it. That is the one place bias does enter, and Section 11.5.1 already flagged it: $\exp(\hat{A}_i/\beta)$ in Chapter 4 and the threshold in Chapter 10 respond to $\hat{A}$ itself, not only to its expectation. The deliberate reward leak of Section 11.5.4 inherits exactly the same caveat — a variance-and-scale choice, not a broken gradient.

Substituting α^* back gives the minimum achievable variance, $\text{Var}(G)(1 - \rho_{GV}^2)$. Read the formulas against your situation. A perfect value function satisfies $G = V + \text{noise}$, so $\text{Cov}(G, V) = \text{Var}(V)$ and $\alpha^* = 1$ — full subtraction, Chapter 2's textbook case. A noisy head inflates σ_V and shrinks ρ_{GV} , and both push α^* below 1. And whenever $\sigma_V > 2\rho_{GV}\sigma_G$, full subtraction is worse than *no baseline at all*.

11.5.3 Worked example

Real numbers, and one sentence of setup so you know which frame we are standing on. Take a frame early in the episode, before any checkpoint has paid, so $R_{acc,t} = 0$ and the return-to-go is exactly the binary outcome of Section 11.4: 1 if the episode eventually succeeds, 0 if it fails. Put the success probability at $p = 0.5$, roughly where a mid-training garment sits. Then G is a coin flip and $\text{Var}(G) = p(1 - p)$. Frames later in the episode, after a checkpoint has banked 0.5, have a return-to-go of $1 - R_{acc,t}$ or $-R_{acc,t}$ and a correspondingly smaller spread; the arithmetic below is the clean early-frame case, which is also the worst case for variance.

One number in what follows deserves a raised eyebrow before you read past it: $\sigma_V = 0.7$ is larger than $\sigma_G = 0.5$. That says the value head's predictions bounce around more than the returns they are predicting. A well-calibrated predictor of a binary outcome cannot do that, because its output is a conditional mean and averaging never adds spread — calibration forces $\sigma_V \leq \sigma_G$. So $\sigma_V > \sigma_G$ is not an incidental choice of numbers; it is itself the diagnostic that the head is miscalibrated or badly scaled. That is precisely the regime this section exists for, which is why the numbers stay as they are.

```

Var(G) = p(1 - p) = 0.5 x 0.5           = 0.25
sigma_G = sqrt(0.25)                     = 0.5

Your value head is noisy:
sigma_V = 0.7   ->   Var(V) = 0.7 x 0.7   = 0.49
rho_GV  = 0.6
Cov(G,V) = rho_GV x sigma_G x sigma_V
           = 0.6 x 0.5 x 0.7               = 0.21

Full subtraction, alpha = 1:
Var = 0.25 - 2(1)(0.21) + (1)(0.49)
     = 0.25 - 0.42 + 0.49               = 0.32
vs no baseline (0.25): 0.32 / 0.25     = 1.28   -> +28% variance

Optimal damping:
alpha* = Cov / Var(V) = 0.21 / 0.49     = 0.4286
check: rho x sigma_G / sigma_V = 0.6 x 0.5 / 0.7 = 0.4286   (agrees)
Var = 0.25 - 2(0.4286)(0.21) + (0.4286)^2 (0.49)
     = 0.25 - 0.18 + 0.09               = 0.16
formula: Var(G)(1 - rho^2) = 0.25 x (1 - 0.36) = 0.16       (agrees)
vs no baseline: 0.16 / 0.25 = 0.64      -> -36% variance

```

So the textbook move makes things 28% worse here, and the correctly scaled move 36% better. The gap is one coefficient. The "worse than nothing" condition is visible directly: $\sigma_V = 0.7$ while $2\rho_{GV}\sigma_G = 0.6$. Figure 11.2 plots the parabola.

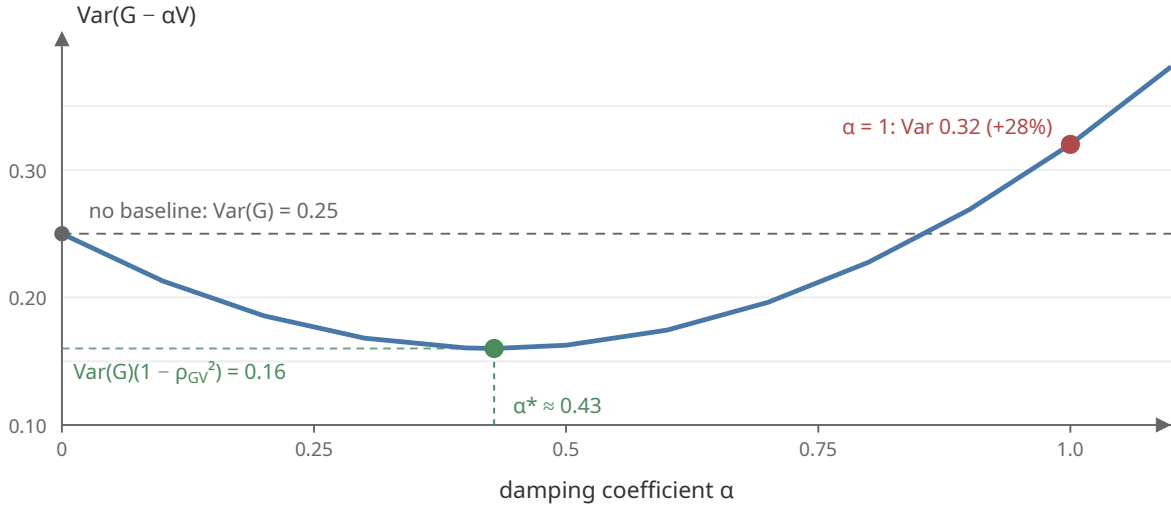


Figure 11.2 — Variance of the advantage estimate versus the damping coefficient α , for the worked example. Notice that the curve at $\alpha = 1$ sits *above* the dashed no-baseline line: full subtraction actively hurts. The parabola itself is perfectly symmetric about α^* — missing by d in either direction costs the same $\text{Var}(V) d^2$ — so the reason to err low is not curvature but range: α cannot go below zero, which caps the damage from under-damping at the no-baseline value $\text{Var}(G)$ marked by the dashed line, while over-damping has no such wall and climbs without limit to the right.

11.5.4 The second, deliberate effect of damping

Damping does something else that is easy to miss, and it is intentional. With $V(s_t) = \alpha(\hat{p}_{succ,t} - R_{acc,t})$, substitute into the TD residual:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \approx (1 - \alpha\gamma) r_t + \alpha(\gamma \hat{p}_{succ,t+1} - \hat{p}_{succ,t})$$

The algebra is short: $R_{acc,t+1} = R_{acc,t} + r_t$ by definition, so those terms almost cancel, leaving a residue $\alpha R_{acc,t}(1 - \gamma)$ negligible at $\gamma = 0.999$ — hence "approximately." Now read the first term. An undamped, accurate baseline makes $(1 - \alpha\gamma) \approx 0$, cancelling checkpoint rewards almost entirely. That is technically correct — the head saw the checkpoint coming, so it was priced in — but it means the dense signal of Section 11.2 is subtracted away by the estimator meant to use it. Damping at $\alpha = 0.5$ leaves $(1 - 0.5 \times 0.999) \approx 0.5$, so half of every checkpoint reward passes through as raw signal, deliberately left un-explained.

IN THE LEHOME SOLUTION

Larchenko damped with $\alpha_s = 0.5$ — his name for this section's α , subscripted for the success channel to keep it apart from the completion-shaping α_c of Section 11.3.4 — chosen by instinct rather than derivation. The paper's v2 works out $\alpha^* = \rho_{GV} \sigma_G / \sigma_V$ and measures it from logged rollouts: the true per-garment optimum ranged roughly 0.4 to 0.8, median near 0.5. He calls it "a mistake that worked" (Larchenko, 2026). You can do better than luck — both V and G already sit in your rollout logs. Estimate ρ_{GV} , σ_G , σ_V from a few thousand frames per garment type, and set α from the formula.

11.6 Stale values and the group anchor

11.6.1 A failure mode created by your own infrastructure

One more problem, born entirely of Chapter 14's asynchronous loop. Two details matter there. Value predictions $\hat{p}_{succ}$ are recorded *at collection time* and never recomputed — re-running the backbone over millions of stored frames every iteration would cost more than training. And old rollout datasets are not deleted; their sampling share decays slowly, so they linger for many iterations. A large fraction of your training data therefore carries value estimates made by a policy several generations old.

This is **stale off-policy data**, and staleness is worse than noise. Noise averages out over a batch; a stale head is *systematically* wrong in one direction, misjudging exactly the states where the policy has recently improved — the regime Section 11.5 warned about.

Think of it as a cached computed column that nobody invalidates. The difference is that a database can detect staleness by comparing cache to source; here, checking a stored prediction means recomputing it, the expense you were avoiding. You cannot fix this by detecting it. You have to build an estimator that does not care.

11.6.2 An advantage with no value function in it

GRPO — Group Relative Policy Optimization (Shao et al., 2024) — does exactly that for language models. For a given prompt it samples a *group* of responses from the same policy, scores them all, and sets each response's advantage to its own reward minus the group mean, divided by the group standard deviation. The group mean is the baseline. No critic is trained, and none can drift, because the baseline is recomputed from fresh samples every time.

The robotics translation swaps "prompt" for "task instance." Group episodes by garment, the closest thing to the same prompt. Split each episode into two segments at the first checkpoint: everything strictly before the frame where the checkpoint paid, and that frame onward. A segment is then a coarse unit of behavior, and each segment is scored against its own baseline.

The two baselines are different quantities, which is the part worth stating explicitly. Segment one is scored against the garment's empirical *checkpoint rate* — the fraction of recent episodes on that garment that reached the checkpoint at all. Reaching it scores **1 – rate**; missing it scores **–rate**. Segment two is scored against the empirical success rate *given* the checkpoint, $P(\text{success} \mid \text{checkpoint})$, because once the checkpoint is banked that conditional is what "typical" means for the rest of the episode. An episode that never reaches the checkpoint has no second segment: it is one segment, the whole episode, scored against the checkpoint rate. All these rates are plain counts over recent rollouts, recomputed every iteration, then scaled to match GAE magnitudes so the channels can be added — the scale factor is the segment's mean discount weight, $(1 - \gamma^n)/(n(1 - \gamma))$ for a segment of n frames, which is what one unit of outcome would be worth after GAE had spread it over those frames. Nothing here is learned, so nothing goes stale. The price is resolution: one number per segment, no per-step credit.

11.6.3 Blending by freshness

Two channels with complementary weaknesses: GAE is fine-grained but leans on an aging value head; the group anchor is coarse but ageless. Blend them by data age, and add Section 11.3's shaping on top as a third channel. Give the shaping its own per-step reward $F_t = \gamma\Phi(s_{t+1}) - \Phi(s_t)$ and run that through the same GAE recursion the reward channel uses, so all three terms arrive on the same footing and can simply be added:

$$\tilde{A}_t = \underbrace{w \hat{A}_t^{\text{GAE}}}_{\substack{\text{per-step credit,} \\ \text{fades as data ages}}} + \underbrace{(1 - w) \hat{A}_t^{\text{grp}}}_{\substack{\text{group outcome anchor,} \\ \text{stale-proof}}} + \underbrace{\hat{A}_t^{\Phi}}_{\substack{\text{potential shaping,} \\ \text{always full strength}}}$$

IN PLAIN WORDS

$\tilde{A}_t$ is the final advantage for frame t , the number everything downstream consumes. The first term is Section 11.4's GAE estimate, which knows about individual timesteps but trusts the value head. The second is the group-relative estimate, which knows only "this segment did better or worse than typical for this garment" but trusts nothing that can age. The weight w , between 0 and 1, decides which you lean on. It is tied to the dataset's **sampling share** — the fraction of each training batch drawn from that dataset, which decays as the dataset ages (Chapter 14). Fresh data gets w near 1 and uses fine-grained credit; decayed data gets w near 0 and degrades into outcome-only anchoring rather than trusting an obsolete prediction. This w is a data-freshness weight and has nothing to do with the AWR weight $w_i = \exp(\hat{A}_i/\beta)$ of Chapter 4. $\hat{A}_t^\Phi$ is the shaping channel just described: the per-step bonus $F_t = \gamma\Phi(s_{t+1}) - \Phi(s_t)$ pushed through that same GAE recursion. It carries no weight of its own because, as Section 11.3.4 argued, it cannot change which policy is optimal — though it does move what the sampler looks at, which is why you added it.

Finally, normalize $\tilde{A}$ by a global standard deviation measured on **unbiased rollouts** — the episodes collected by plain random or curriculum selection, the ones that alone count toward your success statistics, as opposed to the replay and hard-mining rollouts that deliberately oversample particular states (Chapter 14) — and clip to $[-2, 2]$ before it feeds Chapter 4's AWR sampler and Chapter 10's conditioning token. LeHome adds a garnish: the tightest 20% of successes per garment, ranked by the worst per-condition margin at the final frame, get **+0.3** on every frame, nudging the policy from barely-passing folds toward comfortable ones. Figure 11.3 lays out the finished pipeline end to end.

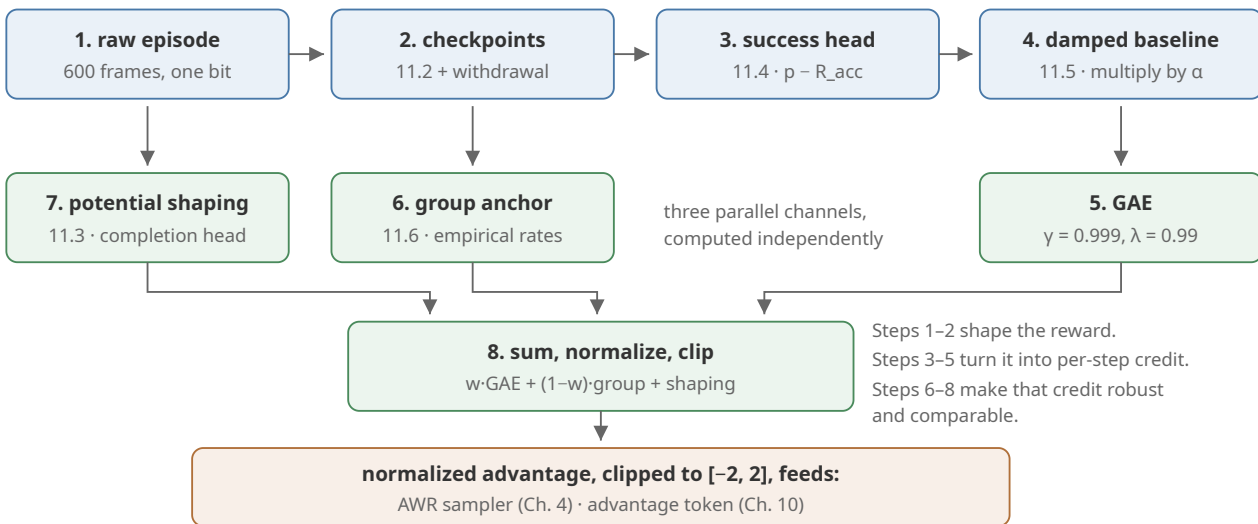
The advantage assembly line

Figure 11.3 — The whole chapter as one pipeline. The top row is a genuine chain, read left to right: the raw episode gets checkpoint rewards, the success head turns those into a value estimate, damping scales it. The three boxes below are *not* a chain — they are three independent channels, computed from different inputs and combined only at step 8, so the numbering is reading order, not data flow. Only steps 3, 4, and 7 consume a learned prediction (the success head and the completion head); everything else is arithmetic on logged numbers, and both heads were evaluated once at collection time, which is why the pipeline is cheap enough to rerun over every stored rollout each iteration. Notice too that the fragile part, the value head, sits between a damping coefficient and a stale-proof anchor.

11.6.4 Second worked example: one frame, all three terms

Take the checkpoint frame of a long-sleeved-top episode that eventually succeeded, so $r_t = 0.5$, with $\alpha = 0.5$, $\gamma = 0.999$, smoothed heads $\hat{p}_{succ,t} = 0.55$ and $\hat{p}_{succ,t+1} = 0.62$, and $R_{acc,t} = 0$ because nothing has paid out yet. Say the checkpoint lands at step 200 of a 600-step episode, so the second segment runs for the remaining $n = 400$ frames — that number is needed for the group anchor's scale factor.

```

V(s_t)    = 0.5 x (0.55 - 0.00)          = 0.275
V(s_{t+1}) = 0.5 x (0.62 - 0.50)          = 0.060    # R_acc,t+1 is now 0.5
delta_t    = 0.5 + 0.999 x 0.060 - 0.275 = 0.28494
cross-check with 11.5.4's shortcut:
  (1 - alpha x gamma) x r_t = 0.5005 x 0.5          = 0.25025
  alpha x (gamma x p_{t+1} - p_t) = 0.5 x 0.06938    = 0.03469
  sum                          = 0.28494    (agrees)

Suppose GAE accumulates this and later residuals to A_GAE = 0.42.
Group anchor: post-checkpoint segment succeeded -> score 1
empirical P(success | checkpoint) for this garment = 0.62
raw = 1 - 0.62                                = 0.38
scale = mean discount weight over the n = 400 frames of the segment
       = (1 - 0.999^400) / (400 x 0.001) = 0.32982 / 0.4 = 0.8246
A_grp = 0.38 x 0.8246                        = 0.3133
Shaping: completion head rose 0.31 -> 0.44, so F_t = gamma x Phi' - Phi
A_phi = 0.5 x (0.999 x 0.44 - 0.31) = 0.5 x 0.12956 = 0.06478
Freshness: data 25 iterations old, decay 0.98/iter
w = 0.98^25                                = 0.60
Blend = 0.60 x 0.42 + 0.40 x 0.3133 + 0.06478 = 0.4421
Normalize by global std 0.35: 0.4421 / 0.35 = 1.26
Tightest 20% of successes, so +0.3          = 1.56 (inside clip)

```

Trace what each number did. Of the 0.285 residual, 0.250 is raw reward leaking through — exactly the $(1 - \alpha\gamma)$ fraction. GAE then contributed the largest share of the blend, 0.252, because this data is only moderately aged; the group anchor contributed 0.125, and more if the data were older; shaping contributed 0.065, small but never zero, which keeps a gradient alive where the other two collapse. Normalization turned an unreadable 0.442 into a 1.26 that means something fixed: about 1.3 standard deviations better than average.

Two lines in that block deserve a note, because both are places where a reader is entitled to ask "where did that come from." The scale factor 0.8246 is not a magic constant: it is the mean discount weight $(1 - \gamma^n)/(n(1 - \gamma))$ over the segment, from Section 11.6.2, and it is the only quantity in the example that depends on how long the segment is. Recompute it for your own horizon rather than copying the number — at $n = 600$ it would be 0.75, at $n = 900$ it would be 0.66. And the shaping line shows this frame's own F_t rather than the full GAE of the shaping channel; running the recursion would fold in later frames as well, but the single step is the piece you can check by hand, and it is the piece that carries the γ most people quietly drop.

PITFALLS

Advantage scale drift across task groups. $\hat{A}$ enters $\exp(\hat{A}/\beta)$ in Chapter 4 and a threshold in Chapter 10, so its *scale* silently acts as a temperature. A near-saturated garment may produce advantages of ± 0.1 while a struggling one swings ± 0.8 . Blend those unnormalized and the struggling type dominates sampler and gradient alike — which sounds desirable until you notice it also dominates when its advantages are pure noise. Use one global normalizer plus clipping, and re-check per-group standard deviations whenever a group saturates.

WORDS YOU WILL HEAR

Dense vs sparse reward — dense pays out most timesteps, sparse only at the end; "sparse-reward task" is shorthand for "hard exploration problem." **Reward hacking** — the policy scores well on your reward function while failing the task you meant. **Potential function** Φ — a state-only score used to build provably safe shaping. **Control variate** — a correlated quantity subtracted to cut an estimator's variance without biasing the thing you actually want to estimate; a baseline is one. **Stale off-policy data** — samples, or predictions stored with them, produced by an older policy than the one being trained. **Group-relative** — advantage computed by comparing samples of the same task instance instead of against a learned critic.

CHECKPOINT

- Explain why a 600-step task with one terminal bit yields a zero gradient, naming the sparsity and horizon-variance failures separately.
- Describe the two exploits checkpoint rewards invite — cyclic farming and bank-then-thrash — say which mechanism blocks each, and state precisely what reward withdrawal preserves and what it changes.
- State the potential-shaping theorem, sketch the telescoping argument, and explain why a bad Φ wastes effort but cannot change the optimal policy.
- Derive $V(s_t) = \hat{p}_{succ}(s_t) - R_{acc,t}$ by cases, and say why $R_{acc,t}$ contributes no estimation error.
- Compute α^* from ρ_{GV} , σ_G , σ_V , show that $\alpha = 1$ can be worse than no baseline, and explain why scaling the baseline is a variance choice that leaves the policy gradient unbiased.
- Explain why asynchronous training makes stored value predictions stale, and how a group anchor computed from empirical rates sidesteps that.

Where to go deeper. Ng, Harada, and Russell (1999) is short and worth reading in full — the necessity direction, the claim that *only* potential-based shaping is safe, is the surprising half most papers skip. Deng et al. (2013) introduces CUPED, the fastest way to internalize control variates. Schulman et al. (2016) covers the GAE machinery this chapter reuses, and Shao et al. (2024) presents GRPO where the group structure is far more obvious than in robotics. Sections 5 and 6 of Larchenko (2026) show every idea here deployed together; Appendix A gives links and a reading order.

Chapter 12 — Inference-time optimization

Every chapter so far has changed the policy's weights, at a cost of GPU-days: Chapter 10 changed what the network is told about each sample, Chapter 11 the numbers the critic hands it. This chapter freezes the checkpoint — not one gradient step, not one new episode of data — and moves the measured success rate by changing only *how it is executed*. In LeHome those choices mattered enough that Larchenko built an online tuning system for them (Larchenko, 2026).

The reason this is possible: a generative chunk policy is not a controller. A controller is a function from observation to action; once you have it there is nothing left to decide. A chunk policy is a *sampler*, and a sampler leaves unfinished business. How many of the $\mathbf{H}$ predicted actions do you execute before asking again? At what speed? What happens at the seam between chunks? How hard do you push the advantage condition from Chapter 10? None of these has an answer inside the weights. Every one of them moves the number.

12.1 Free success rate

Make it concrete. Your policy is Chapter 8's: a vision-language-action model whose action expert emits a chunk $\mathbf{a}_{t:t+\mathbf{H}}$ by integrating a learned velocity field. In LeHome, $\mathbf{H} = 30$ twelve-dimensional joint-delta actions — one second at 30 Hz — from $\mathbf{K} = 10$ Euler steps of the flow ODE of Chapter 7.

Training fixed the *distribution* chunks are drawn from. It did not fix what you do with one. That gap splits into three families. **One:** how much of a chunk you run and how fast (Section 12.2). **Two:** how consecutive chunks are stitched, since each is generated from a stale observation while the arm is moving (Section 12.3). **Three:** how the chunk is drawn at all — how hard to push the advantage condition, how wide the noise you start from is, one candidate or several (Sections 12.4 and 12.5). Seven decisions, each tuned per garment type in LeHome, and Section 12.6 tunes all seven without a grid search.

One framing note. In classical deep RL there is essentially no inference-time surface: your PPO rover policy emits a Gaussian, you take the mean, you are done. A flow policy has an entire iterative sampling procedure between observation and action, all of it adjustable after training. That is new surface area, and it is easy to leave free success rate on the table because your previous tools never had any.

WORDS YOU WILL HEAR

Inference-time compute — work spent per decision at deployment (extra sampler steps, extra candidates) rather than in training; "scaling" it means buying accuracy with latency. **Guidance scale** — the multiplier g on a conditional-minus-unconditional difference; papers also write w , s , or α . **Temperature** — a scale on sampling noise; low means more deterministic, less diverse samples. **Arm** — one option a bandit can choose. **Posterior** — your belief about a quantity, as a distribution, after seeing data. **Verifier** — any model that scores a candidate output; in LLM work, "best-of-N with a verifier."

12.2 Execution length, playback speed, and chunk switching

12.2.1 How much of the chunk to execute

What we want is closed-loop control: the robot should notice when the world stops matching its plan, quickly.

The obvious thing is to execute the chunk: all thirty actions, then ask for the next second. One expensive forward pass buys a full second of seamless behavior.

Here is exactly how it fails. Your arm is 400 ms into a one-second reach toward a sleeve when the gripper brushes the fabric and the sleeve slides two centimeters left. The remaining 600 ms were computed from an observation in which the sleeve was elsewhere, and the policy will not be asked again until they are done — so the arm closes its fingers on

empty table. The judgment was not wrong; it was *stale*. You have built an open-loop controller and called it closed-loop.

The fix is **receding-horizon** execution: run only the first $n_e \leq H$ actions, discard the rest, re-query from a fresh observation. With $n_e = 5$ at 30 Hz the policy re-plans every 167 ms — boundaries at ticks 0, 5, 10, 15 — and the sleeve slid at 400 ms, which is tick 12. So it is noticed at the next boundary, tick 15: 100 ms and three control ticks later, instead of eighteen. Notice why the chunk was worth predicting anyway: predicting a full second forces the network to commit to a *coherent intention* rather than a twitchy per-step average. You keep the coherence and discard the staleness.

You do pay. Smaller n_e means more chunk boundaries, each a potential discontinuity — Section 12.3 exists because of this — more forward passes per second, and less within-chunk smoothness reaching the motors. Larchenko's tuned values were 3 to 5 out of 30: six to ten re-plans per second of predicted motion.

ON YOUR RIG

LeRobot exposes this knob directly and most people never touch it: for an ACT or pi0-style policy on your SO-101, `n_action_steps` versus `chunk_size` is exactly n_e versus H . Sweep it before retraining anything — 1, 2, 4, 8, 16 against a fixed checkpoint, twenty trials each. You will usually find a clear interior optimum, for an afternoon's work instead of a week's. On rl-rover, action repeat and control rate trade reactivity against smoothness the same way. You also already own the verifier of Section 12.5: your HIL-SERL SAC critic. Sample a handful of chunks, score each with the min over the critic ensemble, execute the argmax — and validate against random choice before trusting it, per the pitfall box.

12.2.2 Playback speed

A chunk is a time-indexed trajectory: action 0 is meant for now, action 1 for 33 ms from now. Nothing forces you to honor that timing — you can resample it onto a different number of control ticks. Larchenko's BEHAVIOR-1K entry (Larchenko et al., 2025) resampled 26 predicted actions with cubic splines onto 20 control ticks: 26 ticks of motion become 20, so the same path is replayed $26/20 = 1.3$ times faster and takes $20/26 = 77\%$ as long. Keep those two numbers apart, because it is easy to quote the wrong one: a 1.3x speed increase is a 23% cut in cycle time, not a 30% cut. Speed up and time saved are not the same quantity, and the difference grows as the ratio does. So: 23% less time on the clock for zero training and no new data.

But speed is not uniformly safe, and the interesting part is the exception rule: compression was *disabled whenever the gripper state changed significantly*. Reaching through free space tolerates being rushed. Closing fingers on a garment does not — a grasp replayed at 1.3x gives the fabric and the contact forces 23% less time to settle. So the policy sprints between contacts and walks through them. In LeHome this knob is a per-garment-type playback stretch.

12.2.3 Chunk switching

The third decision: when a new chunk arrives while the old one is still running, how much should the old constrain the new? Larchenko parameterizes this with an **anchor length** n_a — how many of the new chunk's actions are pulled toward the old chunk rather than generated freely, tuned small, 3 to 6 out of 30. Note the word "pulled." This is a soft influence, a weight that fades to nothing across those few slots, not a rule the new chunk has to obey. There is also a hard part, where some actions are copied over exactly and are not negotiable at all, and the two are separate knobs. Both, and the reason a generative model can be made to honor either, are the next section.

12.3 Action-chunk inpainting: executing while you think

12.3.1 The problem is latency, and the arm does not stop

A VLA forward pass is not free: the vision backbone and the language transformer run over three camera images before the action expert takes one step, costing tens to hundreds of milliseconds. Meanwhile the robot has momentum

and will not hover in mid-air while you think. The obvious first attempt is synchronous execution — run the chunk, stop, think, run the next. You see it in demo videos once you know to look: the arm moves, freezes for a beat, moves again. For quasi-static tasks that is merely ugly; with dynamics it is a failure mode, because the world moves during the pause.

So you go asynchronous, the instinct of anyone who has prefetched anything: start generating chunk $k+1$ while chunk k executes, and swap when it is ready.

Here is how that fails. Suppose generation takes 235 ms and your control period is 33 ms. By then the arm has executed seven or eight more actions from chunk k 's tail — but chunk $k+1$ was generated from the observation of 235 ms ago and begins with *its own* idea of what those actions should have been. The two do not match: chunk k drifted left, chunk $k+1$ assumes nothing moved. Swap them and the commanded position jumps. With a delta-action policy — LeHome's are joint deltas — that is worse than aesthetic: a delta is added to the current state, so you apply a correction meant for a configuration the arm has already left. The result is a violent jerk at every boundary, six to ten times a second.

12.3.2 The fix: freeze what is committed, blend the seam, generate the rest

Real-time chunking (Black et al., 2025) fixes this by noticing that some of the disagreement is not disagreement at all — it is already history.

By the time chunk $k+1$ is ready, the first n_f of its actions have *already been executed*, from chunk k 's tail. They are not up for discussion; the arm is where it is. So freeze them: constrain the new chunk's first n_f actions to equal the committed tail exactly. Pull the next few softly toward the old chunk with a weight decaying to zero — the anchor region of length n_a , which makes the seam smooth rather than merely continuous. The rest is generated freely (Figure 12.1). The name comes from image editing: **inpainting** is filling a masked-out region consistently with the rest of the picture — here, the chunk's future given the prefix that already ran.

An analogy from your day job. This is speculative execution with a commit point. A CPU predicts a branch and executes down the predicted path; when the real answer arrives, anything already *retired* is permanent and the rest can be revised. Real-time chunking gives the robot the same structure: retired actions frozen, in-flight actions blended, the not-yet-issued tail free. *Where it breaks down:* a CPU can squash a mispredicted path and lose only cycles. A robot cannot un-move. There is no rollback, which is why the frozen prefix must be respected exactly rather than merely preferred.

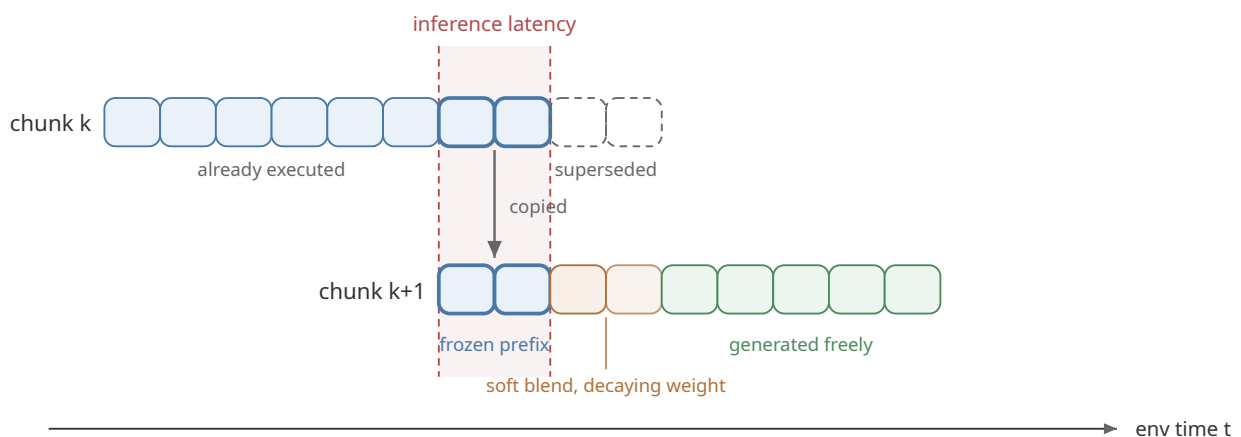


Figure 12.1 — Real-time chunking as inpainting. Notice that the shaded band is not idle time: the robot keeps executing chunk k 's tail while $k+1$ is generated, so those actions are history when it arrives. They are frozen into the new chunk (blue), the next few softly blended with a decaying weight (orange), the rest generated freely (green).

12.3.3 The mechanism: a correction to the velocity field

How do you force a flow model to respect a prefix? Overwriting the finished chunk's first n_f actions afterward does not work: the rest was generated without knowing about the overwrite and would not connect to it. The constraint has to be present *during* generation. Recall from Chapter 7: start from noise $a^0 = z$ and integrate the velocity field $v_\theta(a^u, u, o)$ forward in flow time u from 0 to 1 in K Euler steps. Inpainting adds a correction at each step:

$$a_{0:n_f}^u \leftarrow a_{n'_f:n'_f+n_f}^{\text{prev}}, \quad \tilde{v}_j(a^u, u, o) = v_{\theta,j}(a^u, u, o) + W_j \frac{a_{n'_f+j}^{\text{prev}} - a_j^u}{1 - u}$$

IN PLAIN WORDS

The first line is the hard freeze: slots 0 through $n_f - 1$ of the chunk being generated, written $a_{0:n_f}^u$, are overwritten with n_f entries of the previous chunk a^{prev} , counted from the offset n'_f . The second line is the soft part, per slot j : $v_{\theta,j}$ is what the network wanted there, $\tilde{v}_j$ what you integrate instead. The fraction $(a_{n'_f+j}^{\text{prev}} - a_j^u)/(1 - u)$ is the constant velocity that would carry the current value to the committed one just as integration finishes at $u = 1$: distance remaining over flow time remaining. W_j sets how much of that pull to apply — decaying from 1 to 0 across the anchor region of length n_a , and 0 for the free tail, where the network is left alone.

Two details in that formula bite in practice. The first is the offset n'_f , which is only bookkeeping but is the bookkeeping people get wrong. Slot j of the new chunk is slot $n'_f + j$ of the old one, because the new chunk's clock starts n'_f ticks after the old chunk's; that is why both lines index a^{prev} with the offset and never bare j . Both slices are half-open, so each holds exactly n_f entries, and Section 12.3.4 puts numbers on n_f and n'_f together. Code usually applies the offset once up front, shifting the old chunk onto the new chunk's clock, which is what the `shift(a_prev, nf_off)` line in the Section 12.4 listing does. The second detail is that $1 - u$ goes to zero as integration finishes, so the pull grows without bound at the last step. In exact arithmetic that is harmless, because the distance in the numerator shrinks just as fast. In floating point it is a division by nearly nothing, which is why implementations floor the denominator — the `max(1e-3, 1.0 - u)` in that same listing is the floor, and it is load-bearing rather than decorative.

Two more observations. First, this is *guidance, not retraining*: nothing in the weights knows about chunk boundaries, so any flow or diffusion chunk policy supports it out of the box. Second, the correction need not run for the whole integration. Larchenko gates it by a threshold u_{ip} on the flow time (the tech report writes t_{ip}), active only once u passes it, with tuned values around 0.4 to 0.5 — **soft inpainting**. Early on the sample finds its own shape undragged; the constraint switches on afterward. u_{ip} is one of the seven knobs.

The alternative in older codebases is **temporal ensembling**: averaging the overlapping actions of successive chunks instead of constraining generation. It smooths the seam but blurs genuine changes of intent. Black et al. (2025) show that under injected latencies both naive swapping and temporal ensembling degrade, while real-time chunking keeps success rates flat.

12.3.4 Worked example: a latency budget

The knobs interact through a compute budget. Illustrative timings, LeHome's structural parameters.

```
30 Hz -> tick = 33.3 ms; n_e = 5 -> re-query every 167 ms
expert passes = K x 2 (CFG) x N = 10 x 2 x 3 = 60
60 x 3 ms + one 55 ms VLM prefix pass = 235 ms
235 vs 167 available -> 68 late = 2.04 ticks stalled
N 3 -> 2: 40 x 3 + 55 = 175 ms
n_f = ceil(235 / 33.3) = 8 frozen, free tail 30 - 8 = 22
n_f' = n_e = 5 (new chunk slot 0 = old chunk slot 5)
generations in flight = ceil(235 / 167) = 2
```

Read what that says, one arrangement at a time. Run this synchronously — stop, look, think, move — and you stall the full 235 ms at every boundary, because nothing overlaps: the arm simply waits out the whole forward pass. Pipeline it naively instead, generating chunk $k+1$ while chunk k runs, and you still come up 68 ms short each time, because 235 ms of work does not fit in a 167 ms window. That is about two ticks per boundary and six boundaries per second, so roughly 12 of every 30 control ticks are stalled — 40% of the robot's time spent waiting on a network. Cutting N from 3 to 2 nearly closes the gap, and costs you the veto benefit of Section 12.5.

Now be precise about what real-time chunking does and does not fix here, because this is where careful readers get stuck. It does not make the 68 ms disappear. It makes the *seam* smooth. The throughput deficit is a separate problem with two honest answers: keep two generations in flight at all times, since 235 ms of latency against a 167 ms re-query interval needs depth $\lceil 235/167 \rceil = 2$; or lengthen n_e until one generation fits inside one interval. Inpainting is orthogonal to both. What it buys is that when the new chunk lands, the arm does not jerk.

And the freeze is not free either, which the $n_f = 8$ line makes concrete. Eight slots committed at 33.3 ms each is 267 ms during which the robot will execute what it already decided no matter what it sees — the sliding sleeve of 12.2.1, arriving inside that window, waits. That is exactly the staleness cost a whole section was spent establishing, and it has not been abolished; it has been paid deliberately, in a place where it is smaller than the discontinuity it replaces. The rest is comfortable: with $n'_f = n_e = 5$, the freeze copies $a_{5:13}^{\text{prev}}$ into the new chunk's slots 0 through 7, leaving 22 slots generated freely — far more than the 5 you will actually execute before re-querying.

12.4 Classifier-free guidance: asking for more than the data showed

12.4.1 What we want, and why plain conditioning is not enough

Chapter 10 gave you an advantage-conditioned policy: set the condition to c_A , "this was a good execution," and the policy imitates the good part of its own data.

The obvious thing is to just do that: set the token, sample, done. It helps — less than you would hope, and the reason is in the training procedure. Recall the gating from Chapter 10: chunks with negative advantage always get the neutral token, positive chunks are masked stochastically, the probability of going neutral ramping from 0.5 down to 0.1. So a large share of training frames — plausibly most, once the plain behavior-cloning and DAgger data are counted alongside the advantage-labeled frames — carry a neutral condition. Be careful with that claim: the tech report gives the gating rule but not the resulting fraction, so this is the reading that best explains what follows rather than a reported number. Such a network learns to be a good unconditional model first and leans on the advantage token weakly. Switching it on nudges the velocity field; it does not transform it.

What we want is stronger: a policy that acts *more* like the good episodes than any single good episode did. Here is what makes that possible. The same stochastic masking means one network has learned two velocity fields at once: $v_\theta(a^u, u, o, \emptyset)$, typical behavior, and $v_\theta(a^u, u, o, c_A)$, good behavior. The gating is exactly the **condition dropout** of **classifier-free guidance** (Ho and Salimans, 2022) — randomly replacing the condition with a null token during training so one model serves as both generators.

12.4.2 The fix: extrapolate along the difference

If you have both fields, their difference is meaningful: $v_\theta(\cdot, c_A) - v_\theta(\cdot, \emptyset)$ points from "what the policy would do anyway" toward "what it does when told to be good." Rather than taking one step of that difference, take g . At each of the K integration steps, run the action expert twice and step along the extrapolated field:

$$\tilde{v}(a^u, u, o) = v_\theta(a^u, u, o, \emptyset) + g[v_\theta(a^u, u, o, c_A) - v_\theta(a^u, u, o, \emptyset)]$$

IN PLAIN WORDS

The left side, $\tilde{v}$, is the velocity you actually integrate with — the direction the candidate chunk moves this sampling step. $\mathbf{a}^u$ is the partially-formed chunk, u its flow time, $\mathbf{o}$ the observation. $\mathbf{v}_\theta(\dots, \emptyset)$ is the output with the condition nulled (average behavior), $\mathbf{v}_\theta(\dots, \mathbf{c}_A)$ the output with it set to "good" (above-average behavior). The bracket is the difference, a small vector pointing from average toward good; g , the guidance scale, says how many copies of it to add. At $g = 1$ the two $\mathbf{v}_\theta(\dots, \emptyset)$ terms cancel and you get plain conditional sampling; at $g = 5$ you move five times as far in the good direction as the conditional model proposed.

Figure 12.2 draws all three vectors at one point: guidance is not a new direction, it is the same one walked further. Why does that help rather than just being wrong? The answer needs one step of Bayes, so let us take it slowly rather than assert it.

Start with what conditioning already means. Bayes says $p(\mathbf{a} \mid \mathbf{o}, \mathbf{c}_A) \propto p(\mathbf{a} \mid \mathbf{o})p(\text{good} \mid \mathbf{a}, \mathbf{o})$: the probability of chunk $\mathbf{a}$ given that we asked for a good execution is the probability of $\mathbf{a}$ under the policy's ordinary behavior, reweighted by how likely $\mathbf{a}$ is to turn out well. Read the pieces. $p(\mathbf{a} \mid \mathbf{o})$ is the prior, what the policy would do anyway. $p(\text{good} \mid \mathbf{a}, \mathbf{o})$ is a classifier: given this observation and this chunk, how probable is a good outcome? Nobody trained that classifier; it is implied by the two velocity fields the network already learned. Ordinary conditional sampling carries exactly one factor of it.

Guided sampling with scale g draws instead from a distribution proportional to $p(\mathbf{a} \mid \mathbf{o}, \mathbf{c}_A)p(\text{good} \mid \mathbf{a}, \mathbf{o})^{g-1}$ — the conditional distribution, tilted by $g - 1$ extra copies of that classifier. Substitute the Bayes expression for the first factor and the classifier terms combine, since $\mathbf{x}^1 \mathbf{x}^{g-1} = \mathbf{x}^g$:

$$p(\mathbf{a} \mid \mathbf{o})p(\text{good} \mid \mathbf{a}, \mathbf{o}) \cdot p(\text{good} \mid \mathbf{a}, \mathbf{o})^{g-1} = p(\mathbf{a} \mid \mathbf{o})p(\text{good} \mid \mathbf{a}, \mathbf{o})^{1+(g-1)} = p(\mathbf{a} \mid \mathbf{o})p(\text{good} \mid \mathbf{a}, \mathbf{o})^g$$

So the whole effect of guidance is to sample from the policy's ordinary distribution with an implicit success classifier raised to the power g — sharpening a success detector you never built. Check the reduction two ways. At $g = 1$ the first form's exponent is zero, the extra factor becomes 1, and you are back to ordinary conditional sampling; the second form gives $p(\mathbf{a} \mid \mathbf{o})p(\text{good} \mid \mathbf{a}, \mathbf{o})$, which by Bayes is the same thing. This is the cleanest sense in which a conditioning policy can beat its own average data.

One piece of honesty about that result. It is proved for diffusion models, where the network estimates a score $\nabla_{\mathbf{a}} \log p$ and the algebra of tilted distributions goes through exactly. Flow matching learns a velocity field, which is related to the score but not identical to it, and the derivation is not redone here or in most papers. Carrying the tilted-distribution story across to flow velocity fields as-is is what everyone does, the π -family included, and it works in practice. Hold it as an inherited justification, not one this chapter established.

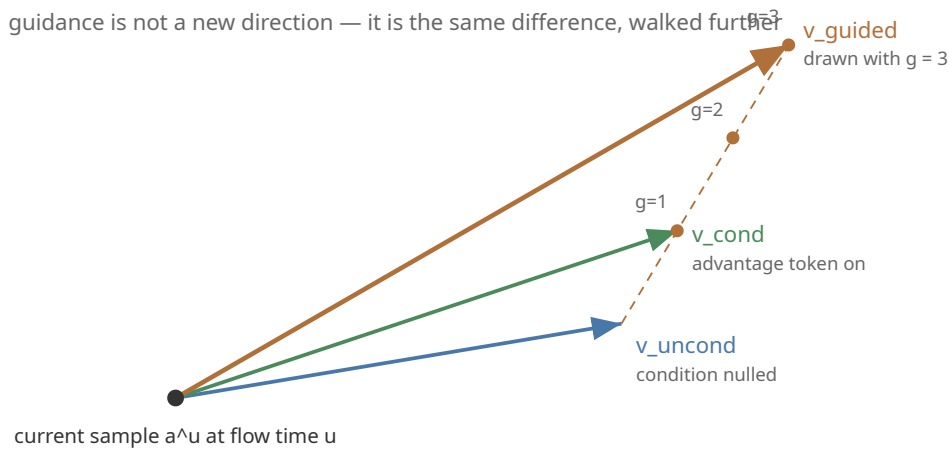


Figure 12.2 — Classifier-free guidance as extrapolation at one integration step. All three vectors start at the same point. Notice how little the conditional vector differs from the unconditional one: that small gap is what guidance multiplies, which is why a large g is needed. The dots mark $g = 1, 2, 3$, and nothing guarantees the model is accurate far along the dashed line.

12.4.3 The surprising empirical result

How far can you push g ? Image generation typically uses 1.5 to 8, with artifacts at the top of that range, and robotics usually far less. Larchenko searched 0 to 2 — a reasonable range. The bandit of Section 12.6 kept pinning the top arm. He extended the ladder; it kept pinning the top arm. It settled at g between 7 and 9 by garment type, which he calls "surprisingly high."

Two explanations, both worth internalizing because they tell you when to expect the same thing. First, the conditioning signal is weak by construction: a large share of training frames carry a masked condition, so the conditional–unconditional difference is a *small* vector needing a large multiplier. Second, the object being generated is forgiving. A natural image has 200,000 dimensions in which a small push off the manifold produces visible garbage; a 30-step chunk of 12-dimensional joint deltas is smooth and heavily constrained, so moderate extrapolation still lands on a feasible motion.

An analogy from photography. Guidance is unsharp masking: to sharpen a photo you subtract a blurred copy and add several multiples of the difference back, **sharp** = **image** + g (**image** – **blur**). Same formula, same intuition — amplify what distinguishes this thing from its bland version. *Where it breaks down:* unsharp masking is applied once, to a finished image. Guidance is applied at every one of the K steps, and each amplified step changes the input to the next, so errors compound. That is why too much guidance gives you an arm saturating its joint limits.

The cost is mild: two action-expert passes per integration step, sharing one VLM prefix cache — the expensive part of the model runs once.

```

prefix = vlm(o)                                # expensive; computed once
a_ref = shift(a_prev, nf_off)                   # old chunk on the NEW chunk's clock
a = T_z * randn(H, d)                           # T_z scales the only noise drawn
for i in range(K):
    u = i / K
    v_unc = expert(a, u, prefix, cond=NULL)      # advantage token masked
    v_con = expert(a, u, prefix, cond=C_A)       # advantage token = "good"
    v = v_unc + g * (v_con - v_unc)             # g = 1 -> plain conditioning
    if u > u_ip:                                 # inpainting switches on late
        v = v + W * (a_ref - a) / max(1e-3, 1.0 - u) # floor: 1-u -> 0
    a = a + v / K                               # Euler step of size 1/K
    a[:d] = a_ref[:d]                           # freeze the committed prefix

```

12.5 Best-of-N with the Q-like head

The last thing we want: never commit to a catastrophic chunk. Not "always pick the optimal chunk" — just never take the one that ends the episode.

The obvious fix is to reduce randomness: lower the noise temperature, take more integration steps, hold the seed fixed so a given observation always yields the same chunk.

That first knob is the seventh on the list, and so far this chapter has only named it, so here it is properly. The **noise temperature** T_z (the tech report writes τ ; this book keeps τ for a trajectory, as in Chapters 1 and 2, so the temperature gets its own letter) is the scale on the initial noise: instead of starting integration from $\mathbf{a}^0 = \mathbf{z}$ with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, you start from $\mathbf{a}^0 = T_z \mathbf{z}$. Nothing else changes — not the weights, not the velocity field, not the number of steps K . Since the finished chunk at $\mathbf{u} = 1$ is a deterministic function of where integration started — integration itself injects no further randomness, as the next page spells out — shrinking the ball you start from shrinks the spread of the chunks you finish with. At $T_z < 1$ all N candidates are drawn from a narrower region of noise space and land nearer the same mode: more repeatable, less diverse. At $T_z > 1$ they spread out and you see more of the model's alternatives, the good ones and the bad ones both.

Note where LeHome landed: T_z between 0.7 and 0.9, which is *below* the 1.0 the model was trained at. That is worth pausing on, because it is a deliberate choice and not a tuning artifact. Training taught the policy a distribution over plausible chunks; deployment sampled deliberately less of it, trading diversity for reliability. Sampling at the temperature you trained at is a convention, not a law.

But turning T_z down helps a little and misses the point. Catastrophic samples are usually not noise around a good plan; they are a *different mode*. At a multimodal decision point — which sleeve to grab first — the sampler occasionally commits to a mode that leads nowhere, and a lower temperature draws that mode more crisply rather than not at all.

So filter instead. Chapter 8's auxiliary heads included an action-conditional one: a **Q-like head** at the tail of the action expert reading the denoised chunk and predicting a success residual — how much this chunk raises or lowers the episode's success probability. That head turns sampling into search. Recall from Chapter 7 that in a flow policy the initial noise $\mathbf{z}$ is the *only* source of stochasticity: unlike DDPM-style diffusion sampling, no fresh noise is injected at each step, so fixing the observation and the seed makes integration deterministic. That is exactly what makes N candidates well-defined — N seeds, N chunks, no other randomness in the room. So draw N chunks from different seeds $\mathbf{z}_1, \dots, \mathbf{z}_N$, score each, execute the argmax (Figure 12.3) — the robot analog of best-of- N sampling in language models, where a reward model rescores candidates before one is emitted.

The economics are favorable: cost grows linearly in N but only in the small action expert, since all candidates share the one VLM prefix pass. In LeHome they are scored by the residual averaged over both guidance passes. Returns diminish fast: $N = 2$ or 3 helped consistently, beyond 3 nothing. And now the finding that should reshape how you think about verifiers. Larchenko reports the head's correlation with final episode outcome was *approximately zero* — and selection still helped. That is not a contradiction. Best-of- N is not fine-ranking good chunks; it is vetoing the

occasional disaster. Telling two plausible folds apart takes a well-calibrated value function; noticing that one candidate drives the gripper into the table does not. A verifier can be useless at ranking and still useful at rejecting. When every candidate scored negative, the client retried with a larger batch.

Now put T_z and N side by side, because they pull against each other and the chapter has been building to this. Best-of- N has something to choose between only if the candidates actually differ, and T_z is the knob that decides how much they differ. Turn T_z far enough down and your N chunks are near-copies of one another: you pay N times the action-expert cost to pick between three renderings of the same motion, and the veto never fires because there is no disaster in the batch to veto. Turn T_z up and you get genuine alternatives, including bad ones — more for the veto to catch, and more chances it misses one. The tuned pair, T_z of 0.7 to 0.9 with N of 2 to 3, sits where a small amount of spread buys a useful veto. Hold on to this pair for Section 12.6: it is the clearest case in the chapter where tuning each knob as if the others did not exist is visibly the wrong model of the problem.

An analogy from your day job. Run a linter over three candidate patches. It cannot say which is most elegant — its correlation with "good code" is near zero — but it reliably catches the one that does not compile. *Where it breaks down:* a linter is sound, its rules deterministic. The Q-like head is a learned estimate that can be confidently wrong in a systematic direction, and then best-of- N selects *for* those errors: $N > 1$ becomes worse than $N = 1$. Validate selection against random choice first.

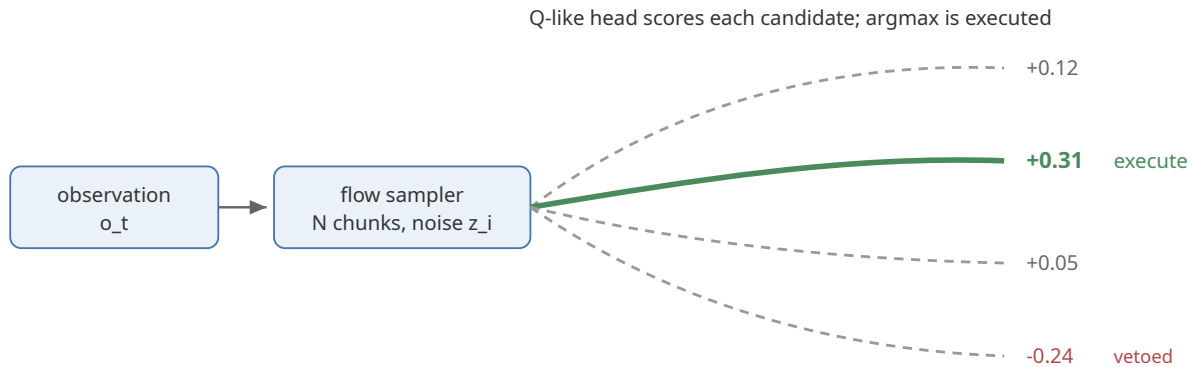


Figure 12.3 — Best-of- N chunk selection. The sampler draws N candidates from different noise seeds, all sharing one expensive prefix pass, and the Q-like head scores each. Notice how little separates the top three: that gap is noise and the head cannot rank them. What it can do is push the fourth clearly negative, and that veto is nearly all the benefit.

12.6 Tuning seven knobs with a bandit

12.6.1 Why grid search is dead on arrival

Count them up: execution length n_e , playback stretch, anchor length n_a , inpainting onset u_{ip} , guidance scale g , noise temperature, best-of- N count N . Seven, each tuned *per garment type* in LeHome, so really twenty-eight.

The obvious approach is a grid search, and it dies immediately. With only four candidate values per knob that is $4^7 \approx 16,000$ configurations per garment type. Each rollout returns one bit, and a bit is a terrible measurement — telling 70% from 75% confidently takes on the order of a thousand trials. Even at a lenient twenty rollouts per configuration you need 320,000 rollouts of about thirty seconds each: 9.6 million seconds, or roughly 2,670 machine-hours. On the four simulator instances a single rollout box runs, that is about 28 days of wall clock — for one garment type, and you have four. Parallel instances bring the wall clock down; they do not change the shape of the problem, and every hour spent on the grid is an hour not spent collecting training data.

But look at what is already happening. Chapter 14's asynchronous flywheel runs rollout workers continuously, hundreds of episodes a day, to feed the training worker — and every one of those episodes has to pick *some* value for each knob anyway. Tuning does not need its own budget. It needs to piggyback.

12.6.2 Bandits and Thompson sampling from scratch

A **multi-armed bandit** is the simplest sequential decision problem: several options ("arms"), each with an unknown success probability, chosen one at a time with the outcome observed after each. No state, no delayed consequence — only the tension between playing what looks best and testing what you are unsure about.

Thompson sampling (Thompson, 1933) resolves that tension without a schedule. For each arm keep a **Beta posterior**, $\text{Beta}(\mathbf{S}, \mathbf{F})$: a distribution over that arm's unknown success rate, $\mathbf{S}$ counting successes plus a prior and $\mathbf{F}$ failures plus a prior. $\text{Beta}(1,1)$ is flat — "could be anything." $\text{Beta}(9,3)$ is a bump around 0.75 with real width; $\text{Beta}(90,30)$ a spike at 0.75. Each episode, draw *one sample from every arm's posterior*, play the arm whose sample is highest, update its counts.

Exploration falls out of the arithmetic. A wide posterior sometimes draws high by luck, so that arm keeps getting tried; a narrow one draws close to its mean. As evidence accumulates the posteriors narrow, lucky draws get rarer, and allocation concentrates on the winner — no epsilon to decay, no schedule to tune, nothing ruled out by one unlucky early failure.

An analogy you have probably lived. This is an A/B test that reallocates traffic as it runs, rather than splitting 50/50 for two weeks and then deciding. *Where it breaks down:* a web A/B test returns a conversion event in seconds from a stationary system. Your feedback arrives thirty seconds later, is one bit, and the checkpoint is being retrained underneath you — the best n_e for a clumsy early policy is not the best for a competent late one.

12.6.3 The four adjustments that make it work in practice

Larchenko runs the bandit **factorized**: one independent bandit per knob, not one over the joint 16,000-cell space. That assumes knob interactions are weak, which is not exactly true — the latency budget above showed $\mathbf{N}$ and n_e fighting over the same milliseconds, and Section 12.5 showed $\mathbf{T}_z$ and $\mathbf{N}$ pulling in opposite directions on candidate diversity. But it needs evidence on only about 7 knobs $\times$ 4 candidate values = 28 arms per garment type, instead of 4^7 joint cells. (That 28 is a different 28 from the one in 12.6.1, which counted 7 knobs $\times$ 4 garment types. The collision is an accident of the numbers; these are arms, those were knob instances.) It is the difference between tractable and impossible. Take the assumption knowingly.

Three more adjustments. *Centered rewards*: reward is success minus the garment type's mean success rate, so an arm is not punished for drawing a hard garment. *Unbiased episodes only*: replay and hard-mining episodes (Chapter 14) start from restored states with wildly different success rates, so letting them update the bandit would corrupt every posterior. *Posterior decay*: each checkpoint iteration, shrink posteriors toward the prior so old evidence fades as the policy drifts. Finally the bandit is **frozen for submission** — you evaluate with fixed knobs, not a still-exploring sampler.

```
# one Beta bandit per knob; arms = candidate values
S = {arm: 1.0 for arm in arms}    # prior successes: Beta(1,1)
F = {arm: 1.0 for arm in arms}    # prior failures

def choose():
    draws = {arm: np.random.beta(S[arm], F[arm]) for arm in arms}
    return max(draws, key=draws.get)

def update(arm, success):          # after the episode ends
    S[arm] += success
    F[arm] += 1 - success

def decay(factor=0.9):            # once per checkpoint iteration
    for arm in arms:              # forget slowly: the world drifts
        S[arm] = 1 + factor * (S[arm] - 1)
        F[arm] = 1 + factor * (F[arm] - 1)
```

12.6.4 Worked example: five pulls by hand

Tune $n_e \in \{3, 5, 10\}$: arms A, B, C, all starting at Beta(1,1), uniform over success probability — we believe nothing. Watch the draws, not the means.

- **Pull 1:** draws 0.62, 0.41, 0.77 → play C; episode fails → C becomes Beta(1,2). All three posteriors were identical, so this choice was pure luck of the draw — what "no information" looks like.
- **Pull 2:** draws 0.35, 0.88, 0.29 → play B; success → B: Beta(2,1). C drew low partly because its posterior shifted down after that failure.
- **Pull 3:** draws 0.71, 0.66, 0.14 → play A; failure → A: Beta(1,2). B just succeeded and still lost: a mean near 0.67 loses to a lucky draw from an untested arm. That is exploration, unscheduled.
- **Pull 4:** draws 0.20, 0.93, 0.55 → play B; success → B: Beta(3,1). Now B wins on merit: two successes and no failures put most of its mass above 0.5.
- **Pull 5:** draws 0.44, 0.72, 0.61 → play B; failure → B: Beta(3,2). One failure does not undo two successes; it widens B's posterior downward, raising how often A and C win future draws.

After five pulls the posterior means are 0.33 (A), 0.60 (B), 0.33 (C) — that is $S/(S+F)$: **1/3, 3/5, 1/3**. But the next pull is not automatically B, and it is worth seeing exactly why rather than taking it on faith. Arm A's posterior is Beta(1,2), whose density is $2(1-x)$; arm B's is Beta(3,2), whose density is $12x^2(1-x)$. The chance A's draw beats B's is one posterior integrated against the other:

$$P(a > b) = \int_0^1 12b^2(1-b) \left[\int_b^1 2(1-x) dx \right] db = \int_0^1 12b^2(1-b)(1-b)^2 db = 12 \int_0^1 b^2(1-b)^3 db = \frac{12}{60} = \frac{1}{5}$$

Take that one bracket at a time. The inner integral is the probability that A's draw lands above some particular value b , and for Beta(1,2) it works out to $(1-b)^2$. The outer integral averages that over every value b that B might draw, weighted by B's own density. The last step uses $\int_0^1 b^2(1-b)^3 db = 1/60$, a standard Beta integral. Arm C is in exactly A's state, so it too wins one draw in five. The chance that *either* beats B is about one in three — not two-fifths, because A and C sometimes win the same round, and the same double integral with both untested arms gives 0.33. So A and C keep being tested while B accumulates evidence. That explore–exploit balance, driven entirely by posterior width, is why five noisy rollouts already produce a sensible allocation while a grid search would still be on its first cell.

PITFALLS

(1) A guidance scale pushed too high walks off the action manifold: confident nonsense, jerky saturated motions. Watch the arm as g grows, not just the success rate. (2) Best-of-N with a miscalibrated Q-like head *amplifies* its errors: if the head systematically prefers a wrong mode, $N > 1$ is worse than $N = 1$. Validate against random choice. (3) Bandit feedback is delayed by a whole episode and confounded with checkpoint updates: decay the posteriors, and never update from replay or hard-mining episodes, whose success rates are not comparable.

IN THE LEHOME SOLUTION

Seven per-garment-type knobs — execution length, playback stretch, anchor length, inpainting onset, guidance scale, noise temperature, best-of-N — tuned by a factorized Thompson bandit rewarded by success minus the garment's mean success rate. Only unbiased episodes update it, posteriors decay each iteration, and it is frozen for submission. Final values: n_e 3–5 of 30, g 7–9, N 2–3, T_z 0.7–0.9, u_{ip} 0.4–0.5. Read T_z below 1 as sampling deliberately narrower than training implied. The high guidance scale surfaced only because the bandit kept pinning the top arm as the ladder was extended; the report gives no per-knob ablations, so treat individual contributions as unmeasured.

CHECKPOINT

- Explain why executing a whole chunk is open-loop control in disguise, and state both sides of the n_e trade-off with a concrete re-planning rate.
- Describe how naive asynchronous chunk swapping fails, why delta actions make it worse, what real-time chunking freezes, blends and leaves free, and what $(a^{\text{prev}} - a^u)/(1 - u)$, W_j and u_{ip} mean in the correction.
- Write the guidance formula, name every symbol, verify by hand that $g = 1$ reduces to plain conditional sampling, and give both reasons $g = 7$ to 9 was viable here when image models use far less.
- Explain how a verifier with near-zero outcome correlation still improves success, when best-of-N makes a policy worse, and why the noise temperature T_z and the candidate count N cannot honestly be tuned as independent knobs.
- Set up a Thompson bandit for one knob, simulate a few pulls by hand, justify factorization and posterior decay, and build a per-query latency budget from control rate, K , N and guidance.

Where to go deeper. Real-time chunking is Black et al. (2025) — short and unusually readable, and worth reading before you implement any asynchronous inference. Classifier-free guidance is Ho and Salimans (2022), written for images but applied verbatim to flow-matching velocity fields. The seven-knob inventory, the surprisingly high guidance scales, and the factorized bandit are Section 7 of Larchenko (2026). For Thompson sampling beyond the Beta–Bernoulli case, the tutorial by Russo et al. (2018) is standard.

Chapter 13 — Imitation and the human in the loop

Every chapter so far has assumed a policy already exists and your job is to improve it — with advantage weights in Chapter 4, with more expressive action distributions in Chapters 6 and 7, with advantage conditioning in Chapter 10, with better rewards in Chapter 11. None of that answers the first question: where does the initial policy come from, and what do you do when it is not good enough?

In practice the answer is a human. Someone teleoperates the robot until there is data to train on, and then someone sits beside the robot while it runs and takes over when it is about to do something stupid. That second activity is the cheapest and highest-leverage resource in robot learning, and it is what this chapter is about.

You already run a pipeline built on exactly this idea: an SO-101 arm, an actor process and a learner process, an Xbox pad in your hands, and a working run in which you were intervening 98% of the time. By the end of this chapter you should understand what each piece is doing, why the intervention rate is the number to watch, and why 98% on an early session is the expected starting point rather than a sign of failure.

WORDS YOU WILL HEAR

Covariate shift — the training inputs and the deployment inputs come from different distributions; in imitation, the demonstrator's states versus the policy's own states. **No-regret** — a property of an online learning algorithm: as rounds accumulate, its average performance approaches that of the best fixed choice in hindsight; vanilla DAgger inherits it, the human-gated variants of Section 13.3 do not, and they trade it for a labeling cost a person can actually pay. **Gate** — the switch that decides who is currently commanding the robot, the policy or the human. **Intervention** — one contiguous segment of human control, from takeover to release. **Replay ratio** (or update-to-data ratio, UTD) — how many gradient steps the learner takes per new environment transition. **Symmetric sampling** — forcing every training batch to be half human-sourced data and half fresh online data.

13.1 Behavior cloning and compounding error

13.1.1 What we want, and the obvious thing to try

What we want is a policy that performs the task, and the cheapest route is to record a human performing it and fit a network to the recording. Chapter 4 called this **behavior cloning** (BC): supervised learning on demonstrated observation–action pairs, identical to AWR's weighted regression with every weight set to one, with no rewards, no rollouts and no credit assignment. Every VLA in Chapter 8 is behavior cloning at scale.

So the obvious thing to try is: collect a hundred demonstrations, train until validation loss stops falling, press run. For short tasks this works better than it has any right to. Then it falls apart, in a way that catches everyone the first time.

13.1.2 How it fails, step by step

Supervised learning gives you one guarantee only: low error on the distribution the training data came from — the states the demonstrator π_β visited. Call that set the *demo tube*, and watch what happens at deployment one step at a time.

At step 0 the robot starts inside the tube, because you reset it the way you always did. The policy predicts an action close to the human's but not identical — a network trained to a finite loss is never exactly right. Say the gripper ends up one millimeter left of where the human's would have been.

At step 1 the robot is therefore slightly off the tube's centerline. It is still inside the tube, so the policy is still reasonably accurate, but a little less accurate than at the center, because this state is a little less represented in the training data. Its next error is a bit larger — say 1.8 millimeters. At step 2 the error is about 3 millimeters, and by now the accumulated offset has carried the gripper out near the tube's edge.

Notice how undramatic that is. Nothing alarming happens on any single step, which is exactly why this failure is hard to catch by watching one frame at a time. The compounding is slow at first, and only becomes visible once the accumulated offset has walked the robot out of the region your demonstrations cover.

At step 3 the robot is outside the tube entirely, and the situation is now qualitatively different. The network has never seen anything like this observation, so its output is not "slightly wrong", it is arbitrary. No gradient anywhere in training ever told it what to do here.

The critical property is that *this loop feeds itself*. Each error moves the robot to a state where the policy is worse, and a worse policy produces a larger error, which moves the robot somewhere worse still. It is not a nuisance that averages out; it is positive feedback. Figure 13.1 is the picture to keep in your head.

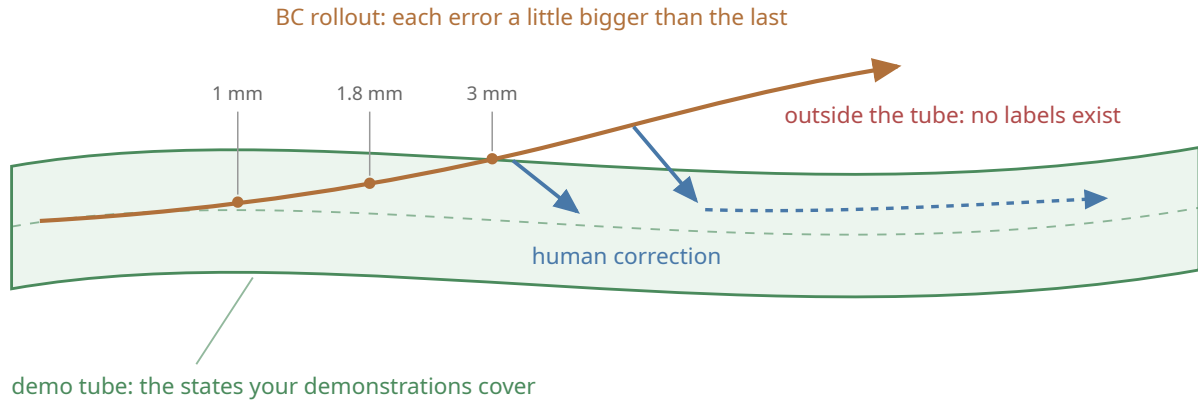


Figure 13.1 — Covariate shift, drawn. Notice two things. First, the labeled errors along the orange rollout: each one is small, and each is a little bigger than the last, which is why the drift is invisible for a while and then suddenly is not. Second, both blue correction arrows start right at the tube's edge rather than far outside it — a human who waits until the robot is hopelessly lost is labeling states the policy will never usefully revisit. The dashed blue path is the rollout after a correction: back inside the tube, where the policy is accurate again.

Two different things are going on here, and it is worth separating them before the next section, because they blur together easily. What you just walked through is a *mechanism*: why one mistake tends to lead to another, larger mistake. The T^2 in the bound below comes from something simpler and more countable — how many mistakes you expect over an episode, multiplied by how long each one keeps hurting. The walkthrough is the intuition; the counting argument is the proof. So do not read the millimeter numbers as a growth law: a policy whose error truly multiplied by four every step would be off the table inside two seconds, and cloning would never work at all.

13.1.3 Why the damage is quadratic

There is a classic result that makes this precise, and it comes in two halves with two different sources: the quadratic side is Ross and Bagnell (2010), and the linear side is the DAGger analysis of Ross et al. (2011), which Section 13.2 comes to. Let ϵ be the per-step mistake rate of the learned policy measured on the demonstrator's distribution — the supervised error rate, which in the imitation literature is universally written ϵ ; this is *not* the PPO clip parameter of Chapter 3, and the collision of symbols is unfortunate but standard. Let T be the episode length in steps, and let J be the expected cost of running a policy for one episode. Then:

$$J(\pi_{\theta}^{\text{BC}}) \leq J(\pi_{\beta}) + \mathcal{O}(\epsilon T^2), \quad J(\pi_{\theta}^{\text{DAGger}}) \leq J(\pi_{\beta}) + \mathcal{O}(\epsilon T).$$

IN PLAIN WORDS

J is the expected cost of one episode — how much worse than perfect the run is, summed over its steps. $J(\pi_\beta)$ is the cost of the human demonstrator: the standard you are trying to match. π_θ^{BC} is the policy you get from plain behavior cloning, $\pi_\theta^{\text{Dagger}}$ the one you get from the method in Section 13.2. ϵ is how often the learned policy picks a bad action on the *demonstrator's* states, as a fraction between 0 and 1. T is the number of steps in an episode. $\mathcal{O}(\cdot)$ means "grows like, up to a constant factor". So the whole statement reads: cloning can be worse than the human by an amount that grows with the *square* of the episode length, while the corrected method pays only in proportion to the episode length. Doubling how long the task takes quadruples one penalty and merely doubles the other.

Where does the square come from? Two factors of T multiply. The first is how many mistakes you expect: at rate ϵ over T steps, roughly ϵT of them. The second is what each mistake costs: a mistake at step t can push you off-distribution, and once off-distribution you may stay there for all $T - t$ remaining steps, so one mistake is charged for up to T bad steps rather than one. Multiply and you get ϵT^2 . DAgger removes the second factor by measuring ϵ on the policy's *own* states, where a mistake is just one bad step in a place the policy was trained — it does not open a trapdoor.

13.1.4 A small worked example: how good would BC have to be?

Put your own numbers on it. Your SO-101 runs autonomously at 30 Hz (Chapter 4's convention: 20 Hz when you are teleoperating through the leader arm, 30 Hz when the policy drives), so a 20-second task is $T = 600$ steps. Suppose your cloned policy is right 99% of the time on demonstration states, so $\epsilon = 0.01$. One line below needs a word of justification before you meet it, the average damage per mistake. The prose above said a mistake can cost you up to T steps, which is the worst case, when it happens on step 1. But a mistake is as likely at step 10 as at step 290, so on average half the episode is still ahead of it, and the expected number of remaining steps is $T/2$. Step through it:

expected mistakes per episode	= $\text{eps} * T$	= $0.01 * 600$	= 6
average damage per mistake	~ half the horizon	= $600 / 2$	= 300 steps
total expected damage (BC)	= $6 * 300$		= 1800 step-units
	(this is the $\text{eps} * T^2 / 2$ term: $0.01 * 360000 / 2 = 1800$)		
total expected damage (DAgger)	= $\text{eps} * T$		= 6 step-units
ratio	= $1800 / 6$		= 300x

Cost here is scaled so a perfectly bad step costs 1, which caps a 600-step episode at 600 no matter what the policy does. A bound of 1800 is therefore three times larger than the worst thing that can possibly happen, and the natural reading of that is wrong. It does not say the episode is 1.5 times lost. It says the guarantee has gone *vacuous*: still true, since 1800 does bound a cost of at most 600, but carrying no information, because you knew the answer was under 600 before doing any analysis. That is the real content of the T^2 at this horizon — cloning comes with no useful promise at all. Now run the question backwards. How accurate would cloning have to be for the expected damage to stay near 6 steps, the level DAgger reaches at $\epsilon = 0.01$? Solve $\epsilon \cdot T^2 / 2 = 6$:

$$\epsilon = \frac{2 \times 6}{600^2} = \frac{12}{360,000} = 3.3 \times 10^{-5}.$$

That is one mistake per 30,000 frames — fifty consecutive flawless episodes. No behavior-cloned policy on a real arm is that accurate. Note also which lever is which: ϵ is what more demonstrations improve, and it enters the bound linearly, while T enters it quadratically. On long-horizon tasks, data volume loses that race. The fix cannot be "train harder"; it has to be "train somewhere else".

13.1.5 An analogy, and where it breaks

You have met a version of this on the rover. Dead reckoning integrates wheel encoders to estimate position; a one-degree heading bias over five meters puts you about nine centimeters off, and the error accumulates the longer you go

without an external fix. Compounding error in BC feels the same: small per-step inaccuracies that integrate into a large end-of-episode failure.

Now the limit, and it matters. In dead reckoning the error lives in your *estimate*; the world is unchanged, and the size of the next error does not depend on how wrong you already are, so the error grows linearly (or as a square root, for a random walk). In behavior cloning the error changes *which state you are actually in*, and the state determines the accuracy of the next prediction. That is a feedback loop, not an integrator, and it is why the growth is quadratic. A drifting odometer gives you a bad map; a drifting policy gives you a robot in a configuration no one has ever labeled.

13.1.6 What it looks like on a real arm

Physically this is mundane and easy to miss. The approach is two centimeters left of every demonstration you recorded. The policy does not explode or thrash; it freezes, or oscillates gently, or closes the gripper on air a centimeter short of the cloth, or drives into the table with mild constant force. Watch it and you think "it almost has it". It does not. It is in a state where its output is arbitrary, and it will stay there until you reset.

PITFALLS

"More demos will fix it" — usually not. Compounding error is about *where* the data lives, not how much of it there is; 500 extra demonstrations thicken the tube without covering the states your policy drifts into. Two more quiet ways to make interactive imitation worse than plain BC: training at full weight on the frames just before a takeover (they *are* the mistake — ramp them to zero, Section 13.4), and discarding earlier rounds when you aggregate (the policy then oscillates between failure modes instead of accumulating fixes).

13.2 DAgger: label where the policy actually goes

The fix is not more demonstrations. It is demonstrations *in the right places* — specifically, in the states your current policy actually reaches. **DAgger** (Dataset Aggregation; Ross et al., 2011) is the algorithm that does this, and it is four steps repeated:

1. **Roll out the current policy** π_θ and record the states it visits. The original algorithm mixes in the expert with a coefficient that decays over rounds, so early rounds stay near the demonstrator's distribution and later rounds are fully autonomous.
2. **Ask the expert for the correct action** at each visited state — what the human *would* have done there, not what the policy did.
3. **Aggregate**: append those new (state, expert action) pairs to everything collected in all previous rounds.
4. **Retrain** the policy on the whole aggregated dataset, and go back to step 1.

After a few rounds the training distribution converges toward the one the policy actually induces. Once the two agree, the supervised error rate ϵ is measured on exactly the states where the policy will be asked to act, and the quadratic blow-up collapses to the linear rate. That is what "DAgger is a no-regret algorithm" buys you.

Step 3 is the one people cut, and it is the one you must not. Train each round only on that round's fresh corrections and the policy fixes the current failure while forgetting the previous one, then fixes that and reintroduces the first. You see it as a success rate that oscillates and never converges. The aggregated dataset is the memory of the process.

13.2.1 The honest cost

Here is the part papers state politely and practitioners feel immediately. Vanilla DAgger requires the expert's action in *every* visited state. On your SO-101 that means the system pauses on a frame and asks: what should the action have been here? The answer is a 12-dimensional vector of joint commands, and no human can write that down. Re-teleoperating the robot back to each visited state so you can demonstrate from there costs about as much as demonstrating from scratch, so you have saved nothing.

A software analogy: your demonstrations are a test suite whose inputs you chose yourself, so they pass and tell you nothing about what production sees; DAgger is fuzzing in production and turning every crash into a new test case. The analogy breaks at the expensive part. A fuzzer detects failures automatically and you write the fix once, whereas DAgger needs a human to supply the correct 12-dimensional output for every frame collected. Automatic discovery, manual labeling. Every interactive variant below exists to spend that human effort only where it pays.

13.3 HG-Dagger: the human as the gate

HG-Dagger (Human-Gated DAgger; Kelly et al., 2019) makes one change, and it is the change that made interactive imitation practical on real robots. It throws away the mixing schedule and replaces it with human judgment. The policy runs autonomously while a human watches. The human takes control only when the robot is heading somewhere bad, drives it back to safety, and releases. Writing π_H for the human policy — the same demonstrator π_β from Section 13.1, now acting online instead of from a recording:

$$\pi_{\text{HG}}(a | s) = \begin{cases} \pi_H(a | s) & \text{human engaged} \\ \pi_\theta(a | s) & \text{otherwise.} \end{cases}$$

IN PLAIN WORDS

$\pi_{\text{HG}}(a | s)$ is the policy that is actually commanding the robot right now: the probability of taking action a in state s . It is defined by cases. When the human has the gate engaged — your finger on the trigger, a pedal down — the commands come from π_H , the human. When the gate is released, they come from π_θ , the network you are training. That is the entire equation, and its content is organizational rather than mathematical: there is no blending coefficient, no schedule, and no randomness deciding who drives. A person decides, moment by moment, and the switch is instantaneous and total.

The labeling rule is the second half of the idea, and it is not the obvious one. Only the intervention segments — states visited while the human was engaged, paired with the human's actions — are added to the training set. Autonomous states are never relabeled. You might expect that to be a compromise forced by the cost of labeling. It is better than that.

Consider where interventions land. A human takes over when, and only when, two things are true at once: the policy reached this state on its own, *and* the policy is handling it badly. That intersection is the current policy's failure boundary. Frames sampled uniformly from an autonomous rollout are mostly frames where the policy is already fine, and training on them teaches it what it already knows. Intervention frames are the most information-dense imitation data you can collect per unit of human time.

Two consequences follow. First, the intervention rate is a live gauge of policy quality that costs nothing to measure — Kelly et al. turn intervention statistics into an explicit risk estimate. Second, you get a curriculum for free: each round's corrections target whatever the previous round still gets wrong, with nobody designing a lesson plan.

13.3.1 What HG-Dagger gives up

Now the price, and be explicit about it, because it is easy to carry Section 13.1's guarantee forward without noticing that it stopped applying. The $\mathcal{O}(\epsilon T)$ half of that bound is a theorem about vanilla DAgger, and its proof leans on two things HG-Dagger deliberately does not do: roll out a controlled mixture of expert and learner, and obtain the expert's action at *every* visited state. HG-Dagger keeps neither. A person takes over when they judge it necessary, and only the states they drove through get labeled. So the intervention-segments-only rule is a well-motivated heuristic, not a theorem, and the same caveat applies to HIL-SERL in Section 13.5, which gates the same way. If you run the pipeline in this chapter, you are not running the algorithm the linear bound was proved about.

What survives is the part that was doing the work. The bound was pointing at a mechanism — train on the states the policy actually reaches, not the states the demonstrator reached — and human gating implements that mechanism

faithfully, just on a human-chosen subset of those states. You trade a proof for a labeling cost a person can actually pay. On a real arm that is almost always the right trade, but it is a trade, and you should know you made it.

13.3.2 “But won't splicing corrections in create multimodal targets?”

This worry stops a lot of people, and you should have it, because Chapter 5 spent a whole chapter on why a policy that averages two valid behaviors produces a third invalid one. If your dataset contains both the policy's own trajectories and human corrections, are you not training on two different behaviors and asking the network to split the difference?

No, and the reason is worth being careful about. The policy learns $\pi_{\theta}(a | o)$ — the action *conditioned on the observation*. Every correction is recorded in the drifted, off-distribution state where it was actually given, and that state is by construction one the original demonstrations never covered. So at that observation there is exactly one label in the dataset: the correction. At the observations the demonstrations do cover there are no corrections, because the human had no reason to intervene. The two sources occupy different regions of observation space rather than competing for the same one.

Genuine multimodality is something else: two *different* actions labeled at the *same* state. Two operators folding the same flat shirt, one starting with the left sleeve and one with the right, both looking at an identical overhead image. That is real, it is common in demonstration data, and it is a large part of why Chapters 6 and 7 exist. But gated corrections do not manufacture it. If anything they reduce it, since a correction commits decisively to one recovery from a state where the policy was dithering between several.

The honest caveat: this argument depends on the observation distinguishing the drifted state from the nominal one. If the observation is a low-resolution overhead image and the drift is two millimeters of gripper position, the network cannot tell them apart, and you do get conflicting labels at effectively the same observation. The fix is a better observation — wrist cameras exist for this — not a different training rule.

13.4 Intervention signals as a quasi-advantage

So far an intervention has been a source of labels. It is more than that: it is a *verdict*. By keeping their hands off the controller for a hundred consecutive frames, the human implicitly scored those hundred policy actions as acceptable; by taking over at frame 101, they scored the recent actions as bad and supplied better ones. You paid a human to evaluate the policy in real time, so it is wasteful to keep the labels and discard the evaluation.

Chapter 4 gave you the machinery. AWR turns an advantage estimate into a per-sample weight on a supervised loss; here the human's gate stands in for the critic:

$$P(\text{sample } i) \propto \exp(\hat{A}_i^{\text{hum}}/\beta), \quad \hat{A}_i^{\text{hum}} = \begin{cases} A_{\text{corr}} & i \in \text{correction segment} \\ 0 & i \text{ autonomous, far from takeovers} \\ -cm_i & i \text{ in the pre-takeover window,} \end{cases}$$

IN PLAIN WORDS

The left side is the probability that frame i gets drawn into a training batch: $\propto$ means "proportional to", so these numbers are normalized across the dataset to sum to one. $\hat{A}_i^{\text{hum}}$ is a stand-in advantage for frame i , inferred from what the human did rather than from a critic, and β is the AWR temperature: small β makes the exponential sharp so only the best frames get drawn, large β flattens it toward uniform sampling. The three cases say: frames where the human was driving get a fixed positive score $A_{\text{corr}} > 0$, so $\exp(A_{\text{corr}}/\beta)$ is a weight above one. Ordinary autonomous frames get zero, so $\exp(0) = 1$, the baseline weight — the human let them pass, so they are neither exemplary nor wrong. Frames in the window just before a takeover get $-cm_i$, where c is a large positive constant and m_i is a ramp multiplier that climbs from 0 at the start of the window to 1 at the moment of takeover; the weight $\exp(-cm_i/\beta)$ therefore decays from 1 down toward 0 as the takeover approaches. In practice you can skip the exponential for that last case and ramp the weight itself linearly — the shape is what matters, not the parameterization.

Two notes on letters, because this section is crowded with them. This β is the AWR temperature of Chapter 4, and it has nothing to do with the β subscript in π_β , which names the demonstrator; they are different objects sharing a Greek letter, and both spellings are standard. And the ramp multiplier is written m_i here rather than the λ_i some papers use, to keep it clear of the GAE λ of Chapter 3 — if you meet λ_i in an intervention-weighting paper, expect it to mean this ramp.

The third case is the one people get wrong, so be explicit about why it is there. The frames immediately before a takeover are the frames in which the policy was doing the thing that made the human reach for the controller. They *are* the mistake. Train on them at full weight and you will faithfully clone the approach that was about to knock the garment off the table — having paid a human to help you do it. Ramping them to zero says: this segment is not a positive example, so rather than scoring how bad it was, stop learning from it.

Note also where the weights are applied. As in Chapter 4, putting them in the *sampler* rather than the loss is equivalent in expectation — drawing a frame twice as often and doubling its gradient do the same thing on average — but the sampler version is far cheaper, because a near-zero-weight frame is never loaded from disk or decoded. When each frame is three camera images, that is the difference between a data loader that keeps up with the GPU and one that does not.

```
import numpy as np

def frame_weights(is_interv, ramp=30, w_corr=4.0, w_auto=1.0):
    """Per-frame sampling weights for one episode. is_interv: bool array."""
    w = np.full(len(is_interv), w_auto, dtype=np.float32)
    w[is_interv] = w_corr
    # a takeover begins where the flag goes False -> True
    prev = np.concatenate([False, is_interv[:-1]])
    starts = np.flatnonzero(is_interv & ~prev)
    for s in starts:
        lo = max(0, s - ramp)
        n = s - lo
        if n == 0:
            continue
        # descending 1.0 -> ~0 over the frames leading INTO the takeover
        ramp_w = np.linspace(1.0, 0.0, n, endpoint=False)
        # only touch frames the POLICY was driving: an earlier correction
        # segment caught inside this window must keep its w_corr
        auto = ~is_interv[lo:s]
        # minimum() so overlapping ramps can lower a weight, never raise one
        w[lo:s] = np.where(auto, np.minimum(w[lo:s], ramp_w), w[lo:s])
    return w
```

The two guards in that loop exist for a case you will hit constantly, not an exotic one. Early in training you correct almost continuously, so takeovers land closer together than **ramp** frames and each ramp window opens on top of the previous correction segment. The **auto** mask is what stops that: frames the human was driving keep their full weight

no matter whose window they fall inside. Where two windows overlap on genuinely autonomous frames, `minimum` keeps the lower value, so a frame implicated in two nearby takeovers is discounted by the harsher of them.

Drop the mask and the scheme inverts itself exactly where you need it most. At the 98% intervention rate that is normal on a first session, nearly every ramp window lands on the preceding correction, so nearly every correction gets crushed from 4.0 down to a ramp value below 1.0: you would train hardest on the frames you meant to discard and discard the frames you paid a human to produce, and nothing in the loss curve would tell you. Worth a unit test — two takeovers ten frames apart, and assert the first correction segment still comes back at `w_corr`.

13.4.1 Worked example: weighting one DAgger round

You run 100 episodes at 30 Hz, 300 steps each — ten seconds of autonomous running apiece. Total frames:

$$100 \times 300 = 30,000 \text{ frames.}$$

The human intervened 20 times over those 100 episodes. Each intervention contributes a 50-frame correction segment and a 30-frame pre-takeover ramp in front of it. Sort every frame into one of three buckets:

bucket	frames		weight	sampling mass
corrections	20 * 50	= 1,000	4.0	1,000 * 4.0 = 4,000
ramps	20 * 30	= 600	0.5	600 * 0.5 = 300
clean auto	30,000 - 1,000 - 600	= 28,400	1.0	28,400 * 1.0 = 28,400
		-----		-----
total		30,000		32,700

The ramp weight of 0.5 is the average of a linear ramp from 1 down to 0 across the 30 frames. Sampling mass is simply frames times weight. Note that a ramp is a duration, not a frame count: 30 frames at 30 Hz is one second of discounted run-up, and if you change the control rate you must rescale it. LeHome used a five-second window, which on its rig is a far longer ramp than this example's.

With a batch size of 256, the expected number of slots each bucket claims is its share of the total mass times 256:

corrections	:	256 * 4,000 / 32,700 = 1,024,000 / 32,700 = 31.31 slots (12.2%)
ramps	:	256 * 300 / 32,700 = 76,800 / 32,700 = 2.35 slots (0.9%)
clean auto	:	256 * 28,400 / 32,700 = 7,270,400 / 32,700 = 222.34 slots (86.9%)
check	:	31.31 + 2.35 + 222.34 = 256.00

Compare that with uniform sampling, where corrections would get their raw share of the dataset:

$$\frac{1,000}{30,000} = 0.0333 = 3.3\%, \quad 256 \times 0.0333 \approx 8.5 \text{ slots.}$$

So the weighting oversamples corrections by $12.2/3.3 \approx 3.7\times$. That is the whole effect of the scheme, in one number.

And the knob runs backwards just as easily. If 12% is not enough and you want corrections to be a full quarter of every batch, set the correction mass $1,000w$ against the $600 \times 0.5 + 28,400 = 28,700$ that everything else contributes, solve $1,000w/(1,000w + 28,700) = 0.25$, and you get $w = 9.57$. That is the practical reason to prefer sampling weights over an opaque loss coefficient: you state your intent as a batch composition and back out the number.

13.5 HIL-SERL: the same gate, wrapped in an RL loop

13.5.1 The ceiling everything so far has been built under

Stop and notice what every method in this chapter has in common. In behavior cloning, in DAgger, in HG-DAgger, and in the weighting scheme you just worked through, every training target is a human action. The network is asked, over and over, to predict what the person would have done here. Section 13.4 did not change that; it only decided

which human actions to count more heavily. So there is a ceiling, and the ceiling is the human. A policy trained this way can approach the demonstrator and, with luck, average away some of their jitter. It cannot beat them, because nothing in the objective knows what "better" would even mean — no term in a weighted regression loss prefers a fast insertion to a slow one.

That ceiling is lower than it sounds, and the way to feel it is to teleoperate the task yourself. Drive the SO-101 through a connector insertion with the gamepad and watch what you actually do. You approach slowly, because you cannot feel contact through the pad. You stop short and creep the last two millimeters by eye. You back off and retry when the pin catches. A faithful clone learns all of it — the caution, the pauses, the retries — because those frames are in the dataset and every one is labeled correct. You did not demonstrate the task; you demonstrated your best attempt at it through a laggy interface with no force feedback, and the policy inherits your handicap with no way to discover that a firmer, faster approach would have worked.

Breaking the ceiling needs a signal that says something other than "match this action" — one that says "this outcome was good", so the policy can search for its own route to good outcomes, including routes no human ever showed it. That signal is reward, and turning reward into policy improvement is what Chapters 3 and 4 were about. The open questions are practical: how do you get a reward on a real robot without instrumenting the world, and how does the robot survive the exploration?

13.5.2 The four ingredients

HIL-SERL (Luo et al., 2024) answers both by putting the same human gate inside a full reinforcement learning loop instead of a supervised one. This is the pipeline you already run, so the goal here is to name each piece you have on disk. Four ingredients:

1. **A handful of teleoperated demonstrations** seeding an offline buffer. Not hundreds — tens. They give the critic something other than failure to look at in the first minutes.
2. **A reward classifier**: a small network trained on human-labeled success and failure images, whose output $\hat{p}_{succ}$ is thresholded into the sparse reward r_t . This is how you get a reward function on a real robot without instrumenting the world — you label a few hundred frames instead.
3. **Human interventions during autonomous rollouts**, stored in the demonstration buffer as corrections, exactly as in HG-Dagger above.
4. **An off-policy learner: RLPD** (Ball et al., 2023), an actor-critic in the style of SAC (soft actor-critic: an off-policy method training a Q-function alongside an entropy-regularized Gaussian actor, so the policy is rewarded for staying stochastic rather than collapsing early). RLPD adds a high update-to-data ratio — many gradient steps per collected transition, which is how an hour of robot time suffices — plus critic ensembling (several Q-networks whose minimum is used) to fight **off-policy overestimation**: a critic trained on its own bootstrapped targets tends to drift upward, because taking a maximum over actions systematically selects whichever Q value happens to be over-estimated, and that inflated number then becomes the target for the next update. It is a new failure mode if your RL experience is on-policy PPO, and it is the reason most of these tricks exist. Layer normalization on the critic (rescaling each layer's activations to zero mean and unit variance) keeps the same drift bounded as the update-to-data ratio climbs. The fourth addition is **symmetric sampling**.

$$\mathcal{B} = \frac{1}{2} \mathcal{B}_{\text{demo+interv}} + \frac{1}{2} \mathcal{B}_{\text{online}}, \quad y = r_t + \gamma \mathbb{E}_{a' \sim \pi_\theta} \left[\min_i Q_i(s_{t+1}, a') \right].$$

IN PLAIN WORDS

The first equation describes how one training batch $\mathcal{B}$ is assembled: half its samples are drawn from $\mathcal{B}_{\text{demo+interv}}$, the buffer holding your seed demonstrations and every human correction, and half from $\mathcal{B}_{\text{online}}$, the replay buffer of transitions the policy has generated recently. The proportion is fixed at one half regardless of how large either buffer has grown — that is the point, since online data will eventually outnumber human data thousands to one and would otherwise drown it out. The second equation is the ordinary temporal-difference target for the critic: y is the number $Q(s_t, a_t)$ is regressed toward, r_t is the reward from the classifier at this step, γ is the discount, s_{t+1} is the next state, and a' is an action *resampled* from the current policy π_θ at that next state. $\min_i Q_i$ takes the smallest estimate across the ensemble of critics, a deliberate pessimism that keeps the bootstrap from inflating itself.

The actor/learner split you run in project hil-serl is the structural point, and Chapter 14 scales this same asynchronous pattern up. The actor executes on the arm and streams transitions; the learner consumes them, takes many gradient steps, and ships weights back; the human sits beside the robot with a gamepad. Interventions do double duty here in a way they do not in pure DAgger: they are imitation targets for the actor *and* on-policy evidence about failure states for the critic. Figure 13.2 draws the cycle.

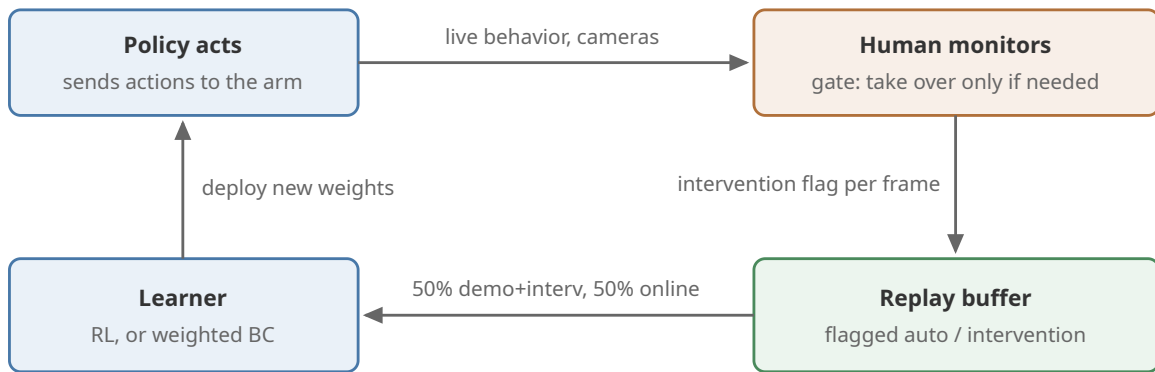


Figure 13.2 — The human-in-the-loop cycle shared by HG-DAgger and HIL-SERL. Follow the flag: every frame enters the buffer tagged with who was driving, and that single bit is what lets the learner sample asymmetrically later. The only box that changes between the two methods is the learner — off-policy RL with a critic in HIL-SERL, weighted behavior cloning in DAgger-on-a-VLA — which is why the same rig and the same gamepad serve both.

The reported results are why this pipeline is worth the trouble. Across real tasks spanning dynamic manipulation, precision assembly (RAM and SSD insertion, timing-belt assembly, IC-chip insertion) and dual-arm coordination, HIL-SERL reaches near-100% success on nearly all of them within **1 to 2.5 hours** of real-world training per task, averaging roughly **2×** the success rate and **1.8×** the execution speed of imitation-learning baselines. Single-digit hours on a physical robot, not days. Note the qualifier: *nearly all*. The hardest tasks in the paper are the ones that do not get there, and if one of your own tasks plateaus at 80% that is a normal outcome rather than evidence you have misconfigured something.

Look again at that speed number, because it is the ceiling argument made numerical. The trained policy runs the task about 1.8 times faster than the demonstrations it was seeded from. No amount of weighted cloning produces that, at any weight, with any sampler, because the demonstrations are the target and the demonstrations are slow. A policy paid for outcomes rather than for imitation is free to find a faster route to the same outcome. That is what "exceeding the demonstrator" looks like with a figure on it, and it is why HIL-SERL is not just a more complicated DAgger.

13.5.3 The intervention rate is the metric

Early in training the human intervenes constantly; late in training, almost never. That trajectory is the primary diagnostic, and it is more informative than the reward curve, which on a sparse-reward real task is mostly zeros for a while. The human is a hard-negative miner that smoothly retires. Figure 13.3 shows what the curve should look like, and what it means when it does not.

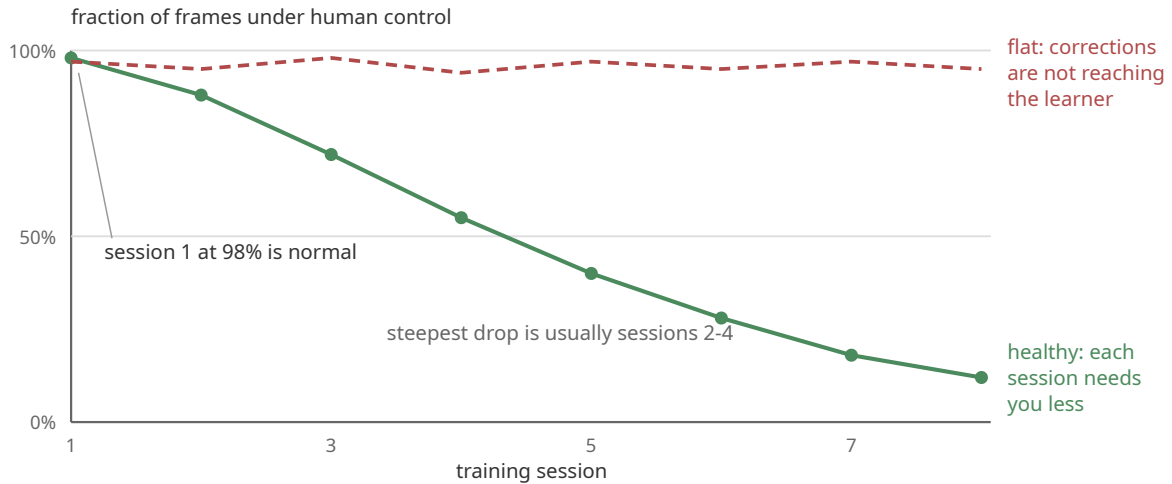


Figure 13.3 — The intervention-rate curve, the single most useful plot in a human-in-the-loop run. The green curve is what success looks like: a very high starting rate, a steep drop once the first few corrections land, then a long tail. The red dashed curve is the alarm — if you are still driving 95% of the time after several sessions, suspect the data path (are intervention frames actually flagged and reaching the buffer?) before you suspect the algorithm.

So your 98% intervention rate on an early run is the expected starting point. On session one the policy has seen a handful of demonstrations and nothing else; of course you are driving. What matters is whether that number falls between sessions. If it does, the loop is working and your job is to keep sitting there. If it is dead flat across four or five sessions, check the plumbing before touching a hyperparameter.

ON YOUR RIG

Your gate is RB on the Xbox pad; Y and B label success and failure for the reward classifier. You have already met the silent failure this design invites: release RB a moment early and the pipeline logs neutral (zero) actions as though a human deliberately chose them, poisoning both buffers with "do nothing" at exactly the states that matter most. The general lesson is bigger than the bug — instrument the data path. Log the intervention flag on every frame, histogram actions by source, and alert when the neutral bin spikes. Every human-in-the-loop pipeline has a bug like this; the good ones make it visible. And do not read a high early intervention rate as failure: 98% on session one is normal.

13.6 Choosing between HIL-RL and DAgger on a VLA

You now have two loops that share a rig, a gamepad and a diagram, and differ only in the learner. Which do you run? The answer depends on three things, none of which is "which is more advanced".

Choose HIL-RL (HIL-SERL) when you are training a small, task-specific policy from scratch; when your policy class has tractable log-probabilities, so a SAC-style actor can be updated and a critic bootstrapped; when you have a rig you can occupy for hours at a stretch and resets cheap enough for hundreds of episodes; and when success is visually obvious enough for a reward classifier to learn from a few hundred labeled frames. Your SO-101 on a fixed task satisfies all of these, which is why it is the right choice for project hil-serl.

Choose Dagger on a pretrained VLA when you start from a strong generalist prior that already almost does the task; when robot time is measured in days, so every human minute must refine the policy rather than train a critic from scratch; and — decisively — when the policy is a flow-matching model whose log-probability you cannot evaluate (Chapter 7). That last one rules out the SAC actor update HIL-SERL needs: that update requires evaluating $\log \pi_{\theta}(\mathbf{a} \mid \mathbf{s})$ for the entropy term and differentiating through a reparameterized sample, and a flow-matching policy gives you neither. It rules out PPO's importance ratio ρ_t too, for the same underlying reason — see Chapter 9 — but that is a separate algorithm, and the two are worth keeping apart in your head: there is no importance ratio anywhere in SAC. The update here is plain weighted behavior cloning with the weights of Section 13.4, with no critic on-robot at all.

The LeHome real-world final is the textbook case for the second: one week of robot time, a flow-matching VLA, and no tractable density to build a policy gradient from, so Larchenko (2026) made human-gated Dagger the main real-world tactic. Chapter 15's capstone will have you make this same choice deliberately, rather than inheriting it.

IN THE LEHOME SOLUTION

Dagger was the backbone of the real-round week: **471 Dagger episodes against 321 plain teleop episodes**, on a bimanual rig where the leader arms continuously track the followers so a foot-pedal takeover is jump-free — no lurch when control changes hands. Per-frame quasi-advantage weights followed Section 13.4: human-correction frames highest, distant autonomous frames low, and the five seconds before each takeover ramped to zero. In simulation, by contrast, Dagger underperformed — the policy had outgrown the operator, and teleoperating through a simulator is genuinely hard — so the sim flywheel instead minimized human idle time with parallel simulators and queued hard states.

CHECKPOINT

- Explain covariate shift as a feedback loop, not merely as "the test set differs from the training set", say where the T^2 in the behavior-cloning regret bound comes from, and say why a bound of 450 on a 300-step episode has gone vacuous rather than tight.
- State the four steps of Dagger, say why aggregating all previous rounds matters, and explain in one sentence why vanilla Dagger is impractical on a 12-DOF arm.
- Write down the HG-Dagger gate equation, say which frames enter the training set, and answer the multimodality objection: corrections are state-conditioned, and genuine multimodality means conflicting actions at the *same* observation. Say also what HG-Dagger gives up relative to vanilla Dagger's guarantee, and what survives the trade.
- Assign per-frame sampling weights to a recorded session — corrections up, clean autonomous frames at baseline, the pre-takeover window ramped to zero — and compute what fraction of a batch each bucket will claim.
- Explain why any loop whose training targets are human actions caps the policy at the demonstrator's skill, and why reward is what lifts the cap — with HIL-SERL's 1.8× execution speed as the concrete evidence.
- Name HIL-SERL's four ingredients, explain symmetric sampling and why the 50/50 split is fixed rather than proportional, read an intervention-rate curve well enough to distinguish "learning" from "the corrections are not reaching the learner", and decide for a given policy and rig whether HIL-RL or Dagger-on-a-VLA is the right loop — justified by log-probability tractability, robot time, and whether a critic can be trained on-robot.

Where to go deeper. Read Ross et al. (2011) for the Dagger analysis and Kelly et al. (2019) for HG-Dagger; between them they contain everything in Sections 13.2 and 13.3. Luo et al. (2024) documents the exact HIL-SERL pipeline you are running, and is worth reading with your own config file open beside it. Larchenko (2026), Section 9, details the bimanual Dagger rig and its intervention weighting in the real round. Appendix A sequences those four; the quadratic-regret setup (Ross and Bagnell, 2010) and RLPD (Ball et al., 2023) are optional background beyond the guide.

Chapter 14 — Simulation, sim-to-real, and the asynchronous RL loop

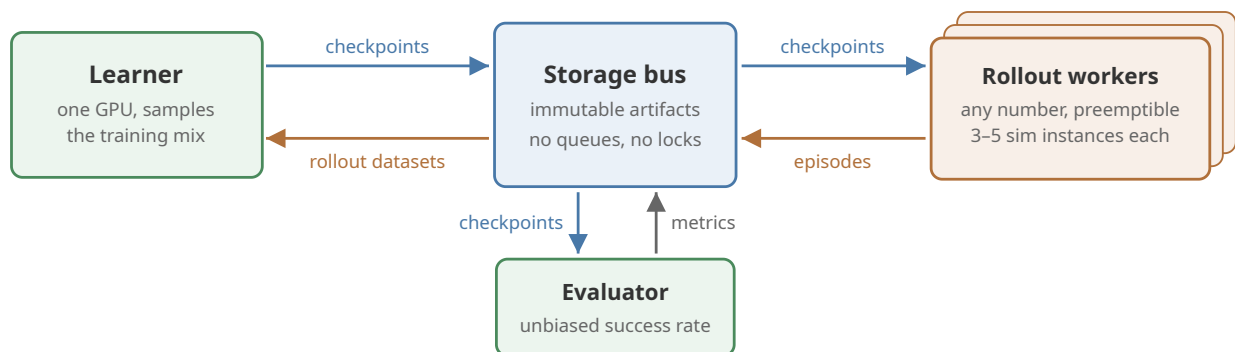
Chapters 11 and 13 both assumed the same thing without saying it out loud: that transitions arrive. Advantages get computed on them, weights get attached, the policy improves. Chapter 12 made a quieter version of the same assumption — that a trained policy already exists to run at inference time. Nobody asked where the transitions come from, how fast, or what happens when the machine producing them dies at three in the morning.

This chapter is about the factory that produces them. It has three rooms: the simulator, where most robot RL data is actually made; the bridge between simulation and reality, which decides whether any of that data was worth making; and the asynchronous training loop that keeps the whole thing running without a human standing over it.

You have built a small version of all three — the rover's MuJoCo port with domain randomization, and hil-serl's actor and learner processes. What this chapter adds is scale, the failure modes that only appear at scale, and one architectural idea, the storage bus, that turns two processes into any number of machines. Figure 14.1 is what we are building toward; we will earn each box.

WORDS YOU WILL HEAR

Domain gap, also called the **reality gap** — the mismatch between the observation and dynamics distribution a policy trained on and the one it is deployed into; papers use the two names interchangeably. **System identification** (sysID) — measuring a real system's physical parameters (masses, friction, motor gains, latency) so a model can match them. **Domain randomization** (DR) — sampling those parameters and visual properties from wide distributions in training so the policy cannot depend on any one; **ADR** (automatic domain randomization) is the variant that widens each range on its own whenever the policy keeps succeeding inside it (OpenAI et al., 2019). **Privileged information** and **teacher–student distillation** — the main alternative to randomization: train a teacher that reads simulator-only state (exact poses, contact forces), then distill it into a student that reads only the cameras the real robot has. **Curriculum** — a sampling rule that biases collection toward task instances of a chosen difficulty instead of sampling them uniformly. **Staleness** — how many gradient steps old the policy that generated a piece of data is.



no shared clock: each component pulls the newest artifact whenever it happens to be ready

Figure 14.1 — The asynchronous RL loop. Notice what is *not* in this picture: there is no arrow between the learner and the workers. Every component talks only to the storage bus, so any of them can be restarted, duplicated, or killed without the others noticing anything worse than a pause in the flow of new artifacts.

14.1 Why simulate at all

14.1.1 What we want, and the obvious thing to try

What we want is millions of transitions covering the states the policy will actually visit, labeled with enough reward signal to tell good behavior from bad. The obvious way to get them is to collect them on the robot. You have a robot. Point it at the task and let it run.

Here is the tally, and you know most of it from *hil-serl*. Every episode ends with a human resetting the scene — on your rig, you. Wall-clock speed is fixed at 1×; physics has no fast-forward button. One robot means one data stream. The reward signal needs a learned success classifier or a human finger on a button (Chapter 13). And an exploring policy — one deliberately trying actions it is unsure about — can strip a gear, knock a mug off the table, or close a gripper on a finger.

14.1.2 Exactly how badly that fails, with numbers

The arithmetic is more brutal than the intuition. Take LeHome's garment folding: a 600-step control horizon at 30 Hz, which is twenty seconds of robot motion, plus the human work between episodes. That work is two jobs, not one. Twenty seconds to flatten the garment back out across the table, and about ten more to reseat it in the same starting position and press start. So thirty seconds of reset against twenty seconds of episode — fifty seconds per attempt — and assume a person can keep that up for eight solid hours, which they cannot.

```
# Real-robot collection, one rig, one human
episode (600 steps at 30 Hz)           = 20 s
reset (flatten 20 s + reseat and start 10 s) = 30 s
episode + reset                        = 20 + 30 = 50 s
episodes per hour                     = 3600 / 50 = 72
episodes per 8-hour day               = 72 * 8 = 576
LeHome's final data pool = 12,500 kept episodes
days of nonstop human work = 12500 / 576 = 21.7 days

# Simulated collection, one rollout machine, 4 Isaac Sim instances
episode (incl. reset) = ~30 s
episodes per instance-hour = 3600 / 30 = 120
episodes per machine-hour = 120 * 4 = 480
three machines = 1440 episodes / hour
12,500 episodes = 12500 / 1440 = 8.7 hours of fleet time
```

Twenty-two person-days against nine hours of unattended machine time, for one data pool of one run. Two honest caveats. Nine hours is throughput, not calendar time: the actual LeHome campaign ran roughly three months across ~140 sessions, because most of a project is debugging, not streaming. And Isaac Sim takes about thirty seconds to produce a twenty-second episode, so it runs *slower* than real time. Simulation does not win by being fast. It wins by being parallel, and by not needing you.

14.1.3 The fix, item by item

Resets become a function call. More than that: you can save a full physics snapshot — every position, velocity, and contact state — at any timestep and restore it later. Section 14.5 turns that one capability into an entire curriculum toolkit. There is no real-world equivalent; you cannot rewind a garment.

Speed and parallelism. One instance may be slower than real time, but M instances give M independent streams, and M is limited by your GPU budget rather than by how many robots you own. (M here, not K : this book keeps K for the sampler's integration steps.)

Ground-truth state. The simulator knows the true state $\mathbf{s}$, not merely the observation $\mathbf{o}$. Chapter 11 built dense rewards out of keypoint distances — how far the left sleeve's tip is from the right shoulder. On hardware those must

be estimated by a perception model that is itself sometimes wrong, or abandoned for a binary classifier; in simulation they are a scene-graph query, exact on every frame, free. The LeHome success checker is literally a conjunction of pairwise keypoint-distance thresholds, five conditions for tops and four for pants (the official checker; Larchenko expanded long pants to seven for his own reward — Chapter 8). So "the simulator gives you keypoints" means Chapter 11's whole reward toolkit — checkpoints, potential shaping on completion, the tightness bonus — becomes cheap, because every input it needs is exact.

Safety. The policy can flail through millions of catastrophic actions and the only cost is electricity.

An analogy from your day job: the simulator is your test suite. Resets are `tearDown`, physics snapshots are fixtures restored before each case, M instances are M parallel test runners, and ground-truth keypoints are the internal assertions you can only write because you hold the object under test rather than its HTTP response. Where the analogy breaks: a unit test is written against a specification *you control*, so passing means correct. A simulator approximates physics nobody controls, so passing in sim means only "passing in sim."

14.1.4 The stack landscape, and a rule for choosing

MuJoCo (Todorov et al., 2012) is the fast, CPU-friendly rigid-body workhorse — what you ported the rover to, still the default for contact-rich rigid manipulation and for anything that must run on a laptop. **NVIDIA Isaac Sim**, with the **Isaac Lab** RL layer on top, trades setup simplicity for GPU-parallel physics, photorealistic rendering, and deformable bodies — cloth in particular, which is why LeHome's garments live there. **Genesis** is a newer GPU-native simulator aiming at speed across several solvers. **SAPIEN**, with **ManiSkill** as its task suite, targets articulated-object manipulation. **PyBullet** is the old, dependency-light option, still fine for prototyping.

The rule for choosing has two clauses. Pick by the physics your task cannot fake: cloth, rope, fluid, or granular material eliminates most of the list immediately. Then pick by the rendering fidelity your observations require: a policy reading joint angles does not care what the render looks like; a policy reading three RGB cameras cares enormously. Everything else is a tiebreaker.

14.2 The two levers for any domain gap

14.2.1 The obvious plan, and why it cannot work

What we want is a policy trained in simulation that works in reality. The obvious plan is to build a simulator that matches reality, so that "works in sim" and "works in reality" become the same statement.

This fails for a reason worth stating precisely. Every sim-to-real problem is a **domain gap**: the joint distribution over observations and dynamics seen in training is not the one seen at deployment. Some of that mismatch you can measure and fix. Much of it you cannot even enumerate — nobody knows the bending stiffness of a particular worn t-shirt, the spectral response of a \$30 webcam under fluorescent light, or the true latency from photon to torque on your USB chain. You cannot fix what you cannot measure.

Worse, the mismatches you would never think to look for are the ones the network finds. Larchenko's auxiliary heads could tell apart two image-resize pipelines — 640 pixels straight down to 224, versus 640 to 320 to 224 — and success dropped when the wrong one was used (Larchenko, 2026). If a resampling kernel is a detectable domain difference, "match reality exactly" is not a plan.

14.2.2 The fix: two levers, and only two

Everything you can do about a domain gap is one of two levers, drawn in Figure 14.2.

Lever one: make the domains more similar. This is **system identification** — measuring real masses, friction coefficients, motor response curves, and control latency, then setting the simulator's parameters to match — plus camera alignment (pose, intrinsics, exposure) and asset matching (scan or rebuild the actual objects you will manipulate).

Lever two: make the training data more diverse. This is **domain randomization** (Tobin et al., 2017; Peng et al., 2018): randomize colors, patterns, textures, lighting, camera pose and intrinsics, and physics parameters, then stack heavy image augmentation on top of the rendered result.

That list is a category list, and category lists are easy to nod along to without knowing what you would actually write. So here is the real randomizer from the LeHome simulation, item by item. Each garment's mesh gets its texture swapped for another drawn from a library, so the same shirt appears in dozens of fabrics. The swapped texture is then remapped in LAB color space, which moves lightness and hue independently of the pattern — a plaid stays plaid but arrives in a color no photograph in the library contained. Every camera's pose is jittered around its nominal mount, and its focal length with it, so no fixed pixel ever means a fixed place on the table. The dome light is redrawn between episodes: direction, color temperature, intensity. And a small color tint is applied *per step*, so the white balance drifts even within a single episode and the policy cannot latch onto one.

Notice the two timescales, because you will have to choose them yourself. Texture, camera pose, and lighting are drawn once per episode and then held, which is what makes each episode look like a different room. The tint is redrawn every step, which is what makes the room flicker. The two teach different invariances, and you want both.

Formally, randomization replaces one MDP with a distribution over MDPs indexed by randomization parameters ξ :

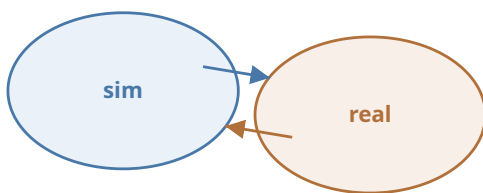
$$J(\theta) = \mathbb{E}_{\xi \sim p(\xi)} \mathbb{E}_{\tau \sim p_{\xi}(\tau | \pi_{\theta})} [G_0], \quad G_0 = \sum_t \gamma^t r_t.$$

IN PLAIN WORDS

The left side, $J(\theta)$, is what we maximize: how good the policy with parameters θ is. Normally that is one expectation over trajectories; here there are two, nested. The outer one, $\mathbb{E}_{\xi \sim p(\xi)}$, says: first roll dice to build a world. ξ is one draw of every randomized quantity at once — this texture, this lighting, this camera pose, this friction — and $p(\xi)$ is the distribution you chose to draw from. The inner expectation says: *inside that world*, run π_{θ} to get a trajectory τ , where $p_{\xi}(\tau | \pi_{\theta})$ is the trajectory distribution that world's physics produces. G_0 is that trajectory's return from step 0: rewards r_t each shrunk by γ^t , summed. So $J(\theta)$ is an *average across a population of worlds*, and one θ must serve all of them.

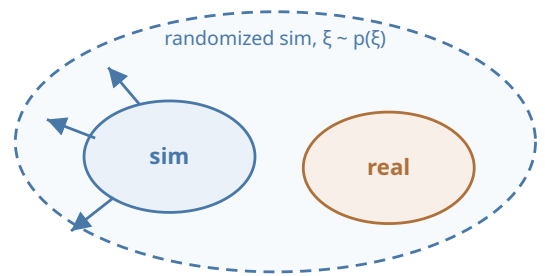
That is why the lever works. One set of weights must succeed for every ξ it will meet, so it cannot key on any feature that varies with ξ . If the shirt is sometimes red and sometimes plaid, "look for red" stops being viable and the network is pushed toward invariant structure — shape, edges, how the fabric deforms. Make the ranges wide enough and reality becomes just another sample of ξ , handled because all the others are.

Lever 1 — align: move the domains together



system ID · camera alignment · asset matching

Lever 2 — diversify: widen sim over real



randomization widens training until real falls inside it

Figure 14.2 — The two levers, and the fact that they solve the same problem in opposite directions. On the left the training distribution moves toward the deployment one and stays narrow; on the right it stays put and grows until the deployment distribution is inside it. The right panel costs capacity, because one policy must now solve every world in the dashed ellipse.

14.2.3 Which lever, when — and what diversification costs

Align when the mismatch is measurable and cheap to fix: a camera mounted three centimeters off, an action scale wrong by a constant factor, forty milliseconds of unmodeled latency. Fixing those removes the gap exactly and costs no policy capacity. Diversify when the mismatch is unmeasurable or high-dimensional: fabric dynamics, room lighting, texture, wear. The working rule is one sentence: **align what you can measure, randomize what you cannot.**

Diversification is not free, and this is the part people skip. The policy now solves a strictly harder problem: $J(\theta)$ is an average over worlds, and the θ maximizing an average is generally not the θ maximizing any single term. Illustratively, a policy trained under one fixed lighting setup might reach 85% there and 5% elsewhere, while the randomized version reaches 78% nearly everywhere. You trade peak performance in one domain for a floor across all of them — deliberately, rather than discovering it on a leaderboard.

ON YOUR RIG

You have pulled both levers already: the rover's MuJoCo port randomized dynamics before the printed chassis ran the policy. What is worth adding is the audit. Write an `audit_baseline.py` that samples 200 draws of ξ from your current ranges and in each runs a scripted expert (or replays a known-good trajectory), recording success. If the expert falls from 95% to 40%, your ranges are not teaching robustness, they are teaching that the world is noise. Run it whenever you widen a range: minutes spent, days saved.

PITFALLS

Randomizing past solvability. This is what the audit above catches. On the rover, overly wide dynamics ranges produced sampled worlds that were effectively undrivable — no policy could have succeeded, so every gradient from those episodes was noise. The symptom is deceptive: training loss looks fine, because the network dutifully predicts the mean of an unpredictable target, and success rate simply stops improving. Widen ranges only as fast as *some* policy can still win in them, and one factor at a time so you know which one broke it.

14.3 Real-to-real: the gap everyone forgets

Sim-to-real is only half the story, because "real" is not one domain either. Two SO-101 arms off the same production run differ in servo zero offsets, gear backlash, and cable tension; two cameras of the same model differ in color response; two 3D-printed mounts differ by a millimeter, which at the end of a 30 cm arm is several millimeters of tool position. In any competition or deployment, **the evaluation rig is not your rig.**

Larchenko faced the whole chain at once: policies trained in Isaac Sim, fine-tuned on his own bimanual rig, evaluated on the organizers' rig, which he never touched. He also discovered, only on physically assembling his setup, that the garments were kids'-sized (Larchenko, 2026). Zero-shot transfer failed outright. The real problem was not sim-to-real; it was `sim` → `his robot` → `their robot`, with the last hop blind.

This is "works on my machine," with physics — and the analogy has a hard limit. In software you end that argument by shipping a container image that pins everything; there is no container for a servo's backlash, a room's lighting, or a garment's age. The only substitute is Section 14.2's two levers applied a second time: align what you can reach, randomize what you cannot.

14.3.1 Three tools that close the real-to-real gap

Tool one: alignment overlay tooling. Build a utility that drives the robot to the joint state recorded in a dataset frame, then draws the live camera image on top of that stored frame. Now you nudge a camera mount until the two images coincide instead of guessing, and the same trick overlays a simulator render on a real view. This is the align

lever applied real-to-real, and it worked well enough that the LeHome organizers adopted the tool to calibrate the official evaluation arenas — evidence that per-rig misalignment was widespread, not personal.

Tool two: per-source speed factors. Datasets recorded on different rigs by different operators move at different tempos. Organizer demonstrations are brisk; your own teleoperation is slower because you are being careful; DAgger corrections — the human-takeover segments of Chapter 13, where an operator grabs the controls mid-episode and drives the robot out of trouble — are slower still, because a human is reacting to a failure already in progress. Train on the raw mixture and the policy inherits an incoherent sense of pace, with nothing in the observation to say which applies. The fix is to resample each source's time axis to a common motion intensity: in the LeHome real mix, organizer demonstrations at $1.0\times$ (the reference), home teleoperation played back $1.5\times$ faster, DAgger corrections $2.0\times$ faster, and simulator data *slowed* to $0.65\times$ because sim motion was the fastest of the four. Above one means too slow relative to the reference; below one, too fast.

Tool three: sim-to-real format conversion. Simulator success replays were rendered into the real data format — same camera layout, resolution, and normalization — so sim experience could sit inside the same training mix as real episodes, sampled by the same code path. The mix used three buckets with fixed batch shares: organizer demonstrations at 60% (500 episodes, $\sim 187\text{k}$ frames), the author's own teleoperation plus DAgger at 30% (321 plus 471 episodes), and converted sim replays at 10% (1,723 episodes, $\sim 625\text{k}$ frames).

On top of all three buckets sits the augmentation stack, applied independently per camera so the three views never agree about what was distorted. It is worth naming in full, since "heavy augmentation" is another phrase that is easy to agree with and hard to implement: color jitter with separate gain and gamma shifts, blur, simulated sensor noise, random crop, rotation and zoom, cutout patches that blank part of the frame, and whole-camera dropout that blanks one view entirely for a stretch of steps. Proprioceptive state gets noise too. Camera dropout is the one people leave out and the one that saves the run: on the evaluation rig a USB camera will eventually stall for half a second, and a policy that has never seen a dead view will do something interesting with it.

The lesson is in the mismatch. The sim bucket holds more than three times the frames of the organizer bucket and receives one sixth of the batch share — per frame, organizer data was sampled at roughly 1.00, home data at 0.21, sim data at 0.05. Batch share states how much you trust a source; it has almost nothing to do with how much of it you have.

14.3.2 The strip-the-model recipe

Fine-tuning for the real world is not just a data change; it is a model change. Four moves, each with a reason.

Start from a **late-but-not-latest checkpoint**. The latest is by construction the most sim-overfit — it has had the most gradient steps to exploit simulator-specific cues. Back up a little and you keep most of the skill with less of the overfitting.

Keep the action objective (Chapter 7's flow-matching loss, which is the actual skill), plus the garment-type and completion heads: their targets exist on real hardware, since a human can label garment type and completion is just elapsed fraction of a successful run. **Freeze** the success-style heads — success, checkpoint, time-to-completion. On hardware there is no per-frame ground-truth success signal, so training them means training on labels you do not have; freezing keeps their features intact without corrupting them.

Drop the keypoint head, both of the report's future-prediction queries, advantage conditioning, guidance, and best-of-N. The keypoint head was Chapter 8's cheap world-model stand-in, and its targets are sim-privileged: no keypoints on hardware, no head. The two future queries go for exactly the same reason — the training-only FAST-token query and the flow-matching Q -like query at the expert's tail (Chapter 8 lists both) are trained against simulator state the real robot has no way to observe. Advantage conditioning (Chapter 10) needs an advantage, and there are no advantages without rewards. Guidance and best-of-N (Chapter 12) depend on those heads being trustworthy, and you have just established that they are not.

Real-world inference is then three things. Chunked actions: predict $\mathbf{a}_{t:t+H}$, execute part of it, predict again. Soft inpainting between chunks — Chapter 12's trick for making a newly generated chunk agree with the actions already committed, by pinning the overlapping prefix during generation instead of letting the new chunk start wherever it likes. And the garment-type bootstrap: a warm-up inference at the start of the episode whose first few chunks each vote on which of the four garment types is on the table, with the majority answer latched and fed back as a conditioning input for the rest of the episode, so the policy cannot change its mind about what it is folding halfway through.

PITFALLS

Sim-format visual leakage. Converting sim data into the real format leaks sim-only cues — a render artifact, a noise floor, a resampling signature. The policy learns "this looks like sim, act like the sim policy," and splits into two modes, one of which never runs at evaluation. Not hypothetical: the auxiliary heads could distinguish two image-resize pipelines. Three defenses. Keep the sim-format share small (10% in LeHome). Augment that bucket hardest — the whole stack listed above, camera dropout and cutout included. And test that behavior does not change when you restyle an input the task does not depend on.

14.4 The asynchronous loop

14.4.1 Why the PPO shape cannot do this

Chapter 3's PPO is a lockstep algorithm, and it is worth being precise about why. Its objective is built on the importance ratio

$$\rho_t(\theta) = \frac{\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)}{\pi_{\theta_{old}}(\mathbf{a}_t \mid \mathbf{s}_t)}.$$

Name the parts, because the whole argument lives in them. The numerator is how likely the *current* policy, the one with parameters θ you are updating right now, is to have taken the action $\mathbf{a}_t$ that was actually taken in state $\mathbf{s}_t$. The denominator is how likely $\pi_{\theta_{old}}$, the policy that was running when that action got collected, was to take it. So ρ_t is exactly 1 when nothing has changed since collection, larger than 1 for actions the new policy now favors more, smaller for ones it favors less, and it drifts further from 1 the further θ moves from θ_{old} .

That ratio is what reweights data collected under $\pi_{\theta_{old}}$ so it can estimate a gradient for π_θ . The correction is only trustworthy while θ stays near θ_{old} ; push far and the ratios become enormous or tiny, the variance explodes, and the clip meant as a safety rail becomes the entire signal. So PPO must alternate: freeze θ_{old} , collect a batch with it, take a few clipped epochs, discard the batch, repeat.

Collection and learning are welded together. Suppose collecting one iteration's data takes eight minutes across your simulator fleet and the update takes two minutes on the GPU. The GPU is idle 80% of the time, the fleet 20%, and every worker also idles at the end of collection waiting for the slowest one — easily a two-to-one spread in episode length once success terminates episodes early. Add a preempted spot instance — a cheap cloud machine the provider may reclaim at any moment, which is why you rented it cheaply — and the whole batch is compromised, because the surviving data is a biased subset of what you meant to collect.

14.4.2 Why weighted behavior cloning can

Chapter 4's AWR family removes the weld. Its update is weighted maximum likelihood on whatever data exists, with weights derived from advantages you can recompute at any time from stored quantities. There is no ratio to invalidate, because there is no ratio. Data from a checkpoint three hours old is still usable — mildly off-policy, and that mildness is priced in through the advantage weights and Chapter 11's staleness blend. Chapter 9 drew the comparison in full; this is its most consequential operational line. **Off-policy tolerance is what makes asynchrony possible.**

14.4.3 The three roles and the storage bus

The **learner** (one GPU) cycles: download new rollouts, recompute advantages over all datasets, train on the sampled mix, upload a checkpoint. The **rollout workers** pull the latest checkpoint — whatever it is, they never check its age — run several simulator instances behind one shared stateless policy server, and upload finished episodes as datasets. The **evaluator** runs unbiased episodes under fixed conditions and publishes success rates: the only numbers anyone is permitted to believe.

All three communicate exclusively through the storage bus — in Larchenko's system the bus is literally the Hugging Face Hub, with background sync daemons and no barrier anywhere. The evaluator was not even a separate process: the unbiased rollout strategies (random, full, and curriculum, all three defined in Section 14.5) doubled as the evaluation stream, and a manual DAGger station (Chapter 13) formed a third, human-paced worker.

You have shipped this architecture before: a build server pushing versioned artifacts to object storage while deploy agents poll for the newest tag — no RPC, no queue with acknowledgements, just immutable named files. Nothing blocks, nothing locks, and a worker offline for an hour catches up by reading. Where the analogy breaks: in CI the artifact is a passive dependency, so a stale one is merely old. Here the checkpoint a worker pulls determines what data it produces, which determines the next checkpoint. That feedback loop is why staleness carries a statistical cost, not just a latency cost — what Chapter 11's blend exists to price.

```
# --- learner: one GPU, runs forever ---
while True:
    hub.download_new_rollout_datasets()      # whatever showed up; may be nothing
    recompute_advantages(all_datasets)      # Ch.11 blend, over ALL data, not just new
    for _ in range(1000):                  # one "iteration" = ~1000 steps
        train_step(sample_mix())           # per-source shares, Section 14.5
    hub.upload_checkpoint()                 # real system: every ~500 steps, background sync

# --- rollout worker: any number of these, all preemptible ---
while True:
    ckpt = hub.latest_checkpoint()          # may be minutes old; we do not care
    eps = run_episodes(ckpt, strategy=pick()) # random / curriculum / replay / hard-mine:
    Section 14.5
    attach_values(eps)                     # value predictions FROZEN at collection time
    hub.upload_dataset(eps)                 # immutable; never merged into anything
```

Two lines deserve a second look. `recompute_advantages(all_datasets)` runs over everything, not only new arrivals, because advantages depend on group statistics that shift as the policy improves. `attach_values(eps)` stores the value predictions made *at collection time* and never re-predicts them — those frozen numbers are exactly the stale-value anchor Chapter 11's blend consumes, and re-predicting them with a newer network would silently destroy the property that makes the blend correct.

14.4.4 Why the operational payoff is the real payoff

Every component **scales independently**: more throughput means starting another rollout machine, with no coordination step and no config to update — it pulls a checkpoint and starts uploading. Every component also **dies independently**. Workers can run on spot instances; a preemption costs throughput and never correctness, because a half-finished episode is simply never uploaded, and the learner restarts from its last checkpoint having lost at most one iteration.

One knob keeps the whole thing from drifting into the past: each rollout dataset's sampling rate decays every iteration. Rate, not fraction — the same kind of number as the per-frame 1.00, 0.21, and 0.05 of Section 14.3, where organizer data was drawn at full rate and sim data at a twentieth of it.

$$\text{share}_k = \max(\text{floor}, \text{share}_0 \cdot 0.98^k), \quad w = \min(\text{share}_k, 1).$$

IN PLAIN WORDS

share_k is this dataset's sampling *rate* **k** learner iterations after collection — not a slice of the batch, but a rate relative to a reference source, like Section 14.3's 1.00 for organizer data and 0.05 for sim. **share₀** is the rate on arrival. Each iteration multiplies it by **0.98**, so it fades geometrically instead of being deleted at a cutoff; **floor**, which is **0.1** in the LeHome system, stops the fade short of zero. A rate above 1 is legal for a source you deliberately oversample, which is why **w** clamps it: sampled more often than the reference, yes; trusted more than fully by Chapter 11's blend, never. One number, two jobs — how often old data is seen, and how far its frozen value predictions are believed. BC datasets are exempt; a demonstration does not age.

Work the decay out, because a decay constant means nothing until it is in hours. Solving $0.98^k = 0.5$ gives $k = \ln(0.5)/\ln(0.98) = -0.693/-0.0202 \approx 34$ iterations. Now convert iterations to wall-clock in two multiplications. The learner uploads a checkpoint roughly every 40 minutes and about two checkpoints go by per iteration, so one iteration is $2 \times 40 = 80$ minutes. Then $34 \times 80 = 2,720$ minutes, and $2,720/60 = 45$ hours — a little under two days.

Do the same for the floor, since that tells you when the fading stops rather than when it is half done. Solving $0.98^k = 0.1$ gives $k = \ln(0.1)/\ln(0.98) = -2.303/-0.0202 \approx 114$ iterations, which at 80 minutes each is $114 \times 80 = 9,120$ minutes, or 152 hours, about six days. Put the two together and you have the life story of a rollout dataset: it halves in influence in two days, keeps sliding for another four, and then sits at a tenth of its arrival rate indefinitely, a permanent thin trickle of old experience. That is the half-life of experience in this system, and it is a number you should be able to state for your own runs.

ON YOUR RIG

You already run this pattern at $K = 1$. HIL-SERL's actor and learner are two processes: transitions stream one way, parameters the other (Luo et al., 2024). The async loop swaps that live socket for a bus of immutable artifacts, so components need not be online at the same time — and the price of that decoupling is exactly the staleness you now know how to blend away. When Chapter 15 has you scale past one SO-101, you change the transport, not the architecture. On paper first: which of your config values would become per-dataset shares?

IN THE LEHOME SOLUTION

One H200 learner uploaded a checkpoint roughly every 40 minutes. Rollout machines each ran 3–5 Isaac Sim instances behind one shared stateless policy server, and a manual DAGger station acted as a third, human-paced worker. Everything moved through the Hugging Face Hub; datasets were never merged, only sampled, with rollout shares decaying $0.98\times$ per iteration and the BC share fixed. Across ~140 sessions this produced about 12,500 kept episodes and ~4.3M frames — averaging ~344 frames, about 11 seconds, per episode, because success terminates episodes early. Four checkpoint rollbacks were used deliberately to escape local optima (Larchenko, 2026).

14.5 Rollout collection as a curriculum

14.5.1 The obvious plan and its arithmetic problem

Once workers can restore snapshots and choose which task instance to attempt, collection becomes a decision problem: which episodes are worth the ~30 wall-clock seconds they cost? The obvious plan is uniform random sampling over all instances. It is unbiased, trivial to implement, and wastes most of your fleet.

Here is why, and it follows from Chapter 11's group-baselined advantages. With a binary outcome and an empirical per-task baseline, an episode's advantage takes one of exactly two values, and the signal a task offers scales with how far apart they are:

$$\hat{A}_{\text{ep}} \in \{1 - \hat{p}_{\text{succ}}, -\hat{p}_{\text{succ}}\}, \quad \text{std}(\hat{A}_{\text{ep}}) = \sqrt{\hat{p}_{\text{succ}}(1 - \hat{p}_{\text{succ}})}.$$

IN PLAIN WORDS

$\hat{p}_{\text{succ}}$ is the policy's measured success rate on this particular task instance — this garment, say — and it plays the role of the baseline. $\hat{A}_{\text{ep}}$ is the advantage the whole episode receives, and there are only two possibilities. If the episode succeeds, the outcome was 1 against an expectation of $\hat{p}_{\text{succ}}$, so the advantage is the positive number $1 - \hat{p}_{\text{succ}}$. If it fails, the outcome was 0 and the advantage is $-\hat{p}_{\text{succ}}$. The second expression is the standard deviation of that two-valued quantity: how much the advantage varies between episodes, which is the same thing as how much information a fresh rollout carries. $\sqrt{\hat{p}(1 - \hat{p})}$ is largest at $\hat{p}_{\text{succ}} = 0.5$ and zero at both ends.

That standard deviation is worth deriving rather than quoting, partly because it takes three lines and partly because you should not take a variance on trust. Write $\hat{p}$ for $\hat{p}_{\text{succ}}$ to keep the algebra short. Start with the mean: the episode succeeds with probability $\hat{p}$ and then contributes $1 - \hat{p}$, and it fails with probability $1 - \hat{p}$ and then contributes $-\hat{p}$.

$$\mathbb{E}[\hat{A}_{\text{ep}}] = \hat{p}(1 - \hat{p}) + (1 - \hat{p})(-\hat{p}) = 0.$$

The mean is exactly zero, which is the whole point of subtracting a baseline. Because the mean is zero, the variance is simply the mean of the square, with no correction term to carry:

$$\text{Var}(\hat{A}_{\text{ep}}) = \hat{p}(1 - \hat{p})^2 + (1 - \hat{p})\hat{p}^2.$$

Both terms carry a factor $\hat{p}(1 - \hat{p})$, so pull it out and watch what is left collapse:

$$\hat{p}(1 - \hat{p})[(1 - \hat{p}) + \hat{p}] = \hat{p}(1 - \hat{p}) \cdot 1 = \hat{p}(1 - \hat{p}).$$

Take the square root and you have $\text{std}(\hat{A}_{\text{ep}}) = \sqrt{\hat{p}(1 - \hat{p})}$, the expression printed above. It is the standard deviation of a single coin flip, which is what a binary-outcome episode is.

Put numbers on it. At $\hat{p}_{\text{succ}} = 0.9$ a success carries $+0.1$ and a failure -0.9 , with standard deviation $\sqrt{0.9 \times 0.1} = 0.30$. Nine rollouts in ten come back carrying exactly the same advantage, $+0.1$. It is worth being exact about why that is fatal rather than merely wasteful, because "low-variance advantage" sounds like a mild complaint and it is not.

Recall from Chapter 4 that the update is weighted maximum likelihood, with each sample's weight $w_i = \exp(\hat{A}_i/\beta)$ — the exponential of its advantage divided by the temperature β . Take $\beta = 0.5$. At $\hat{p}_{\text{succ}} = 0.9$ each of the nine successful episodes gets $w = \exp(0.1/0.5) = \exp(0.2) = 1.22$, and the single failure gets $w = \exp(-0.9/0.5) = \exp(-1.8) = 0.17$. So ninety percent of that garment's frames arrive at the optimizer carrying an identical weight. A weighted regression in which every weight is the same is an unweighted regression, and an unweighted regression onto samples the policy itself just produced is a gradient of approximately zero — you are teaching the network to do what it already does. Only the one failing episode in ten carries a weight that differs from its neighbors, and it is that contrast, not the absolute size of any weight, that moves θ anywhere.

Now run the same computation at $\hat{p}_{\text{succ}} = 0.5$. The advantages are ± 0.5 , the standard deviation is 0.50 , and the weights are $\exp(0.5/0.5) = 2.72$ for a success against $\exp(-0.5/0.5) = 0.37$ for a failure. The ratio between the two weights has not changed — with a binary outcome the gap between the two advantages is always exactly 1, so the ratio is always $\exp(1/\beta)$, here 7.4 . What changed is the composition of the batch. Half the frames now sit on each side of that gap instead of one frame in ten, so the contrast is present in half the gradient mass rather than a tenth of it. The same thirty seconds of simulator time buys $1.67\times$ the spread in advantage, and it buys a batch that actually contains a comparison.

At $\hat{p}_{\text{succ}} = 0.05$ you are back to a standard deviation of $\sqrt{0.05 \times 0.95} = 0.218$, paying for 19 failures per success, and the same collapse happens with the signs reversed. A garment the policy always folds teaches nothing; one it never folds teaches nothing. The frontier carries the gradient, as Figure 14.3 shows.

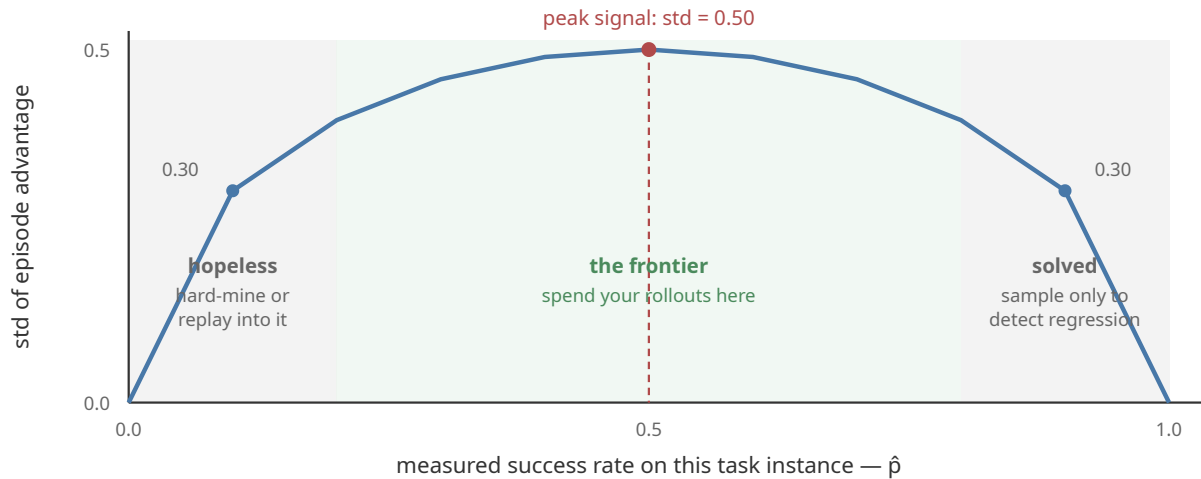


Figure 14.3 — How much one rollout teaches you, against how well the policy already does on that task. Notice the flat top, shaded green: anywhere between about 0.2 and 0.8 you get most of the available signal, so the curriculum target needs no precise tuning. Notice too that both tails reach exactly zero — which is why a fleet sampling uniformly over solved and hopeless tasks looks busy while learning almost nothing.

Sun et al. (2025) make this precise for GRPO-style RL in language models — targeting moderate-difficulty items and replaying recent rollouts cut fine-tuning time by 23–62% — and LeHome's **curriculum** strategy (a sampling rule biasing collection toward instances of a chosen difficulty) generalizes it to robotics.

14.5.2 The strategies, one at a time

Random samples instances uniformly: unbiased, and therefore your measurement instrument rather than your workhorse. **Full** sweeps every instance in order for even coverage. Both are honest and both are inefficient. **Curriculum** prefers instances whose running $\hat{p}_{succ}$ is nearest a target — the green band in Figure 14.3, and the direct implementation of the variance argument above.

Physics snapshots buy three sharper strategies with no real-world equivalent at all.

Success replay. Restore an early snapshot (LeHome used step 5) of an episode that succeeded, re-run from there under much heavier randomization, and keep only the replays that also succeed. This is a machine for manufacturing positive examples in states you know are winnable. Snapshots are saved with probability $1 - \hat{p}_{succ}$, so hard instances are deliberately overrepresented in the bank.

Semi-success replay. Take an episode that nearly worked, replay its recorded actions open-loop to reach the near-success state, then hand control to the live policy — endgame practice at the last few seconds of a task the policy rarely reaches on its own.

Hard mining. Watch the smoothed success-head prediction during a rollout — Chapter 8's auxiliary value head moonlighting as a difficulty detector. When it drops sharply from a running maximum (LeHome: more than 0.12 below a running max above 0.25), snapshot that state. You have caught a failure precursor: the moment the policy's own estimate of its prospects fell off a cliff. Restore those states later for targeted practice.

14.5.3 Two hygiene rules

First, **only unbiased strategies may update success statistics**. Random, full, and curriculum rollouts count; replayed and hard-mined episodes do not. A success replay starts from a state selected *because* it led to a success, so its outcomes are wildly optimistic; a hard-mined episode starts from a state selected because it looked bad. Let either into $\hat{p}_{succ}$ and you are steering the curriculum with a broken instrument. Keep two counters, and never let them touch.

Curriculum sitting on the permitted side of that line looks like a contradiction, since curriculum is defined by preferring some instances over others. The distinction is worth holding onto, because it is what makes the rule usable rather than arbitrary. Curriculum biases *which garment gets attempted*. Once the garment is chosen, the episode starts from an ordinary initial state, drawn the same way it would have been under random sampling. So every trial of that garment is still a fair trial of it, and that garment's own $\hat{p}_{succ}$ is estimated from fair trials. Success replay and hard mining choose the starting *state* on the basis of how some earlier episode turned out, which conditions the estimate on the outcome it is supposed to be measuring.

One consequence follows immediately and is easy to trip over. The fleet-wide average success rate *is* biased under curriculum, and biased downward, because the fleet is deliberately spending most of its time on the instances it finds hardest. That number is not wrong so much as meaningless: it measures your sampling policy as much as your robot policy. This is why the evaluator reports per-instance success rates rather than one headline figure, and why a rising per-garment curve alongside a flat fleet average is a normal, healthy picture rather than a contradiction to debug.

Second, **the buffers must stay fresh**. That is what Section 14.4's 0.98 decay enforces: as a dataset ages its collection-time value predictions drift further from the current policy, which is why Chapter 11 fades stale GAE advantages toward the outcome-only group anchor as the share falls.

14.5.4 Worked example: one training mix, and what happens when it drifts

Suppose the learner draws batches of 200 frames from four buckets.

Bucket	Share Frames		What it contributes
Success replays	40%	80	Dense positive supervision; keeps $\hat{p}_{succ}$ -style heads calibrated; heavy augmentation turns wins into robustness data
Semi-success replays	20%	40	Endgame practice at states the unaided policy rarely reaches
Fresh on-policy	25%	50	The only unbiased view of the current policy; feeds curriculum statistics and anchors the advantage blend
Dagger corrections	15%	30	Human labels on the failure boundary (Chapter 13); highest value per frame, scarcest supply

Now suppose success replays creep from 40% to 60% — not by decision, but because replays are cheap and the share was never audited. The other three buckets split the remaining 40% in their old proportions, so each is scaled by $40/60 = 0.667$. Here is every step.

Success replays	60%	-> $0.60 * 200$	= 120 frames
Semi-success replays	$20\% * 0.667 = 13.3\%$	-> $0.133 * 200$	= 26.7 frames
Fresh on-policy	$25\% * 0.667 = 16.7\%$	-> $0.167 * 200$	= 33.3 frames
Dagger corrections	$15\% * 0.667 = 10.0\%$	-> $0.100 * 200$	= 20.0 frames
		check: $120 + 26.7 + 33.3 + 20$	= 200
Unbiased frames per batch:	50.0	-> 33.3	(a 33% cut)
Per 1000-step iteration:	50,000	-> 33,300	unbiased frames

Stop before the next step, because there is a link in the chain that is easy to assume and worth making explicit. Batch shares say which frames the gradient sees. They do not, by themselves, say how many episodes the fleet collects. The two are connected because the workers are told to collect in roughly the proportion the learner intends to consume — collecting data that nobody samples is machine time thrown away, so the strategy mix tracks the training mix. When the replay share creeps up in the batch, the collection mix creeps up with it: the fleet spends more of its thirty-second slots restoring snapshots and fewer running fresh episodes from ordinary initial states. Fewer unbiased episodes come back per iteration, in about the same proportion.

That is the chain, stated in order: the batch share drifts, the collection mix follows it, and the statistic the curriculum steers by ends up estimated from fewer trials. It is also the place to cut if you do not want measurement quality riding on a training-mix knob — pin a fixed fraction of worker time to unbiased rollouts and let the batch shares drift where they like.

Now finish the computation, with the episode count derived rather than assumed. Section 14.4's LeHome average is about 344 frames per episode, so turn frames into episodes:

before: $50,000/344 = 145.3$ unbiased episodes per iteration,

after: $33,300/344 = 96.8$ unbiased episodes per iteration.

Those episodes are spread across the instances the curriculum tracks. Suppose there are about ten of them: four garment types, each with two or three distinct starting layouts that the curriculum scores separately. Divide:

$$145.3/10 = 14.5 \quad \text{versus} \quad 96.8/10 = 9.7$$

unbiased episodes per instance per iteration. A success rate estimated from n fair trials has standard error $\sqrt{\hat{p}(1-\hat{p})/n}$, which is worst at $\hat{p} = 0.5$, so use that:

$$\text{before: SE} = \sqrt{0.25/14.5} = \sqrt{0.017241} = 0.1313,$$

$$\text{after: SE} = \sqrt{0.25/9.7} = \sqrt{0.025773} = 0.1605,$$

$$0.1605 / 0.1313 = 1.22.$$

The noise on $\hat{p}_{succ}$ has risen by 22%, and nobody touched a setting called "measurement".

Read what that does, in order. The curriculum picks instances by comparing $\hat{p}_{succ}$ to a target, and a noisier $\hat{p}_{succ}$ means instances land in the wrong zone of Figure 14.3 more often — you start spending rollouts on garments you only believe are at the frontier. Meanwhile 60% of the gradient mass comes from states that a snapshot taken by an older checkpoint selected: the mix has quietly become behavior cloning of a past self. And the tell is nasty — training loss *falls*, because replay frames are repetitive, positive, and easy to fit, while the evaluator's success rate goes flat.

Batch shares are a knob you audit on a schedule, not a constant you set once. Put the realized per-bucket frame counts on your dashboard next to the loss, with unbiased episodes per garment per iteration beside them.

CHECKPOINT

- Run the real-versus-simulated tally with your own numbers — episode length, reset time, instances per machine — and say what a 10,000-episode pool would cost you in person-days.
- Explain what "the simulator gives you keypoints" means, and name three pieces of Chapter 11's reward machinery that become cheap because of it.
- Write down the randomized-MDP objective, name every symbol, and use it to explain why randomization lowers peak performance in any single domain — then describe the audit that catches ranges pushed past solvability.
- List the three real-to-real tools and the four moves of the strip-the-model recipe, giving the reason each head is kept, frozen, or dropped.
- Explain why PPO's importance ratio forbids an asynchronous loop and why weighted behavior cloning permits one, then sketch the learner and worker loops from memory, including where value predictions are frozen.
- Derive $\text{std}(\hat{A}_{ep}) = \sqrt{\hat{p}_{succ}(1-\hat{p}_{succ})}$, state which rollout strategies may update success statistics and why, and compute a batch's frame counts from its shares.

Where to go deeper. Sun et al. (2025) is the short difficulty-targeted selection paper behind Section 14.5. Larchenko (2026), sections 2–3 and 9, documents the Hub-based flywheel, the rollout strategies, and the one-week sim-to-real sprint in engineering detail — read section 9 with Section 14.3 open beside it. The canonical domain-randomization papers, Tobin et al. (2017) and Peng et al. (2018), are optional context; OpenAI et al. (2019) takes randomization to

its automated extreme under the name automatic domain randomization (ADR), which is the phrase to search for. Appendix A places Sun et al. and the LeHome report in reading order; the randomization papers are optional context outside the guide. Appendix B's ladder has you build a miniature Figure 14.1 on your own hardware.



PART V

Doing it

Turning the whole book into one staged project on hardware you own.

Chapter 15 — Capstone: replicating LeHome

You now hold every piece of the machine: policy gradients and advantage estimation (Chapters 2 and 3), why they do not transfer to flow-matching policies (Chapter 9), generative action models and the VLA that wraps one (Chapters 5 through 8), the two ways such a system consumes an advantage (Chapters 4 and 10), reward and advantage engineering (Chapter 11), inference-time optimization (Chapter 12), the human in the loop (Chapter 13), and the asynchronous simulation flywheel (Chapter 14).

This chapter contains almost no new theory. It contains a plan: how to assemble all of that into one project you can finish on hardware you own, ending in a portfolio artifact a hiring manager can verify. The project is to replicate Ilia Larchenko's prizewinning LeHome solution. Each stage has a goal, a deliverable, and a numeric gate. You advance on numbers, not on the feeling that things are going well.

15.1 What you are replicating, and how you will know

15.1.1 The challenge and the two rounds

The LeHome Challenge 2026, held at ICRA, asked teams to fold garments — long-sleeved tops, short-sleeved tops, long pants, shorts — on a bimanual SO-ARM101 rig: two six-degree-of-freedom arms, a twelve-dimensional joint action space, three RGB cameras. Success is binary and geometric. Each garment carries labeled keypoints, and an episode succeeds when a conjunction of pairwise keypoint-distance conditions holds: certain pairs must come within a threshold, certain others must stay apart. Five conditions for tops, four for pants (the official checker; Larchenko expanded long pants to seven for his own reward — Chapter 8). The garment category is not given at evaluation time; the policy must infer what it is looking at.

There were two rounds. The online round ran inside an NVIDIA Isaac Lab simulation released by the organizers, ranked by overall success rate with no partial credit; sixty-two teams submitted, and Larchenko won it at 79.63% overall, with per-type rates of 74.5, 70.0, 80.5, and 93.5 percent. The real-world final at ICRA in Vienna took the top eight teams under different rules — partial credit, jury-scored quality, a bonus on unseen garments — and he placed second (Larchenko, 2026). His solution is the subject of most of this book: a $\pi_{0.5}$ -derived VLA (Physical Intelligence, 2025a) with auxiliary value heads, sampler-side AWR, RECAP-style advantage conditioning from $\pi_{0.6}^*$ (Physical Intelligence, 2025b), and an asynchronous flywheel on the Hugging Face Hub.

15.1.2 Why this is replicable at all

Most impressive results are not reproducible by one person with one GPU. This one nearly is, because four things are public at once: the tech report, the Apache-2.0 solution repo, the organizers' environment and success checker, and both sim and real checkpoints on the Hub. Open code plus open weights plus an open evaluator turns replication into an engineering task rather than an admiration exercise. The software-engineering analogy: a bug report saying "it crashes sometimes" is useless, while one with a minimal reproduction script and exact dependency versions is something you can act on in an afternoon. LeHome is the second kind. *Where the analogy breaks down*: a reproduction script is deterministic — you run it, it crashes or it does not. A policy evaluation is a noisy estimate: run the identical checkpoint on the identical environment twice and you get different numbers, and that difference is not a bug. Which brings us to the part of this chapter that matters most.

15.1.3 The measurement problem, felt before it is solved

What you want is simple: a way to tell whether a change helped. The obvious way to get it is equally simple. Run your policy, count successes, write down the rate. Change something. Run it again, count again, compare. If the second number is bigger, the change helped.

Now watch that fail with real numbers. Suppose the change you are about to make does nothing at all: the true success rate is 50% before and 50% after. On Monday you evaluate 20 episodes with the old settings and get 10 successes. You flip the hyperparameter, retrain, evaluate 20 more on Tuesday, and get 13. That is 65% versus 50%, a 15-point gain, and you are pleased. But a fair coin flipped 20 times gives between 6 and 14 heads about 95% of the time. Both days are ordinary outcomes for a coin that never changed. You have measured nothing — and you will now build three weeks of work on a hyperparameter that did nothing.

The fix is to attach an uncertainty to every number. The unit of everything in this chapter is a success rate $\hat{p}_{succ}$ over n scored episodes, and the resolution of that estimate is set by the binomial standard error:

$$SE(\hat{p}_{succ}) = \sqrt{\frac{\hat{p}_{succ}(1 - \hat{p}_{succ})}{n}}$$

IN PLAIN WORDS

The left side is the typical amount by which your measured success rate misses the true one, purely from luck of the draw. $\hat{p}_{succ}$ is the fraction of episodes that succeeded; the hat means "estimated from data" rather than "the true value." n is how many episodes you scored. The product $\hat{p}_{succ}(1 - \hat{p}_{succ})$ is the variance of a single coin flip with that success probability: largest at $\hat{p}_{succ} = 0.5$, shrinking toward zero as the rate approaches 0 or 1. Dividing by n is averaging — more episodes, less noise. The square root converts a variance back into the units of the rate itself, so the answer is in success-rate points.

Two consequences shape this chapter. Your error bar shrinks like $1/\sqrt{n}$, not like $1/n$: to halve it you need four times the episodes, not twice as many. And near a 50% success rate — where robot policies live during the interesting phase of a project — you sit at the worst point on the curve. A 95% confidence interval is roughly $\hat{p}_{succ} \pm 1.96 SE$.

15.1.4 Worked example: is a +8 point improvement real?

This is the calculation you run before every gate below, so do it once slowly. Stage B, your behavior-cloning baseline, lands at $\hat{p}_{succ} = 0.42$ over $n = 200$ episodes. Stage C, after all the RL machinery, lands at **0.50** over another 200. Is the 8-point gain real?

Step 1, the standard error of the baseline. $0.42 \times 0.58 = 0.2436$. Divide by 200: $0.2436/200 = 0.001218$. Square root: $\sqrt{0.001218} = 0.0349$.

Step 2, the standard error of the new number. $0.50 \times 0.50 = 0.25$. Divide by 200: 0.00125 . Square root: $\sqrt{0.00125} = 0.0354$.

Step 3, the standard error of the *difference*. Independent measurements have their variances add, not their standard errors, so add the squares and take one square root: $0.001218 + 0.00125 = 0.002468$, and $\sqrt{0.002468} = 0.0497$. That is about $\sqrt{2}$ times either individual error: comparing two conditions costs precision.

Step 4, the interval. $1.96 \times 0.0497 = 0.097$, so the 95% interval on the difference runs from -1.7 points to $+17.7$ points. The interval contains zero. Your improvement is not resolved: it might be real, it might be nothing, and 200 episodes per condition cannot tell you which.

Step 5, how many episodes would have told you. Invert the arithmetic: the half-width $1.96 SE_{diff}$ should be no larger than the difference Δ you want to detect.

$$n \approx \frac{2 z^2 \hat{p}_{succ}(1 - \hat{p}_{succ})}{\Delta^2}, \quad z = 1.96$$

IN PLAIN WORDS

n is the number of episodes you need *in each of the two conditions*, not in total. Δ is the difference you care about, in the same units as the rate — here **0.08** for eight percentage points. $z = 1.96$ turns a standard error into a two-sided 95% interval. $\hat{p}_{succ}(1 - \hat{p}_{succ})$ is the same per-episode variance as before, evaluated near where you expect both conditions to sit. The factor of 2 is the price of comparing two noisy numbers instead of measuring one. The Δ^2 in the denominator is the harsh part: halving the effect you want to detect quadruples the episodes you need.

Plug in $\hat{p}_{succ} = 0.46$ (midway between the conditions) and $\Delta = 0.08$. First $z^2 = 1.96^2 = 3.8416$, so $2z^2 = 7.6832$. Then $0.46 \times 0.54 = 0.2484$, and $7.6832 \times 0.2484 = 1.9085$. Then $\Delta^2 = 0.08^2 = 0.0064$. Finally $1.9085/0.0064 = 298.2$. Round up: about 300 episodes per condition, 600 total.

Now price it. A LeHome episode takes roughly 30 seconds in Isaac Sim, so $600 \times 30 = 18,000$ seconds, or 5 hours on one simulator instance. That is the real cost of one honest claim; budget it like training time.

One honest footnote to that 300. It is the point at which a difference of exactly 8 points would just barely clear zero — so if the true gain really is 8 points, an experiment at $n = 300$ resolves it roughly half the time, because your measured difference lands above or below 8 with equal ease. To be confident of catching a real 8-point gain, roughly double it: swap $z^2 = 3.84$ for $(1.96 + 0.84)^2 = 7.84$ and the same formula gives $2 \times 7.84 \times 0.2484/0.0064 = 608.6$, about 600 per condition, ten hours of sim. Budget 300 when exploring, 600 when making a claim you will publish.

A second, smaller example. Stage A asks you to reproduce his per-type numbers on the 12 garments per type you can see, each run 10 times, so $n = 120$. At his long-sleeve number of **0.745**: $0.745 \times 0.255 = 0.1900$, divided by 120 is **0.001583**, square root **0.0398**, times 1.96 gives ± 7.8 points. Your reproduction can land anywhere from 67% to 82% and still be consistent with 74.5%. That is why the Stage A gate says "within a few points" — and why you should not celebrate at 78%.

One caveat on that ± 7.8 , and it makes things worse rather than better. Those 120 episodes are 12 garments run 10 times each, not 120 independent draws. If the policy simply cannot fold one particular long-sleeved top, that garment contributes ten correlated failures rather than ten unlucky coin flips, and the binomial formula does not know that. The honest quick fix is to treat the garment as the unit: compute a per-garment rate over its 10 runs, then take the standard error across the 12 garment means. Expect an interval wider than ± 7.8 , not narrower. The same caveat applies to the 300-per-condition figure in Step 5.

```
from math import sqrt
Z = 1.96                                # two-sided 95% normal quantile

def delta_ci(p1, n1, p2, n2):           # 95% interval on (p2 - p1)
    var = p1*(1-p1)/n1 + p2*(1-p2)/n2   # variances add, standard errors do not
    half = Z * sqrt(var)
    return (p2 - p1 - half, p2 - p1 + half)

def n_needed(p, delta, z=Z):            # episodes PER condition, not total
    return 2 * z**2 * p * (1 - p) / delta**2

print(delta_ci(0.42, 200, 0.50, 200))  # (-0.017, 0.177) -> contains 0: not resolved
print(n_needed(0.46, 0.08))            # 298.2 -> 300 per condition, 5 h of sim
print(n_needed(0.46, 0.08, z=2.80))    # 608.6 -> 600 if you also want the power
```

WORDS YOU WILL HEAR

Checkpoint — two things: a saved snapshot of model weights, and a mid-episode reward milestone (Chapter 11).

Context disambiguates; papers rarely bother. **Eval harness** — the code that runs a policy for n episodes, scores them, and reports rates; infrastructure, not an experiment. **LoRA** — low-rank adaptation: freeze the base weights, train small added matrices, ship those. **VRAM** — memory on the GPU board itself; the constraint deciding whether you fine-tune fully or with adapters. **Confidence interval** — the range that would cover the true value in 95 of 100 repeated experiments.

Ablation — removing one component to measure what it contributed.

Put these in your harness on day one and make every table print intervals automatically. If a number cannot print an interval, it does not go in the table. Figure 15.1 shows the pipeline those intervals guard.

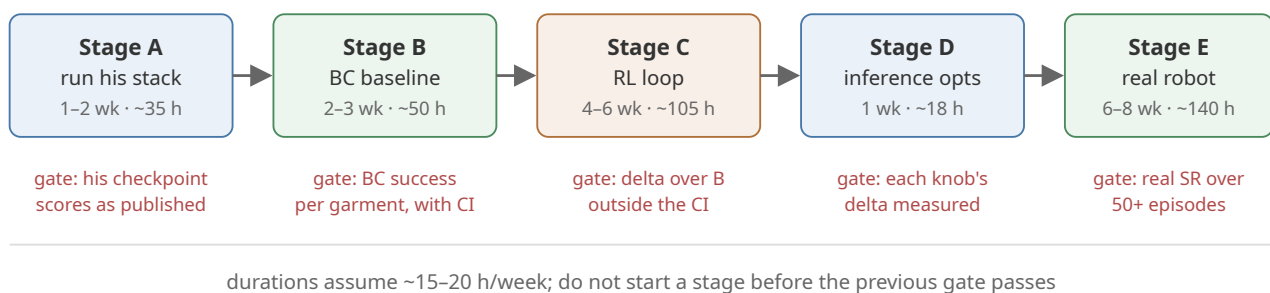


Figure 15.1 — The staged replication pipeline. Notice that every arrow is guarded by a red gate written in units you can measure, and that most gates mention an interval or a sample size rather than a bare number. A stage that has not passed its gate is not a foundation; it is a liability you pay for two stages later, when you can no longer tell which layer broke.

15.2 Stage A — run before you build (1-2 weeks)

The goal, in one sentence: prove his published result runs on your machine and scores what he says, before you change a line.

What you actually do. Clone the organizers' challenge repository and get Isaac Lab rendering on your GPU — one consumer card runs one simulation instance comfortably. Clone the solution repository. Provision disk: the repo asks for 500 GB or more. Download his sim checkpoint from the Hub. Run the official evaluation with *his* weights on the garments you can see — 10 seen plus 2 public unseen per type, 10 runs each — and record the four per-type rates with intervals.

The deliverable is two things, and the second matters as much as the first. One: a table of your reproduced per-type success rates. Two: a one-page map of the codebase, in your own words. Where does the model live? Where does each auxiliary head attach? Where are reward and advantages computed, in which process? How does the sampler apply AWR priorities? You are about to modify this system; Stage A is where you earn the right to.

The gate: your per-type numbers land within a few points of his published 74.5 / 70.0 / 80.5 / 93.5. You will not reproduce 79.63% exactly, because eight garments per type were private and are not in your evaluation set.

How it most likely goes wrong. Not the science — the plumbing. Installing Isaac Lab against your driver and CUDA version is the largest time sink, and the worst failure mode is silent: everything runs, no error appears, and your numbers come out fifteen points low because an image-resize path differs. If your reproduction is far off, resist the urge to "improve" anything; diff your config against his first.

15.3 Stage B — the behavior-cloning baseline (2–3 weeks)

The goal, in one sentence: train your own policy on the same data with none of the RL machinery, so every later claim has something honest to be compared against.

Be explicit about why this stage exists, because it is the one people skip. You want to know what the RL machinery buys, and the obvious approach is to build the full system and compare against his published number. That fails subtly: his number reflects his compute budget, his pool of roughly 12,500 rollout episodes, and three months of iteration. If your system scores 55%, you have learned nothing about whether your advantage computation is correct — only that you have less compute. A baseline you trained yourself is the only comparison that isolates what you are testing. In software terms: you would never accept a performance optimization without benchmarking the unoptimized build on the same machine.

What you actually do. Fine-tune the base policy on the organizers' scripted-demonstration dataset with plain flow-matching behavior cloning, exactly as Chapter 7 described it: sample a flow time u , form the interpolant, regress the velocity field. No advantage signal, no reward pipeline, no conditioning. If you start from his checkpoint rather than a clean $\pi_{0.5}$ base, keep its advantage token masked to neutral throughout — otherwise the policy is quietly conditioning on a signal you are not computing, and your "baseline" is not one. A clean $\pi_{0.5}$ base has no advantage token and needs no such care.

Stage B also forces you through his training code at your scale: the multi-dataset sampler, the smooth per-timestep action normalization, the frozen SigLIP encoder (a contrastively pretrained vision encoder that stays fixed while the rest trains). Downscale deliberately — LoRA adapters (low-rank adapters: freeze the original weights, train small added matrices instead) or a reduced batch with gradient accumulation.

LoRA deserves an analogy, since it is everywhere in this literature. A LoRA adapter is the patch file, not the fork: instead of shipping a modified copy of a 2.7-billion-parameter model, you ship a small diff applied at load time, and you can keep several diffs against one base. *Where the analogy breaks down:* a patch file can express any change, given enough lines. A low-rank adapter cannot — restricted to updates of limited rank, it genuinely has less capacity than full fine-tuning. Usually fine, sometimes even helpful in limiting drift, but a real constraint and not just a storage trick.

The deliverable: per-type success rates with intervals at the sample sizes from Section 15.1, plus training throughput — steps per second, VRAM used (the memory on the GPU card itself, the binding constraint here), wall-clock hours per 10,000 steps. Record it even though it feels like bookkeeping; Stage C's feasibility is a function of that number.

The gate: a per-type BC success rate with a confidence interval, written down and committed. Every later stage must beat it or explain itself.

How it most likely goes wrong. Data pipeline mismatches that show up as mediocre-but-plausible scores: wrong action normalization, a camera in the wrong slot, a chunk horizon disagreeing with the dataset's. And the human failure: declaring the baseline done at $n = 40$ because training was slow and you were impatient.

PITFALLS

Skipping Stages A and B to "start with the interesting part" means debugging reinforcement learning on top of an unverified stack. When success rates stall, you face three unknowns at once: the advantage math might be wrong, the evaluation harness might be wrong, or the policy might never have trained correctly. Each is a week to rule out, and they interact. A reproduced checkpoint score and a measured BC baseline collapse three unknowns into one. That is the whole argument for staging.

15.4 Stage C — the RL loop (4–6 weeks)

The goal, in one sentence: add the learning loop in dependency order, so when something breaks you know which layer broke it.

Dependency order is not a preference: each piece consumes the output of the previous one, and a bug in an early piece looks exactly like a bug in a later one.

15.4.1 Layer one: the auxiliary heads

Four of the five heads (Chapter 8) are linear probes: one matrix multiply from a representation the network has already computed to the quantity you want. That makes them nearly free, and it makes them informative — if a linear map can read the answer off the representation, the representation already contained it.

All four hang off the shared query token, the single vector the backbone hands to the action expert. The success head predicts the probability that this episode ends in success. The garment-type head predicts which of the four categories the policy is looking at, which matters because the category is withheld at evaluation time. The completion head predicts how far through the fold the episode is, as a fraction. The keypoint-distance head predicts the pairwise distances the success checker will eventually measure.

The fifth head is different in kind. It sits at the tail of the action expert rather than on the query token, and it reads the sampled action chunk, which makes it action-conditional: it answers "if I execute *this* chunk, what happens?" rather than "from this state, what tends to happen?" That is the distinction between $Q^\pi(s, a)$ and $V^\pi(s)$ from Chapter 3, which is why the report calls it Q-like. It predicts future keypoint distances and a success residual — how much this particular chunk moves the episode's success probability.

The keypoint targets need sim-privileged state, meaning ground truth the simulator knows and your cameras do not; that is why this layer belongs to Stage C and could not have been built in Stage B. Verify each head alone before anything consumes it. Garment type should reach very high accuracy within a few thousand steps; if it does not, suspect your data pipeline before your head. And write down the success head's correlation with actual outcomes on held-out episodes, because a weak correlation there is your earliest warning that layer three will produce noise.

15.4.2 Layer two: reward

The reward is derived from the success checker rather than invented alongside it (Chapter 11). Any reward you write by hand is a second specification of the task, and the moment it disagrees with the checker your policy optimizes the wrong one. Three mechanisms sit on top of it.

Checkpoint densification pays partial credit at one intermediate milestone, exactly as Chapter 11 §11.2 specifies. Tops and long pants get a mid checkpoint worth 0.5 — for a long-sleeved top, "sleeves folded onto the body" — with the remaining 0.5 paid at full success, so cumulative reward tops out at 1.0 on precisely the episodes the official checker accepts. Shorts get no mid checkpoint: one fold has no halfway point. For tops that first 0.5 is not a lump but is granted gradually, in proportion to the reduction of the primary proximity distance, under the running-min rule that stops a policy farming it by sliding the sleeve back and forth. Implement Chapter 11's version verbatim; a policy that half-folds and then thrashes must end the episode at zero, which is what reward withdrawal is for.

Gradual proximity credit softens that further for tops, where the conditions are hardest. Instead of paying only when a distance crosses its threshold, you pay on the running minimum: track the smallest value that pair has reached in this episode, and pay whenever that minimum improves. Running minimum rather than current value, for two reasons — a policy is not punished when a later motion disturbs a pair it already brought together, and it cannot farm reward by oscillating across a threshold.

Failure withdrawal takes it all back. If the episode ends in failure, every partial payment made during it is subtracted, so the episode's return is exactly zero or exactly the success value. Densification is allowed to change *where inside an episode* credit lands; it must not change *what an episode is worth*. If it does, a long failure that

collected four partial payments outranks a short clean success, and your advantages will faithfully teach the policy to fail slowly.

15.4.3 Layer three: advantages

Four decisions go into the advantage, and they are easier to hold separately than together. First, the value estimate comes from the success head rather than from a separately trained critic — you trained that head in layer one, and it already predicts what a value function predicts. Second, that prediction becomes a value by subtracting the reward already collected and then damping the result. Third, per-step credit comes from GAE (Chapter 3) with $\gamma = 0.999$ and $\lambda = 0.99$: a long horizon and a long trace, because a fold runs for hundreds of frames and the credit for a good grasp is genuinely spread across all of them. Fourth, because rollouts stay in the pool while the policy moves on, that per-step term is blended toward a GRPO-style anchor comparing a segment against the empirical average for its garment type, with the blend shifting toward the anchor as the data ages. Potential-based completion shaping is added on top.

$$\hat{V}_t = \alpha (\hat{p}_{succ}(s_t) - R_{acc,t}), \quad \tilde{A}_t = w \hat{A}_t^{GAE} + (1 - w) \hat{A}_t^{GRPO} + \hat{A}_t^{\Phi}$$

IN PLAIN WORDS

Careful — $\hat{p}_{succ}$ means something narrower here than it did in Section 15.1. There it was the fraction of episodes that succeeded, measured after the fact over a whole condition. Here $\hat{p}_{succ}(s_t)$ is the success head's prediction at one frame: the model's own guess, made while the episode is still running, that this episode will end in success. Same underlying quantity — a probability of success — but one is measured over many episodes and the other is predicted at a single frame, which is why this one carries s_t . Both terms on the right are read at that same frame: the head's prediction at t , and $R_{acc,t}$, the reward already paid out in the frames before t . Their difference is the reward still available to collect. The damping coefficient $\alpha = 0.5$ shrinks that estimate deliberately, because a noisy baseline subtracted at full strength adds more variance than it removes.

IN PLAIN WORDS

The second equation is the final advantage for frame t , a weighted sum of three estimates of one thing. $\hat{A}_t^{GAE}$ is per-step credit from Chapter 3: accurate, but it leans on value predictions that go stale as the policy moves away from the one that collected the rollout. $\hat{A}_t^{GRPO}$ compares a segment against the empirical average for that garment type — coarser, but an outcome stays an outcome however old the policy is. w is the blend weight between them, tied to how heavily that dataset is still sampled; it is unrelated to the AWR weight w_i below, despite the letter. $\hat{A}_t^{\Phi}$ is potential-based shaping from the completion head, added at full strength because potential-based shaping (Chapter 11) provably leaves the optimal policy unchanged, so turning it up cannot quietly bend the objective.

Use $\alpha = 0.5$, with w tied to the dataset's decaying sampling share — Chapter 11's formula verbatim, the $\hat{A}^{GRPO}$ anchor computed per garment-type segment.

15.4.4 Layer four: spending the advantage twice

The same $\tilde{A}_t$ is spent twice, in the two ways Chapters 4 and 10 describe. One step the formula below hides: before clipping, divide the advantages by a single global standard deviation computed over the current pool of rollouts. Without it the clip range $[-2, 2]$ means something different every iteration, because raw advantage magnitudes drift as the policy improves. Normalize, then clip.

The first consumer is AWR through the sampler: $P(i) \propto e^{\text{clip}(\tilde{A}_i, -2, 2)}$, where $P(i)$ is the probability that training sample i is drawn into a batch — not a weight in the loss, a probability in the sampler. That is Chapter 4's AWR weight $w_i = \exp(\tilde{A}_i/\beta)$ with temperature $\beta = 1$, moved out of the loss and into the sampler, where it costs nothing because you were going to draw a batch anyway. The clip at ± 2 caps the ratio between the most and least favored sample at about $e^4 \approx 55$, so one freak episode cannot take over training.

The second consumer is the RECAP advantage token from Chapter 10: the advantage is quantized and fed back to the model as an input, so the policy learns what a high-advantage action looks like rather than merely being pushed toward one. Mask that token stochastically during training so the model also learns the unconditional distribution — that is what makes classifier-free guidance available in Stage D. Without the masking there is no unconditional mode to guide away from.

15.4.5 Layer five: the flywheel, and the case you will actually build

Conceptually the flywheel (Chapter 14) is three processes that never wait for each other. A *trainer* reads rollout datasets and writes checkpoints. *Rollout workers* read the newest checkpoint and write scored episodes. A *Dagger station* lets a human take over failing episodes and writes corrections. All three talk only through the Hub, and what the absence of synchronization barriers buys is concrete: a slow simulator never leaves the GPU idle, and a long training step never leaves the simulators idle.

On one workstation you cannot buy that, because your single GPU can only do one of those jobs at a time. So build the flywheel's degenerate case: the same three roles, the same interfaces, run sequentially in one process — collect, compute, train, repeat. The pseudocode below is exactly that case, which is why it looks strictly sequential. Keep the roles separate anyway and going asynchronous later is a scheduling change, not a rewrite. When Section 15.8 tells you to cut the flywheel, it is confirming this choice rather than reversing it.

```
# one Stage C iteration on a single workstation (pseudocode)
roll = collect(policy=ckpt, n_episodes=300, garments=["top_long"]) # ~2.5 h of sim
rew = densify(success_checker, roll) # checkpoints + failure withdrawal
val = 0.5 * (roll.p_succ_at_collection - roll.R_acc) # damped baseline, alpha=0.5
adv = (w * gae(rew, val, gamma=0.999, lam=0.99) # predictions are NOT re-run
      + (1 - w) * group_anchor(roll) # stale-proof outcome term
      + potential(roll.completion)) # shaping, full strength
adv = clip(adv / global_std, -2, 2)
prob = softmax(adv) # AWR lives in the sampler, not the loss
ckpt = train(ckpt, sample_by=prob, adv_token=adv, steps=1000) # ~6 h at LoRA scale
```

Scoping this to one workstation, honestly. Full fine-tuning of the roughly 2.7-billion-parameter backbone-plus-expert with AdamW wants 40–50 GB of VRAM; LoRA with the frozen encoder fits in 24 GB. Run one or two Isaac instances alternating with training, and restrict the curriculum to one or two garment types. Expect one full iteration — collect rollouts, recompute advantages, a thousand steps, push a checkpoint — to take a day, not the forty-minute cadence his H200 sustained.

The deliverable is three artifacts. The trained checkpoint. A per-head validation report — garment-type accuracy, the success head's correlation with outcomes, keypoint-distance error, each on held-out episodes and each plotted over training. And unit tests for the advantage pipeline, the part you will want to skip and the part that costs a month when it is wrong: a fabricated episode with constant return must give zero advantage everywhere, a shaping-only episode must leave the episode's total return unchanged, and a stale-data episode must use the predictions recorded at collection time rather than fresh ones.

The gate: a statistically resolved improvement over Stage B on at least one garment type, at the sample sizes from Section 15.1. One type, one resolved delta. That is a real result.

How it most likely goes wrong. Turning on all five layers in one weekend. Forgetting that predictions must be recorded at collection time and never re-predicted, which silently corrupts advantages. And a subtler trap about which episodes are allowed to count.

Only rollouts started from the ordinary initial-state distribution count toward success statistics. Three strategies qualify: *random*, which draws a random initial garment pose; *full*, which runs a whole episode from the standard reset; and *curriculum*, which biases which garments appear but not where an episode begins. Two others deliberately start somewhere else. *Success replay* restarts from a saved physics snapshot of an earlier success and re-runs it under

heavier augmentation. *Hard mining* restarts from snapshots of states where the value prediction collapsed. Both make excellent training data and neither makes a measurement, because their starting states were chosen by looking at outcomes; mix them into a gate and you manufacture improvements that do not exist.

The codebase calls the first three the *unbiased* strategies, and that word is worth flagging before it misleads you. It has nothing to do with the estimators in Section 15.1: here "unbiased" means "not selection-biased in which state the episode starts from." An episode from an unbiased strategy still gives you a noisy rate with all the same error bars.

IN THE LEHOME SOLUTION

Training ran on a single H200: batch 192, bf16, frozen SigLIP, about 300k steps, pushing a new checkpoint roughly every 40 minutes. Separate RTX PRO 6000 machines each ran 3–5 Isaac Sim instances behind one shared stateless policy server, and scaling up meant starting another machine; the repo asks for 500+ GB of disk. The final pool was about 12,500 rollout episodes over roughly 140 sessions. Your single-GPU loop iterates perhaps ten times slower; scope Stage C to match. One garment type and one resolved improvement beats four types and no measurements.

15.5 Stage D — inference-time optimization (1 week)

The goal, in one sentence: with the policy frozen, harvest Chapter 12's cheap wins one at a time, each with an attributable delta.

What you actually do, in this order. First, sweep the classifier-free guidance scale g — the knob that controls how hard the sampler is pushed toward the high-advantage mode and away from the unconditional one it also learned (Chapter 12). Sweep it wide. His bandit pinned values of 7 to 9 after he had initially searched only 0 to 2, so sweep at least 0 to 12; the setting that worked was outside the range he first thought reasonable.

Second, add best-of- N : sample N candidate action chunks instead of one, score each with the action-conditional success-residual head you built in Stage C, and execute the winner. $N = 2$ or 3 helped consistently and larger did not. The cost is linear — three candidates means three passes through the action expert per chunk — while the benefit saturates as soon as the scoring head runs out of discrimination.

Only then let the bandit loose. A factorized Thompson-sampling bandit is an online tuner: it keeps a probability distribution over where each knob's best value lies, samples a setting from it, runs episodes, and updates. "Factorized" means it treats the knobs as independent rather than searching their joint space, which is the only reason seven of them are tractable. It tunes the seven per-garment-type knobs — four sets of seven, one set per garment type, not one per garment — updating only on episodes started from the ordinary initial-state distribution, for the reason Section 15.4 gave. The seven, with what each physically does:

- **Execution length** — how many actions of each predicted chunk you actually execute before replanning. Shorter means more reactive and more compute.
- **Playback stretch** — a time-scaling factor on the chunk as it is executed; stretching plays the same motion more slowly.
- **Anchor length** — how many actions from the end of the previous chunk are handed to the sampler as context, so consecutive chunks join smoothly.
- **Inpainting onset** — the flow time u below which the previous chunk's tail is held fixed while the rest of the new chunk is generated around it.
- **Guidance scale g** — as above.
- **Noise temperature** — how much of the initial noise z survives into the sample; lower makes the sampler more deterministic and its chunks more alike.
- **N** — the best-of- N width.

The ordering is the lesson: sweeps first, bandit second. A bandit finds a good joint setting but tells you nothing about why. Start there and you end the stage with a config file and no understanding, unable to answer "which of those

mattered?" Attribution first, optimization second.

The deliverable and gate are the same object: a table with one measured delta per accepted knob, each with its interval. Section 15.1's arithmetic applies here — a 2-point guidance gain needs far more episodes than an 8-point one, and some knobs will not be resolvable on your budget. "Not resolved at $n = 300$ " is a legitimate entry, more honest than a number without an interval.

How it most likely goes wrong. You sweep a knob at $n = 60$ per setting, see a 2-point win, and keep it. Run Section 15.1's arithmetic before you do. Near $\hat{p}_{succ} = 0.5$ the standard error of the difference is $\sqrt{2 \times 0.25/60} = 0.091$, so the interval on your 2-point win runs $+2 \pm 17.9$ points. You have measured nothing, and you have now frozen a knob on it. This is Section 15.1's lesson in a new costume, and it bites hardest here precisely because the stage is cheap: a sweep is a config change and an hour of sim, so you can freeze twenty knobs on noise in a week. The same arithmetic explains the ordering above — sweeping all seven at honest sample sizes is more simulation than Stage C used, so you buy attribution for two and let the bandit have the rest.

15.6 Stage E — to the real robot (6–8 weeks)

The goal, in one sentence: get a policy running on physical hardware, measured, with the protocol written down before the first episode. Here is the honest fork: LeHome is bimanual, and you own one SO-101 leader/follower pair.

Path (a): replicate properly. A second leader/follower pair costs roughly \$250–450 in kit form, plus two wrist cameras. Then follow his sim-to-real recipe. It is five separate mechanisms, and each one removes a specific mismatch between the simulator your policy grew up in and the room it now has to work in.

Mount the arms 46 cm apart, matching the simulation. This sounds too trivial to write down and it is not: the policy learned a workspace geometry along with everything else, and arms 5 cm too far apart shift every reachable point by an amount no reasonable quantity of fine-tuning data will politely absorb.

Make takeover jump-free. During DAGger the leader arms track the followers continuously, so at the moment you grab a leader it is already at the follower's pose. Without that, takeover is a discontinuity — the robot snaps from where it was to wherever your hands were resting — and every snap is recorded as a demonstrated action that the policy will faithfully learn. He arms and releases takeover with a three-pedal foot switch, which keeps both hands on the leaders; the pedal is not the point, the hands-free arming is.

Build the camera-overlay alignment tool. It drives the robot to the joint state recorded in a chosen dataset frame, then overlays the live camera image on the recorded one so you can nudge each camera until the two coincide. Camera placement is the mismatch you are least likely to notice and most likely to be destroyed by, because a policy trained on one viewpoint does not see a shifted viewpoint as the same scene from slightly elsewhere. It sees a different state.

Apply per-source speed factors. Demonstrations from different sources — your teleoperation, his, scripted trajectories — move at systematically different speeds, and a policy trained on the mixture learns a blurred average pace that matches nobody. A per-source scaling factor normalizes motion intensity, so the mixture teaches what to do rather than how fast whoever recorded it happened to move.

Strip the model, and here the reason comes first because everything follows from it. On a real robot there is no success checker; no checker means no reward, and no reward means no advantage. Everything in your architecture that consumes an advantage therefore has nothing to consume: the advantage token has no value to be set to, classifier-free guidance has no high-advantage mode to steer toward, and best-of- N has no way to rank candidates, because the success-residual head scores against a signal you can no longer produce. Take all three out. What remains is the action objective plus the garment-type and completion heads, both predictable from images alone and both unaffected.

Path (b): adapt. A single-arm garment task — a towel fold with one arm plus a fixture, a clamp or weighted edge standing in for the second gripper — reusing the same machinery: teleoperated demonstrations on your rig, a sim

scene aligned to your cameras, DAgger corrections with intervention weighting (Chapter 13), then a real fine-tune under heavy augmentation. This is your hil-serl workflow with a VLA policy in place of SAC.

Either path exercises every skill a robotics-RL interview probes; path (b) costs less and demands more judgment, which is not a downside. **The gate:** a real-robot success rate over at least 50 scored episodes, with the protocol — garment set, reset procedure, what counts as success, who scores it — written down *before* you run it.

How it most likely goes wrong. Not the policy — the protocol. Three specific failures manufacture success rate on real hardware, and all three feel like diligence while you are committing them. *Protocol drift:* the way you set up an episode in week two is not quite the way you set it up in week one, because by week two you have learned what the policy likes; it happens one small accommodation at a time and turns a fixed measurement into a moving one. *Asymmetric resets:* after a failure you smooth the garment out carefully and after a success you leave it as it fell, or the reverse, and the two outcomes are no longer drawn from the same distribution of starting states. *Scoring your own episodes:* you know which condition is the new one, folds near the threshold are genuinely ambiguous, and they drift the way you are hoping.

The countermeasures are dull and they work. Write the protocol before the first episode. Photograph the reset state and match the photograph every time. Randomize the order conditions are run in. Have someone who does not know which condition is which score the videos — or, failing that, score them yourself from video, shuffled and unlabeled, days later.

ON YOUR RIG

You have already run the hardest part of path (b) once. Your hil-serl setup is actor and learner as separate processes, Xbox-pad interventions on a physical SO-101, and a working run at a 98% intervention rate — the human doing the task, the policy almost nothing. That is the correct starting point, not a failure: Chapter 13's argument is that intervention rate is your progress metric, and watching it fall toward 30% is Section 15.1's discipline in a different unit. Swap SAC for the VLA, keep the topology and keep the pad — whatever your hand is already trained on for takeover is the right takeover device.

PITFALLS

Letting simulation work expand forever. Larchenko spent roughly three months in sim and one week on sim-to-real — and zero-shot transfer still failed, because the latest checkpoints had overfit the simulator down to the image-resize pipeline: 640 → 224 and 640 → 320 → 224 were distinguishable by the auxiliary heads, and the difference cost real success rate. Timebox Stages C and D, start transfer from a late-but-not-latest checkpoint, and accept the trade: touching the real arm two weeks earlier beats one more point in sim.

15.7 What to publish

The portfolio artifact is the point of the capstone, so plan it from week one rather than assembling it from memory at the end. Publish a repository with four things in it. Each one answers a question a reviewer is already asking, whether or not he says so out loud.

One: the per-stage evaluation table. One row per stage and garment type, five columns — condition, *n*, success rate, 95% interval, and which rollout strategies produced the episodes. Two rows are enough to show the shape:

condition	n	rate	95% CI	rollouts
B: BC baseline, top_short	300	0.41	[0.354, 0.466]	random+full
C: +AWR +advantage token	300	0.51	[0.453, 0.567]	random+full

The columns are the argument. The *n* column says you knew sample size was a decision rather than an accident. The interval column says you know the rate is an estimate. The rollout column says you knew which episodes were allowed to count, and it is the one that most sharply separates someone who has run this system from someone who

has read about it. A table like this takes ten minutes to produce if your harness printed intervals from day one, and two weeks to reconstruct if it did not.

Two: a two-page technical note, in the style of his report. Section by section: what you set out to reproduce and what the published numbers were; what your reproduction actually measured, with intervals; what you changed, in what order, and what each change bought; one small figure per claim, each showing a rate with an interval rather than a training curve; a paragraph on what your setup could not test and why — one GPU, one garment type, no access to the private garments; and a short list of what you would do next given more compute. Two pages, not ten. A reviewer reads it in four minutes and decides from it whether to open your code.

Three: rollout videos with reward, value, and advantage overlays — the debug views his pipeline saved at collection time. Thirty seconds of that shows a reviewer things no table can. He watches the value trace fall seconds before the failure is visible, which tells him your success head learned something real rather than an average. He watches the advantage change sign at the frame where the grasp slips, which tells him your credit assignment lands on the right frames. And he sees that the overlay exists at all, which tells him you instrumented the system instead of running it. A number can be copied off a leaderboard; an overlay video can only come from a system you built and can still run today.

Four: the negative results, written as results rather than apologies. "Best-of- N at $N = 4$ measured -1.2 ± 8.0 points against $N = 2$ at $n = 300$ per condition: not resolved" is a stronger sentence than most success rates you could report, and it is worth saying why outright. To write it you had to fix a protocol in advance, hold everything else constant, run two conditions at an honest sample size, and then report a number you were hoping would come out differently. That is a controlled experiment. Reporting only wins is fully compatible with having run someone else's repository until something looked good, and a reviewer knows it. The interval is the proof, and it is the single thing that separates an engineer from an operator of other people's code.

This mirrors how Larchenko's own open-sourcing built credibility. His report is an engineering case study with almost no ablations — by academic standards a weakness — yet it became a calling card, because every claim ships with runnable code, checkpoints, and an evaluator anyone can rerun. Verifiability substituted for ablations. For a job search, copy that model — and after this book you can.

15.8 If you only have three months

Figure 15.2 lays the five stages out on a calendar: at 15–20 hours a week they occupy 22 weeks and about 380 hours in total, roughly five months. If your job search does not have five months in it, you need triage, and deliberate triage beats panic triage in week 11.

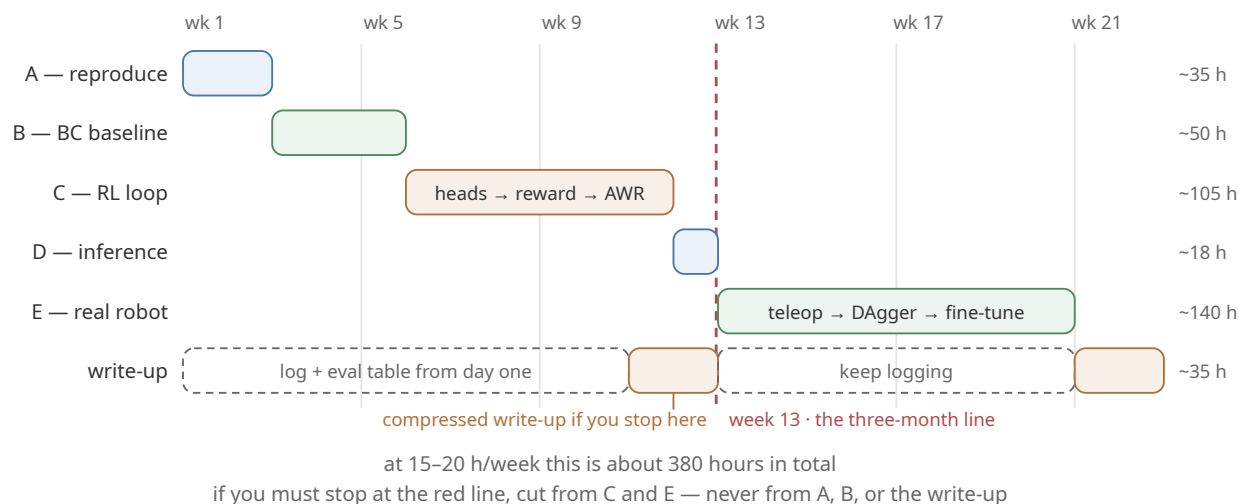


Figure 15.2 — The same five stages on a calendar at 15–20 hours per week. Notice how little of it is the part that sounds exciting: Stage C, the actual reinforcement learning, is six weeks out of twenty-two, and everything before it exists to make its result interpretable. The red line marks three months; Section 15.8 explains what survives to its left. Notice also that the write-up row has two solid blocks, not one — the small block at weeks 11–12 is there because a write-up is on the never-cut list, so a plan that stops at the red line has to budget one before the line, not after it.

What to cut. Cut Stage C's full asynchronous flywheel, which is a confirmation of Section 15.4.5 rather than a reversal of it: build only the degenerate case described there, the single-process loop that collects, stops, recomputes advantages offline, trains, and repeats. On one workstation you have nothing to be asynchronous *with*, so the asynchronous version is engineering you would be doing for its own sake. Cut multi-garment coverage: pick one type and stay there. Cut Stage D's bandit, keeping the two sweeps. Cut Stage E path (a) — do not buy a second arm pair under time pressure — and run a reduced path (b) with one task and one fixture.

What never to cut. Stage A's reproduction, because without it nothing you report is anchored. Stage B's baseline, because without it your improvement has no denominator. One measured improvement with a real interval, the only part that shows you can do research rather than follow instructions. And the write-up, because unpublished work does not exist for a hiring manager.

The minimum coherent story, and it is a good one: "I reproduced the winning LeHome sim checkpoint on my own hardware and matched the published per-type rates to within measurement error. I trained my own behavior-cloning baseline from the organizers' data and measured it at $41\% \pm 5.6$ over 300 episodes on short-sleeved tops. I added advantage-conditioned sampler-side AWR with a densified checkpoint reward and measured $51\% \pm 5.7$ over another 300; the 95% interval on the difference is $+10 \pm 7.9$ points, excluding zero. Here is the code, the protocol I fixed in advance, the rollout videos with value overlays, and two things that did not work." That is worth more than four garment types with no error bars, and it fits in three months.

CHECKPOINT

- Compute the binomial standard error of a success rate, turn it into a 95% interval, and combine two into an interval on a difference — without looking anything up.
- Decide before an experiment how many episodes per condition resolve a given difference, and price that in hours of simulation.
- Explain why Stages A and B exist, in terms of the three unknowns they collapse into one.
- State Stage C's dependency order — heads, reward, advantages, AWR and conditioning, flywheel — and what breaks if you reorder it.
- Scope the project to one workstation: what fits in 24 GB with LoRA, what needs 40–50 GB, how many simulator instances run alongside training.
- Choose between the two Stage E paths for your hardware and budget, and write the evaluation protocol before collecting a real episode.

Where to go deeper. Read Larchenko (2026) end to end now that you have the vocabulary for all of it; it will read as a checklist rather than a wall. Read it alongside $\pi_{0.5}$ (Physical Intelligence, 2025a) and $\pi_{0.6}^*$ (Physical Intelligence, 2025b), with his BEHAVIOR-1K report (Larchenko et al., 2025) for the machinery he carried over rather than invented. Appendix A gives the reading order; Appendix B's experiment ladder scales this discipline down to the rover and the arm you already own.

Appendix A — Paper-by-paper reading guide

This appendix turns the book's load-bearing citations into a study plan — the papers worth hours of your time, not every work the chapters cite in passing. Every entry tells you three things: what the paper contributes, which sections to read, and which sections you may skip without guilt. The skipping instructions matter as much as the reading ones.

Each entry carries a **tier**, a reading-depth label that says how much of your week the paper deserves. **Tier 1** means read deeply, with a pen: work the derivations the book only sketched, and reproduce one figure or equation yourself. Budget three to five hours. **Tier 2** means read once, for the single idea the book borrows; do not fight the proofs. Budget about an hour. **Tier 3** means skim, or file it as a reference for when a detail matters. Budget twenty minutes, and zero if the week is full.

If you have been dropped into an unfamiliar codebase, the tiers will feel familiar. Tier 1 is stepping through a function in a debugger until you could rewrite it. Tier 2 is reading the docstring and one call site. Tier 3 is knowing the file exists so you can grep it in six weeks. The analogy has one limit: code tells you when you have misunderstood it, because the test goes red. A paper never does; you can misread one for a year and feel fine. Hence the pitfall box below.

Read in *book order*, not chronological order: the list is a dependency graph, and book order is one topological sort of it, each paper placed where you first have the vocabulary for it. Chronological order is closer to `git log`, abandoned branches and all — but paper dependencies, unlike imports, are partial, so you can often read half a paper long before you can read all of it.

WORDS YOU WILL HEAR

Ablation. An experiment that removes one component to show it mattered; often the most informative table in a paper.

Preprint vs camera-ready. The arXiv and conference versions may differ; when numbers disagree, the later one wins.

ELBO. Evidence lower bound — the variational objective diffusion papers derive, then simplify away; you can skip it.

Reduction. A proof technique turning one problem into another with known guarantees, as DAGger reduces imitation to online learning. **Supplementary.** The appendix-after-the-appendix, where hyperparameters hide.

A.1 Foundations and policy gradients (Chapters 1–2)

REINFORCE (Williams, 1992) — Machine Learning 8(3–4). Tier 2. The original score-function gradient estimator and the first proof that subtracting a baseline leaves the gradient unbiased. Read Sections 1–2 and the algorithm statement; skip the episodic variants and the 1992-era network machinery. Chapter 2 rederives what you need.

Policy gradient theorem (Sutton, McAllester, Singh, and Mansour, 1999) — NeurIPS 1999. Tier 2. Shows that the gradient of $J(\theta)$ needs no derivative of the state distribution, which is why policy gradients are computable at all. Read the theorem and the function-approximation section; skim the convergence result.

A.2 Trust regions, PPO, and GAE (Chapter 3)

TRPO (Schulman et al., 2015) — arXiv 1502.05477. Tier 3. Read Sections 1–4 for the monotonic-improvement bound and the KL-constraint idea; skip the conjugate-gradient implementation. PPO replaced it in practice, and the bound is what survives.

PPO (Schulman et al., 2017) — arXiv 1707.06347. Tier 1. The clipped surrogate objective you have already trained with. The paper is short; read all of it, and hand-trace the clip logic for one ratio ρ_t above $1 + \epsilon$ and one below $1 - \epsilon$. Read it with your rl-rover TensorBoard curves open; that closes the loop.

GAE (Schulman et al., 2016) — arXiv 1506.02438. Tier 1. The λ -weighted sum of TD residuals δ_t as a bias–variance dial for $\hat{A}_t$ — used verbatim in LeHome. Read Sections 1–3 with a pen; skim the trust-region value-fitting section.

A.3 RL as weighted behavior cloning (Chapter 4)

AWR (Peng et al., 2019) — arXiv 1910.00177. Tier 1. The hinge of this book: RL as supervised regression with weights $w_i = \exp(\hat{A}_i/\beta)$. Work through the constrained-policy-search derivation in Section 3 and the appendix — it is the one derivation everything after Chapter 4 stands on.

AWAC (Nair et al., 2020) — arXiv 2006.09359. Tier 2. AWR with an off-policy Q-function critic, built for offline pretraining followed by online fine-tuning — the shape of every modern robot pipeline, including yours. Read the introduction and Algorithm 1; skip the benchmark tables.

A.4 Generative policies I: the diffusion lineage (Chapter 6, with background for Chapter 5)

Chapter 5 gets no reading slot of its own, and that is deliberate: it surveys the generative families rather than following one paper. Two entries below still bear on it — ACT for the conditional-VAE alternative, DDPM for the iterative-refinement argument. Its other distinctive citations, the GAN paper (Goodfellow et al., 2014) and RT-2 (Brohan et al., 2023), are worth knowing by name but not worth hours on this timeline.

DDPM (Ho et al., 2020) — arXiv 2006.11239. Tier 2. The paper that made diffusion practical: a fixed forward noising process, a network trained to predict the noise added, and the sampler that walks back down the noise levels k . Read Sections 2 and 3, especially where the variational objective collapses into a plain mean-squared-error loss on predicted noise; skip the ELBO derivation.

Diffusion Policy (Chi et al., 2023) — arXiv 2303.04137. Tier 1. The primary source for Chapter 6: a visuomotor policy — camera pixels in, motor commands out — expressed as a conditional denoising process over a whole action chunk $a_{t:t+H}$, executed receding-horizon: predict a long chunk, execute only its first few actions, then re-plan. Read the method and ablations closely, especially prediction horizon versus execution horizon — that one table explains more about chunked control than any prose can; skim the benchmark task descriptions unless you mean to reproduce one.

DDIM (Song et al., 2020) — arXiv 2010.02502. Tier 3. A deterministic, non-Markovian sampler that reaches comparable samples in far fewer steps K . Read the sampling section only. When a diffusion policy is too slow at inference, this is the first knob, and Chapter 12 assumes it exists.

ACT / action chunking (Zhao et al., 2023) — arXiv 2304.13705. Tier 2. The ALOHA paper, which introduced Action Chunking with Transformers: predict a block of H actions at once, then blend overlapping predictions by temporal ensembling. Its generative model is a conditional VAE — an encoder-decoder that samples from a learned latent distribution — not a diffusion model, which cleanly separates what chunking buys you from what the generative family buys you. Read Section 3 for the CVAE formulation and the temporal-ensembling description; skip the ALOHA hardware sections and the bimanual teleoperation setup, which do not transfer to a single SO-101 pair.

ON YOUR RIG

Diffusion Policy and ACT are the two entries you can run the week you read them. LeRobot ships both, and your frozen 24-episode cube dataset is enough for either — no new teleoperation session needed. Train ACT and a diffusion policy on that *same* dataset, then vary only the execution horizon. You will feel the difference between committing to eight actions and committing to two, and every later discussion of chunking will have a memory attached. Appendix B's E5 and E6 are this experiment written out, with hour budgets and a gate.

A.5 Generative policies II: flow matching (Chapter 7)

Flow matching (Lipman et al., 2023) — arXiv 2210.02747. Tier 2. Trains a velocity field $v_\theta(a^u, u, o)$ by regression along straight-line interpolants — no likelihoods, no ODE solves during training. Read Sections 1–3 for the

conditional flow-matching trick, where an intractable target becomes tractable because the conditional and marginal objectives share a gradient, and the optimal-transport part of Section 4 for the straight-line interpolant; skip the appendix proofs.

Classifier-free guidance (Ho and Salimans, 2022) — arXiv 2207.12598. Tier 2. Train conditional and unconditional models jointly by dropping the condition at random, then extrapolate between the two predictions with scale g at sampling time. A short workshop paper; read it all. Chapter 10 reuses the trick on advantage conditioning, and Chapter 12 turns g into an inference-time knob.

A.6 Vision-language-action models (Chapter 8)

Pi0 (Black et al., 2024) — arXiv 2410.24164. Tier 2. The first of the family: a pretrained vision-language backbone with a flow-matching action expert grafted on, trained on a large cross-embodiment dataset — many robot types pooled into one training set. Read the architecture section and how the action expert exchanges information with the backbone; skip the evaluation tables and the dataset-composition breakdown. Reading it first makes $\pi_{0.5}$ half as long, because $\pi_{0.5}$ assumes it.

$\pi_{0.5}$ (Physical Intelligence, 2025a) — arXiv 2504.16054. Tier 1. The vision-language-action model underneath the LeHome solution: a VLM backbone with a flow-matching action expert emitting chunks $\mathbf{a}_{t:t+H}$. Read the architecture and co-training sections closely, including hierarchical inference, where the model first emits a language subtask and then acts on it; skim the evaluation.

A.7 The comparison chapter (Chapter 9)

Chapter 9 has no primary source: nobody has written the paper that sits PPO and VLA-RL side by side. Build it yourself: read PPO (Section A.2) and $\pi_{0.5}$ (Section A.6) back to back, and write down every place where the second has no equivalent of something the first needs. That list — no per-action log-probability, no on-policy data, no ratio ρ_t to clip — is the chapter in raw form.

A.8 Learning by conditioning (Chapter 10)

$\pi_{0.6}^*$ / RECAP (Physical Intelligence, 2025b) — arXiv 2511.14759. Tier 1. Advantage-conditioned policies: instead of reweighting data, label each chunk with an advantage condition $\mathbf{c}$ — in the paper a binary indicator thresholded at ϵ_A and written into the input as a text token reading “Advantage: positive” or “Advantage: negative” — then ask for the positive label at inference. Read the RECAP method section twice — it is Chapter 10 in primary-source form — plus the value-learning details and how the per-task threshold is set, by percentile rather than by sign; skim the fleet results.

A.9 Reward and advantage engineering (Chapter 11)

Potential-based shaping (Ng, Harada, and Russell, 1999) — ICML 1999. Tier 2. Shaping rewards of the form $\gamma\Phi(\mathbf{s}_{t+1}) - \Phi(\mathbf{s}_t)$ are the only ones guaranteed to preserve the optimal policy. Read the theorem and the bicycle example, the clearest published account of reward hacking; skip the proofs.

CUPED (Deng et al., 2013) — WSDM 2013. Tier 3. Control variates — subtracting a correlated quantity you can predict, to shrink variance — applied to A/B testing; LeHome’s damped baseline $\alpha\mathbf{V}$ is this estimator transplanted into advantage estimation. Read Section 3 for the variance-optimal coefficient; the rest is web experimentation.

GRPO (Shao et al., 2024) — arXiv 2402.03300. Tier 3. The DeepSeekMath paper. Only the group-relative advantage matters here — scoring each rollout against the mean of its siblings — the anchor Chapter 11 uses against stale value heads. Read Section 4; skip the benchmarks.

A.10 Inference-time optimization (Chapter 12)

Thompson sampling (Thompson, 1933) — Biometrika 25. Tier 3, a historical pleasure worth one evening. Pair it with the **tutorial** (Russo et al., 2018) — arXiv 1707.02038, Tier 2: read Chapters 1–4 on Bernoulli bandits with Beta posteriors, exactly the machinery that tunes LeHome's seven inference knobs online; skip the approximate-inference chapters.

Real-Time Chunking (Black et al., 2025) — arXiv 2506.07339. Tier 2. Frames chunk switching as inpainting: freeze the overlap with the chunk currently executing, then generate the rest consistently with it. Read the method; skim the latency analysis. This is the origin of Chapter 12's inpainting knob.

A.11 Imitation and the human in the loop (Chapter 13)

Dagger (Ross, Gordon, and Bagnell, 2011) — arXiv 1011.0686. Tier 1. Why behavior cloning compounds errors quadratically in horizon, and how on-policy relabeling makes the growth linear. Read the full paper, including the reduction argument; the no-regret machinery can be skimmed on a first pass.

HG-Dagger (Kelly et al., 2019) — arXiv 1810.02890. Tier 2. Human-gated Dagger: the expert decides when to seize control, so every intervention marks a state the policy handled badly. This is your RB-to-intervene gesture, written up. Read the gating section and the risk metric; skip the driving-simulator specifics.

HIL-SERL (Luo et al., 2024) — arXiv 2410.21845. Tier 1. The pipeline you run on the SO-101. Read the full system section against your own actor/learner logs — intervention handling, reward interface, replay design — and the ablations on human corrections. Nothing to skip; it is all operational, and your 98% intervention rate is a number it helps you interpret.

A.12 Simulation and rollout collection (Chapter 14)

Difficulty-targeted selection (Sun et al., 2025) — arXiv 2506.05316. Tier 3. Training signal peaks on examples the policy succeeds at roughly half the time; LeHome's hard mining applies this to rollout selection. Read the introduction and the selection rule; the LLM details do not transfer.

A.13 The capstone sources (Chapter 15)

LeHome tech report (Larchenko, 2026) — arXiv 2606.27163. Tier 1. The case study this book is built around: $\pi_{0.5}$ plus AWR plus RECAP, auxiliary heads, the full advantage formula, inference-time bandits, and the sim-to-real playbook. Read it twice: once after Chapter 10, when the training story will parse, and again line by line before starting the capstone.

BEHAVIOR-1K report (Larchenko et al., 2025) — arXiv 2512.06951. Tier 2. The earlier win that seeded LeHome's architecture choices and chunk post-processing logic. Read for lineage after the LeHome report; skim the benchmark engineering.

IN THE LEHOME SOLUTION

The minimum viable reading list behind LeHome is six entries: GAE for $\hat{A}_t$, AWR for the weights w_i , $\pi_{0.5}$ for the architecture, RECAP for advantage conditioning, CUPED for the damped baseline αV , and the Russo tutorial for the inference-time bandit. Everything else explains why those six are reasonable rather than arbitrary. That is no argument for skipping the rest — a system you can only copy is a system you cannot debug — but it says where to start if a deadline compresses the plan.

A.14 Beyond papers

Sutton and Barto, *Reinforcement Learning: An Introduction*, 2nd edition (2018). Reference. When a foundation wobbles, its own Chapters 3–6 (MDPs, dynamic programming, TD learning) and its own Chapter 13 (policy gradients) are the canonical patch — those are Sutton and Barto's chapter numbers, not this book's. Do not read it cover to cover on this timeline.

CS224R (Chelsea Finn, Stanford). Lecture-format coverage of deep RL for robotics: offline RL, imitation, and real-robot case studies. Use the offline-RL lecture as a second angle on Chapters 4 and 10 whenever the papers feel too compressed.

LeRobot (Hugging Face). The codebase your hil-serl project already runs. Treat it as a primary source: when a paper's description of intervention handling or chunk execution is ambiguous, the resolution is in this repository, not in another paper.

PITFALLS

Reading papers without running code. A week of passive reading produces the feeling of understanding; twenty minutes of predicting what a change does to your own runs — then checking — produces the real thing. Pair every Tier 1 paper with an experiment on rl-rover or the SO-101 — Appendix B names twelve rungs across the fifteen chapters, each with an hour budget and a gate — or improvise: change λ , change β , break the baseline on purpose. If you cannot name the experiment a paper suggests, you have not finished reading it. Second pitfall: tier drift. Everything feels Tier 1 while you are reading it. Set a timer.

A.15 A twelve-week cadence

Does this list fit the time you have? The raw count of twenty-eight entries misleads, because a Tier 3 skim and a Tier 1 study are not the same unit of work. So weight them: count a Tier 1 paper as one full *paper-equivalent*, a Tier 2 as one half, a Tier 3 as one quarter.

$$N_{\text{eq}} = 1.0 n_1 + 0.5 n_2 + 0.25 n_3$$

IN PLAIN WORDS

The left side, N_{eq} , is the size of the reading list in “one deep paper” units — the currency you budget in. On the right, n_1 counts the Tier 1 entries, n_2 the Tier 2, n_3 the Tier 3. The coefficients **1.0**, **0.5**, **0.25** say a Tier 2 read costs about half a Tier 1 and a Tier 3 skim about a quarter — a deliberately conservative rounding, not a reading of the hour budgets above, which at face value would make Tier 2 a third to a fifth of Tier 1 and Tier 3 under a tenth. The rounding gives the lighter tiers room, because skims expand. Change them if your pace differs, and redo the arithmetic.

Counting the entries above gives 9 Tier 1, 13 Tier 2, and 6 Tier 3. Substituting, one term at a time:

$$N_{\text{eq}} = 1.0 \times 9 + 0.5 \times 13 + 0.25 \times 6 = 9 + 6.5 + 1.5 = 17.0$$

IN PLAIN WORDS

Seventeen paper-equivalents of work. The nine Tier 1 papers contribute nine units, since each counts fully. The thirteen Tier 2 papers contribute six and a half, half of thirteen. The six Tier 3 papers contribute one and a half, a quarter of six. Adding the subtotals gives seventeen. Notice that the Tier 2 entries dominate by count yet cost less than the Tier 1 ones: your week is decided by the Tier 1 column.

Spread over twelve weeks, $17.0/12 \approx 1.42$ paper-equivalents per week. In practice that is three deep papers every four weeks — the nine Tier 1 entries over twelve weeks — plus roughly one Tier 2 read a week and a Tier 3 skim

every other week. Priced at the Tier 1 rate of three to five hours per equivalent, that is about four to seven hours of reading a week; summing the raw hour budgets instead gives three and a half to five, the gap being the conservatism in the weights. Either way most of your 15 to 20 hours a week is left for building and debugging. The hiring signal comes from the rigs, not the reading list.

The same count, section by section, each line multiplying entries by their tier weights.

section	entries (tier)	weighted	subtotal
A.1	REINFORCE(2) PGtheorem(2)	.5+.5	1.00
A.2	TRPO(3) PPO(1) GAE(1)	.25+1+1	2.25
A.3	AWR(1) AWAC(2)	1+.5	1.50
A.4	DDPM(2) DiffPolicy(1) DDIM(3) ACT(2)	.5+1+.25+.5	2.25
A.5	FlowMatching(2) CFG(2)	.5+.5	1.00
A.6	Pi0(2) $\backslash(\backslash\pi_{0.5}\backslash)(1)$	.5+1	1.50
A.7	(no primary source)	none	0.00
A.8	RECAP(1)	1	1.00
A.9	Ng(2) CUPED(3) GRPO(3)	.5+.25+.25	1.00
A.10	Thompson(3) Russo(2) RTC(2)	.25+.5+.5	1.25
A.11	Dagger(1) HGDAgger(2) HILSERL(1)	1+.5+1	2.50
A.12	Sun(3)	.25	0.25
A.13	LeHome(1) BEHAVIOR(2)	1+.5	1.50
		TOTAL	17.00

Now group those subtotals into three blocks of four weeks. **Weeks 1–4** take Sections A.1–A.3: $1.00 + 2.25 + 1.50 = 4.75$ equivalents, about 1.19 per week, while you rebuild advantage estimation on rl-rover. **Weeks 5–8** take Sections A.4–A.8: $2.25 + 1.00 + 1.50 + 0.00 + 1.00 = 5.75$, about 1.44 per week — the generative block, read while the capstone's simulation stage trains in the background. **Weeks 9–12** take Sections A.9–A.13: $1.00 + 1.25 + 2.50 + 0.25 + 1.50 = 6.50$, about 1.63 per week, while you fine-tune on the real SO-101. Check the sum: $4.75 + 5.75 + 6.50 = 17.00$, matching the total above. Each block carries exactly three Tier 1 papers, which is the three-every-four-weeks rate made concrete.

Be honest about that curve: each block is heavier than the one before it, and the last is the heaviest — the block where real-robot debugging will eat your evenings. If weeks 9 through 12 slip, move the Sun et al. skim and CUPED forward; both are Tier 3, worth 0.25, readable in one sitting. Do not move DAGger or HIL-SERL: Section A.11 is 2.50 equivalents on its own, and it lands exactly where your hands will be.

CHECKPOINT

- Explain what each tier asks of you, and estimate the hours it implies.
- Say why the list is ordered by chapter rather than by publication date.
- Name the primary source for each of Chapters 6, 7, 8, 10, and 13 without looking back.
- State the six-entry minimal reading list behind LeHome, and what each contributes.
- Compute the paper-equivalent total for any subset, and rebalance the plan when a week is lost.
- For any Tier 1 paper here, name the experiment you would run on rl-rover or the SO-101.

Where to go deeper. The highest-return habit is a reading log: one paragraph per paper, written from memory a day later, naming the one idea and the one experiment. Keep it in the repository with the code it explains. When a paper resists two honest attempts, find the matching CS224R lecture or the LeRobot implementation and return with the mechanism already in your head.

Appendix B — The experiment ladder

Reading this book produces vocabulary. Only experiments produce skill. You can finish every chapter and still freeze the first time a run does something the book did not describe, because reading builds recognition and only running builds reflex.

So this appendix turns most chapters into one concrete experiment on hardware and code you already own: the `rl`-boarding gridworld, the `rl-rover` PPO stack, the LeRobot checkout and the SO-101 arm in `hil-serl`, and finally the LeHome stack itself.

Two chapters get no rung of their own, and that is deliberate rather than an oversight. Chapter 5's multimodality is not a separate experiment; it is the thing E5 measures, so running E5 is running Chapter 5. Chapter 9's missing log-probability is not something you can run at all — it is the gate on E5 and E6, the question you answer once both are done. Chapter 8 does get a task, but it lives inside E6 as part (b), because one forward pass through a released checkpoint is an afternoon's work rather than a rung of its own.

The rungs are ordered, and a rung's failure mode is only legible when the rung below it is solid. Times assume your 15–20 hours per week. Each rung also maps onto a phase of the companion roadmap — `ROADMAP.md`, the five-month schedule described in the front matter, whose phases are numbered 0 through 5 — and Section B.7 lines that up in one table. There are twelve rungs, E1 through E12. Two of them, E5 and E6, cover the generative chapters, which is where most of your new ground is.

B.1 Three rules that bind every rung

Rule one: one question per experiment. Write the question down before you touch the keyboard, as one sentence ending in a question mark, then design the smallest run that answers it. This sounds like bureaucracy and is not: it prevents the common failure where you change three things, get a better number, and cannot say which change caused it.

Rule two: measure, don't watch. Every rung ends in a number or a plot, never an impression. "It looked smoother" is not a result. Where runs are cheap — E2 and E3 especially — use 3–5 seeds, because a single-seed RL result is noise wearing the costume of a finding: two seeds of one configuration can differ by more than two genuinely different configurations do.

Rule three: log like you already do. One dated note per rung, in the `NEXT.md` style you use in your own repos: question, setup, result, surprise. The surprise line is the valuable one, because what surprised you is what your mental model got wrong. Those notes become the tech notes of roadmap Phase 5 almost verbatim, so writing them is not overhead.

An analogy, with its limit. The ladder is a test suite for your own understanding. E1–E3 are unit tests: fast, cheap, run five times, assert something narrow. E10 and E12 are integration tests: slow, expensive, failing for reasons the unit tests cannot localize. Where it breaks down: a unit test is deterministic and asserts one value, whereas every rung here returns a *distribution*. A red unit test means a bug; a "red" rung might just mean an unlucky seed, which is why "run it once" is never a gate.

WORDS YOU WILL HEAR

Ablation. Removing one component and re-measuring, to attribute a result to it — papers say "we ablate the value baseline". **Sweep.** Running one setup across a grid of a hyperparameter's values; a *seed sweep* varies only the random seed. **Open-loop rollout.** Running the policy's output without feeding consequences back — you sample chunks and inspect them, the robot never moves. Closed-loop means the world answers. **Wilson interval.** A confidence interval for a success rate that stays sane near 0% and 100%.

B.2 Foundations rungs (Chapters 1–3)

E1 · Values by hand

Chapter 1; rl-boarding; ~3 h. *Question:* do empirical returns really converge to a value function obeying the Bellman recursion? Run the gridworld under a fixed random policy for a few thousand episodes. Pick two cells, and for each average the discounted returns observed from it to estimate $V(s)$. Then check the recursion using the empirical successor frequencies you logged:

$$V(s) \approx \mathbb{E}[r + \gamma V(s')]$$

IN PLAIN WORDS

The left side, $V(s)$, is the number you just estimated by averaging returns out of cell s . The right side says: take one step from s , collect the immediate reward r , look up the value of wherever you landed, $V(s')$, shrink it by the discount γ , and add. $\mathbb{E}[\cdot]$ means average that over all the steps you took out of s , weighted by how often each successor came up. If the two sides agree, the long-horizon average and the one-step-plus-lookup agree — which is the whole content of the Bellman equation.

Repeat at $\gamma = 0.9$ and $\gamma = 0.99$. *You understood it if* you can predict, before rerunning, which cells' values move most when γ changes — and say why cells far from the goal feel the discount hardest.

E2 · REINFORCE from scratch, and the variance cure

Chapter 2; CartPole, already in the rover venv via `gymnasium[classic-control]`; ~5 h. Write REINFORCE in about sixty lines: a softmax policy and the estimator of Section 2.3, no library. Train 5 seeds with no baseline, then 5 with a learned value baseline. Plot all ten curves on one axis, and separately the per-batch standard deviation of the gradient estimate. *You understood it if* the baseline curves are visibly tighter, you can point at the line where G_t became $G_t - b(s_t)$, and you can explain why that subtraction cannot change the gradient's expectation.

E3 · PPO forensics on the rover

Chapter 3; rl-rover; ~5 h. Your `train_rover.py` runs are the corpus, so this rung is mostly reading. Read a healthy run's TensorBoard against Section 3.6: `approx_kl`, `clip_fraction`, `entropy_loss`, `value_loss`. Then break it twice on 300k-step runs: `clip_range=10`, and watch for the collapse Section 3.1 predicts; then `gae_lambda=0.0`, and watch early learning starve while the critic is still bad. Finally dump one episode's rewards and `policy.predict_values()` outputs and recompute $\hat{A}_t$ by hand at $\lambda \in \{0, 0.95\}$, reproducing Section 3.5's arithmetic on your own data. *You understood it if* you can diagnose, from curves alone, which knob a stranger's broken PPO run needs.

B.3 The pivot rungs (Chapters 4, 6, 7–8, 10)

E4 · AWR on your own logged data

Chapter 4; rl-rover; ~7 h. Build an offline dataset first: roll out a deliberately *mediocre* checkpoint — an early `ppo_rover_v0*.zip` — for about 500 episodes, storing observations, actions, and returns. Then train two fresh policies by supervised regression onto those actions: one unweighted, which is plain behavior cloning, and one weighting each sample:

$$w_i = \exp\left(\frac{\hat{A}_i}{\beta}\right), \quad \hat{A}_i = G_i - V(s_i)$$

IN PLAIN WORDS

w_i is how much sample i counts in the regression loss; 1 is neutral, 5 means it counts five times. $\hat{A}_i$ is that sample's advantage: the return G_i that actually followed it, minus what the old checkpoint's critic predicted for that state, $V(s_i)$ — positive when the sample went better than expected. β is a temperature setting how sharply you care: large β flattens every weight toward 1 and you are back at behavior cloning, small β makes the exponential explode and only a few lucky samples survive. This is ordinary supervised learning with one multiplier bolted on.

Normalize the advantages before exponentiating, and clip the weights at 20. Evaluate both policies over 100 episodes each — and write the uncertainty down beside the number, because 100 episodes is not many. For a mean return, the 95% interval is about $1.96 \sigma / \sqrt{100} = 0.196 \sigma$, a fifth of one episode's standard deviation, so compute σ over those episodes before you decide the gap is real. If you score success rather than return, the right tool is a Wilson interval instead, and near a 50% success rate 100 episodes buy you roughly plus or minus 10 points. *You understood it if* weighted beats unweighted, and you can show the weight histogram and say what β does to its shape — then push β toward zero and watch Chapter 4's dead-data failure appear in front of you.

E5 • Diffusion, multimodality, and the latency bill

Chapter 6; LeRobot in `hil-ser1`, GPU; ~8 h. Train LeRobot's diffusion policy on an existing SO-101 dataset — your 24-episode cube set is enough. This is the first rung whose policy has no density you can write down. E2's softmax and E3's Gaussian both hand you $\log \pi_\theta(a|s)$ on request; this one only hands you samples. Everything awkward in Chapters 9 through 12 follows from that one sentence, and the point of this rung is to meet it on your own hardware rather than take it from Chapter 5.

Two measurements. The first needs more care than it looks like it needs.

First, multimodality. Freeze one observation, draw 50 action chunks from it, and plot a **histogram** of the first action dimension. Look at the shape, not at the standard deviation. Spread and modality are not the same thing: one broad unimodal sampler scatters exactly as much as two tight modes do, and a single number like `.std()` cannot tell them apart. Separated clusters are the evidence you want. A single hump, however wide, is not.

Choose the observation deliberately as well, because your fixture can decide the answer before your model does. Pick a frame where *your own demonstrations diverge* — the moment before the grasp, where you sometimes came in from the left and sometimes from the right. If all 24 of your episodes did the same thing at that frame, then one tight cluster is the *correct* result: it means your dataset is unimodal there, not that training failed. Multimodality is a property of the data first and the model second, and a rung that cannot tell those apart is not measuring anything. If no frame in your 24 episodes qualifies, the cheapest fix is to record six more episodes approaching the cube from the other side, so the rung has a mode to find.

Second, the trade-off of Chapter 6's last section: sweep the denoising step count K and measure both action quality and per-query latency against a 30 Hz budget. "Action quality" has to be an actual number or the sweep means nothing, so fix one now and reuse it in E6. Hold out four episodes. For each of 200 held-out observations, sample 10 chunks and record the distance from the demonstrated chunk to its *nearest* sample; report the mean of those 200 distances. Nearest, not mean-of-samples — a policy that correctly covers both modes must not be penalized for the mode it did not take, which is precisely the scoring mistake that would make a mode-averaging policy look good.

That open-loop metric is a proxy, and you should say so in your notes. The honest measurement is closed-loop: 30 scored rollouts per value of K on the arm, which costs an afternoon and is worth paying once. Chapter 6's rig box already asks you for the small version of it — train at 100 denoising steps, sample at 10, confirm the success rate holds.

```

# NOT select_action(): it pops one action from an internal queue and only runs
# the denoiser when that queue empties, so 50 calls would time ~7 generations.
obs = dataset[0] # one frozen frame, reused everywhere
batch = {k: v[None].to(device) for k, v in obs.items()}
            if k.startswith("observation."): # real keys: observation.state,
                                                # observation.images.CAM -- batched

for K in [2, 4, 10, 50, 100]:
    policy.diffusion.num_inference_steps = K # set it where the SAMPLER reads it;
    policy.config.num_inference_steps = K    # the model copies this field at build
                                                # time, so config alone may do nothing

    samples, t0 = [], time.perf_counter()
    for _ in range(50):
        policy.reset() # clear any queued actions per draw
        with torch.no_grad():
            chunk = policy.predict_action_chunk(batch) # one full denoiser run
            samples.append(chunk[0, 0, 0].item()) # first action of the chunk, dim 0
        dt = (time.perf_counter() - t0) / 50 # true seconds per generation
    print(K, dt)
    plt.hist(samples, bins=30) # the SHAPE, not the std

```

Three things in that block are the rung rather than boilerplate, and none of them raises an exception when you get them wrong. That is why they are named out loud here instead of left in a comment.

One: `select_action` is the wrong entry point. It serves one action at a time out of an internal queue, and it only runs the denoiser when that queue is empty — every `n_action_steps` calls, so every eighth call at the default. Fifty calls therefore trigger about seven real generations, not fifty. Both of the rung's measurements break as a result. The reported latency comes out roughly eight times too low, so a genuine 100 ms query prints as 13 ms and ***K* = 100** looks affordable. And the values you collect mix variation across timesteps *inside* one chunk with variation across seven samples, which is not sample diversity at all. Use `predict_action_chunk`, which runs the denoiser once per call and hands back the chunk. (LeRobot's diffusion implementation slices that chunk to `n_action_steps` rows before returning it, which is harmless here since you only read row 0.)

Two: the step-count field may not be connected. It is read into the sampler when the model is constructed, so assigning to `policy.config` after the fact can silently change nothing at all, and your whole sweep quietly becomes five repetitions of one setting. Grep for where the field is actually read in your checkout. Then sanity-check it: print the latency at ***K* = 2** and at ***K* = 100** before you trust a single row of output, and if the two are the same number, the knob is not connected.

Three: those are not the observation keys. LeRobot's are `observation.state` and `observation.images.<your_camera>`, batched and moved to the device; there is no key called simply `observation`. That one at least fails loudly. The first two do not, and both are exactly the kind of silent measurement bug Rule Two exists to prevent.

Worked example: does *K* = 10 fit? Suppose the loop reports 100 ms per query at ***K* = 10**, matching the 0.1 s Chi et al. (2023) report on an RTX 3080. Split it in two. The observation encoder runs once per query; call it 12 ms. The remaining **100 – 12 = 88** ms is the denoising loop, so one denoiser pass costs **88/10 = 8.8** ms. Now predict any ***K*** instead of measuring it:

```

K = 4    -> 12 + 4 x 8.8 = 12 + 35.2 = 47.2 ms
K = 10   -> 12 + 10 x 8.8 = 12 + 88.0 = 100.0 ms
K = 100  -> 12 + 100x 8.8 = 12 + 880.0 = 892.0 ms

```

The budget is not one control period. It is the wall-clock length of the segment you execute between queries. At 30 Hz one period is **1/30 = 33.3** ms, and with `horizon = 16` and `n_action_steps = 8` you execute **8 × 33.3 = 266.7** ms of motion per query. So ***K* = 10** costs 100 ms of 266.7 ms, a **duty cycle** — the fraction of each execution window spent computing rather than moving — of **100/266.7 = 0.375**, meaning the GPU is busy 37% of the window and idle for the rest, which is comfortable. Low is good here: the idle fraction is your margin for a slow frame. ***K* = 100**

costs 892 ms, needing $892/33.3 = 26.8$, so 27 executed steps; but `n_action_steps` cannot exceed `horizon = 16`, so $K = 100$ is not merely slow here, it is impossible.

Now the other half, which is why you cannot just take the cheapest K . Say the 50 samples at $K = 10$ form two clusters at -0.31 and $+0.42$ rad, holding 45% and 55% of the draws. Their mixture mean is $0.45 \times (-0.31) + 0.55 \times (0.42) = -0.1395 + 0.2310 = 0.0915$. At $K = 4$ the samples collapse into one cluster at $+0.09$ rad — within 0.002 of that mixture mean, which is the uncomfortable part. That is the between-modes average Chapter 6 warns about: the policy has not become slightly worse, it has started emitting an action no demonstrator ever performed. Cheap K bought 53 ms and cost the property you adopted diffusion for.

E6 · Flow matching head-to-head, and Chapter 8's VLA shape trace

Chapters 7 and 8; same data, GPU; ~8 h. Two parts, both comparisons rather than new training runs. Loading a released checkpoint is the intended path: fine-tuning a π_0 -family model on 24 episodes is a weekend, not an evening, and it is not what this rung is measuring.

Part (a), flow versus diffusion. Load a π_0 -family flow-matching policy, point it at the same cube dataset, and run E5's two measurements on it: sample diversity from a frozen observation, and the quality-versus- K curve, using the same nearest-sample distance you fixed in E5 so the two rungs are comparable.

Now the part that will mislead you if nobody warns you. Do *not* compare the two policies on wall-clock milliseconds per query and call the winner a win for the formalism. LeRobot's diffusion policy is a U-Net of roughly 100M parameters; a π_0 -family policy is a 3B backbone plus a 300M action expert. Those differ by more than an order of magnitude, so the flow policy will be dramatically *slower* per query on your GPU even when its K is equal or lower. That is a parameter-count fact and has nothing to do with flow matching. Chapter 7's actual claim is about network *evaluations* per sample — about straight-line paths tolerating few steps — not about wall clock across different-sized networks.

So split the measurement in two and label which is which. Measure (i) *steps to reach a given quality*: the smallest K at which each policy's nearest-sample distance stops improving. This is Chapter 7's claim, and flow should win or tie. Measure (ii) *milliseconds per network evaluation*, and record it as a size fact about your two checkpoints, not as a verdict. A fair latency comparison of the two formalisms would need matched-size networks, which you are not going to train. What you can honestly compare is step count at equal quality, and the absence of schedule tables in the config.

Confirm too where the three knobs live, because the naming is inconsistent across the two families and everyone loses an hour to it once. The chunk horizon H is `chunk_size` in the π_0 family and `horizon` in diffusion. The step count K is `num_steps` in the π_0 -family config and `num_inference_steps` in the diffusion config — the same split naming as H , in the same two files. The execution length is `n_action_steps` in both, the one mercy. Field names drift between LeRobot releases, so open the two config dataclasses in your own checkout and read the names off them rather than trusting this paragraph; locating them is part of the rung.

Part (b), the shape trace. Load a `pi0` or `pi05` checkpoint in your LeRobot checkout, run one inference call, and print tensor shapes at each stage of Chapter 8's worked example: patches per camera, total prefix tokens, the suffix of noisy action tokens, the emitted chunk.

Expect three deliberate mismatches against the book's 783-token prefix, and do not spend an evening concluding you loaded the wrong model. First, there is no advantage token and no garment-type token: those are LeHome's own additions — the advantage token from Chapter 10, the garment-type token from Section 8.7's walkthrough — not part of the released architecture. Second, a stock π_0 carries robot state as a single projected token in the *suffix*, whereas LeHome bins each of its 12 joints into its own token in the *prefix*. Third, π_0 tokenizes a language prompt, and LeHome uses no language at all. What must line up is the image accounting — 256 tokens per 224×224 camera, so 768 for three — and the suffix shape: H noisy action tokens plus one flow-matching query token. *You understood it if*

you can say which tensors are computed once per decision and which are recomputed K times; that gate survives all three mismatches, which is why it is the gate.

You understood E5 and E6 if you can explain why neither policy has a usable `log_prob` method while E2's sixty-liner did — and why that missing method is what Chapter 9 is about.

ON YOUR RIG

You own one SO-101 leader/follower pair and one GPU, enough for every rung here but forcing serialization: you cannot train E5's policy while E10 holds the arm, so batch the GPU rungs into evenings and the arm rungs into weekends. Keep the 24-episode cube dataset frozen and versioned — it is the shared fixture that makes E5, E6, and E7 comparable. Record it midway and every earlier number becomes uncomparable.

E7 • Minimal RECAP: one conditioning bit

Chapter 10; `hil-serl` data, GPU; ~8 h. Conditioning needs failures, and your cube dataset is 24 successes out of 24 — so mix in failures from your HIL-SERL run buffers, or record about ten deliberate near-misses. Label every episode's chunks $c \in \{+, -\}$ by outcome and append the bit as an extra policy input.

Then train with roughly 30% **condition dropout**, and be exact about what that means, because getting it wrong here costs a retraining rather than a tuning pass. On about a third of the samples you replace the label with a *third, neutral value* — a null token, a specific vector meaning "no label given" — so that the same network also learns the unconditioned distribution. Dropout does not mean feeding the minus label on those samples, and it does not mean feeding nothing at all. Chapter 10 describes two structurally different ways to end up holding a second conditional; this rung implements the stochastic-replacement one with a null token, not $\pi_{0.6}^*$'s always-on two-term objective anchored at the negative conditional. Without that third value there is no unconditioned branch to take a difference against, and Chapter 12's guidance has nothing to work with.

Evaluate the policy queried at $c = +$ against an unconditioned twin trained on the same mixed data. *You understood it if $c = +$ beats the clean-data policy while the twin is dragged down — conditioning turned bad data into signal instead of poison.*

B.4 Engineering rungs (Chapters 11–12)

E8 • Shaping honestly, and CUPED on your logs

Chapter 11; `rl-rover`; ~6 h. Two parts. In part (a) you add a deliberately broken progress bonus — a raw "+k whenever distance to the goal shrinks", not potential-based — and short-train until you catch the policy farming it, oscillating toward and away from the goal. Then rewrite the same idea as $\gamma\Phi(s') - \Phi(s)$, where Φ is a **potential**: any function of the state alone, here minus the distance to the goal. The bonus becomes the *change* in potential rather than a raw payment, so a step out and back cancels exactly instead of paying twice. Confirm the exploit dies. Section 11.3's theorem will never land as hard as a rover that learned to twitch.

In part (b) you take E3's logged episodes, compute the correlation ρ_{GV} between returns and critic predictions, then the damping coefficient. This is **CUPED** — *controlled-experiment using pre-experiment data*, the A/B-testing trick of Section 11.5 that subtracts a correlated covariate scaled by a fitted coefficient rather than by 1 (Deng et al., 2013) — wearing RL clothes:

$$\alpha^* = \rho_{GV} \frac{\sigma_G}{\sigma_V}$$

IN PLAIN WORDS

α^* is how much of the critic's prediction to subtract from the return — not all of it, only the informative fraction. ρ_{GV} is the correlation between returns G and critic values V across your logged episodes: 1 means the critic tracks returns perfectly, 0 means it is noise. σ_G and σ_V are their standard deviations, and the ratio rescales V into G 's units. A trustworthy critic earns α^* near 1, so subtract it fully; a noisy one earns α^* near 0, meaning ignore it rather than let its noise contaminate your advantage.

A small worked example. Suppose your logs give $\sigma_G = 6.0$, $\sigma_V = 4.0$, $\rho_{GV} = 0.5$. Then $\alpha^* = 0.5 \times 6.0/4.0 = 0.5 \times 1.5 = 0.75$.

Compare variances with $\text{Var}(G - \alpha V) = \sigma_G^2 - 2\alpha\rho_{GV}\sigma_G\sigma_V + \alpha^2\sigma_V^2$. That expression is not a new fact, so unpack it once rather than reciting it. It is the ordinary variance of a difference, $\text{Var}(X - Y) = \text{Var}(X) - 2\text{Cov}(X, Y) + \text{Var}(Y)$, applied with $X = G$ and $Y = \alpha V$. Scaling V by α scales its covariance term by α and its variance term by α^2 , and $\text{Cov}(G, V)$ has been written out as $\rho_{GV}\sigma_G\sigma_V$. Every symbol earns its place: σ_G^2 is the noise you started with, the middle term is the payoff and is linear in α , the last term is the cost and is quadratic, and the whole thing is an upward parabola with one minimum at α^* .

Three numbers, then. No baseline at all: 36. Full subtraction, $\alpha = 1$:
 $36 - 2(1)(0.5)(6)(4) + 1(16) = 36 - 24 + 16 = 28$. At $\alpha^* = 0.75$:
 $36 - 2(0.75)(0.5)(6)(4) + 0.5625(16) = 36 - 18 + 9 = 27$.

Report both comparisons separately, because they answer different questions and it is easy to quote one while meaning the other. Against no baseline, $36 \rightarrow 27$, a cut of $(36 - 27)/36 = 25\%$. Against full subtraction — the naive move damping is supposed to beat — $28 \rightarrow 27$, a cut of $(28 - 27)/28 = 3.6\%$. The second number is the one that measures what damping actually bought, and here it is nearly nothing.

That is not a defect in the method; it is a fact about the regime, and Chapter 11 hands you the line that makes the regime legible. Full subtraction is worse than no baseline at all whenever $\sigma_V > 2\rho_{GV}\sigma_G$. Here that reads 4 against $2(0.5)(6) = 6$, and $4 < 6$, so you are in the benign regime where the textbook move already works and damping is a rounding error.

So run it again with a critic worth damping, because that is the case the chapter is actually about. Take $\sigma_G = 6.0$, $\sigma_V = 8.0$, $\rho_{GV} = 0.3$ — a head that bounces around more than the returns it is predicting, which is what E3's critic looks like in its first 50k steps. The diagnostic now reads 8 against $2(0.3)(6) = 3.6$, and $8 > 3.6$, so full subtraction should hurt.

It does. At $\alpha = 1$: $36 - 2(1)(0.3)(6)(8) + 64 = 36 - 28.8 + 64 = 71.2$ — nearly double the raw variance of 36, from the move Chapter 2 taught you as standard. At $\alpha^* = 0.3 \times 6.0/8.0 = 0.225$: $36 - 28.8(0.225) + 64(0.050625) = 36 - 6.48 + 3.24 = 32.76$, which agrees with the closed form $\sigma_G^2(1 - \rho_{GV}^2) = 36 \times 0.91 = 32.76$.

Report both comparisons here too. Against no baseline, $36 \rightarrow 32.76$, a 9% cut. Against full subtraction, $71.2 \rightarrow 32.76$, a cut of $(71.2 - 32.76)/71.2 = 54\%$. Same formula, same three symbols, same coefficient — and the point of the whole section is only visible in the second regime. Which regime your own logs land in is an empirical question you can settle in one line, so settle it before you decide damping was not worth the trouble.

You understood it if you can say which of E3's critics, early or late, deserves α closer to 1 — and predict which of the two regimes each one lands in before computing anything.

E9 • Inference knobs and a bandit to tune them

Chapter 12; E5's or E6's policy, in sim or on the arm; ~5 h. The checkpoint is frozen and only execution changes, which is what makes this rung cheap.

Sweep `n_action_steps` over four *distinct* values at about 30 scored episodes each, and keep the curve. For E5's diffusion policy, whose horizon is 16, use **{1, 4, 8, 16}**; note that at $H = 16$ the generic choice $H/2$ coincides with 8, so pick the four by hand rather than from a formula, or you will run two identical arms against each other and split the evidence between them. On a π_0 -family policy H is much larger — 50 in the released configs, 30 in LeHome — so the same four fractions land on entirely different integers.

Then be honest about what 30 episodes per arm can resolve, because this is where Rule Two bites the rung that states it. Near a 50% success rate, 30 episodes give a 95% Wilson interval of roughly plus or minus 18 points. So treat the sweep as a coarse ordering, not a ranking: the comparison you can actually make is between the best arm and the worst, not between neighbors. Chapter 12 makes the same point at scale — separating 70% from 75% takes on the order of a thousand trials — and Chapter 15 does the arithmetic in full. That imprecision is itself the argument for the bandit, which spends its episodes where the ordering is still in doubt rather than spreading them evenly across arms it has already separated.

Now hand the same four-way choice to the Beta-Thompson bandit of Section 12.6 — the fifteen-line implementation — and let it allocate the same total budget online. Skip best-of- N : you have no Q -like head until the capstone. *You understood it if* the bandit's posteriors put the same arm on top as your sweep did, having spent most of its episodes on the contenders — and if you can say why "the same arm on top" is the right gate here, while "the same full ordering" is a gate your sample size cannot support.

B.5 Human-and-real-world rungs (Chapters 13–14)

E10 · Close the HIL-SERL loop, and watch the human retire

Chapter 13; `hil-serl`, the run your `NEXT.md` says is one USB cable away; 2–3 robot sessions. Finish the cube run to a clean success rate over at least 50 scored episodes. Then instrument what Chapter 13 predicts: log the intervention flag per frame and plot intervention rate per 10-episode window. Your first working run sat at 98%, which is what the start of that curve should look like; it should decay as the policy absorbs your corrections. Keep the neutral-action histogram from your RB-release bug as a permanent dashboard check. Finally implement Section 13.4's weighting in one DAgger-style refit: corrections up-weighted, pre-takeover frames ramped to zero. *You understood it if* you can show the decay curve and say what a *flat* one would have told you.

E11 · Async with two workers

Chapter 14; `rover-mujoco`; ~9 h. Split rover training into Chapter 14's actor/learner roles on one machine: a learner consuming episode files, plus rollout workers that load the newest checkpoint from a shared directory. The filesystem stands in for the model hub, and that substitution is the second analogy worth naming: the shared directory *is* a message queue, checkpoints are messages, staleness is queue lag. Where it breaks down: a real queue delivers every message exactly once, whereas a checkpoint directory is a mailbox where workers read only the newest item and skip the rest — which is why staleness is a correctness question, not a latency one. Measure episodes per hour at 1 versus 2 workers, then kill a worker mid-run and confirm nothing but throughput suffers. Use AWR-style weighted updates on the learner, then note why PPO here would be quietly wrong as checkpoints lag. Your `audit_baseline.py` solvability check is the companion piece: rerun it whenever you widen randomization. *You understood it if* the crash cost throughput and nothing else, and you can name the property of the update rule that made that true.

B.6 The capstone gate (Chapter 15)

E12 · Stage A, exactly as written

Chapter 15; LeHome repos, GPU; 1–2 weeks. Clone the challenge environment and the solution repo, get Isaac Lab rendering, run *his* checkpoint through the official evaluator on the public garments, and land within a few points of the published per-type numbers: 74.5 / 70.0 / 80.5 / 93.5. Write the one-page codebase map Chapter 15 demands. This

rung is deliberately unoriginal, and that is its value: it converts the book into a running system you have verified. *You understood it if* your reproduced score has a confidence interval attached and you can point, in his code, at where each auxiliary head attaches and where the sampler applies the AWR priorities.

IN THE LEHOME SOLUTION

Nearly every rung has a counterpart in the LeHome stack, which is why the ladder ends there. E5 and E6's step-count and execution-length knobs are the inference-time parameters LeHome tunes; E7's conditioning bit is RECAP-style advantage conditioning in miniature; E8's damping is its α -damped value baseline; E9's bandit is how its inference knobs were set; E11's split is its actor/learner topology. E12 is where those toy pieces become one system trained on a single H200 (Larchenko, 2026).

B.7 The ladder at a glance

Rung	Chapter	Where	Time	Roadmap phase
E1 values by hand	1	rl-boarding	~3 h	Phase 0
E2 REINFORCE + baseline	2	CartPole	~5 h	Phase 0
E3 PPO forensics	3	rl-rover	~5 h	Phase 0
E4 AWR on logged data	4	rl-rover	~7 h	Phase 1
E5 diffusion + latency sweep	6	LeRobot + GPU	~8 h	Phase 1
E6 flow matching + VLA shape trace	7–8	LeRobot + GPU	~8 h	Phase 1
E7 minimal RECAP	10	hil-serl data	~8 h	Phase 1
E8 shaping + CUPED	11	rl-rover	~6 h	Phase 2–3
E9 knobs + bandit	12	E5/E6's policy	~5 h	Phase 3
E10 HIL-SERL close-out	13	SO-101	2–3 sessions	Phases 1, 4
E11 async, two workers	14	rover-mujoco	~9 h	Phase 2–3
E12 capstone Stage A	15	LeHome stack	1–2 wk	Phase 2

Do the addition rather than trusting a round number, because a schedule built on the wrong one costs weeks. The ten rungs quoted in hours add up to 64: **3 + 5 + 5 + 7 + 8 + 8 + 8 + 6 + 5 + 9**. E10's two or three robot sessions add roughly 10 to 15 more. And E12 is one to two weeks, which at your rate is another 15 to 40 — note that E12 is Stage A, so it comes before the capstone's Stage B rather than after it, and cannot be left out of the total. The ladder through Stage A therefore runs to roughly 90 to 120 hours of bench time. At 15 to 20 hours a week that divides out to five to eight weeks.

Those are bench hours only. The reading that goes with them is separate and roughly as large, which is why the roadmap gives Phases 0 through 2 ten weeks rather than five. E1–E3 compress into Phase 0's two weeks because the tooling already exists. E10 straddles the plan deliberately: start it early, because the robot teaches faster than the book does.

PITFALLS

Climbing rungs without the gates. The temptation is to "run" E4 or E7, see a plausible curve, and move on. The gate is the stated measurement plus the *you understood it if* test, spoken aloud or written down. An experiment that produced a number you cannot explain has not been finished; it has been visited.

CHECKPOINT

- State the three rules — one question, measure don't watch, log it — and what failure each prevents.
- Compute whether a chunked policy fits its real-time budget at a given step count, and say what dropping below the safe count does to sample quality.
- Say where H , K , and the execution length live in a LeRobot config, in both the diffusion and π_0 -family namings.
- Explain why a single-seed result on E2 or E3 is not a result, and what to run instead.
- Attach an interval to every success rate you report, and say what 30 scored episodes per arm can and cannot resolve.
- Name, for each rung you climbed, the surprise it produced and where you wrote it down.

Where to go deeper. The best next reading is your own logs: reread E3's TensorBoard after finishing E9, and you will see things you could not see the first time. For papers, Appendix A sequences them. For experimental hygiene, the reproducibility literature around deep RL — seed counts, confidence intervals, how often published gains fail to survive them — is worth an evening.